15-Day Django ORM & models.py Mastery — Nexora CRM

Oct 20, 2026 · @tanvik

15 days, 40 code tasks a day (600 total). Every task pairs Django code with the SQL it produces, run against PostgreSQL and a realistically sized Nexora dataset — so the ORM becomes a tool you can check, not a black box. It ends where analytics engineering begins: reporting queries, window functions and a verified query library.

How each day works

Each Day tab = 40 tasks in four blocks, solutions at the bottom. Tick each box once your code runs AND you have read its SQL.

Block	Tasks	Focus
A. Drills	1–10	One concept each, with the SQL printed
B. Core	11–20	The day's main ORM feature, built properly
C. Nexora CRM	21–30	Business questions on leads, calls, brokers, deposits and trades
D. Challenge	31–40	Bugs, wrong results, slow queries, edge cases

Rules:

- Type the code, don't paste. Solve first, then check the solutions.
- For every query, print its SQL (`str(qs.query)` or the SQL log) before trusting the result.
- When a result looks surprising, write the same query in `psql` and compare — your SQL is the referee.
- End every day with `python manage.py test` and `git commit -m "orm day N"`.
- Target: Django 5.2 LTS, Python 3.10+, PostgreSQL 15+.
- Days arrive 3 at a time; say "next" for the following batch.

Practice project: nexora-orm on PostgreSQL

One app, `crm`. The schema grows day by day (Day 1: desks, leads, brokers, SOP rules; Day 3: the full Nexora schema plus a seed of thousands of rows). PostgreSQL from the first day — SQLite hides too much (it doesn't enforce `max_length`, has no row locks and no real `DISTINCT ON`).

```
mkdir nexora-orm && cd nexora-orm
python -m venv .venv
.venv\Scripts\Activate.ps1                    # macOS/Linux: source
.venv/bin/activate
python -m pip install "django~=5.2" "psycopg[binary]" sqlparse
django-admin startproject django_project .
python manage.py startapp crm
createdb nexora_orm                           # your PG user also needs
CREATEDB (test databases)
```

```
# django_project/settings.py
import os

INSTALLED_APPS += ["crm"]
TIME_ZONE = "Asia/Dubai"
USE_TZ = True

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ.get("PGDATABASE", "nexora_orm"),
        "USER": os.environ.get("PGUSER", "postgres"),
        "PASSWORD": os.environ.get("PGPASSWORD", ""),
        "HOST": os.environ.get("PGHOST", "localhost"),
        "PORT": os.environ.get("PGPORT", "5432"),
    }
}

# SHOW_SQL=1 python manage.py shell  -> every query is printed as it runs
LOGGING = {
    "version": 1,
    "handlers": {"console": {"class": "logging.StreamHandler"}},
```

```
    "loggers": {"django.db.backends": {"handlers": ["console"],
                                       "level": "DEBUG" if
os.environ.get("SHOW_SQL") else "INFO"}},
}
```

```
# crm/sqltools.py -- the two helpers used on every day of the program
from contextlib import contextmanager

import sqlparse
from django.db import connection
from django.test.utils import CaptureQueriesContext


def show(queryset):
    """Print a queryset's SQL, formatted. (Parameters are inlined for READING
only.)"""
    print(sqlparse.format(str(queryset.query), reindent=True,
keyword_case="upper"))


@contextmanager
def queries(label=""):
    """with queries("list"): ...  -> prints how many queries ran and each
one's SQL."""
    with CaptureQueriesContext(connection) as ctx:
        yield ctx
    print(f"--- {label} {len(ctx)} quer{'y' if len(ctx) == 1 else 'ies'}")
    for q in ctx.captured_queries:
        print(f"[{q['time']}s] {q['sql']}")
```

```
python manage.py shell
>>> from crm.sqltools import show, queries
>>> from crm.models import Lead
>>> show(Lead.objects.filter(status="new"))
>>> with queries("count"):
...     Lead.objects.count()
```

`str(qs.query)` shows the SQL with parameters pasted in for reading — it is NOT always valid SQL to copy into `psql` (quoting may differ). For an exact plan use `qs.explain(analyze=True)` (PostgreSQL).

15-day roadmap

Day	Topic	By the end of the day you can
1	Models from zero	Turn a class into a table, create/read/update/delete rows, and read the SQL of each
2	Fields and constraints	Pick the right field for money, time, status and JSON; enforce rules in the database with constraints and indexes
3	Relationships	Model FK, one-to-one, many-to-many with `through`, and the Brf referral hierarchy; seed thousands of rows
4	Migrations	Read, write, reverse and squash migrations; data migrations; safe changes on big tables
5	QuerySet basics	`filter` / `exclude` / `Q`, every common lookup, laziness, slicing, ordering, `explain()`
6	Aggregation	`aggregate` vs `annotate`, GROUP BY with `values()`, conditional counts, HAVING
7	Expressions	`F`, `Value`, `Case` / `When`, `Coalesce`, `Cast`, date truncation and extraction, updates with expressions
8	Subqueries and window functions	`Subquery`, `OuterRef`, `Exists`, `Window` with `RowNumber` / `Lag` / running totals — first deposit per client, latest call per lead
9	Performance	N+1 hunting, `select_related` / `prefetch_related` / `Prefetch`, `only`, `values`, `iterator`, bulk operations, indexes with `EXPLAIN ANALYZE`
10	Transactions and concurrency	`atomic`, savepoints, `select_for_update`, `get_or_create` races, isolation levels, `on_commit`
11	Managers and custom QuerySets	Chainable domain methods (`Lead.objects.open().for_desk(d)`), soft-delete managers, where logic belongs

Day	Topic	By the end of the day you can
12	Raw SQL and PostgreSQL features	`raw()`, cursors, `RawSQL`, `ArrayField`, JSON queries, full-text search, `DISTINCT ON`, partial and GIN indexes
13	Model design	Abstract bases, multi-table and proxy inheritance, `clean()` / `full_clean()`, signals vs overriding `save()`
14	Reporting with the ORM	Cohorts, FTD funnels, Brf downline revenue with a recursive CTE, materialised views via migrations, unmanaged models, a read-only analytics database
15	Capstone	A Nexora query library: 20 business questions answered in the ORM, verified against hand-written SQL, with locked query counts

Quick reference: ORM → SQL

You already think in SQL. This table is the bridge; every row is drilled somewhere in the 15 days.

ORM	SQL it produces
`Lead.objects.filter(status="new")`	`SELECT ... FROM crm_lead WHERE status = 'new'`
`.exclude(status="lost")`	`WHERE NOT (status = 'lost')`
`.filter(Q(a) | Q(b))`	`WHERE a OR b`
`.filter(desk__slug="vip")`	`INNER JOIN crm_desk ... WHERE crm_desk.slug = 'vip'`
`.select_related("desk")`	the same JOIN, columns of both tables in one row
`.prefetch_related("calls")`	a second query: `... WHERE lead_id IN (1, 2, 3, ...)`

ORM	SQL it produces
`.order_by("-created_at")[:25]`	`ORDER BY created_at DESC LIMIT 25`
`.count()` / `.exists()`	`SELECT COUNT(*)` / `SELECT 1 ... LIMIT 1`
`.aggregate(total=Sum("amount"))`	one row: `SELECT SUM(amount)`
`.values("desk").annotate(n=Count("id"))`	`GROUP BY desk_id`
`Count("id", filter=Q(status="ftd"))`	`COUNT(id) FILTER (WHERE status = 'ftd')`
`.annotate(n=Count("calls")).filter(n__gt=3)`	`GROUP BY ... HAVING COUNT(...) > 3`
`F("call_attempts") + 1`	`call_attempts + 1` (a column reference, not a Python value)
`Case(When(...), default=...)`	`CASE WHEN ... THEN ... ELSE ... END`
`Subquery(...)` with `OuterRef("pk")`	a correlated subquery in SELECT or WHERE
`Exists(...)`	`WHERE EXISTS (SELECT 1 ... )`
`Window(RowNumber(), partition_by=..., order_by=...)`	`ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`
`.distinct("lead_id")` (PostgreSQL)	`DISTINCT ON (lead_id)`
`.update(status="lost")`	one `UPDATE ... WHERE ...`, no `save()`, no signals
`.select_for_update(skip_locked=True)`	`SELECT ... FOR UPDATE SKIP LOCKED`

The habit that makes all of this stick: write the question in SQL first, then make the ORM produce that SQL, then compare the two with `show()` and `explain()`.

Day 1 — Models from zero: a class is a table

Goal: see a model as exactly what it is — a table definition plus a Python API over it. By tonight `crm` has desks, leads, brokers and SOP rules, and you can predict the SQL of every create, read, update and delete before you run it.

```
class Lead(models.Model)        ->  CREATE TABLE crm_lead (
    name = CharField(100)       ->      name varchar(100) NOT NULL,
    desk = ForeignKey(Desk)     ->      desk_id bigint NOT NULL REFERENCES
crm_desk (id),
                                ->      id bigint GENERATED BY DEFAULT AS
IDENTITY PRIMARY KEY )
Lead.objects                    ->  the door to the table: every query starts
here
lead = Lead(...)                ->  one row, in memory only, until .save()
```

A. Drills (1–10) — table, row, query

- [ ] **1.** Model `Desk` (slug unique, name, is_active, created_at). `makemigrations`, then `sqlmigrate crm 0001`: read the `CREATE TABLE` line by line.
- [ ] **2.** Add `__str__` and a `Meta` with `ordering` and `verbose_name_plural`. Which of these changes the database?
- [ ] **3.** Create desks three ways: instance + `save()`, `objects.create()`, and `bulk_create()`. Print the SQL of each.
- [ ] **4.** Read them back with `all()`, `get()`, `first()` and `filter()`. Which ones hit the database, and when?
- [ ] **5.** Update a desk with `save()`, with `save(update_fields=[...])` and with `QuerySet.update()`. Compare the three UPDATE statements.
- [ ] **6.** Delete one desk with `instance.delete()` and several with `QuerySet.delete()`. What do they return?
- [ ] **7.** Where does `id` come from? Find `DEFAULT_AUTO_FIELD`, then give a test model a `UUIDField` primary key and compare the SQL.
- [ ] **8.** Model `Lead`: name, phone (unique), status (`TextChoices`: new, callback, ftd, lost), desk (FK, PROTECT), `created_at`, `updated_at`.
- [ ] **9.** Register both models in the admin with `list_display`, `list_filter` and `search_fields`; add a few rows by hand.

- [ ] **10.** In `python manage.py dbshell`, run `\d crm_lead` and match every column, constraint and index to your model.

B. Core (11–20) — the first Nexora tables

- [ ] **11.** Models `Broker` (code unique like BRF-0042, name, `commission_rate` as a decimal) and `SOPRule` (code, title, `max_hours`, severity as `IntegerChoices`).
- [ ] **12.** Model methods on `Lead`: an `is_open` property and `mark_status(new_status)` that saves only the changed columns.
- [ ] **13.** A classmethod `Lead.normalize_phone(raw)` used everywhere a phone enters the system.
- [ ] **14.** Show the `ORDER BY` that `Meta.ordering` adds to every query — and one query where it's unwanted.
- [ ] **15.** `help_text`, `verbose_name`, `db_comment` and `db_table_comment`. Which appear in PostgreSQL (`\d+ crm_lead`)?
- [ ] **16.** Use the status choices: `get_status_display()`, `Lead.Status.values`, `Lead.Status.FTD.label`. Is anything stopping `status="banana"` from reaching the database today?
- [ ] **17.** Override `save()` to normalise the phone. Then show three ways to write a lead that skip your `save()`.
- [ ] **18.** Add `clean()` to `Lead` (name not blank after trimming). Prove `save()` doesn't call it and `full_clean()` does.
- [ ] **19.** Timestamps: how is `created_at` stored in PostgreSQL, and what do you see when you read it back in Dubai time?
- [ ] **20.** Management command `seed_basics`: 3 desks, 2 brokers, 4 SOP rules, 30 leads. Safe to run twice.

C. Nexora CRM (21–30) — everyday table work

- [ ] **21.** Insert 1,000 test leads with a loop of `create()`, then with `bulk_create()`. Count the INSERT statements and time both.
- [ ] **22.** Find a lead from a phone typed any way (`050 000 0007`, `+971500000007`). Handle "not found" and "found twice" properly.
- [ ] **23.** `get_or_create` a desk by slug; `update_or_create` SOP rules from a Python list (a config sync you can run every deploy).

- [ ] **24.** Mark every `new` lead older than 7 days as `lost` in ONE statement. How many rows changed?
- [ ] **25.** Delete all leads whose name starts with `TEST` in one statement. What does it return?
- [ ] **26.** "Has this phone ever been seen?" — the cheapest possible query.
- [ ] **27.** `values()` and `values_list(flat=True)` for a quick export of desk names and lead phones.
- [ ] **28.** Order leads by status, then newest first, with leads that have no `updated_at` last.
- [ ] **29.** Page through leads 25 at a time with slicing. What SQL does `[50:75]` produce? What happens with `[-1]`?
- [ ] **30.** Test the model methods and the seed command. Commit `orm day 1`.

D. Challenge (31–40) — first-day traps

- [ ] **31.** Add a field to `Lead` and query it without making a migration. Copy the error.
- [ ] **32.** A model change with a migration that was never applied: find it with `showmigrations`; make CI fail on missing migrations.
- [ ] **33.** Why must you never edit a migration that has already been applied somewhere else?
- [ ] **34.** Add a required field to `Lead` while rows exist. What does `makemigrations` ask, and what are the consequences of each answer?
- [ ] **35.** Two shells load the same lead; each changes a different field and calls `save()`. Which change survives?
- [ ] **36.** Prove QuerySets are lazy: build one, chain it, and find the exact line where SQL runs. What does `repr()` in the shell do?
- [ ] **37.** Evaluate the same QuerySet twice, then reuse a cached one. When does Django query again?
- [ ] **38.** `get()` with no match and with two matches. Copy both exceptions; when is `first()` the better tool?
- [ ] **39.** Save a 101-character name into `CharField(max_length=100)` on PostgreSQL. Copy the error. Why might SQLite have accepted it?
- [ ] **40.** Extend `crm/sqltools.py` with `explain(qs)` that prints `EXPLAIN ANALYZE` output and the total time.

Solutions

Run the shell with `SHOW_SQL=1 python manage.py shell` (PowerShell: `$env:SHOW_SQL=1; python manage.py shell`) so every statement prints as it runs. Exact constraint and index names in the SQL below contain hashes that will differ on your machine.

A. Drills

1–2.

```
# crm/models.py
from django.db import models


class Desk(models.Model):
    slug = models.SlugField(unique=True)
    name = models.CharField(max_length=60)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:                                  # 2
        ordering = ["name"]
        verbose_name_plural = "desks"

    def __str__(self):
        return self.name
```

```
-- python manage.py sqlmigrate crm 0001
CREATE TABLE "crm_desk" (
  "id" bigint NOT NULL PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  "slug" varchar(50) NOT NULL UNIQUE,          -- SlugField defaults to
max_length=50
  "name" varchar(60) NOT NULL,
  "is_active" boolean NOT NULL,                -- no DEFAULT: Django fills
defaults in Python
  "created_at" timestamp with time zone NOT NULL
);
CREATE INDEX "crm_desk_slug_..._like" ON "crm_desk" ("slug"
varchar_pattern_ops);   -- for LIKE 'abc%'
```

2: `__str__`, `ordering` and `verbose_name_plural` are Python-side only. `makemigrations` still writes an `AlterModelOptions` migration (Django tracks the whole model state), but `sqlmigrate` shows no SQL for it.

3–4.

```
>>> d = Desk(slug="retention", name="Retention")     # nothing yet: a row in
memory
>>> d.save()
INSERT INTO "crm_desk" ("slug", "name", "is_active", "created_at")
  VALUES ('retention', 'Retention', true, '2026-10-
20T06:15:02+00:00'::timestamptz) RETURNING "crm_desk"."id"
>>> Desk.objects.create(slug="conversion", name="Conversion")     # the same
INSERT, one line
>>> Desk.objects.bulk_create([Desk(slug="vip", name="VIP"), Desk(slug="test",
name="Test")])
INSERT INTO "crm_desk" (...) VALUES (...), (...) RETURNING "crm_desk"."id"
-- ONE statement

>>> qs = Desk.objects.all()                  # no SQL
>>> list(qs)                                 # SELECT ... FROM crm_desk ORDER
BY name ASC
>>> Desk.objects.get(slug="vip")             # SELECT ... WHERE slug = 'vip'
LIMIT 21
>>> Desk.objects.first()                     # SELECT ... ORDER BY name ASC
LIMIT 1
>>> active = Desk.objects.filter(is_active=True)   # no SQL until used
```

4: `get()` asks for up to 21 rows so that a duplicate can be reported as "returned 2" (or "more than 20"). `first()` always orders — by `Meta.ordering`, or by `pk` when there is none.

5–7.

```
# 5
d.name = "VIP Desk"; d.save()
  UPDATE crm_desk SET slug='vip', name='VIP Desk', is_active=true,
created_at='...' WHERE id = 3
  -> EVERY column, from the values in memory (task 35 shows why that's
dangerous)
d.save(update_fields=["name"])
  UPDATE crm_desk SET name = 'VIP Desk' WHERE id = 3
Desk.objects.filter(slug="vip").update(name="VIP Desk")
  UPDATE crm_desk SET name = 'VIP Desk' WHERE slug = 'vip'      -> returns
the row count, no instance

# 6
d.delete()                                  -> (1, {'crm.Desk': 1})
Desk.objects.filter(slug="test").delete() -> (1, {'crm.Desk': 1})
  the dict lists every model affected, cascades included. Django first
SELECTs the related
  rows it must handle (cascade, protect, set null), then runs the DELETEs.

# 7
settings.DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"   (startproject
sets it;
apps.py's default_auto_field overrides it per app) -> bigint IDENTITY primary
keys.
```

```
# 7: a UUID primary key, for ids that leave the system (APIs, the trading
platform)
import uuid


class ExternalRef(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4,
editable=False)
# SQL: "id" uuid NOT NULL PRIMARY KEY -- generated in PYTHON by uuid4(), not
by the database
```

8–10.

```
# crm/models.py
class Lead(models.Model):
    class Status(models.TextChoices):
        NEW = "new", "New"
```

```
        CALLBACK = "callback", "Callback"
        FTD = "ftd", "First deposit"
        LOST = "lost", "Lost"

    name = models.CharField(max_length=100)
    phone = models.CharField(max_length=20, unique=True)
    status = models.CharField(max_length=20, choices=Status.choices,
default=Status.NEW)
    desk = models.ForeignKey(Desk, on_delete=models.PROTECT,
related_name="leads")
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return f"{self.name} ({self.phone})"


# crm/admin.py  (9)
from django.contrib import admin

from .models import Desk, Lead


@admin.register(Lead)
class LeadAdmin(admin.ModelAdmin):
    list_display = ("name", "phone", "status", "desk", "created_at")
    list_filter = ("status", "desk")
    search_fields = ("name", "phone")


admin.site.register(Desk)
```

```
# 10  python manage.py dbshell  ->  \d crm_lead
 Column     | Type                     | Nullable | Default
 id         | bigint                   | not null | generated by default as
identity
 name       | character varying(100)   | not null |
 phone      | character varying(20)    | not null |
 status     | character varying(20)    | not null |
 created_at | timestamp with time zone | not null |
 updated_at | timestamp with time zone | not null |
 desk_id    | bigint                   | not null |
Indexes:
  "crm_lead_pkey" PRIMARY KEY (id)
  "crm_lead_phone_key" UNIQUE CONSTRAINT (phone)           <- unique=True
  "crm_lead_desk_id_.." btree (desk_id)                    <- every FK gets
an index
  "crm_lead_phone_..._like" btree (phone varchar_pattern_ops)
Foreign-key constraints:
  "crm_lead_desk_id_..._fk_crm_desk_id" FOREIGN KEY (desk_id) REFERENCES
crm_desk(id)
      DEFERRABLE INITIALLY DEFERRED                          <- checked at
COMMIT, not per statement
```

B. Core

11–13.

```
# crm/models.py
import re
from decimal import Decimal


class Broker(models.Model):                                              #
11
    code = models.CharField(max_length=20, unique=True)                  #
BRF-0042
    name = models.CharField(max_length=100)
    commission_rate = models.DecimalField(max_digits=5, decimal_places=4,
default=Decimal("0.0500"))

    def __str__(self):
        return self.code
```

```
class SOPRule(models.Model):
    class Severity(models.IntegerChoices):
        LOW = 1, "Low"
        MEDIUM = 2, "Medium"
        HIGH = 3, "High"

    code = models.SlugField(unique=True)                                    #
first-contact-24h
    title = models.CharField(max_length=120)
    max_hours = models.PositiveIntegerField(default=24)    # PG: integer +
CHECK (max_hours >= 0)
    severity = models.PositiveSmallIntegerField(choices=Severity.choices,
default=Severity.MEDIUM)

    def __str__(self):
        return self.code


class Lead(models.Model):
    ...
    @property
    def is_open(self):                                                      #
12
        return self.status in {self.Status.NEW, self.Status.CALLBACK}

    def mark_status(self, new_status):
        self.status = new_status
        self.save(update_fields=["status", "updated_at"])   # auto_now only
updates if LISTED

    @classmethod
    def normalize_phone(cls, raw):                                          #
13
        digits = re.sub(r"\D", "", raw or "")
        if digits.startswith("00"):
            digits = digits[2:]
        elif digits.startswith("0"):
            digits = "971" + digits[1:]
        return "+" + digits
```

12: `update_fields=["status"]` alone would leave `updated_at` untouched — `auto_now` fires only for fields actually written.

14–16.

```
>>> show(Lead.objects.filter(status="new"))
SELECT ... FROM "crm_lead" WHERE "crm_lead"."status" = new ORDER BY
"crm_lead"."created_at" DESC

# 14: where Meta.ordering hurts -- DISTINCT plus an ordering on another
column
>>> list(Lead.objects.values_list("desk_id", flat=True).distinct())
# SELECT DISTINCT desk_id, created_at ... ORDER BY created_at DESC  ->
created_at joins the
# DISTINCT, so you get one row per lead, not one per desk.
>>> list(Lead.objects.order_by().values_list("desk_id",
flat=True).distinct())   # [1, 2, 3]
```

14: an empty `order_by()` removes the default ordering. (Since Django 3.1, `Meta.ordering` is no longer applied to `GROUP BY` queries, and `count()` / `exists()` drop it themselves.)

```
# 15
name = models.CharField(max_length=100, verbose_name="client name",
help_text="As on the ID",
                        db_comment="Client full name as written on the ID
document")
class Meta:
    db_table_comment = "Leads from all desks. One row per client phone
number."
# verbose_name / help_text: Python only (admin, forms). db_comment /
db_table_comment
# (Django 4.2+): COMMENT ON COLUMN / TABLE -> visible in \d+ crm_lead.
```

```
# 16
>>> lead.get_status_display()            # 'First deposit'
>>> Lead.Status.values                   # ['new', 'callback', 'ftd', 'lost']
>>> Lead.Status.FTD.label                # 'First deposit'
>>> Lead.objects.filter(pk=lead.pk).update(status="banana")    # 1 -- the
database accepts it!
# choices are checked only by full_clean() and forms. Day 2 adds a
CheckConstraint so the
# DATABASE refuses "banana" too.
```

17–20.

```
# crm/models.py -- Lead
from django.core.exceptions import ValidationError

    def save(self, *args, **kwargs):                                        #
17
        self.phone = self.normalize_phone(self.phone)
        super().save(*args, **kwargs)                    # never forget to
call it

    def clean(self):                                                        #
18
        if not (self.name or "").strip():
            raise ValidationError({"name": "Name can't be blank."})
```

```
# 17  three ways to write a lead WITHOUT your save():
Lead.objects.filter(pk=7).update(phone="050 1")     one UPDATE statement; no
instance, no save()
Lead.objects.bulk_create([Lead(...), ...])          one INSERT; save() is
never called
Lead.objects.bulk_update(leads, ["phone"])          one UPDATE ... CASE;
save() is never called
(and raw SQL / loaddata). Rules that MUST always hold belong in the database
(Day 2).

# 18
>>> lead = Lead(name="   ", phone="0500000999", desk=desk)
>>> lead.save()            # saved! save() never calls clean()
>>> lead.full_clean()
ValidationError: {'name': ["Name can't be blank."]}
# full_clean() = clean_fields() + clean() + validate_unique() +
validate_constraints().
# ModelForms call it for you; plain code must call it itself.
```

```
# 19
>>> lead.created_at
datetime.datetime(2026, 10, 20, 6, 15, 2, tzinfo=datetime.timezone.utc)
>>> from django.utils import timezone
>>> timezone.localtime(lead.created_at)
datetime.datetime(2026, 10, 20, 10, 15, 2,
tzinfo=zoneinfo.ZoneInfo(key='Asia/Dubai'))
# PostgreSQL stores timestamptz as an absolute instant; Django sets the
connection to UTC.
# Templates and localtime() convert to TIME_ZONE for display. In psql:
#   SET TIME ZONE 'Asia/Dubai'; SELECT created_at FROM crm_lead;   -> ...
10:15:02+04
```

```
# crm/management/commands/seed_basics.py  (20) + empty __init__.py files in
management/ and commands/
from django.core.management.base import BaseCommand

from crm.models import Broker, Desk, Lead, SOPRule

DESKS = [("retention", "Retention"), ("conversion", "Conversion"), ("vip",
"VIP")]
RULES = [("first-contact-24h", "First contact within 24 hours", 24, 3),
         ("no-guaranteed-returns", "Never promise returns", 0, 3),
         ("kyc-before-ftd", "KYC approved before first deposit", 0, 3),
         ("call-recording-consent", "Recording consent on every call", 0, 2)]


class Command(BaseCommand):
    help = "Small, repeatable starter data"

    def handle(self, *args, **opts):
        desks = [Desk.objects.get_or_create(slug=s, defaults={"name": n})[0]
for s, n in DESKS]
        for code, name in [("BRF-0001", "Gulf Partners"), ("BRF-0042",
"Desert IB")]:
            Broker.objects.get_or_create(code=code, defaults={"name": name})
        for code, title, hours, severity in RULES:
            SOPRule.objects.update_or_create(code=code, defaults={
                "title": title, "max_hours": hours, "severity": severity})
        for n in range(1, 31):
```

```
            Lead.objects.get_or_create(phone=f"+9715{n:08d}", defaults={
                "name": f"Lead {n:03d}", "desk": desks[n % 3],
                "status": Lead.Status.values[n % 4]})
        self.stdout.write(self.style.SUCCESS("seeded"))
```

20: `get_or_create` / `update_or_create` keyed on the natural keys (slug, code, phone) make the command idempotent — run it twice and nothing is duplicated. `update_or_create` also pushes changed titles on every run, which is what a config sync wants.

C. Nexora CRM

21–23.

```
# 21: python manage.py shell
import time

from django.db import connection
from django.test.utils import CaptureQueriesContext

from crm.models import Desk, Lead


def timed(label, fn):
    start = time.perf_counter()
    with CaptureQueriesContext(connection) as ctx:
        fn()
    print(f"{label}: {len(ctx)} queries, {time.perf_counter() - start:.2f}s")


desk = Desk.objects.get(slug="retention")
timed("create() x1000", lambda: [Lead.objects.create(name=f"Bulk {n}",
phone=f"+9715{n:08d}", desk=desk)
                                 for n in range(10_001, 11_001)])
timed("bulk_create", lambda: Lead.objects.bulk_create(
    [Lead(name=f"Bulk {n}", phone=f"+9715{n:08d}", desk=desk) for n in
range(11_001, 12_001)]))
# create() x1000: 1000 queries, several seconds (each INSERT is its own round
trip + commit)
# bulk_create:    1 query, tens of milliseconds   (add batch_size=500 for
very large lists)
Lead.objects.filter(name__startswith="Bulk ").delete()      # clean up
```

21: `bulk_create` skips `save()` — the phone normalisation from task 17 does NOT run, so pass already-clean values (they are, here).

```
# 22
def find_lead(raw_phone):
    try:
        return Lead.objects.get(phone=Lead.normalize_phone(raw_phone))
    except Lead.DoesNotExist:
        return None
# MultipleObjectsReturned can't happen: phone is UNIQUE, so the database
guarantees at most
# one row. For a non-unique lookup (by name) use .filter() and decide what
"two" means.

# 23
desk, created = Desk.objects.get_or_create(slug="vip", defaults={"name":
"VIP"})
for code, title, hours, severity in RULES:
    rule, created = SOPRule.objects.update_or_create(
        code=code, defaults={"title": title, "max_hours": hours, "severity":
severity})
```

23: `get_or_create` = SELECT, then INSERT if missing (inside a savepoint, retrying the SELECT if a parallel INSERT won the race). `update_or_create` runs `SELECT ... FOR UPDATE` in a transaction, then UPDATE or INSERT. `defaults` are used for both paths; Django 5.0's `create_defaults` lets the INSERT use different values.

24–27.

```
from datetime import timedelta

from django.utils import timezone

# 24 -- make some leads old enough first (the seed created them just now)
Lead.objects.filter(pk__lte=5).update(created_at=timezone.now() -
timedelta(days=10))
cutoff = timezone.now() - timedelta(days=7)
changed = (Lead.objects.filter(status=Lead.Status.NEW, created_at__lt=cutoff)
           .update(status=Lead.Status.LOST, updated_at=timezone.now()))
# UPDATE crm_lead SET status = 'lost', updated_at = '...' WHERE status =
'new' AND created_at < '...'
# changed -> number of rows. update() skips auto_now, so updated_at is set by
hand.
```

```
# 25
Lead.objects.filter(name__startswith="TEST").delete()      # (3, {'crm.Lead':
3})
# today: ONE statement -- DELETE FROM crm_lead WHERE name LIKE 'TEST%'. Once
other tables
# point at Lead (Day 3), Django must first SELECT the ids to cascade, then
delete by id.

# 26
Lead.objects.filter(phone=Lead.normalize_phone(raw)).exists()
# SELECT 1 AS "a" FROM crm_lead WHERE phone = '+971500000007' LIMIT 1   --
served by the unique index

# 27
list(Desk.objects.values("slug", "name"))       # [{'slug': 'conversion',
'name': 'Conversion'}, ...]
list(Lead.objects.values_list("phone", flat=True)[:5])
list(Lead.objects.values_list("name", "desk__slug"))       # follows the FK
with a JOIN
```

28–29.

```
# 28 -- updated_at is NOT NULL, so add the nullable field Day 3 relies on
anyway:
#   last_contacted_at = models.DateTimeField(null=True, blank=True)   +
makemigrations/migrate
from django.db.models import F

Lead.objects.order_by("status", F("last_contacted_at").desc(nulls_last=True))
# ORDER BY status ASC, last_contacted_at DESC NULLS LAST
# (PostgreSQL puts NULLs FIRST in descending order by default -- nulls_last
overrides it)

# 29
Lead.objects.order_by("id")[50:75]       # ... ORDER BY id ASC LIMIT 25
OFFSET 50  (still lazy)
Lead.objects.all()[-1]                   # ValueError: Negative indexing is
not supported.
Lead.objects.order_by("-id").first()     # how to get "the last one"
```

30.

```
# crm/tests.py
from io import StringIO

from django.core.management import call_command
from django.db import connection
from django.test import TestCase
from django.test.utils import CaptureQueriesContext

from .models import Desk, Lead, SOPRule


class Day1Tests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_basics", stdout=StringIO())

    def test_seed_is_idempotent(self):
        call_command("seed_basics", stdout=StringIO())
        self.assertEqual((Desk.objects.count(), Lead.objects.count(),
SOPRule.objects.count()), (3, 30, 4))

    def test_phone_normalized_on_save(self):
        lead = Lead.objects.create(name="Sara", phone="050 123 4567",
desk=Desk.objects.first())
        self.assertEqual(lead.phone, "+971501234567")

    def test_mark_status_writes_two_columns(self):
        lead = Lead.objects.filter(status=Lead.Status.NEW).first()
        with CaptureQueriesContext(connection) as ctx:
            lead.mark_status(Lead.Status.CALLBACK)
        self.assertEqual(len(ctx), 1)
        self.assertNotIn('"name"', ctx.captured_queries[0]["sql"])    # only
status + updated_at

    def test_is_open(self):
        self.assertTrue(Lead(status=Lead.Status.CALLBACK).is_open)
        self.assertFalse(Lead(status=Lead.Status.LOST).is_open)
```

```
python manage.py test crm && git add -A && git commit -m "orm day 1"
```

D. Challenge

31–34.

```
# 31  a new field in models.py, no migration
django.db.utils.ProgrammingError: column crm_lead.source does not exist
-> Django's SELECT lists every field the MODEL has; the TABLE doesn't have it
yet.

# 32
python manage.py showmigrations crm            [X] 0001_initial  [ ]
0002_lead_source
python manage.py makemigrations --check --dry-run   exits non-zero if models
changed without a migration
python manage.py migrate --check                    exits non-zero if
migrations are unapplied
-> put the first one in CI: nobody can merge a model change without its
migration.

# 33  editing an applied migration
The database records migrations by NAME in django_migrations. Databases that
already ran
the old version never run your edit; fresh databases (tests, new laptops) run
the new one.
Schemas silently diverge -- "works on my machine". Change things with a NEW
migration.

# 34  a required field on a table that has rows
makemigrations: "1) Provide a one-off default now (will be set on all
existing rows)
                 2) Quit and manually define a default value in models.py."
1 -> existing rows get a value nobody chose on purpose (often wrong for
reports later);
2 -> a real default that new rows get too. Often the honest answer is
null=True first,
     backfill with a data migration, then make it NOT NULL (Day 4).
```

35–38.

```
# 35  shell A and shell B load lead 7
A: lead.status = "callback"; lead.save()
B: lead.name = "Omar Khalid"; lead.save()     <- B's object still holds
status='new'
B's save() writes EVERY column, so status goes back to 'new'. A's change is
lost, silently.
fixes: save(update_fields=[...]); QuerySet.update(); F() expressions; a
version column (Day 10).
```

```
# 36 -- laziness
qs = Lead.objects.filter(status="new")     # no SQL
qs = qs.exclude(desk__slug="vip")          # no SQL
qs = qs.order_by("name")                   # no SQL
for lead in qs:                            # <- SQL runs HERE, once
    ...
# Also evaluated by: list(), len(), bool()/if qs, slicing with a step,
pickling -- and repr():
# typing `qs` at the >>> prompt prints its repr, which runs SELECT ... LIMIT
21.

# 37 -- the result cache
qs = Lead.objects.filter(status="new")
list(qs); list(qs)                         # ONE query: the second loop reads
the cache
qs.count()                                 # no query either: len() of the
filled cache
list(Lead.objects.filter(status="new"))    # a NEW QuerySet -> a new query
qs.all()                                   # also a new QuerySet: .all()
never reuses the cache
```

```
# 38
Lead.objects.get(phone="+971599999999")
  crm.models.Lead.DoesNotExist: Lead matching query does not exist.
Lead.objects.get(status="new")
  crm.models.Lead.MultipleObjectsReturned: get() returned more than one Lead
-- it returned 8!
get():   exactly one MUST exist (a unique key) -- anything else is a bug
worth an exception.
first(): zero is a normal answer and any one row will do ("the newest new
lead").
```

39–40.

```
# 39
Lead.objects.create(name="x" * 101, phone="050 000 0101", desk=desk)
django.db.utils.DataError: value too long for type character varying(100)
SQLite ignores declared VARCHAR lengths and would have stored all 101
characters --
one reason this program uses PostgreSQL from the first day.
```

```
# crm/sqltools.py  (40)
def explain(queryset):
    """EXPLAIN ANALYZE with sequential scans flagged. Runs the query for
real."""
    plan = queryset.explain(analyze=True, buffers=True)
    for line in plan.splitlines():
        print(">>" if "Seq Scan" in line else "  ", line)

# >>> explain(Lead.objects.filter(phone="+971500000007"))
#    Index Scan using crm_lead_phone_key on crm_lead ... (actual
time=0.02..0.02 rows=1 loops=1)
#    Planning Time: 0.1 ms
#    Execution Time: 0.05 ms
```

40: `analyze=True` executes the query — never run it on an UPDATE/DELETE you don't want applied. On a 30-row table PostgreSQL may prefer a Seq Scan even when an index exists; that's correct, not a problem. The plans start to matter on Day 9's 200,000 rows.

Day 2 — Fields and constraints

Goal: choose every column's type and rules on purpose — and push the rules that must ALWAYS hold into PostgreSQL, where no `update()`, `bulk_create()` or other system can bypass them. By tonight Nexora has trading accounts and deposits modelled to the cent.

```
Where a rule can live                        Who can bypass it
form / full_clean() validators               QuerySet.update(), bulk_create(),
raw SQL, other systems
Model.save() override                        the same list
Database constraint (CHECK, UNIQUE, FK)      nobody -- that's the point
```

A. Drills (1–10) — field types and their PostgreSQL types

- [ ] **1.** `CharField` vs `TextField`: which PostgreSQL type does each create? When do you choose each?
- [ ] **2.** Show the float problem with money (`0.1 + 0.2`), then do the same sums with `Decimal` and `quantize()`.
- [ ] **3.** The integer family: `SmallIntegerField`, `IntegerField`, `BigIntegerField`, the `Positive...` versions. Should a trading account number be an integer or a string?
- [ ] **4.** A three-state KYC flag (approved / rejected / not checked yet) with `BooleanField(null=True)`. Why not two booleans?
- [ ] **5.** Call length as `DurationField` vs a seconds integer. Compare their SQL types and how each sums.
- [ ] **6.** `Lead.kyc = JSONField(default=dict, blank=True)`: store nationality and passport expiry; filter on a key.
- [ ] **7.** `UUIDField` for an external reference and `GenericIPAddressField` for a login IP. Which PostgreSQL types?
- [ ] **8.** Save `"not-an-email"` into an `EmailField` with `.save()`, then call `full_clean()`. What does that tell you about validators?
- [ ] **9.** `IntegerChoices` for leverage (50, 100, 200) and `TextChoices` for deposit method.
- [ ] **10.** A throwaway model with one field of every type above; read `sqlmigrate` and list each field's SQL type.

B. Core (11–20) — null, defaults, constraints, indexes

- [ ] **11.** The `null` / `blank` matrix: four combinations, what each means for the database and for forms.
- [ ] **12.** Defaults: a constant, a callable (`timezone.now`), and the mutable trap (`default={}`). Read the check warning.
- [ ] **13.** `db_default` (Django 5.0+): a default the DATABASE applies. Compare the SQL with `default`.
- [ ] **14.** `unique=True` vs `UniqueConstraint`; a composite unique key: account number + platform.
- [ ] **15.** A conditional `UniqueConstraint`: at most one PRIMARY trading account per lead.
- [ ] **16.** `CheckConstraint`s: status in its choices, commission rate between 0 and 0.5, deposit amount above 0, fx rate above 0.
- [ ] **17.** `violation_error_message` on a constraint; see it from `full_clean()`, and see what the database says instead.
- [ ] **18.** Indexes: composite, descending, partial (open leads only) and functional (`Upper("name")`). Read each one's SQL.
- [ ] **19.** `GeneratedField` (Django 5.0+): `Deposit.amount_usd = amount × fx_rate`, computed and stored by PostgreSQL.
- [ ] **20.** `db_comment` on the money columns; a custom `db_table` for a table another system also reads.

C. Nexora CRM (21–30) — accounts and money

- [ ] **21.** `TradingAccount`: account number, lead (FK), currency (3 upper-case letters, enforced in the DB), leverage, `is_primary`, `opened_at`.
- [ ] **22.** `Deposit`: account (FK, PROTECT), amount, currency, fx rate, `amount_usd` (generated), method, status, `created_at`.
- [ ] **23.** KYC queries on the JSON column: leads with `kyc.status = "approved"`, leads that have a passport number, leads from a list of nationalities.
- [ ] **24.** Make every constraint fire once in the shell. Copy each error and find the constraint's name in it.
- [ ] **25.** Turn constraint failures into friendly messages with `full_clean()` before saving.
- [ ] **26.** Assign `100.555` to a 2-decimal amount. What does `full_clean()` say? Who rounds on save, and how should YOU round?

- [ ] **27.** Money helpers in `crm/money.py`: `to_usd(amount, rate)` and `total(amounts)`, Decimal-only, explicit rounding.
- [ ] **28.** For three real Nexora queries, predict which index each will use; check with `explain()`.
- [ ] **29.** Extend `seed_basics`: one or two accounts per lead and a few deposits each.
- [ ] **30.** Constraint tests with `assertRaises(IntegrityError)`. Commit `orm day 2`.

D. Challenge (31–40) — data-type traps

- [ ] **31.** Store deposits in a `FloatField`, add up 10,000 of them, compare with the Decimal total.
- [ ] **32.** `CharField(null=True)`: show the two different "empty" values and a query that misses one.
- [ ] **33.** A unique column that allows NULL: how many NULLs can it hold in PostgreSQL? Make it allow only one.
- [ ] **34.** `default=timezone.now()` WITH parentheses. What goes wrong, and where does it show up?
- [ ] **35.** Mutate the `kyc` dict of one unsaved lead created with a shared mutable default. Who else changes?
- [ ] **36.** Lengthen and then shorten a `CharField` on a big table. Which one rewrites the table?
- [ ] **37.** Add a new status to `TextChoices` but forget the `CheckConstraint`. What happens on insert?
- [ ] **38.** Save a naive `datetime`. Copy the warning and explain what was stored.
- [ ] **39.** `kyc__age=30` vs `kyc__age="30"` vs `kyc__contains={...}`: which match, and why?
- [ ] **40.** Create a deposit and print `amount_usd` straight away. What do you see? Fix it.

Solutions

The complete `TradingAccount` and `Deposit` models are in block C (tasks 21–22); block B shows the pieces as they are introduced.

A. Drills

1–4.

```
# 1
CharField(max_length=100) -> varchar(100)   bounded identifiers and labels:
names, phones, codes
TextField()               -> text           free text: call notes, KYC
remarks
In PostgreSQL both perform the same; varchar(n) simply adds a length check.

# 3
SmallIntegerField    smallint   -32,768 .. 32,767
IntegerField         integer    about +-2.1 billion
BigIntegerField      bigint     about +-9.2 x 10^18
Positive...Field     the same type + CHECK (col >= 0)
Account number: the platform issues numeric logins and nobody does arithmetic
on them,
but they're never formatted with leading zeros -> BigIntegerField is fine.
Anything that
can carry leading zeros, a '+' or letters (phones, IBANs, BRF codes) is a
CharField.

# 4  kyc_approved = models.BooleanField(null=True)  -> boolean, NULL allowed
True = approved, False = rejected, None = not checked yet.
Two booleans (approved, rejected) allow an impossible fourth state: both
True.
(When a fourth state is likely one day -- "expired" -- a TextChoices status
is clearer still.)
```

```
# 2
>>> 0.1 + 0.2
0.30000000000000004
>>> from decimal import Decimal, ROUND_HALF_UP
>>> Decimal("0.1") + Decimal("0.2")
Decimal('0.3')
>>> (Decimal("100") * Decimal("3.6725")).quantize(Decimal("0.01"),
rounding=ROUND_HALF_UP)
Decimal('367.25')
>>> Decimal(0.1)            # built from a FLOAT: the error is already inside
Decimal('0.1000000000000000055511151231257827021181583404541015625')
# always build Decimals from strings (or from the database), never from
floats
```

5–9.

```
import uuid

from django.db import models


class CallSample(models.Model):                                          #
5
    length = models.DurationField()              # PostgreSQL interval ->
Sum() gives a timedelta
    duration_sec = models.PositiveIntegerField()  # integer             ->
Sum() gives an int


# Lead  (6)
kyc = models.JSONField(default=dict, blank=True)  # jsonb

>>> lead.kyc = {"status": "approved", "nationality": "AE", "passport_expiry":
"2031-04-30"}
>>> lead.save(update_fields=["kyc"])
>>> show(Lead.objects.filter(kyc__nationality="AE"))
# WHERE ("crm_lead"."kyc" -> 'nationality') = '"AE"'::jsonb


class LoginEvent(models.Model):                                          #
7
    ref = models.UUIDField(default=uuid.uuid4, unique=True)    # uuid
    ip = models.GenericIPAddressField()                        # inet


class TradingAccount(models.Model):                                      #
9 (full model: task 21)
    class Leverage(models.IntegerChoices):
        X50 = 50, "1:50"
        X100 = 100, "1:100"
        X200 = 200, "1:200"

    leverage = models.PositiveSmallIntegerField(choices=Leverage.choices,
default=Leverage.X100)


class Deposit(models.Model):
```

```
    class Method(models.TextChoices):
        CARD = "card", "Card"
        BANK = "bank", "Bank transfer"
        CRYPTO = "crypto", "Crypto"

    method = models.CharField(max_length=10, choices=Method.choices)
```

5: Nexora keeps `duration_sec` as an integer: BI tools, CSV exports and plain SQL sum it without interval handling. Use `DurationField` when the app itself does time arithmetic.

```
# 8
>>> s = Subscriber(email="not-an-email"); s.save()       # saved --
EmailField is just varchar(254)
>>> s.full_clean()
ValidationError: {'email': ['Enter a valid email address.']}
Validators run in full_clean() and forms only. The database never sees them.

# 10  a FieldZoo model through sqlmigrate
CharField(100) varchar(100) · TextField text · DecimalField(14,2) numeric(14,
2) ·
IntegerField integer · BigIntegerField bigint · PositiveSmallIntegerField
smallint + CHECK >= 0 ·
BooleanField boolean · DateField date · DateTimeField timestamp with time
zone ·
DurationField interval · JSONField jsonb · UUIDField uuid ·
GenericIPAddressField inet ·
EmailField varchar(254) · SlugField varchar(50) + LIKE index
```

B. Core

11–13.

```
# 11                        database                  forms / full_clean()
null=False, blank=False     NOT NULL                  required            --
most fields
null=False, blank=True      NOT NULL, stores ''       optional            --
optional TEXT (notes)
null=True,  blank=True      NULL allowed              optional            --
optional dates, numbers, FKs
null=True,  blank=False     NULL allowed              required            --
rare: filled in later by code
Rule: text fields use '' for "empty" (never null=True, task 32); everything
else uses NULL.
```

```
# 12
from django.utils import timezone

created_at = models.DateTimeField(default=timezone.now)   # callable:
evaluated for EACH row
kyc = models.JSONField(default=dict)                      # callable: a fresh
dict each time
kyc = models.JSONField(default={})                        # one dict shared
by every instance
# python manage.py check:
# crm.Lead.kyc: (fields.E010) JSONField default should be a callable instead
of an instance
#   so that it's not shared between all field instances.
#   HINT: Use a callable instead, e.g., use `dict` instead of `{}`.


# 13
from django.db.models.functions import Now

received_at = models.DateTimeField(db_default=Now())
# SQL: "received_at" timestamp with time zone DEFAULT (STATEMENT_TIMESTAMP())
NOT NULL
# default=timezone.now never appears in the SQL: Django fills it in Python.
db_default lives
# IN the table, so rows inserted by other systems (the trading platform's
ETL) get it too.
```

14–17 — constraints.

```
# crm/models.py
from decimal import Decimal

from django.db.models import Q

LEAD_STATUSES = ["new", "callback", "ftd", "lost"]     # see the note below


class Lead(models.Model):
    ...
    class Meta:
        ordering = ["-created_at"]
        constraints = [
            models.CheckConstraint(condition=Q(status__in=LEAD_STATUSES),
# 16
                                   name="lead_status_valid"),
        ]


class Broker(models.Model):
    ...
    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=Q(commission_rate__gte=0) &
Q(commission_rate__lte=Decimal("0.5")),
                name="broker_commission_0_to_50_pct"),
        ]


class TradingAccount(models.Model):
    ...
    class Meta:
        constraints = [
            models.UniqueConstraint(fields=["platform", "account_no"],
# 14
                                    name="unique_account_per_platform"),
            models.UniqueConstraint(fields=["lead"],
condition=Q(is_primary=True),    # 15
                                    name="one_primary_account_per_lead"),
        ]
```

```
class Deposit(models.Model):
    ...
    class Meta:
        constraints = [
            models.CheckConstraint(condition=Q(amount__gt=0),
name="deposit_amount_positive",
                                   violation_error_message="A deposit must be
more than zero."),  # 17
            models.CheckConstraint(condition=Q(fx_rate__gt=0),
name="deposit_fx_rate_positive"),
        ]
```

```
-- what PostgreSQL gets
ALTER TABLE crm_lead ADD CONSTRAINT lead_status_valid
  CHECK (status IN ('new', 'callback', 'ftd', 'lost'));
ALTER TABLE crm_tradingaccount ADD CONSTRAINT unique_account_per_platform
UNIQUE (platform, account_no);
CREATE UNIQUE INDEX one_primary_account_per_lead ON crm_tradingaccount
(lead_id) WHERE is_primary;
```

16: inside `class Meta`, the name `Status` from the enclosing class body is NOT visible (Python class scopes don't nest), so `Q(status__in=Status.values)` fails with `NameError`. Keep the values in a module-level list — or define the `TextChoices` at module level and use `LeadStatus.values`. Django 5.1 renamed `CheckConstraint(check=...)` to `condition=`; the old name is deprecated.

```
# 17
>>> Deposit(account=acc, amount=Decimal("-5"), currency="USD", fx_rate=1,
method="card").full_clean()
ValidationError: {'__all__': ['A deposit must be more than zero.']}      <-
your message (Django 4.1+)
>>> Deposit.objects.create(account=acc, amount=Decimal("-5"), currency="USD",
fx_rate=1, method="card")
IntegrityError: new row for relation "crm_deposit" violates check constraint
"deposit_amount_positive"
```

18–20.

```
# crm/models.py -- Lead.Meta  (18)
from django.db.models.functions import Upper

indexes = [
    models.Index(fields=["desk", "-created_at"],
name="lead_desk_recent_idx"),         # composite, DESC
    models.Index(fields=["created_at"], condition=Q(status__in=["new",
"callback"]),
                 name="lead_open_created_idx"),
# partial
    models.Index(Upper("name"), name="lead_name_upper_idx"),
# functional
]
```

```
CREATE INDEX lead_desk_recent_idx ON crm_lead (desk_id, created_at DESC);
CREATE INDEX lead_open_created_idx ON crm_lead (created_at) WHERE status IN
('new', 'callback');
CREATE INDEX lead_name_upper_idx ON crm_lead ((UPPER(name)));
```

18: a partial index is small (open leads only) and serves only queries whose WHERE implies its condition. A functional index serves only queries using the SAME expression — `annotate(u=Upper("name")).filter(u="OMAR K.")` — so check with `explain()` rather than assuming `name__iexact` uses it.

```
# crm/models.py -- Deposit  (19, 20)
from django.db.models import F
from django.db.models.functions import Round

amount = models.DecimalField(max_digits=14, decimal_places=2,
                             db_comment="Amount in the account currency")
fx_rate = models.DecimalField(max_digits=12, decimal_places=6,
                              db_comment="Units of USD per 1 unit of currency
at deposit time")
amount_usd = models.GeneratedField(
    expression=Round(F("amount") * F("fx_rate"), 2),
    output_field=models.DecimalField(max_digits=16, decimal_places=2),
    db_persist=True,                        # STORED: computed on write,
indexable, free to read
)

class Meta:
    db_table = "nexora_deposits"           # 20: a stable name the BI team's
queries can rely on
```

```
"amount_usd" numeric(16, 2) GENERATED ALWAYS AS (ROUND(("amount" *
"fx_rate"), 2)) STORED
```

19: PostgreSQL recomputes `amount_usd` on every insert and update — nobody can store a USD value that disagrees with `amount × fx_rate`. 20: `db_comment`s travel with the table into `\d+` and into most BI tools' catalog views, where analysts actually look.

C. Nexora CRM

21–22 — the two models in full.

```
# crm/models.py
from django.utils import timezone


class TradingAccount(models.Model):
    class Leverage(models.IntegerChoices):
        X50 = 50, "1:50"
        X100 = 100, "1:100"
        X200 = 200, "1:200"

    account_no = models.BigIntegerField()
```

```
    platform = models.CharField(max_length=10, default="mt5")
    lead = models.ForeignKey(Lead, on_delete=models.PROTECT,
related_name="accounts")
    currency = models.CharField(max_length=3, default="USD")
    leverage = models.PositiveSmallIntegerField(choices=Leverage.choices,
default=Leverage.X100)
    is_primary = models.BooleanField(default=False)
    opened_at = models.DateTimeField(default=timezone.now)

    class Meta:
        constraints = [
            models.UniqueConstraint(fields=["platform", "account_no"],
name="unique_account_per_platform"),
            models.UniqueConstraint(fields=["lead"],
condition=Q(is_primary=True),
                                    name="one_primary_account_per_lead"),
            models.CheckConstraint(condition=Q(currency__regex=r"^[A-Z]
{3}$"), name="account_currency_iso"),
        ]

    def __str__(self):
        return f"{self.platform}:{self.account_no}"


class Deposit(models.Model):
    class Method(models.TextChoices):
        CARD = "card", "Card"
        BANK = "bank", "Bank transfer"
        CRYPTO = "crypto", "Crypto"

    class Status(models.TextChoices):
        PENDING = "pending", "Pending"
        APPROVED = "approved", "Approved"
        REJECTED = "rejected", "Rejected"

    account = models.ForeignKey(TradingAccount, on_delete=models.PROTECT,
related_name="deposits")
    amount = models.DecimalField(max_digits=14, decimal_places=2,
db_comment="Amount in the account currency")
    currency = models.CharField(max_length=3)
    fx_rate = models.DecimalField(max_digits=12, decimal_places=6,
                                  db_comment="USD per 1 unit of currency at
deposit time")
```

```
    amount_usd = models.GeneratedField(expression=Round(F("amount") *
F("fx_rate"), 2),

output_field=models.DecimalField(max_digits=16, decimal_places=2),
                                        db_persist=True)
    method = models.CharField(max_length=10, choices=Method.choices)
    status = models.CharField(max_length=10, choices=Status.choices,
default=Status.PENDING)
    created_at = models.DateTimeField(default=timezone.now)

    class Meta:
        db_table = "nexora_deposits"
        constraints = [
            models.CheckConstraint(condition=Q(amount__gt=0),
name="deposit_amount_positive",
                                   violation_error_message="A deposit must be
more than zero."),
            models.CheckConstraint(condition=Q(fx_rate__gt=0),
name="deposit_fx_rate_positive"),
            models.CheckConstraint(condition=Q(currency__regex=r"^[A-Z]
{3}$"), name="deposit_currency_iso"),
            models.CheckConstraint(condition=Q(status__in=["pending",
"approved", "rejected"]),
                                   name="deposit_status_valid"),
        ]
        indexes = [
            models.Index(fields=["account", "-created_at"],
name="deposit_account_recent_idx"),
            models.Index(fields=["created_at"],
condition=Q(status="approved"), name="deposit_approved_idx"),
        ]
```

21: `Q(currency__regex=...)` inside a `CheckConstraint` becomes `CHECK (currency::text ~ '^[A-Z]{3}$')` — "usd" and "US" are rejected by PostgreSQL itself.

23–26.

```
# 23
Lead.objects.filter(kyc__status="approved")                 # (kyc ->
'status') = '"approved"'::jsonb
Lead.objects.filter(kyc__has_key="passport_no")             # kyc ?
'passport_no'
Lead.objects.filter(kyc__nationality__in=["AE", "IN", "PK"])  # (kyc ->
```

```
'nationality') IN (...)


# 24 -- one failure at a time, each in its own savepoint
from django.db import IntegrityError, transaction


def attempt(label, fn):
    try:
        with transaction.atomic():
            fn()
    except IntegrityError as exc:
        print(f"{label}: {exc}")


lead, acc = Lead.objects.first(),
TradingAccount.objects.filter(is_primary=True).first()
attempt("status", lambda:
Lead.objects.filter(pk=lead.pk).update(status="banana"))
attempt("commission", lambda: Broker.objects.create(code="BRF-9999",
name="x", commission_rate=Decimal("0.9")))
attempt("duplicate account", lambda:
TradingAccount.objects.create(account_no=acc.account_no, lead=lead))
attempt("second primary", lambda: TradingAccount.objects.create(account_no=1,
lead=acc.lead, is_primary=True))
attempt("currency", lambda: TradingAccount.objects.create(account_no=2,
lead=lead, currency="usd"))
attempt("amount", lambda: Deposit.objects.create(account=acc, amount=0,
currency="USD", fx_rate=1, method="card"))
# status: new row for relation "crm_lead" violates check constraint
"lead_status_valid"
# duplicate account: duplicate key value violates unique constraint
"unique_account_per_platform"
# second primary: duplicate key value violates unique constraint
"one_primary_account_per_lead"
# ...each message names the constraint -- that's why every constraint gets a
deliberate name


# 25
def save_validated(obj):
    obj.full_clean()          # field validators + clean() + unique AND check
constraints, in Python
```

```
    obj.save()

# save_validated(Deposit(account=acc, amount=0, ...))
# ValidationError: {'__all__': ['A deposit must be more than zero.']}
```

```
# 26  deposit.amount = Decimal("100.555")
full_clean() -> {'amount': ['Ensure that there are no more than 2 decimal
places.']}
save() WITHOUT full_clean(): the value is rounded to 2 places on the way to
numeric(14,2) --
by a rule you didn't choose. Quantize it yourself, with the rule Finance
agreed, before assigning:
  Decimal("100.555").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)  ->
Decimal('100.56')
```

27–29.

```
# crm/money.py  (27)
from decimal import ROUND_HALF_UP, Decimal

CENT = Decimal("0.01")


def money(value):
    if isinstance(value, float):
        raise TypeError("Money is never a float -- pass a Decimal, an int or
a string.")
    return Decimal(value).quantize(CENT, rounding=ROUND_HALF_UP)


def to_usd(amount, rate):
    return money(Decimal(amount) * Decimal(rate))


def total(amounts):
    return money(sum((Decimal(a) for a in amounts), Decimal("0")))

# to_usd("100", "1.083333") -> Decimal('108.33')     total(["0.10"] * 3) ->
Decimal('0.30')
```

```
# 28 -- on a tiny table PostgreSQL rightly prefers Seq Scan; force it to show
index plans
from django.db import connection

with connection.cursor() as cur:
    cur.execute("SET enable_seqscan = off")          # this session only -- a
diagnostic, never in code
explain(Lead.objects.filter(desk__slug="vip").order_by("-created_at")[:25])
#   -> Index Scan using lead_desk_recent_idx
explain(Lead.objects.filter(status="new", created_at__gte=week_ago))
#   -> Index Scan using lead_open_created_idx   (WHERE implies the partial
index's condition)
explain(Deposit.objects.filter(account=acc, status="approved").order_by("-
created_at"))
#   -> Index Scan using deposit_account_recent_idx
```

```
# seed_basics.py -- additions  (29)
for i, lead in enumerate(Lead.objects.order_by("id")):
    primary, _ = TradingAccount.objects.get_or_create(
        platform="mt5", account_no=880_000 + lead.pk, defaults={"lead": lead,
"is_primary": True})
    if i % 3 == 0:
        TradingAccount.objects.get_or_create(
            platform="mt5", account_no=890_000 + lead.pk, defaults={"lead":
lead, "currency": "EUR"})
    if lead.status == Lead.Status.FTD and not primary.deposits.exists():
        Deposit.objects.create(account=primary, amount=Decimal("250.00"),
currency="USD",
                               fx_rate=Decimal("1"),
method=Deposit.Method.CARD,
                               status=Deposit.Status.APPROVED)
```

30.

```
# crm/tests.py
from decimal import Decimal

from django.db import IntegrityError, transaction

from .models import Deposit, TradingAccount
```

```
class ConstraintTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_basics", stdout=StringIO())
        cls.account = TradingAccount.objects.filter(is_primary=True).first()

    def assertViolates(self, constraint, fn):
        with self.assertRaisesMessage(IntegrityError, constraint):
            with transaction.atomic():               # keep the test's
transaction usable
                fn()

    def test_database_refuses_bad_rows(self):
        acc = self.account
        self.assertViolates("deposit_amount_positive", lambda:
Deposit.objects.create(
            account=acc, amount=0, currency="USD", fx_rate=1, method="card"))
        self.assertViolates("one_primary_account_per_lead", lambda:
TradingAccount.objects.create(
            account_no=1, lead=acc.lead, is_primary=True))
        self.assertViolates("account_currency_iso", lambda:
TradingAccount.objects.create(
            account_no=2, lead=acc.lead, currency="usd"))
        self.assertViolates("lead_status_valid", lambda:
Lead.objects.filter(pk=acc.lead_id).update(status="banana"))

    def test_amount_usd_is_generated(self):
        d = Deposit.objects.create(account=self.account,
amount=Decimal("100.00"), currency="EUR",
                                   fx_rate=Decimal("1.083333"),
method="bank")
        d.refresh_from_db()
        self.assertEqual(d.amount_usd, Decimal("108.33"))
```

```
python manage.py test crm && git add -A && git commit -m "orm day 2"
```

D. Challenge

31–35.

```
# 31
>>> sum([0.1] * 10_000)
1000.0000000001588
>>> sum([Decimal("0.10")] * 10_000)
Decimal('1000.00')
# a FloatField (double precision) drifts like this inside every SUM --
reports end up
# cents off, and reconciliation with the payment provider never matches.
```

```
# 32  notes = CharField(null=True, blank=True)
Some rows hold NULL, others '' -- both mean "no notes".
filter(notes="") misses the NULLs; filter(notes__isnull=True) misses the ''s.
Django's convention: text columns are NOT NULL and use '' for empty.
(Exception: an optional
UNIQUE text column, where '' twice would collide -- there NULL is right, see
33.)

# 33  unique + null=True
PostgreSQL treats NULLs as distinct: a UNIQUE column can hold any number of
NULLs.
To allow only ONE: models.UniqueConstraint(fields=["ref"],
name="one_null_ref", nulls_distinct=False)
-> UNIQUE NULLS NOT DISTINCT (Django 5.0+, PostgreSQL 15+)

# 34  created_at = DateTimeField(default=timezone.now())     <- parentheses
The function runs ONCE when models.py is imported: every row gets the
server's start time
until the next restart. makemigrations also bakes that fixed datetime into
the migration and
warns: fields.W161 "Fixed default value provided." Pass the function:
default=timezone.now
```

```
# 35 -- with kyc = JSONField(default={}) (the trap)
>>> a = Lead(name="A", phone="050 000 0201", desk=desk)
>>> b = Lead(name="B", phone="050 000 0202", desk=desk)
>>> a.kyc["status"] = "approved"
>>> b.kyc
{'status': 'approved'}           # the SAME dict object -- every new lead
shares it
# with default=dict each instance gets its own {}
```

36–40.

```
# 36
varchar(100) -> varchar(200)   metadata-only in PostgreSQL: instant, no
rewrite
varchar(100) -> text           also metadata-only
varchar(200) -> varchar(50)    rewrites the whole table under an exclusive
lock, and fails
                               if any existing value is longer than 50

# 37  a new status "duplicate" in TextChoices, LEAD_STATUSES not updated
IntegrityError: ... violates check constraint "lead_status_valid" on the
first insert.
Add it to the list -> makemigrations writes RemoveConstraint + AddConstraint.
Build the
constraint from LeadStatus.values (module-level choices) and makemigrations
notices for you.

# 38  Deposit.objects.create(..., created_at=datetime(2026, 10, 20, 10, 0))
RuntimeWarning: DateTimeField Deposit.created_at received a naive datetime
(2026-10-20 10:00:00) while time zone support is active.
Django assumes the CURRENT time zone (Asia/Dubai): stored as 06:00 UTC. If
the value came
from a UTC system, it's now 4 hours wrong. Use timezone.now(), or
make_aware(dt, tz) explicitly.
```

```
# 39 -- jsonb is typed
>>> lead.kyc = {"age": 30, "status": "approved"}; lead.save(update_fields=
["kyc"])
>>> Lead.objects.filter(kyc__age=30).count()           # 1   (the JSON number
30)
>>> Lead.objects.filter(kyc__age="30").count()         # 0   (the JSON string
"30" -- different)
>>> Lead.objects.filter(kyc__contains={"status": "approved"}).count()   # 1
kyc @> '{"status": "approved"}'
# containment (@>) can use a GIN index (Day 12); key-by-key comparisons are
type-sensitive.


# 40
>>> d = Deposit.objects.create(account=acc, amount=Decimal("100.00"),
currency="EUR",
...                            fx_rate=Decimal("1.083333"), method="bank")
>>> d.amount_usd
# not the computed 108.33 you expect: the DATABASE computed it, and the
instance in memory
# was never told. The documented answer -- reload before reading:
>>> d.refresh_from_db(fields=["amount_usd"])
>>> d.amount_usd
Decimal('108.33')
```

Day 3 — Relationships

Goal: model how Nexora's data actually connects — agents, desks, the Brf referral hierarchy, leads, calls, accounts, deposits, trades, SOP breaches — and query across those links knowing which JOIN each `__` produces. By tonight the full schema exists and holds thousands of seeded rows; every later day queries it.

```
Desk ──< Lead >── Broker ── parent ──> Broker            (Brf hierarchy: self
FK)
  |         | \── assigned_to ──> User ── AgentProfile (1:1) ──> Desk
  |         ├──< CallLog >── agent (User)
  |         └──< TradingAccount ──< Deposit
  |                           └──< Trade
  └── managers (M2M through DeskMembership) ──> User
SOPRule ──< SOPBreach >── agent (User), lead (nullable)
──< = one-to-many (the many side holds the FK)
```

A. Drills (1–10) — the relationship toolkit

- [ ] **1.** Try every `on_delete` option on a scratch FK: CASCADE, PROTECT, RESTRICT, SET_NULL, SET_DEFAULT, DO_NOTHING. Where is each enforced — Python or PostgreSQL?
- [ ] **2.** `related_name`: compare `lead.calls.all()` with the default `lead.calllog_set.all()`. What does `related_query_name` change?
- [ ] **3.** `lead.desk_id`, `lead.desk.id` and `lead.desk.name`: which ones run a query?
- [ ] **4.** `AgentProfile` one-to-one with the user: `user.profile`, `profile.user`, and the error when a user has no profile.
- [ ] **5.** A plain many-to-many `Desk.managers`: find the auto-created table; `add`, `remove`, `set`, `clear` — read the SQL of each.
- [ ] **6.** Replace it with a `through` model `DeskMembership` (role, since). What changes in `add()`?
- [ ] **7.** `Broker.parent` (self FK): a broker's children, its parent, and its parent's parent.
- [ ] **8.** Filter forward across a FK: leads whose desk slug is `vip`. Read the JOIN.
- [ ] **9.** Filter backward: desks that have at least one FTD lead. Why do desks repeat, and how do you fix it?

- [ ] **10.** On the nullable `Lead.broker`: `broker__isnull=True` vs `broker__code="BRF-0042"`. Which JOIN type does each produce?

B. Core (11–20) — the full Nexora schema

- [ ] **11.** `AgentProfile` (user 1:1, desk, extension, hired_on) and the new `Lead` columns: `broker`, `assigned_to`.
- [ ] **12.** `CallLog`: lead (CASCADE), agent (PROTECT), started_at, duration_sec, outcome; an index for "an agent's calls by time".
- [ ] **13.** `Trade`: account (CASCADE), symbol, side, volume, open/close price and time, `pnl_usd` (null while open); a CHECK that a closed trade has a close price.
- [ ] **14.** `SOPBreach`: rule (PROTECT), agent (PROTECT), lead (SET_NULL), occurred_at, note.
- [ ] **15.** `DeskMembership` with a role (manager / deputy), `since`, and a unique (desk, user) pair.
- [ ] **16.** The Brf hierarchy: `Broker.parent`, plus a database CHECK that a broker isn't its own parent.
- [ ] **17.** `seed_nexora`: 3 desks, 12 agents, 2 managers, 40 brokers on 3 levels, 5,000 leads, about 20,000 calls, accounts, deposits, trades and breaches spread over 180 days. Repeatable (fixed random seed) and fast (`bulk_create`).
- [ ] **18.** Run it and print the row count of every table and the total time.
- [ ] **19.** Admin: calls and accounts as inlines on a lead; `autocomplete_fields` for agent and broker; `list_select_related`.
- [ ] **20.** Write the schema as a Mermaid `erDiagram` in `docs/schema.md`.

C. Nexora CRM (21–30) — questions that cross tables

- [ ] **21.** Leads that came through brokers directly under BRF-0001.
- [ ] **22.** Leads two levels under BRF-0001 (brokers of brokers).
- [ ] **23.** Trading accounts that have never received a deposit.
- [ ] **24.** Agents who made no calls today.
- [ ] **25.** The desks a given user manages (with their role), and the managers of a given desk.
- [ ] **26.** Calls where the agent's desk is different from the lead's desk.

- [ ] **27.** Leads under BRF-0001's downline with at least one approved deposit above 1,000 USD.
- [ ] **28.** How many DISTINCT leads made at least one approved deposit last month? Compare with the naive count.
- [ ] **29.** Add a manager to a desk with a role, change the role, remove them — through the M2M API and through the model.
- [ ] **30.** Tests for the relationships and the seed sizes. Commit `orm day 3`.

D. Challenge (31–40) — relationship traps

- [ ] **31.** Delete a desk that has leads, then a lead that has calls. What happens, and which queries does Django run first?
- [ ] **32.** Two FKs from one model to `User` without `related_name`. Copy the system check error and fix it.
- [ ] **33.** Two models that reference each other across files. How do string references (`"crm.Lead"`) avoid the circular import?
- [ ] **34.** "Leads with a call by agent1 that was `interested`": compare `filter(a, b)` with `filter(a).filter(b)`. Explain the different counts.
- [ ] **35.** `exclude()` across a multi-valued relation with two conditions. Is it the opposite of the matching `filter()`? Prove it.
- [ ] **36.** `desk.managers.add(user)` on the through model without `through_defaults`. What happens, and why?
- [ ] **37.** Make BRF-A the parent of BRF-B and BRF-B the parent of BRF-A. Does the database stop you? Add a check that does.
- [ ] **38.** Delete a lead with 2,000 calls, then the same with a `post_delete` signal on `CallLog`. Compare the queries.
- [ ] **39.** Why `settings.AUTH_USER_MODEL` (or `get_user_model()`) instead of importing `User` in models?
- [ ] **40.** "The latest call of each lead" for 50 leads, done naively. Count the queries; keep the number for Day 8.

Solutions

The full set of new models is in block B (tasks 11–16); block A uses pieces of it.

A. Drills

1–3.

```
# 1  on_delete -- Django enforces it in PYTHON. The PostgreSQL FK has no ON
DELETE clause
#    (Django creates it DEFERRABLE INITIALLY DEFERRED and does the work
itself).
CASCADE      Django deletes the children first, then the parent
PROTECT      raises ProtectedError before deleting anything
RESTRICT     like PROTECT, but allowed when the child is ALSO being deleted
through another
             CASCADE path in the same delete()
SET_NULL     UPDATE child SET fk = NULL (the FK must be null=True)
SET_DEFAULT  UPDATE child SET fk = <default>
DO_NOTHING   Django does nothing -> the database FK constraint fails at
COMMIT with an
             IntegrityError, unless you clean up yourself
Consequence: a row deleted by raw SQL or another system gets NO cascade --
the FK simply
refuses the delete.
```

```
# 2
class CallLog(models.Model):
    lead = models.ForeignKey(Lead, on_delete=models.CASCADE,
related_name="calls")

lead.calls.all()                                    # with
related_name="calls"
lead.calllog_set.all()                              # the default when
related_name is omitted
Lead.objects.filter(calls__outcome="interested")   # the FILTER name =
related_query_name,
                                                    # which defaults to
related_name
# related_query_name="call" -> Lead.objects.filter(call__outcome=...) while
lead.calls stays

# 3
lead.desk_id        # no query: the column is on the lead row itself
lead.desk.id        # one query to load the Desk (unless
select_related("desk") was used)
lead.desk.name      # the same Desk object, now cached on the lead: no second
query
```

4–7.

```
# 4
>>> user.profile                # SELECT ... FROM crm_agentprofile WHERE
user_id = 5
>>> profile.user                # SELECT ... FROM auth_user WHERE id = 5
>>> admin_user.profile
User.profile.RelatedObjectDoesNotExist: User has no profile.
>>> hasattr(admin_user, "profile")    # False -- the exception subclasses
AttributeError
# SQL: user_id bigint NOT NULL UNIQUE + FK -> a one-to-one is a unique
foreign key
```

```
# 5  managers = models.ManyToManyField(settings.AUTH_USER_MODEL,
related_name="managed_desks")
auto table crm_desk_managers (id, desk_id, user_id, UNIQUE (desk_id,
user_id))
desk.managers.add(u1, u2)   INSERT INTO crm_desk_managers ... ON CONFLICT DO
NOTHING
                            (or SELECT existing + INSERT missing when
m2m_changed receivers listen)
desk.managers.remove(u1)    DELETE FROM crm_desk_managers WHERE desk_id = 1
AND user_id IN (5)
desk.managers.set([u2, u3]) SELECT current ids, then DELETE the missing ones,
INSERT the new ones
desk.managers.clear()       DELETE FROM crm_desk_managers WHERE desk_id = 1

# 6  with through="DeskMembership" (task 15)
desk.managers.add(user, through_defaults={"role": "manager"})
the extra columns need values -- through_defaults supplies them (task 36
shows what happens without).
Switching an EXISTING plain M2M to a through model is a delicate migration
(Day 4); in this
practice project, delete the old field's migration state and recreate it.
```

```
# 7
>>> top = Broker.objects.get(code="BRF-0001")
>>> top.children.all()                      # SELECT ... WHERE parent_id = 1
>>> leaf = Broker.objects.get(code="BRF-0017")
>>> leaf.parent, leaf.parent.parent         # two queries: one per level
```

8–10.

```
# 8
>>> show(Lead.objects.filter(desk__slug="vip"))
SELECT ... FROM crm_lead INNER JOIN crm_desk ON (crm_lead.desk_id =
crm_desk.id)
WHERE crm_desk.slug = vip

# 9
>>> Desk.objects.filter(leads__status="ftd").count()            # e.g. 1003
-- one row PER FTD LEAD
>>> Desk.objects.filter(leads__status="ftd").distinct().count()  # 3
# the JOIN multiplies desk rows by matching leads. .distinct() collapses
them; Day 8's
```

```
# Exists() avoids the multiplication in the first place.

# 10
>>> show(Lead.objects.filter(broker__isnull=True))
SELECT ... FROM crm_lead WHERE crm_lead.broker_id IS NULL          -- no
JOIN: the column is local
>>> show(Lead.objects.filter(broker__code="BRF-0042"))
... INNER JOIN crm_broker ON (crm_lead.broker_id = crm_broker.id) WHERE
crm_broker.code = BRF-0042
>>> show(Lead.objects.filter(broker__parent__isnull=True))
... LEFT OUTER JOIN crm_broker ON (...) WHERE crm_broker.parent_id IS NULL
# careful: that last one ALSO returns leads with no broker at all (the LEFT
JOIN gives NULL).
# "leads of top-level brokers" = filter(broker__isnull=False,
broker__parent__isnull=True)
```

B. Core

11–16 — the schema.

```
# crm/models.py -- additions and changes
from datetime import date

from django.conf import settings


class Desk(models.Model):
    ...
    managers = models.ManyToManyField(settings.AUTH_USER_MODEL,
through="DeskMembership",
                                      related_name="managed_desks",
blank=True)


class DeskMembership(models.Model):                                        #
15
    class Role(models.TextChoices):
        MANAGER = "manager", "Manager"
        DEPUTY = "deputy", "Deputy"

    desk = models.ForeignKey(Desk, on_delete=models.CASCADE,
related_name="memberships")
    user = models.ForeignKey(settings.AUTH_USER_MODEL,
```

```
on_delete=models.CASCADE, related_name="desk_memberships")
    role = models.CharField(max_length=10, choices=Role.choices)
    since = models.DateField(default=date.today)

    class Meta:
        constraints = [
            models.UniqueConstraint(fields=["desk", "user"],
name="one_membership_per_desk_user"),
            models.CheckConstraint(condition=Q(role__in=["manager",
"deputy"]), name="membership_role_valid"),
        ]


class AgentProfile(models.Model):                                          #
11
    user = models.OneToOneField(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name="profile")
    desk = models.ForeignKey(Desk, on_delete=models.PROTECT,
related_name="agents")
    extension = models.CharField(max_length=10, unique=True)
    hired_on = models.DateField()


class Broker(models.Model):                                                #
16
    ...
    parent = models.ForeignKey("self", null=True, blank=True,
on_delete=models.SET_NULL,
                               related_name="children")

    class Meta:
        constraints = [
            ...,  # the commission CHECK from Day 2
            models.CheckConstraint(condition=~Q(parent=F("id")),
name="broker_not_own_parent"),
        ]


class Lead(models.Model):                                                  #
11
    ...
    created_at = models.DateTimeField(default=timezone.now)     # WAS
auto_now_add -- see below
```

```
    broker = models.ForeignKey(Broker, null=True, blank=True,
on_delete=models.SET_NULL, related_name="leads")
    assigned_to = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                                    on_delete=models.SET_NULL,
related_name="leads")


class CallLog(models.Model):                                                #
12
    lead = models.ForeignKey(Lead, on_delete=models.CASCADE,
related_name="calls")
    agent = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.PROTECT, related_name="calls")
    started_at = models.DateTimeField()
    duration_sec = models.PositiveIntegerField(default=0)
    outcome = models.CharField(max_length=20, default="no_answer")

    class Meta:
        indexes = [models.Index(fields=["agent", "-started_at"],
name="call_agent_recent_idx")]


class Trade(models.Model):                                                  #
13
    class Side(models.TextChoices):
        BUY = "buy", "Buy"
        SELL = "sell", "Sell"

    account = models.ForeignKey(TradingAccount, on_delete=models.CASCADE,
related_name="trades")
    symbol = models.CharField(max_length=20)
    side = models.CharField(max_length=4, choices=Side.choices)
    volume = models.DecimalField(max_digits=12, decimal_places=2)
    open_price = models.DecimalField(max_digits=18, decimal_places=6)
    close_price = models.DecimalField(max_digits=18, decimal_places=6,
null=True, blank=True)
    opened_at = models.DateTimeField()
    closed_at = models.DateTimeField(null=True, blank=True)
    pnl_usd = models.DecimalField(max_digits=14, decimal_places=2, null=True,
blank=True)

    class Meta:
```

```
        constraints = [
            models.CheckConstraint(condition=Q(volume__gt=0),
name="trade_volume_positive"),
            models.CheckConstraint(
                condition=Q(closed_at__isnull=True) |
Q(close_price__isnull=False, pnl_usd__isnull=False),
                name="closed_trade_has_close_price"),
        ]
        indexes = [models.Index(fields=["account", "-opened_at"],
name="trade_account_recent_idx")]


class SOPBreach(models.Model):                                          #
14
    rule = models.ForeignKey(SOPRule, on_delete=models.PROTECT,
related_name="breaches")
    agent = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.PROTECT, related_name="sop_breaches")
    lead = models.ForeignKey(Lead, null=True, blank=True,
on_delete=models.SET_NULL, related_name="sop_breaches")
    occurred_at = models.DateTimeField()
    note = models.TextField(blank=True)
```

11: `created_at` changes from `auto_now_add=True` to `default=timezone.now`. `auto_now_add` OVERWRITES any value you pass on insert — so imports of historical leads, seeds and back-filled data become impossible. `default` gives the same behaviour for new rows while staying settable. 16: `~Q(parent=F("id"))` becomes a CHECK comparing two columns of the same row — it stops self-parenting but not longer cycles (task 37).

17–18 — the seed.

```
# crm/management/commands/seed_nexora.py
import random
import time
from datetime import date, timedelta
from decimal import Decimal

from django.contrib.auth import get_user_model
from django.contrib.auth.hashers import make_password
from django.core.management.base import BaseCommand
from django.db import transaction
from django.db.models import Q
```

```
from django.utils import timezone

from crm.management.commands.seed_basics import DESKS, RULES
from crm.models import (AgentProfile, Broker, CallLog, Deposit, Desk,
DeskMembership, Lead,
                        SOPBreach, SOPRule, Trade, TradingAccount)

SYMBOLS = ["XAUUSD", "EURUSD", "GBPUSD", "USDJPY", "BTCUSD", "US30"]
OUTCOMES = ["no_answer", "callback", "interested", "not_interested"]
FX = {"USD": Decimal("1"), "EUR": Decimal("1.083333")}


class Command(BaseCommand):
    help = "Rebuild realistic Nexora data (random but repeatable)"

    def add_arguments(self, parser):
        parser.add_argument("--leads", type=int, default=5000)

    @transaction.atomic
    def handle(self, *args, **opts):
        started = time.perf_counter()
        rng, now = random.Random(7), timezone.now()
        User = get_user_model()

        # wipe in dependency order (PROTECT foreign keys first)
        for model in (SOPBreach, Trade, Deposit, TradingAccount, CallLog,
Lead,
                      DeskMembership, AgentProfile, Broker):
            model.objects.all().delete()
        User.objects.filter(Q(username__startswith="agent") |
Q(username__startswith="manager")).delete()

        desks = [Desk.objects.get_or_create(slug=s, defaults={"name": n})[0]
for s, n in DESKS]
        for code, title, hours, severity in RULES:
            SOPRule.objects.update_or_create(code=code, defaults={"title":
title, "max_hours": hours,
                                                                  "severity":
severity})
        rules = list(SOPRule.objects.all())

        pw = make_password("pw-123456")             # hash ONCE, not once per
user
```

```
        agents = User.objects.bulk_create([User(username=f"agent{i:02d}",
password=pw) for i in range(1, 13)])
        managers = User.objects.bulk_create([User(username=f"manager{i}",
password=pw) for i in (1, 2)])
        AgentProfile.objects.bulk_create([
            AgentProfile(user=a, desk=desks[i % 3], extension=f"2{i:03d}",
                         hired_on=date(2024, 1, 1) + timedelta(days=30 * i))
            for i, a in enumerate(agents)])
        DeskMembership.objects.bulk_create([
            DeskMembership(desk=desks[0], user=managers[0], role="manager"),
            DeskMembership(desk=desks[1], user=managers[0], role="manager"),
            DeskMembership(desk=desks[2], user=managers[1], role="manager"),
            DeskMembership(desk=desks[1], user=managers[1], role="deputy")])

        numbers = iter(range(1, 41))                # 4 top, 12 middle, 24
leaf brokers

        def broker(parent):
            n = next(numbers)
            return Broker(code=f"BRF-{n:04d}", name=f"Partner {n:02d}",
parent=parent)

        tops = Broker.objects.bulk_create([broker(None) for _ in range(4)])
        mids = Broker.objects.bulk_create([broker(t) for t in tops for _ in
range(3)])
        leaves = Broker.objects.bulk_create([broker(m) for m in mids for _ in
range(2)])
        brokers = tops + mids + leaves

        weights = ["new"] * 35 + ["callback"] * 25 + ["ftd"] * 20 + ["lost"]
* 20
        leads = Lead.objects.bulk_create([
            Lead(name=f"Lead {n:05d}", phone=f"+9715{n:08d}",
status=rng.choice(weights),
                 desk=rng.choice(desks), assigned_to=rng.choice(agents),
                 broker=rng.choice(brokers) if rng.random() < 0.7 else None,
                 created_at=now - timedelta(days=rng.uniform(0, 180)))
            for n in range(1, opts["leads"] + 1)], batch_size=1000)

        calls, last_contact = [], {}
        for lead in leads:
            for _ in range(rng.choice([0, 1, 2, 3, 4, 5, 6, 8])):
                started_at = lead.created_at +
```

```
timedelta(minutes=rng.randint(5, 60 * 24 * 30))
                if started_at > now:
                    continue
                agent = lead.assigned_to if rng.random() < 0.85 else
rng.choice(agents)
                calls.append(CallLog(lead=lead, agent=agent,
started_at=started_at,
                                     duration_sec=rng.randint(0, 900),
outcome=rng.choice(OUTCOMES)))
                last_contact[lead.pk] = max(last_contact.get(lead.pk,
started_at), started_at)
        CallLog.objects.bulk_create(calls, batch_size=2000)
        for lead in leads:
            lead.last_contacted_at = last_contact.get(lead.pk)
        Lead.objects.bulk_update(leads, ["last_contacted_at"],
batch_size=1000)

        accounts = []
        for lead in leads:
            if rng.random() < {"ftd": 1.0, "callback": 0.3}.get(lead.status,
0.1):
                opened_at = min(lead.created_at +
timedelta(days=rng.uniform(0, 10)), now)
                accounts.append(TradingAccount(account_no=880_000 + lead.pk,
lead=lead,
                                               is_primary=True,
opened_at=opened_at))
                if rng.random() < 0.2:
                    accounts.append(TradingAccount(account_no=990_000 +
lead.pk, lead=lead,
                                                   currency="EUR",
opened_at=opened_at))
        TradingAccount.objects.bulk_create(accounts, batch_size=1000)

        deposits, first_funded = [], {}
        for acc in accounts:
            if acc.lead.status != "ftd":
                continue
            when = acc.opened_at
            for i in range(rng.randint(1, 4)):
                when += timedelta(days=rng.uniform(0, 20))
                if when > now:
                    break
```

```
                status = "approved" if i == 0 or rng.random() < 0.8 else
rng.choice(["pending", "rejected"])
                deposits.append(Deposit(account=acc,
amount=Decimal(rng.choice([100, 250, 500, 1000, 2500, 5000])),
                                        currency=acc.currency,
fx_rate=FX[acc.currency],
                                        method=rng.choice(["card", "bank",
"crypto"]),
                                        status=status, created_at=when))
                if status == "approved":
                    first_funded.setdefault(acc.pk, when)
        Deposit.objects.bulk_create(deposits, batch_size=2000)

        trades = []                                     # random demo prices:
floats are fine HERE only
        for acc in accounts:
            start = first_funded.get(acc.pk)
            if start is None:
                continue
            for _ in range(rng.randint(0, 10)):
                opened_at = start + timedelta(hours=rng.uniform(1, 24 * 60))
                if opened_at > now:
                    continue
                closed_at = opened_at + timedelta(hours=rng.uniform(0.1, 72))
                closed = closed_at < now and rng.random() < 0.9
                price = Decimal(str(round(rng.uniform(1, 2500), 4)))
                trades.append(Trade(
                    account=acc, symbol=rng.choice(SYMBOLS),
side=rng.choice(["buy", "sell"]),
                    volume=Decimal(rng.choice(["0.10", "0.50", "1.00",
"2.00"])),
                    open_price=price, opened_at=opened_at,
                    close_price=(price * Decimal(str(round(rng.uniform(0.97,
1.03), 4)))).quantize(Decimal("0.000001"))
                    if closed else None,
                    closed_at=closed_at if closed else None,
                    pnl_usd=Decimal(str(round(rng.uniform(-500, 600), 2))) if
closed else None))
        Trade.objects.bulk_create(trades, batch_size=2000)

        SOPBreach.objects.bulk_create([
            SOPBreach(rule=rng.choice(rules), agent=rng.choice(agents),
                      lead=rng.choice(leads) if rng.random() < 0.9 else None,
```

```
                        occurred_at=now - timedelta(days=rng.uniform(0, 180)))
            for _ in range(300)])

        self.stdout.write(self.style.SUCCESS(
            f"leads {len(leads)}, calls {len(calls)}, accounts
{len(accounts)}, deposits {len(deposits)}, "
            f"trades {len(trades)} in {time.perf_counter() - started:.1f}s"))
```

```
# 18: python manage.py seed_nexora, then in the shell
>>> from django.apps import apps
>>> for model in apps.get_app_config("crm").get_models():
...     print(f"{model.__name__:<16} {model.objects.count():>7,}")
Desk                  3
Lead              5,000
CallLog          ~18,000
TradingAccount    ~1,800
Deposit           ~2,400
Trade             ~6,000
SOPBreach            300
...
```

17: three things keep it fast: one transaction for the whole run, `bulk_create` in batches, and a password hashed once (`create_user` would hash 14 times — PBKDF2 is slow on purpose). `random.Random(7)` makes every run produce the same data, so the numbers in later days' solutions match yours. Your exact counts depend on the RNG; the shape is what matters.

19–20.

```
# crm/admin.py  (19)
from django.contrib import admin

from .models import Broker, CallLog, Lead, TradingAccount


class AccountInline(admin.TabularInline):
    model = TradingAccount
    extra = 0


class CallInline(admin.TabularInline):
    model = CallLog
```

```
    extra = 0
    autocomplete_fields = ("agent",)


@admin.register(Lead)
class LeadAdmin(admin.ModelAdmin):
    list_display = ("name", "phone", "status", "desk", "broker",
"assigned_to", "created_at")
    list_filter = ("status", "desk")
    search_fields = ("name", "phone")
    autocomplete_fields = ("broker", "assigned_to")       # a search box, not
a 5,000-option <select>
    list_select_related = ("desk", "broker", "assigned_to")  # the changelist
runs 1 query, not 100
    inlines = [AccountInline, CallInline]


@admin.register(Broker)
class BrokerAdmin(admin.ModelAdmin):
    search_fields = ("code", "name")                      # required by
autocomplete_fields
```

```
<!-- docs/schema.md  (20) -->
```mermaid
erDiagram
    DESK ||--o{ LEAD : holds
    DESK ||--o{ DESKMEMBERSHIP : has
    USER ||--o{ DESKMEMBERSHIP : manages
    USER ||--|| AGENTPROFILE : has
    DESK ||--o{ AGENTPROFILE : employs
    BROKER ||--o{ BROKER : refers
    BROKER ||--o{ LEAD : introduces
    USER ||--o{ LEAD : "assigned to"
    LEAD ||--o{ CALLLOG : receives
    USER ||--o{ CALLLOG : makes
    LEAD ||--o{ TRADINGACCOUNT : opens
    TRADINGACCOUNT ||--o{ DEPOSIT : funds
    TRADINGACCOUNT ||--o{ TRADE : places
    SOPRULE ||--o{ SOPBREACH : "broken in"
    USER ||--o{ SOPBREACH : commits
    LEAD |o--o{ SOPBREACH : concerns
```
```

C. Nexora CRM

21–24.

```
from django.contrib.auth import get_user_model
from django.db.models import F, Q
from django.utils import timezone

from crm.models import Broker, CallLog, Desk, DeskMembership, Lead,
TradingAccount

User = get_user_model()
X = "BRF-0001"

# 21 -- brokers directly under X
Lead.objects.filter(broker__parent__code=X)
# INNER JOIN crm_broker ON lead.broker_id = broker.id
# INNER JOIN crm_broker T3 ON broker.parent_id = T3.id  WHERE T3.code = 'BRF-
0001'

# 22 -- brokers of brokers under X
Lead.objects.filter(broker__parent__parent__code=X)

# the whole downline (3 levels) -- OR across nullable relations: Django turns
the JOINs into
# LEFT OUTER JOINs so a lead whose broker has no grandparent can still match
the first branch
downline = Q(broker__code=X) | Q(broker__parent__code=X) |
Q(broker__parent__parent__code=X)
Lead.objects.filter(downline)

# 23 -- accounts that never received a deposit: an anti-join
TradingAccount.objects.filter(deposits__isnull=True)
# LEFT OUTER JOIN nexora_deposits ON (account.id = deposit.account_id) WHERE
deposit.id IS NULL

# 24 -- agents with no calls today (Dubai's today)
today = timezone.localdate()
User.objects.filter(profile__isnull=False).exclude(calls__started_at__date=to
day)
# WHERE NOT (auth_user.id IN (SELECT agent_id FROM crm_calllog WHERE
started_at::date = '...'))
```

23: the LEFT JOIN + `IS NULL` pattern is correct and usually fast; Day 8 rewrites it as `~Exists(...)`, which reads closer to the question and never multiplies rows. 24: a single-condition `exclude()` across a multi-valued relation becomes a clean `NOT IN (subquery)` — exactly "no such call exists".

25–27.

```
# 25
manager = User.objects.get(username="manager1")
list(manager.managed_desks.values_list("name", flat=True))            #
through the M2M
list(DeskMembership.objects.filter(user=manager)
     .values_list("desk__name", "role"))                              # with
the role -- the through model
list(DeskMembership.objects.filter(desk__slug="conversion")
     .values("user__username", "role", "since"))

# 26 -- compare two foreign keys of the SAME row with F()
CallLog.objects.exclude(agent__profile__desk=F("lead__desk"))
# JOIN crm_agentprofile ON (calllog.agent_id = agentprofile.user_id)
# JOIN crm_lead ON (calllog.lead_id = lead.id)
# WHERE NOT (agentprofile.desk_id = lead.desk_id ...)

# 27 -- downline leads with an approved deposit above 1,000 USD
big = (Lead.objects.filter(downline,
                           accounts__deposits__status="approved",
                           accounts__deposits__amount_usd__gt=1000)   # SAME
filter() = SAME deposit
       .distinct())
```

28–29.

```
# 28 -- "last month" as Dubai calendar boundaries, compared as timestamps
(index-friendly)
from datetime import datetime, time, timedelta

first_this = timezone.localdate().replace(day=1)
first_last = (first_this - timedelta(days=1)).replace(day=1)
start = timezone.make_aware(datetime.combine(first_last, time.min))
end = timezone.make_aware(datetime.combine(first_this, time.min))

qs = Lead.objects.filter(accounts__deposits__status="approved",
                         accounts__deposits__created_at__gte=start,
                         accounts__deposits__created_at__lt=end)
qs.count()                 # counts JOINED ROWS: a lead with 3 such deposits
counts 3 times
qs.distinct().count()      # distinct leads -- the real answer
(Deposit.objects.filter(status="approved", created_at__gte=start,
created_at__lt=end)
 .values("account__lead").distinct().count())      # the same answer,
starting from deposits
```

28: prefer `created_at >= start AND created_at < end` over `created_at__date__range=(...)` — casting the column to a date in SQL stops PostgreSQL from using an index on `created_at`.

```
# 29
vip = Desk.objects.get(slug="vip")
manager = User.objects.get(username="manager1")
vip.managers.add(manager, through_defaults={"role": "deputy"})        # M2M
API + extra columns
DeskMembership.objects.filter(desk=vip, user=manager).update(role="manager")
# change: the MODEL
vip.managers.remove(manager)                                          # M2M
API
# the same through the model only:
m = DeskMembership.objects.create(desk=vip, user=manager, role="deputy")
m.role = "manager"; m.save(update_fields=["role"])
m.delete()
```

30.

```
# crm/tests.py
class Day3Tests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=300, stdout=StringIO())    # small
but complete

    def test_broker_hierarchy_shape(self):
        self.assertEqual(Broker.objects.filter(parent__isnull=True).count(),
4)

self.assertEqual(Broker.objects.filter(parent__parent__isnull=False).count(),
24)

    def test_every_ftd_lead_has_a_primary_account(self):

self.assertFalse(Lead.objects.filter(status="ftd").exclude(accounts__is_prima
ry=True).exists())

    def test_broker_cannot_be_its_own_parent(self):
        b = Broker.objects.first()
        with self.assertRaisesMessage(IntegrityError,
"broker_not_own_parent"):
            with transaction.atomic():
                Broker.objects.filter(pk=b.pk).update(parent=b)

    def test_same_filter_means_same_call(self):
        agent = User.objects.get(username="agent01")
        together = Lead.objects.filter(calls__agent=agent,
calls__outcome="interested").distinct()
        apart =
Lead.objects.filter(calls__agent=agent).filter(calls__outcome="interested").d
istinct()
        self.assertLessEqual(together.count(), apart.count())
```

```
python manage.py test crm && git add -A && git commit -m "orm day 3"
```

D. Challenge

31–33.

```
# 31
>>> Desk.objects.get(slug="vip").delete()
ProtectedError: ("Cannot delete some instances of model 'Desk' because they
are referenced
through protected foreign keys: 'Lead.desk'.", {<Lead: ...>, ...})

>>> lead = Lead.objects.filter(accounts__isnull=True,
calls__isnull=False).first()
>>> with queries("delete lead"): lead.delete()
SELECT ... FROM crm_tradingaccount WHERE lead_id IN (7)        -- PROTECT:
anything blocking?
UPDATE crm_sopbreach SET lead_id = NULL WHERE lead_id IN (7)    -- SET_NULL
DELETE FROM crm_calllog WHERE lead_id IN (7)                    -- CASCADE,
"fast delete": no rows loaded
DELETE FROM crm_lead WHERE id IN (7)
(a lead WITH accounts raises ProtectedError: TradingAccount.lead is PROTECT)

# 32  Lead.assigned_to and Lead.created_by, both FK to User, no related_name
crm.Lead.assigned_to: (fields.E304) Reverse accessor 'User.lead_set' for
'crm.Lead.assigned_to'
clashes with reverse accessor for 'crm.Lead.created_by'.
  HINT: Add or change a related_name argument to the definition for
'crm.Lead.assigned_to' or
  'crm.Lead.created_by'.
(+ fields.E305 for the query name) -> related_name="leads" and
related_name="created_leads".

# 33
ForeignKey("crm.Deposit", ...) / ForeignKey("Deposit", ...) in the same app
is resolved LATER,
after every app's models are loaded. Two modules can refer to each other
without importing
each other; across apps use "app_label.Model" (e.g. "reports.Snapshot").
```

34–35 — multi-valued relations.

```
agent = User.objects.get(username="agent01")

# 34
together = Lead.objects.filter(calls__agent=agent,
calls__outcome="interested").distinct()
apart =
Lead.objects.filter(calls__agent=agent).filter(calls__outcome="interested").d
```

```
istinct()
together.count(), apart.count()           # e.g. (310, 395)
show(apart)                               # TWO joins: crm_calllog and T3 (a
second copy)
# together: ONE call that is by agent01 AND interested
# apart:    a call by agent01, and SOME call (maybe another agent's) that was
interested

# 35
total = Lead.objects.count()
has_it = Lead.objects.filter(calls__agent=agent,
calls__outcome="interested").distinct().count()
not_it = Lead.objects.exclude(calls__agent=agent,
calls__outcome="interested").count()
has_it + not_it == total                  # False
# Django docs: the conditions inside ONE exclude() don't necessarily refer to
the same call.
# The exact opposite of task 34's "together":
exact_not =
Lead.objects.exclude(pk__in=Lead.objects.filter(calls__agent=agent,

calls__outcome="interested"))
has_it + exact_not.count() == total       # True (Day 8: ~Exists(...) says it
more directly)
```

36–40.

```
# 36  vip.managers.add(manager)   -- no through_defaults
role has no default, and a CharField's implicit default is '' -> INSERT ...
role = ''
IntegrityError: ... violates check constraint "membership_role_valid"
Without task 15's CHECK, a membership with an EMPTY role would exist silently
-- the
constraint turned a quiet data bug into a loud error.

# 37  A.parent = B, B.parent = A
The database accepts it: the CHECK compares a row only with ITSELF. Any code
that walks up
the chain ("who gets commission?") now loops forever.
```

```
# crm/models.py -- Broker  (37)
from django.core.exceptions import ValidationError

    def clean(self):
        node, seen = self.parent, {self.pk}
        while node is not None:
            if node.pk in seen:
                raise ValidationError({"parent": "This would create a
referral loop."})
            seen.add(node.pk)
            node = node.parent
# call full_clean() before saving (forms do). Day 14's recursive CTE can
audit the whole
# table for loops in one SQL statement.


# 38
from django.db.models.signals import post_delete

lead = Lead.objects.filter(accounts__isnull=True).first()
CallLog.objects.bulk_create([CallLog(lead=lead, agent=lead.assigned_to,
started_at=timezone.now())
                             for _ in range(2000)])
with queries("fast"):
    lead.delete()                  # DELETE FROM crm_calllog WHERE lead_id IN
(...) -- nothing loaded

def on_call_deleted(sender, instance, **kwargs): ...
post_delete.connect(on_call_deleted, sender=CallLog)
# repeat the setup with another lead, then:
with queries("with a signal"):
    lead2.delete()                 # SELECT all 2,000 calls, then DELETE ...
WHERE id IN (...)
                                   # in batches, and 2,000 Python signal
calls
post_delete.disconnect(on_call_deleted, sender=CallLog)
# a receiver needs the objects, so Django can no longer delete by a single
WHERE clause
```

```
# 39  settings.AUTH_USER_MODEL instead of `from django.contrib.auth.models
import User`
A project can swap in its own user model (and most should, from day one). A
hard import
ties your models to the default one. In models.py use the string
settings.AUTH_USER_MODEL
(resolved lazily); in runtime code use get_user_model().
```

```
# 40 -- naive "latest call per lead"
from django.db import connection
from django.test.utils import CaptureQueriesContext

leads = list(Lead.objects.order_by("-created_at")[:50])
with CaptureQueriesContext(connection) as ctx:
    latest = {lead.pk: lead.calls.order_by("-started_at").first() for lead in
leads}
len(ctx)                            # 50 -- one query per lead
# Day 8 answers it in ONE query (Subquery + OuterRef, or DISTINCT ON); Day 9
in two (Prefetch
# with a sliced queryset). Write your 50 down -- you'll beat it twice.
```

Day 4 — Migrations

Goal: treat migrations as versioned, reviewable schema changes. Read them, write them by hand when `makemigrations` can't guess, move data safely, roll back, and change big Nexora tables without locking the CRM during trading hours.

```
models.py  --makemigrations-->  0007_xxx.py  --migrate-->  PostgreSQL schema
(the state you want)            (the steps, in git)        (+ a row in
django_migrations)
The golden rule of big tables, "expand / contract":
  1 expand    add the new thing (nullable column, new table, new index
CONCURRENTLY)
  2 migrate   backfill the data in batches; deploy code that writes both /
reads new
  3 contract  only now remove the old thing or tighten it (NOT NULL, drop
column)
```

A. Drills (1–10) — how migrations work

- [ ] **1.** Open `crm/migrations/0001_initial.py`: find `dependencies`, `operations`, and the model state inside `CreateModel`.
- [ ] **2.** `showmigrations`, then roll `crm` back two migrations and forward again. Read what `migrate` prints at each step.
- [ ] **3.** `makemigrations --name lead_source` and `makemigrations crm --empty --name backfill_lead_source`.
- [ ] **4.** Which operations can Django reverse automatically? Write one that it can't and copy the error on rollback.
- [ ] **5.** Look at the `django_migrations` table in `psql`. What exactly does Django record?
- [ ] **6.** `migrate --fake` and `--fake-initial`: what do they do, and the one legitimate use of each.
- [ ] **7.** Rename `CallLog.outcome` to `result`. Answer `makemigrations`' question both ways and compare the SQL.
- [ ] **8.** Change `Lead.name` to `max_length=150`. Read the `AlterField` SQL.
- [ ] **9.** Add an index with `AddIndexConcurrently`. Why must that migration say `atomic = False`?

- [ ] **10.** `migrate --plan` and `makemigrations --check --dry-run`: what each tells you before you touch the database.

B. Core (11–20) — data migrations and safe changes

- [ ] **11.** Add `Lead.source` (direct, broker, campaign, referral) and backfill it with `RunPython`: "broker" when the lead has a broker, else "direct".
- [ ] **12.** Give that `RunPython` a real `reverse_code`, and know when `RunPython.noop` is the honest reverse.
- [ ] **13.** Rewrite the backfill to update in batches of 1,000 ids, so it can run on millions of rows.
- [ ] **14.** The same backfill as `RunSQL` with `reverse_sql`. When is SQL the better tool?
- [ ] **15.** A NOT NULL column on a full table in three migrations: add nullable → backfill → make NOT NULL.
- [ ] **16.** Move KYC out of the JSON column into a one-to-one `KycProfile` (expand / migrate / contract).
- [ ] **17.** A second app, `reports`, whose migration depends on a specific `crm` migration.
- [ ] **18.** `squashmigrations crm 0001 0010`: read `replaces`, run the tests, and know when the old files can be deleted.
- [ ] **19.** `SeparateDatabaseAndState`: tell Django the model changed without touching the database (and the reverse).
- [ ] **20.** Test a data migration: migrate to the old state, insert rows with historical models, migrate forward, assert.

C. Nexora CRM (21–30) — change requests on live tables

- [ ] **21.** Add an optional `CallLog.recording_url`. Confirm with `sqlmigrate` that it won't rewrite the table.
- [ ] **22.** Add `Lead.country` NOT NULL with default "AE". Is that fast on PostgreSQL 11+? Why?
- [ ] **23.** Split `Lead.name` into `first_name` / `last_name` without breaking the running app.
- [ ] **24.** Add a "wallet" deposit method: choices, the CHECK constraint and the migration together.
- [ ] **25.** Make `(lead, started_at)` unique on `CallLog` when duplicates already exist: find them, fix them, then constrain.

- [ ] **26.** Index `CallLog(started_at)` concurrently on the live table.
- [ ] **27.** Rename `Broker.code` to `brf_code`. The SQL is instant — so what can still break?
- [ ] **28.** Drop `Lead.last_contacted_at` safely across two deploys.
- [ ] **29.** Reference data (SOP rules): a data migration or a command? Decide and implement.
- [ ] **30.** Migration tests pass; commit `orm day 4`.

D. Challenge (31–40) — migrations that hurt

- [ ] **31.** Two branches both add `0012_...`. Copy the error after merging and fix it.
- [ ] **32.** A `RunPython` that imports `from crm.models import Lead`. Why does it break months later?
- [ ] **33.** Your `RunPython` calls `Lead.objects.open()` (a custom manager) or a model method. What happens?
- [ ] **34.** A data migration that updates 2 million rows in one transaction. What's locked, for how long? Fix it.
- [ ] **35.** Add `external_ref = UUIDField(default=uuid.uuid4, unique=True)` to a table with rows. Copy the error and explain it.
- [ ] **36.** Roll back a migration that dropped a column. What comes back?
- [ ] **37.** A CHECK constraint that passes on the empty test database but fails on production data. Make it safe with `NOT VALID` + `VALIDATE`.
- [ ] **38.** An `ALTER TABLE` waiting behind a long report query blocks every CRM user. Set a `lock_timeout`.
- [ ] **39.** Someone ran `migrate --fake` by mistake. Detect the mismatch and repair it.
- [ ] **40.** Write Nexora's 10-point migration review checklist.

Solutions

Migration numbers below (`0009`, `0010`, …) are examples — yours depend on how many you've made so far. Always check with `showmigrations crm`.

A. Drills

1–5.

```
# 1 -- crm/migrations/0001_initial.py (shape)
class Migration(migrations.Migration):
    initial = True
    dependencies = []                                   # later ones: [("crm",
"0008_...")] and
                                                        #
migrations.swappable_dependency(settings.AUTH_USER_MODEL)
    operations = [
        migrations.CreateModel(
            name="Desk",
            fields=[("id", models.BigAutoField(auto_created=True,
primary_key=True, serialize=False,
                                               verbose_name="ID")),
                    ("slug", models.SlugField(unique=True)), ...],
            options={"ordering": ["name"]},
        ),
    ]
# operations describe CHANGES to a model state; Django replays them in order
to know what
# your models looked like at every point in history (the "historical
models").
```

```
# 2
python manage.py showmigrations crm      [X] 0001_initial ... [X]
0009_lead_source
python manage.py migrate crm 0007        Unapplying crm.0009_lead_source...
OK
                                         Unapplying crm.0008_...... OK
python manage.py migrate crm             Applying crm.0008_...... OK  (and
0009)
Rolling back RemoveField re-creates the column EMPTY -- the data is gone
(task 36).

# 3
python manage.py makemigrations crm --name lead_source
python manage.py makemigrations crm --empty --name backfill_lead_source
-> named files are reviewable; "0014_auto_20261020_1015" tells a reviewer
nothing.

# 4
Reversible by themselves: CreateModel, AddField, RemoveField*, AlterField,
RenameField,
```

```
AddIndex, AddConstraint ...       (*the column comes back, the data doesn't)
Reversible only if you say how:   RunPython(forwards, reverse_code),
RunSQL(sql, reverse_sql)
$ python manage.py migrate crm 0009
django.db.migrations.exceptions.IrreversibleError: Operation <RunPython
<function forwards ...>>
in crm.0010_backfill_lead_source is not reversible

# 5  psql: SELECT app, name, applied FROM django_migrations WHERE app = 'crm'
ORDER BY id;
 crm | 0001_initial      | 2026-10-20 06:02:11+00
 crm | 0009_lead_source  | 2026-10-24 09:40:55+00
Only the app, the NAME and a timestamp. No checksum, no content -- which is
why editing an
applied migration (Day 1, task 33) goes unnoticed.
```

6–10.

```
# 6
migrate crm 0009 --fake     records 0009 as applied WITHOUT running it.
                            Legit use: the change is already in the database
(applied by hand
                            in an emergency) and Django's records must catch
up.
migrate --fake-initial      skips an INITIAL migration whose tables already
exist.
                            Legit use: adopting an existing database (a
legacy reporting schema).
                            It only checks table/column NAMES exist -- not
types or constraints.

# 7  rename CallLog.outcome -> result
"Was calllog.outcome renamed to calllog.result (a CharField)? [y/N]"
 y  RenameField  -> ALTER TABLE crm_calllog RENAME COLUMN outcome TO result;
(instant, data kept)
 N  RemoveField + AddField -> DROP COLUMN outcome; ADD COLUMN result ...;
(every outcome LOST)
Read the question every time. (Rename it back afterwards -- later days use
`outcome`.)

# 8
ALTER TABLE crm_lead ALTER COLUMN name TYPE varchar(150);   -- longer =
metadata-only, instant
```

```
# 9 -- crm/migrations/0011_call_outcome_idx.py
from django.contrib.postgres.operations import AddIndexConcurrently
from django.db import migrations, models


class Migration(migrations.Migration):
    atomic = False          # CREATE INDEX CONCURRENTLY cannot run inside a
transaction block
    dependencies = [("crm", "0010_backfill_lead_source")]
    operations = [
        AddIndexConcurrently("calllog", models.Index(fields=["outcome"],
name="call_outcome_idx")),
    ]
# put the same Index in CallLog.Meta.indexes so models and migration state
agree.
# CONCURRENTLY builds without blocking writes (slower, and a failed build
leaves an INVALID
# index you must drop and retry).
```

```
# 10
python manage.py migrate --plan
  Planned operations:
  crm.0011_call_outcome_idx
      Concurrently create index call_outcome_idx on field(s) outcome of model
calllog
python manage.py makemigrations --check --dry-run    -> non-zero exit if a
migration is missing
```

B. Core

11–14 — one backfill, four ways.

```
# models.py -- Lead
class Source(models.TextChoices):
    DIRECT = "direct", "Direct"
    BROKER = "broker", "Broker"
    CAMPAIGN = "campaign", "Campaign"
    REFERRAL = "referral", "Referral"

source = models.CharField(max_length=10, choices=Source.choices,
default=Source.DIRECT)
# 0009_lead_source (makemigrations) adds it; every existing row gets 'direct'
```

```
# 0010_backfill_lead_source.py  (11 + 12)
from django.db import migrations


def forwards(apps, schema_editor):
    Lead = apps.get_model("crm", "Lead")          # the HISTORICAL model, as
of this migration
    Lead.objects.filter(broker__isnull=False).update(source="broker")


def backwards(apps, schema_editor):
    Lead = apps.get_model("crm", "Lead")
    Lead.objects.update(source="direct")          # exactly the state before
forwards ran


class Migration(migrations.Migration):
    dependencies = [("crm", "0009_lead_source")]
    operations = [migrations.RunPython(forwards, backwards)]
```

12: `reverse_code=migrations.RunPython.noop` is honest when undoing isn't needed for correctness — e.g. the previous migration's reverse drops the column anyway. A missing reverse just makes the whole app impossible to roll back past this point.

```
# 13 -- batched: each UPDATE commits on its own, locks stay short, safe to
re-run
BATCH = 1000


def forwards(apps, schema_editor):
    Lead = apps.get_model("crm", "Lead")
    last_pk = 0
    while True:
        ids = list(Lead.objects.filter(pk__gt=last_pk, broker__isnull=False)
                   .order_by("pk").values_list("pk", flat=True)[:BATCH])
        if not ids:
            break
        Lead.objects.filter(pk__in=ids).update(source="broker")
        last_pk = ids[-1]
```

```
class Migration(migrations.Migration):
    atomic = False            # otherwise ALL batches still run inside one
big transaction
    dependencies = [("crm", "0009_lead_source")]
    operations = [migrations.RunPython(forwards, migrations.RunPython.noop)]


# 14 -- the same as SQL
migrations.RunSQL(
    sql="UPDATE crm_lead SET source = 'broker' WHERE broker_id IS NOT NULL",
    reverse_sql="UPDATE crm_lead SET source = 'direct'",
)
```

14: SQL wins for set-based updates (one statement, no rows travel to Python) and never breaks when models change later. Python wins when the logic needs Python (parsing, normalising phones) or must be database-portable.

15–16 — expand / migrate / contract.

```
# 15: CallLog.desk = the desk at the time of the call (reports must not move
when a lead moves)
# 0012_calllog_desk          AddField desk ForeignKey(null=True)        ->
expand
# 0013_backfill_calllog_desk
migrations.RunSQL(
    sql="""UPDATE crm_calllog c SET desk_id = l.desk_id
           FROM crm_lead l WHERE c.lead_id = l.id AND c.desk_id IS NULL""",
# batch it on big tables
    reverse_sql=migrations.RunSQL.noop,
)
# 0014_calllog_desk_not_null  AlterField desk ForeignKey(null=False)    ->
contract
#   ALTER TABLE crm_calllog ALTER COLUMN desk_id SET NOT NULL;
#   (scans the table under an exclusive lock; on big tables add a validated
#    CHECK (desk_id IS NOT NULL) first -- PostgreSQL 12+ then skips the scan)
# and the code that creates calls sets desk=lead.desk from now on.
```

```
# 16 -- crm/models.py
class KycProfile(models.Model):
    lead = models.OneToOneField(Lead, on_delete=models.CASCADE,
related_name="kyc_profile")
    status = models.CharField(max_length=10, default="pending")
    nationality = models.CharField(max_length=2, blank=True)
```

```
    passport_no = models.CharField(max_length=20, blank=True)
    passport_expiry = models.DateField(null=True, blank=True)


# 0015_kycprofile (CreateModel)  -> expand
# 0016_copy_kyc                   -> migrate
def copy_kyc(apps, schema_editor):
    Lead, KycProfile = apps.get_model("crm", "Lead"), apps.get_model("crm",
"KycProfile")
    batch = []
    for lead in Lead.objects.exclude(kyc={}).only("pk",
"kyc").iterator(chunk_size=2000):
        k = lead.kyc
        batch.append(KycProfile(lead_id=lead.pk, status=k.get("status",
"pending"),
                                nationality=k.get("nationality", ""),
passport_no=k.get("passport_no", ""),
                                passport_expiry=k.get("passport_expiry")))
        if len(batch) == 2000:
            KycProfile.objects.bulk_create(batch, ignore_conflicts=True);
batch = []
    KycProfile.objects.bulk_create(batch, ignore_conflicts=True)
# then deploy code that reads lead.kyc_profile; a LATER release removes
Lead.kyc (contract).
# In this practice project keep Lead.kyc -- Day 12's JSON drills use it.
```

16: `passport_expiry` arrives from JSON as a string like "2031-04-30"; Django converts ISO date strings on save for a `DateField`, but bad strings fail the whole batch — validate them in real data.

17–20.

```
# 17 -- python manage.py startapp reports ; add "reports" to INSTALLED_APPS
# reports/models.py
class DailySnapshot(models.Model):
    desk = models.ForeignKey("crm.Desk", on_delete=models.PROTECT)
    day = models.DateField()
    ftd_count = models.PositiveIntegerField(default=0)

# reports/migrations/0001_initial.py -- generated with a cross-app
dependency:
#   dependencies = [("crm", "0016_copy_kyc")]
# Django will always migrate crm up to 0016 before reports.0001, on every
database.
```

```
# 18
python manage.py squashmigrations crm 0001 0010
-> crm/migrations/0001_squashed_0010_backfill_lead_source.py
   replaces = [("crm", "0001_initial"), ..., ("crm",
"0010_backfill_lead_source")]
New databases run the squash; old databases that ran 0001..0010 are treated
as having it.
RunPython functions are not copied automatically -- Django warns "Manual
porting required";
move those functions into the squashed file. When EVERY environment (laptops,
staging,
production) is past 0010: delete 0001..0010, remove `replaces`, update
dependencies on them.
```

```
# 19 -- change Django's state only (first half of task 28's safe drop)
migrations.SeparateDatabaseAndState(
    state_operations=[migrations.RemoveField("lead", "last_contacted_at")],
    database_operations=[],                # the column stays in PostgreSQL
for now
)
# the reverse use: database_operations=[RunSQL(...)] with state_operations=[]
when SQL made
# a change Django must not repeat; or both lists filled to switch a plain M2M
onto an
# explicit through model that reuses the existing table.


# 20 -- crm/tests/test_migrations.py
from django.db import connection
```

```
from django.db.migrations.executor import MigrationExecutor
from django.test import TransactionTestCase


class BackfillLeadSourceTest(TransactionTestCase):
    migrate_from = [("crm", "0009_lead_source")]
    migrate_to = [("crm", "0010_backfill_lead_source")]

    def setUp(self):
        executor = MigrationExecutor(connection)
        executor.migrate(self.migrate_from)
        old = executor.loader.project_state(self.migrate_from).apps
        Desk, Broker, Lead = (old.get_model("crm", m) for m in ("Desk",
"Broker", "Lead"))
        desk = Desk.objects.create(slug="t", name="T")
        broker = Broker.objects.create(code="BRF-T", name="T")
        self.brokered = Lead.objects.create(name="A", phone="+971500000001",
desk=desk, broker=broker).pk
        self.direct = Lead.objects.create(name="B", phone="+971500000002",
desk=desk).pk

        executor = MigrationExecutor(connection)          # fresh executor:
reload the graph
        executor.migrate(self.migrate_to)
        self.apps = executor.loader.project_state(self.migrate_to).apps

    def tearDown(self):
        executor = MigrationExecutor(connection)
        executor.migrate(executor.loader.graph.leaf_nodes())   # back to the
latest state

    def test_backfill(self):
        Lead = self.apps.get_model("crm", "Lead")
        self.assertEqual(Lead.objects.get(pk=self.brokered).source, "broker")
        self.assertEqual(Lead.objects.get(pk=self.direct).source, "direct")
```

20: historical models have the fields and defaults of THAT moment but none of your custom methods or `save()` (task 33) — so the test passes an already-normalised phone.

C. Nexora CRM

21–22 — adding columns to big tables.

```
-- 21  recording_url = models.URLField(blank=True)     python manage.py
sqlmigrate crm 0017
ALTER TABLE crm_calllog ADD COLUMN recording_url varchar(200) DEFAULT '' NOT
NULL;
ALTER TABLE crm_calllog ALTER COLUMN recording_url DROP DEFAULT;

-- 22  country = models.CharField(max_length=2, default="AE")
ALTER TABLE crm_lead ADD COLUMN country varchar(2) DEFAULT 'AE' NOT NULL;
ALTER TABLE crm_lead ALTER COLUMN country DROP DEFAULT;
```

21–22: since PostgreSQL 11, `ADD COLUMN ... DEFAULT <constant>` is metadata-only — the value is stored once in the catalog and existing rows read it from there, so no rewrite and only a brief lock. Django then drops the database default (it keeps defaults in Python). Use `db_default="AE"` instead if other systems insert rows. A VOLATILE default (`random()`, `gen_random_uuid()`) still rewrites the whole table.

23–25.

```
# 23 -- expand: first_name / last_name (blank=True, default "")
# migrate: a batched RunPython (task 13's pattern) doing
#     lead.first_name, _, lead.last_name = lead.name.strip().partition(" ")
# code: while both exist, every write keeps them in sync --
class Lead(models.Model):
    def save(self, *args, **kwargs):
        self.phone = self.normalize_phone(self.phone)
        if self.name and not self.first_name:
            self.first_name, _, self.last_name =
self.name.strip().partition(" ")
        super().save(*args, **kwargs)
# contract (a later release, after every reader uses the new columns):
RemoveField name.
# Practice project: stop before contract -- `name` is used everywhere else.


# 24 -- module level, so Meta can see it (Day 2, task 16)
DEPOSIT_METHODS = ["card", "bank", "crypto", "wallet"]

class Deposit(models.Model):
    class Method(models.TextChoices):
        ...
        WALLET = "wallet", "E-wallet"
```

```
    class Meta:
        constraints = [...,
models.CheckConstraint(condition=Q(method__in=DEPOSIT_METHODS),

name="deposit_method_valid")]
# makemigrations: AlterField method (choices -- no SQL) + AddConstraint
deposit_method_valid.
# The choice and the CHECK ship in ONE migration, so they can never disagree.


# 25 -- find duplicates first (a Day 6 query)
from django.db.models import Count

dups = (CallLog.objects.values("lead", "started_at").annotate(n=Count("id"))
        .filter(n__gt=1).order_by())

# 0018_dedupe_calls.py
migrations.RunSQL(
    sql="""DELETE FROM crm_calllog c USING crm_calllog d
           WHERE c.lead_id = d.lead_id AND c.started_at = d.started_at AND
c.id > d.id""",
    reverse_sql=migrations.RunSQL.noop,          # deleted duplicates don't
come back
)
# 0019: AddConstraint UniqueConstraint(fields=["lead", "started_at"],
name="one_call_per_lead_instant")
```

25: on a really big table, build the unique index with `CREATE UNIQUE INDEX CONCURRENTLY` first, then `ALTER TABLE ... ADD CONSTRAINT ... UNIQUE USING INDEX ...` (RunSQL with matching `state_operations`) — the constraint then attaches instantly.

26–29.

```
# 26 -- same shape as task 9
class Migration(migrations.Migration):
    atomic = False
    dependencies = [("crm", "0019_one_call_per_lead_instant")]
    operations = [AddIndexConcurrently("calllog", models.Index(fields=
["started_at"], name="call_started_idx"))]


# 27 -- the column rename itself is instant: ALTER TABLE crm_broker RENAME
COLUMN code TO brf_code
# What breaks: during a rolling deploy the OLD code still selects "code"
until it's replaced;
# raw SQL, BI dashboards, ETL jobs and database views all reference the
column name.
# Safer: rename only the Python attribute and keep the column --
brf_code = models.CharField(max_length=20, unique=True, db_column="code")
# makemigrations -> RenameField/AlterField with NO SQL. Python says brf_code,
SQL still says code.
```

27: after the drill, rename the attribute back to `code` (one more no-SQL migration) — every later day queries `broker__code`.

```
# 28  dropping a column in two releases
(practise on a throwaway Lead.legacy_score -- Day 5 still uses
last_contacted_at; task 19's
 snippet works the same with "legacy_score")
Release 1: remove every use in code +
SeparateDatabaseAndState(state_operations=[RemoveField],
           database_operations=[]). Django stops SELECTing and INSERTing the
column; it
           must be nullable (or have a db default) so INSERTs that omit it
still work.
Release 2: RunSQL("ALTER TABLE crm_lead DROP COLUMN legacy_score",
state_operations=[])
           -- by now no running process knows the column exists.
Dropping in ONE release breaks every old process still running during the
deploy.
```

```
# 29 -- SOP rules: the CRM can't run without them, and tests need them -> a
data migration
def load_rules(apps, schema_editor):
    SOPRule = apps.get_model("crm", "SOPRule")
    for code, title, hours, severity in [
        ("first-contact-24h", "First contact within 24 hours", 24, 3),
        ("no-guaranteed-returns", "Never promise returns", 0, 3),
        ("kyc-before-ftd", "KYC approved before first deposit", 0, 3),
        ("call-recording-consent", "Recording consent on every call", 0, 2),
    ]:
        SOPRule.objects.update_or_create(code=code, defaults={"title": title,
"max_hours": hours,
                                                              "severity":
severity})

operations = [migrations.RunPython(load_rules, migrations.RunPython.noop)]
# the list is COPIED into the migration (frozen in time), not imported from
seed_basics.
# A command suits data that differs per environment (demo leads) or changes
daily.
```

30. Two code updates the new columns require, then test and commit:

```
# seed_nexora.py
Lead(..., source="broker" if broker else "direct")              # pick broker
first, then build
CallLog(lead=lead, agent=agent, desk_id=lead.desk_id, ...)       # desk is
NOT NULL since task 15
# Day 3, task 38's test setup also creates calls: add desk_id=lead.desk_id
there too.
```

```
python manage.py makemigrations --check --dry-run && python manage.py test
crm
git add -A && git commit -m "orm day 4"
```

D. Challenge

31–35.

```
# 31
CommandError: Conflicting migrations detected; multiple leaf nodes in the
migration graph:
(0012_lead_country, 0012_calllog_recording_url in crm).
To fix them run 'python manage.py makemigrations --merge'
-> 0013_merge_... depends on both. If both branches changed the SAME field,
resolve it by
   hand. Number collisions are harmless; the graph is what matters.

# 32  from crm.models import Lead   inside RunPython
It uses the model as it is TODAY. A fresh database (CI, a new laptop) runs
this migration at
its point in history, when later columns don't exist yet -> "column
crm_lead.country does not
exist". apps.get_model("crm", "Lead") gives the model as of this migration.

# 33  historical models have fields, defaults and Meta -- but no custom
methods, no custom
#     save(), and only managers marked use_in_migrations = True
AttributeError: 'Manager' object has no attribute 'open'
-> copy the logic into the migration. Migrations are frozen in time; your app
code is not.

# 34  one transaction, 2,000,000 updated rows
Every updated row stays locked until COMMIT: agents editing those leads wait
(or time out).
The transaction blocks VACUUM, writes a huge WAL burst, and a failure at row
1,999,999 rolls
back everything. Fix: atomic = False + keyset batches (task 13), optionally a
short sleep
between batches -- or run the backfill as a command outside the deploy.

# 35  external_ref = UUIDField(default=uuid.uuid4, unique=True) on a table
with rows
django.db.utils.IntegrityError: could not create unique index
"crm_lead_external_ref_key"
DETAIL:  Key (external_ref)=(3f1c...) is duplicated.
The migration evaluates uuid4() ONCE and gives that one value to every
existing row.
Fix (Django's documented recipe): 1) add it null=True, no unique  2)
RunPython: a uuid4() per
row, in batches  3) AlterField unique=True, null=False.
PostgreSQL-only alternative: db_default=RandomUUID()
```

```
(django.contrib.postgres.functions) --
computed per row, but a volatile default rewrites the table.

# 36  rolling back RemoveField
The column is re-created, empty (or with its default). The data is gone; only
a backup brings
it back. That's why "contract" steps run last, separately, and after a
backup.
```

37–38.

```
# 37 -- find violators first
Trade.objects.filter(closed_at__lt=F("opened_at")).count()

# 0020_trade_close_after_open.py
migrations.RunSQL(
    sql="""ALTER TABLE crm_trade ADD CONSTRAINT trade_close_after_open
           CHECK (closed_at IS NULL OR closed_at >= opened_at) NOT VALID""",
    reverse_sql="ALTER TABLE crm_trade DROP CONSTRAINT
trade_close_after_open",
    state_operations=[migrations.AddConstraint("trade",
models.CheckConstraint(
        condition=Q(closed_at__isnull=True) |
Q(closed_at__gte=F("opened_at")),
        name="trade_close_after_open"))],
)
# NOT VALID: enforced for new and updated rows at once; old rows are not
checked (instant).
# 0021, after fixing the old rows:
migrations.RunSQL("ALTER TABLE crm_trade VALIDATE CONSTRAINT
trade_close_after_open",
                  migrations.RunSQL.noop)
# VALIDATE scans the table under a lighter lock that doesn't block reads or
writes.
# Add the same CheckConstraint to Trade.Meta so models and state agree.


# 38
class Migration(migrations.Migration):
    dependencies = [...]
    operations = [
        migrations.RunSQL("SET LOCAL lock_timeout = '5s'",
migrations.RunSQL.noop),
```

```
        migrations.AddField("lead", "country", models.CharField(max_length=2,
default="AE")),
    ]
# ALTER TABLE needs an ACCESS EXCLUSIVE lock. Behind a 10-minute report query
it WAITS -- and
# every new query on crm_lead queues behind the waiting ALTER: the whole CRM
freezes.
# lock_timeout makes the migration fail fast instead; retry when the report
is done.
# SET LOCAL lasts until the migration's transaction ends (atomic migrations
only).
```

39–40.

```
# 39
Symptom: showmigrations says [X] 0017_calllog_recording_url, yet
  ProgrammingError: column crm_calllog.recording_url does not exist
Confirm: compare `sqlmigrate crm 0017` with `\d crm_calllog`.
Repair: python manage.py migrate crm 0016 --fake    (un-record 0017 only)
        python manage.py migrate crm                (run it for real)
If it was PARTLY applied, finish or undo the missing parts by hand before re-
running.

# 40  Nexora migration review checklist
 1. Named, small, one purpose; the file is in the same PR as the model
change.
 2. sqlmigrate read by the reviewer -- no surprise DROP, rewrite or table
lock.
 3. Renames confirmed as RenameField (task 7), never remove + add.
 4. Big tables: expand -> migrate -> contract, across releases.
 5. New NOT NULL: nullable + backfill + tighten, or a constant default.
 6. Indexes on big tables: AddIndexConcurrently, atomic = False.
 7. Constraints on existing data: violators checked; NOT VALID + VALIDATE
when large.
 8. Data migrations: apps.get_model, batched, re-runnable, with a reverse (or
a reason for noop).
 9. lock_timeout on anything taking an exclusive lock on a busy table.
10. Tests run from an empty database (CI) AND a copy of production data
before release.
```

Day 5 — QuerySet basics

Goal: answer any "which rows?" question in one QuerySet — the right lookup, the right combination of conditions, built dynamically from user input, paginated, and checked with `explain()`. This is the `WHERE` clause, mastered.

```
field__lookup=value             WHERE clause
status="new"                    status = 'new'
name__icontains="omar"          UPPER(name::text) LIKE UPPER('%omar%')
created_at__gte=start           created_at >= '...'
broker__isnull=True             broker_id IS NULL
Q(a) | Q(b), ~Q(c)              a OR b, NOT c
filter(a, b) == filter(a).filter(b)   ...except across multi-valued relations
(Day 3, task 34)
```

A. Drills (1–10) — the lookup catalogue

- [ ] **1.** `exact` and `iexact` on status and name. What does `filter(broker=None)` become?
- [ ] **2.** `contains` vs `icontains` on a lead name. Read the SQL of both.
- [ ] **3.** `startswith` / `endswith` on phone numbers. Search for a name containing `%` or `_` — does Django escape it?
- [ ] **4.** `in` with a Python list, then with a QuerySet. Count the queries each time.
- [ ] **5.** `range`, `gt`, `gte`, `lt`, `lte` on dates and on `Decimal` amounts.
- [ ] **6.** Date transforms: `__date`, `__year`, `__month`, `__week_day`, `__hour`. In which time zone are they computed?
- [ ] **7.** `isnull` on a column and on a reverse relation.
- [ ] **8.** `Q` objects: OR, AND, NOT; mixing `Q` with keyword arguments in one `filter()` call.
- [ ] **9.** `exclude(x)` vs `filter(~Q(x))`, on a nullable column. Compare with a hand-written `WHERE col <> 'x'`.
- [ ] **10.** `earliest` / `latest`, `first` / `last`, `in_bulk`, `reverse()`.

B. Core (11–20) — queries built from input

- [ ] **11.** `filter_leads(params)`: build one QuerySet from optional status, desk, broker, source, date-from and date-to inputs.
- [ ] **12.** Add a free-text search over name, phone and broker code.

- [ ] **13.** Reusable base QuerySets: `open_leads`, `this_week(qs)`. Combine them; `union`, `intersection`, `difference` and their limits.
- [ ] **14.** Write functions that return QuerySets, not lists. Show what a list breaks.
- [ ] **15.** `filter(pk__in=...)` with a subquery vs with `list(values_list(...))`: one query vs two, and the size limit of the second.
- [ ] **16.** `count()`, `exists()`, `len()`, `bool()`: when each is right.
- [ ] **17.** Pagination twice: `Paginator` (OFFSET) and a keyset cursor on `(created_at, id)`.
- [ ] **18.** Ordering by a related field, by two fields with NULLs, and the cost of `order_by("?")`.
- [ ] **19.** Read three `explain(analyze=True)` plans: Seq Scan, Index Scan, Bitmap Heap Scan. Estimated vs actual rows.
- [ ] **20.** `crm/queries.py`: named, documented, tested query functions for the rest of the program.

C. Nexora CRM (21–30) — the questions operations asks

- [ ] **21.** agent01's callbacks contacted today.
- [ ] **22.** Open leads not contacted for 3 days, including those never contacted.
- [ ] **23.** Leads created on the night shift (22:00–08:00 Dubai time).
- [ ] **24.** Calls longer than 10 minutes this week on the conversion desk.
- [ ] **25.** Approved deposits between 1,000 and 5,000 USD in Q3 2026.
- [ ] **26.** Leads whose phone is on the UAE mobile prefixes 050, 052, 054, 055, 056 or 058.
- [ ] **27.** XAUUSD trades still open after 7 days.
- [ ] **28.** High-severity SOP breaches in the last 30 days by agents of the conversion desk.
- [ ] **29.** Leads with KYC approved but no deposit at all.
- [ ] **30.** Tests for `crm/queries.py`. Commit `orm day 5`.

D. Challenge (31–40) — queries that lie

- [ ] **31.** `filter(x=None)`, `exclude(x=None)`, `exclude(x="a")` on a nullable column: predict each SQL, then check.
- [ ] **32.** `filter(pk__in=[])`: what SQL runs?
- [ ] **33.** `created_at__date=today` on 200,000 rows with an index on `created_at`. Compare the plan with a range.

- [ ] **34.** `icontains` on 5,000 rows vs 5 million. Why does it stop scaling, and what's the fix (Day 12)?
- [ ] **35.** "One random new lead" with `order_by("?")`. What does PostgreSQL do? Write a cheaper version.
- [ ] **36.** `Lead.objects.all()[:10].filter(status="new")`. Copy the error; write what was meant.
- [ ] **37.** `if Lead.objects.filter(...):` vs `.exists()`. Count rows fetched by each.
- [ ] **38.** `__date` and `__hour` near midnight: compare Django's answer with `psql` in UTC.
- [ ] **39.** `distinct()` + `order_by("desk__name")` + `values("broker")` returns duplicates. Why?
- [ ] **40.** A QuerySet built in a loop with `|` that grows to 500 `OR`s. Rewrite it.

Solutions

All on the `seed_nexora` data (`python manage.py seed_nexora`). After any big seed run `ANALYZE` in `psql` (or wait for autovacuum) — plans built on stale statistics mislead you. SQL below is simplified to the parts that matter.

A. Drills

1–5.

```
# 1
Lead.objects.filter(status="new")                    # WHERE status = 'new'
Lead.objects.filter(name__iexact="omar khalid")      # WHERE
UPPER(name::text) = UPPER('omar khalid')
Lead.objects.filter(broker=None)                     # WHERE broker_id IS
NULL   (=None means isnull)

# 2
Lead.objects.filter(name__contains="Lead 00")        # name::text LIKE
'%Lead 00%'      case-sensitive
Lead.objects.filter(name__icontains="lead 00")       # UPPER(name::text)
LIKE UPPER('%lead 00%')

# 3
Lead.objects.filter(phone__startswith="+97150")      # phone::text LIKE
'+97150%' -- can use the
                                                     # varchar_pattern_ops
```

```
("_like") index from Day 1
Lead.objects.filter(name__contains="50%")             # LIKE '%50\%%' --
Django escapes % and _ for you

# 4
Lead.objects.filter(status__in=["new", "callback"])   # 1 query: status IN
('new', 'callback')
Lead.objects.filter(desk__in=Desk.objects.filter(is_active=True))
# still 1 query: desk_id IN (SELECT id FROM crm_desk WHERE is_active)

# 5
from decimal import Decimal
Deposit.objects.filter(amount_usd__range=(Decimal("1000"), Decimal("5000")))
# BETWEEN -- both ends included
Deposit.objects.filter(created_at__gte=start, created_at__lt=end)
# half-open: the safe date range
```

5: `range` on DATETIMES is a trap — `(date(2026, 9, 1), date(2026, 9, 30))` stops at 30 Sept 00:00 and misses the last day. Use `>= start` and `< next_start`.

6–10.

```
# 6 -- computed in the CURRENT time zone (Asia/Dubai), not UTC
Lead.objects.filter(created_at__date=date(2026, 10, 20))
# WHERE (created_at AT TIME ZONE 'Asia/Dubai')::date = '2026-10-20'
CallLog.objects.filter(started_at__hour__gte=22)       # EXTRACT(HOUR FROM
started_at AT TIME ZONE 'Asia/Dubai')
Lead.objects.filter(created_at__week_day=7)            # 1 = Sunday ... 7 =
Saturday (Django's numbering)
Lead.objects.filter(created_at__iso_week_day=6)        # 1 = Monday ... 7 =
Sunday (ISO) -- Saturday here

# 7
Lead.objects.filter(last_contacted_at__isnull=True)    # a column: IS NULL
TradingAccount.objects.filter(deposits__isnull=True)   # a reverse relation:
LEFT JOIN ... IS NULL

# 8
Lead.objects.filter(Q(status="new") | Q(status="callback"), desk__slug="vip")
# Q first, then kwargs
Lead.objects.filter(Q(status="ftd") & Q(broker__isnull=False))
Lead.objects.filter(~Q(source="broker"))
# Lead.objects.filter(desk__slug="vip", Q(...)) -> SyntaxError: positional
argument follows keyword argument
```

```
# 9  a NULLABLE relation: Lead.broker
Lead.objects.exclude(broker__code="BRF-0001")      (filter(~Q(...)) is
identical)
  LEFT OUTER JOIN crm_broker ON ...
  WHERE NOT (crm_broker.code = 'BRF-0001' AND crm_broker.code IS NOT NULL)
  -> leads WITHOUT a broker are INCLUDED: "not referred by BRF-0001" is true
for them.
Hand-written: ... JOIN crm_broker b ... WHERE b.code <> 'BRF-0001'
  -> NULL <> 'BRF-0001' is NULL, not TRUE: leads without a broker silently
DISAPPEAR.
Django's version is usually what the business means. When comparing an ORM
report with a
SQL report, this is the first place to look.
```

```
# 10
Lead.objects.earliest("created_at"), Lead.objects.latest("created_at")   #
DoesNotExist if empty
Lead.objects.order_by("created_at").first()             # None if empty
Lead.objects.in_bulk([1, 2, 3])                         # {1: <Lead>, 2:
<Lead>, 3: <Lead>}
Lead.objects.in_bulk(["+971500000001"], field_name="phone")   # field must be
unique
Lead.objects.order_by("created_at").reverse()           # flips the ORDER BY;
no extra query
```

B. Core

11–14 — crm/queries.py, part 1.

```
# crm/queries.py
import re
from datetime import datetime, time, timedelta

from django.db.models import Q
from django.utils import timezone
from django.utils.dateparse import parse_date

from .models import Lead

OPEN = ["new", "callback"]


def day_start(d):
    """00:00 Dubai time on date d, as an aware datetime."""
    return timezone.make_aware(datetime.combine(d, time.min))


def _date(value):
    try:
        return parse_date(value or "")
    except ValueError:                      # well-formed but impossible,
e.g. 2026-02-30
        return None


def search_q(text):                                                    #
```

```
12
    """Name, phone digits or exact broker code."""
    text = text.strip()
    q = Q(name__icontains=text) | Q(broker__code__iexact=text)
    digits = re.sub(r"\D", "", text)
    if len(digits) >= 4:
        q |= Q(phone__contains=digits)
    return q


def filter_leads(params, qs=None):                                      #
11
    """Apply only the filters that were given. Returns a QuerySet."""
    qs = Lead.objects.all() if qs is None else qs
    if status := params.get("status"):
        qs = qs.filter(status__in=status.split(","))
    if desk := params.get("desk"):
        qs = qs.filter(desk__slug=desk)
    if broker := params.get("broker"):
        qs = qs.filter(broker__code=broker)
    if source := params.get("source"):
        qs = qs.filter(source=source)
    if d := _date(params.get("from")):
        qs = qs.filter(created_at__gte=day_start(d))
    if d := _date(params.get("to")):
        qs = qs.filter(created_at__lt=day_start(d + timedelta(days=1)))   #
"to" includes that day
    if text := (params.get("q") or "").strip():
        qs = qs.filter(search_q(text))
    return qs


def open_leads(qs=None):                                                #
13
    qs = Lead.objects.all() if qs is None else qs     # NOT `qs or
Lead.objects` -- see below
    return qs.filter(status__in=OPEN)


def created_since(qs, days):
    return qs.filter(created_at__gte=timezone.now() - timedelta(days=days))
```

11: `filter_leads({"status": "new,callback", "desk": "vip", "q": "0500"})` — every branch adds a WHERE condition to one lazy QuerySet; nothing runs until it's used.
13: `qs or Lead.objects` calls `bool(qs)`, which RUNS the query and loads every row just to test truthiness.

```
# 13 -- combining
vip_open = open_leads().filter(desk__slug="vip")
recent_ftd = created_since(Lead.objects.filter(status="ftd"), 7)
both = vip_open.union(recent_ftd)                  # SQL UNION (duplicates
removed); all=True -> UNION ALL
both.order_by("-created_at")[:50]                  # allowed: order_by (by
column name), slicing, count()
both.filter(source="broker")
# NotSupportedError: Calling QuerySet.filter() after union() is not
supported.
# both sides must select the same columns. Usually a Q(...) | Q(...) is
simpler and filterable.


# 14 -- a list ends the conversation with the database
def open_vip_list():
    return list(open_leads().filter(desk__slug="vip"))   # every row loaded
NOW
len(open_vip_list())                # loads thousands of rows to count them
open_vip_list().filter(...)         # AttributeError: 'list' object has no
attribute 'filter'
# return the QuerySet: callers can still filter, count in SQL, paginate,
select_related.
```

15–18.

```
# 15
ids = list(Broker.objects.filter(parent__code="BRF-0001").values_list("id",
flat=True))  # query 1
Lead.objects.filter(broker_id__in=ids)
# query 2
# vs ONE query with a subquery the planner can turn into a semi-join:
Lead.objects.filter(broker__in=Broker.objects.filter(parent__code="BRF-
0001"))
# the list version ships every id through Python and back; with 50,000 ids
the SQL text
# itself becomes huge (and drivers cap the number of parameters).
```

```
# 16
qs.count()    SELECT COUNT(*)            -> you need the number
qs.exists()   SELECT 1 ... LIMIT 1       -> you need yes / no
len(qs)       fetches EVERY row          -> only if you'll loop over the rows
anyway
if qs: / bool(qs)   fetches every row too (fills the cache) -> only right
before using them
```

```
# 17 -- OFFSET pagination
from django.core.paginator import Paginator

page = Paginator(Lead.objects.order_by("-created_at", "-id"), 25).get_page(3)
# SELECT COUNT(*) ...; then ... ORDER BY created_at DESC, id DESC LIMIT 25
OFFSET 50


# 17 -- keyset pagination (crm/queries.py)
def leads_page(cursor=None, size=25, qs=None):
    """cursor = (created_at, id) of the last row of the previous page."""
    qs = (Lead.objects.all() if qs is None else qs).order_by("-created_at",
"-id")
    if cursor:
        created_at, pk = cursor
        qs = qs.filter(Q(created_at__lt=created_at) |
Q(created_at=created_at, id__lt=pk))
    rows = list(qs[:size])
    next_cursor = (rows[-1].created_at, rows[-1].pk) if len(rows) == size
else None
    return rows, next_cursor
# add Index(fields=["-created_at", "-id"], name="lead_recent_keyset_idx") to
Lead.Meta.
# OFFSET 100000 makes PostgreSQL read and throw away 100,000 rows; a keyset
page costs the
# same at page 1 and page 4,000 -- and doesn't skip or repeat rows when new
leads arrive.


# 18
Lead.objects.order_by("desk__name", "-created_at")                # a JOIN
just for ordering
```

```
Lead.objects.order_by(F("last_contacted_at").asc(nulls_first=True), "name")
# never-contacted first
Lead.objects.order_by("?")          # ORDER BY RANDOM(): reads and sorts the
WHOLE table (task 35)
```

19–20.

```
# 19  explain(...) from Day 1's sqltools
explain(Lead.objects.filter(status="lost"))
  Seq Scan on crm_lead (cost=0.00..123.50 rows=1010 width=...) (actual ...
rows=1003 loops=1)
  -> about 20% of rows match: reading the whole table is cheaper than an
index. Correct!
explain(Lead.objects.filter(phone="+971500000007"))
  Index Scan using crm_lead_phone_key on crm_lead (... rows=1) (actual ...
rows=1)
explain(CallLog.objects.filter(agent_id=3, started_at__gte=week_ago))
  Bitmap Heap Scan on crm_calllog (... rows=95) (actual ... rows=88)
    -> Bitmap Index Scan on call_agent_recent_idx
  -> many rows scattered over the table: collect their locations first, then
read pages in order.
Read `rows=` (the planner's estimate) next to `actual ... rows=`. Off by 10x
or more?
Run ANALYZE crm_calllog; -- the statistics are stale (typical right after a
bulk seed).
```

20: `crm/queries.py` now holds `day_start`, `search_q`, `filter_leads`, `open_leads`, `created_since` and `leads_page`; block C adds one function per question. One rule for the file: every function takes optional inputs, returns a QuerySet, and has a one-line docstring saying which business question it answers.

C. Nexora CRM

21–25 — crm/queries.py, part 2.

```
# crm/queries.py (continued)
from datetime import date
from decimal import Decimal

from .models import CallLog, Deposit, SOPBreach, SOPRule, Trade


def today_bounds():
```

```
    today = timezone.localdate()                          # Dubai's today
    return day_start(today), day_start(today + timedelta(days=1))


def callbacks_contacted_today(agent):                                      #
21
    """An agent's callback leads that were contacted today."""
    start, end = today_bounds()
    return Lead.objects.filter(assigned_to=agent, status="callback",
                               last_contacted_at__gte=start,
last_contacted_at__lt=end)


def stale_open_leads(days=3):                                              #
22
    """Open leads not contacted for `days` days, or never contacted."""
    cutoff = timezone.now() - timedelta(days=days)
    return open_leads().filter(Q(last_contacted_at__lt=cutoff) |
Q(last_contacted_at__isnull=True))


def night_shift_leads():                                                   #
23
    """Leads created 22:00-08:00 Dubai time."""
    return Lead.objects.filter(Q(created_at__hour__gte=22) |
Q(created_at__hour__lt=8))


def long_calls_this_week(desk_slug, minutes=10):                           #
24
    """Calls over N minutes since Monday 00:00, on a desk (the desk AT CALL
TIME)."""
    today = timezone.localdate()
    monday = day_start(today - timedelta(days=today.weekday()))
    return CallLog.objects.filter(duration_sec__gt=minutes * 60,
started_at__gte=monday,
                                  desk__slug=desk_slug)


def approved_deposits_between(low, high, start, end):                      #
25
    """Approved deposits with low <= USD amount <= high, start <= day <
end."""
```

```
    return Deposit.objects.filter(status="approved", amount_usd__range=(low,
high),
                                  created_at__gte=day_start(start),
created_at__lt=day_start(end))

# approved_deposits_between(Decimal("1000"), Decimal("5000"), date(2026, 7,
1), date(2026, 10, 1))
```

22: `__lt` alone silently drops the never-contacted leads — NULL is not less than anything. 24: `desk__slug` uses Day 4's `CallLog.desk` (the desk when the call happened); `lead__desk__slug` would be the lead's CURRENT desk. Ask which one the business means — they give different numbers once leads move.

26–29.

```
UAE_MOBILE = ["50", "52", "54", "55", "56", "58"]


def uae_mobile_leads():                                                  #
26
    q = Q()
    for prefix in UAE_MOBILE:
        q |= Q(phone__startswith=f"+971{prefix}")       # each one can use
the LIKE index
    return Lead.objects.filter(q)
# alternative: Lead.objects.filter(phone__regex=r"^\+9715[024568]")  --
shorter, but a regex
# can't use the index. (The seed's phones are all +97150..., so everything
matches.)


def stale_open_trades(symbol="XAUUSD", days=7):                          #
27
    return Trade.objects.filter(symbol=symbol, closed_at__isnull=True,
                                opened_at__lt=timezone.now() -
timedelta(days=days))


def recent_high_breaches(desk_slug, days=30):                            #
28
    return SOPBreach.objects.filter(rule__severity=SOPRule.Severity.HIGH,
                                    occurred_at__gte=timezone.now() -
timedelta(days=days),
```

```
                                        agent__profile__desk__slug=desk_slug)


def kyc_approved_without_deposit():                                          #
29
    return
Lead.objects.filter(kyc__status="approved").exclude(accounts__deposits__isnul
l=False)
```

29: the tempting `filter(kyc__status="approved", accounts__deposits__isnull=True)` is WRONG. It joins every account: a lead with one funded account and one empty account produces a joined row with a NULL deposit, so it matches although it HAS deposits. `exclude(accounts__deposits__isnull=False)` becomes `NOT IN (leads that have any deposit)` — exactly the question. (Day 8's `~Exists()` says it best.) To get data, mark some leads first:

```
Lead.objects.filter(pk__in=Lead.objects.filter(status__in=["callback",
"ftd"]).values("pk")[:300]) \
    .update(kyc={"status": "approved"})       # a sliced subquery: LIMIT
inside IN (...) works on PostgreSQL
```

30.

```
# crm/tests.py
from .queries import OPEN, filter_leads, kyc_approved_without_deposit,
leads_page


class QueryTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=400, stdout=StringIO())

    def test_filters_combine(self):
        qs = filter_leads({"status": "new,callback", "desk": "vip"})
        self.assertEqual(qs.count(), Lead.objects.filter(status__in=OPEN,
desk__slug="vip").count())

    def test_impossible_date_is_ignored(self):
        self.assertEqual(filter_leads({"from": "2026-02-30"}).count(),
Lead.objects.count())
```

```
    def test_keyset_pages_cover_every_lead_once(self):
        seen, cursor = [], None
        while True:
            rows, cursor = leads_page(cursor, size=50)
            seen += [r.pk for r in rows]
            if cursor is None:
                break
        self.assertEqual(sorted(seen), sorted(Lead.objects.values_list("pk",
flat=True)))

    def test_kyc_query_skips_funded_leads(self):
        funded =
Lead.objects.filter(accounts__deposits__isnull=False).first()
        Lead.objects.filter(pk=funded.pk).update(kyc={"status": "approved"})
        self.assertNotIn(funded, kyc_approved_without_deposit())
```

```
python manage.py test crm && git add -A && git commit -m "orm day 5"
```

D. Challenge

31–34.

```
# 31
filter(last_contacted_at=None)            WHERE last_contacted_at IS NULL
exclude(last_contacted_at=None)           WHERE NOT (last_contacted_at IS
NULL)
exclude(close_price=Decimal("1.5"))       WHERE NOT (close_price = 1.5 AND
close_price IS NOT NULL)
  (Trade, a nullable column)              -> open trades (NULL close_price)
are KEPT, unlike
                                             a hand-written close_price <>
1.5 (task 9's lesson)

# 32
list(Lead.objects.filter(pk__in=[]))      -> []  and ZERO queries: Django
knows an empty IN
                                             can't match, so it never asks
the database.
```

```
# 33 -- re-seed bigger first: python manage.py seed_nexora --leads 50000
(~180,000 calls); ANALYZE;
today = timezone.localdate()
explain(CallLog.objects.filter(started_at__date=today))
#   Seq Scan on crm_calllog  Filter: ((started_at AT TIME ZONE
'Asia/Dubai'))::date = ...
#   a FUNCTION of the column -> the index on started_at can't be used
start, end = today_bounds()
explain(CallLog.objects.filter(started_at__gte=start, started_at__lt=end))
#   Index Scan using call_started_idx ... Index Cond: (started_at >= ... AND
started_at < ...)
# same rows, a fraction of the work. Ranges on the raw column are "sargable";
transforms aren't.
```

```
# 34
name__icontains="omar"  ->  UPPER(name) LIKE '%OMAR%'. A leading % defeats a
B-tree index,
so PostgreSQL reads every row: milliseconds at 5,000 rows, seconds at 5
million.
Fixes (Day 12): a pg_trgm GIN index (GinIndex(..., opclasses=
["gin_trgm_ops"])) for
substring search, or full-text search (SearchVector) for word search.
```

35–40.

```
# 35
Lead.objects.filter(status="new").order_by("?").first()
# ORDER BY RANDOM() LIMIT 1 -> PostgreSQL reads EVERY new lead and sorts them
all to keep one.
import random

qs = Lead.objects.filter(status="new")
bounds = qs.aggregate(lo=Min("pk"), hi=Max("pk"))
lead = (qs.filter(pk__gte=random.randint(bounds["lo"],
bounds["hi"])).order_by("pk").first()
        if bounds["lo"] is not None else None)
# two cheap index lookups. Slightly biased (a lead after a big gap in ids is
picked more
# often) -- fine for "give an idle agent any new lead", not for a statistical
sample.
```

```
# 36
Lead.objects.all()[:10].filter(status="new")
TypeError: Cannot filter a query once a slice has been taken.
meant: Lead.objects.filter(status="new")[:10]   -- filter first, slice last
(LIMIT is the
last thing SQL applies; Django won't guess what "filter the first 10" should
mean)

# 37
if Lead.objects.filter(status="callback"):        SELECT ... (every column,
every matching row)
                                                   -> ~1,250 rows fetched to
learn "yes"
if Lead.objects.filter(status="callback").exists():  SELECT 1 ... LIMIT 1 ->
1 row

# 38  a call at 2026-10-19 21:30 UTC = 2026-10-20 01:30 in Dubai
Django:   started_at__date = 2026-10-20     (current time zone: Asia/Dubai)
psql:     started_at::date = 2026-10-19     (the session's time zone, often
UTC)
Same row, different day -> "the ORM report and the SQL report disagree" near
midnight.
In psql: SET TIME ZONE 'Asia/Dubai';  or  (started_at AT TIME ZONE
'Asia/Dubai')::date

# 39
Lead.objects.values("broker").distinct().order_by("desk__name")
SELECT DISTINCT broker_id, crm_desk.name ... ORDER BY crm_desk.name
-> to ORDER BY a column, PostgreSQL needs it in a SELECT DISTINCT, so Django
adds it --
   and now DISTINCT is over (broker, desk name): each broker repeats once per
desk.
Fix: order by what you select (.order_by("broker")), or drop the ordering
(.order_by()).
```

```
# 40
q = Q()
for phone in phones:                # 500 phones from an uploaded file
    q |= Q(phone=phone)
Lead.objects.filter(q)              # WHERE phone = ... OR phone = ...
(x500) -- slow to plan
Lead.objects.filter(phone__in=phones)           # ONE IN (...) list -- same
rows, one condition
Lead.objects.filter(phone__in=[Lead.normalize_phone(p) for p in phones])   #
and normalise first
```

Day 6 — Aggregation

Goal: produce Nexora's reports — desk scoreboards, agent activity, broker revenue — as single GROUP BY queries, and know exactly when an aggregate gives a wrong number. This is the day the ORM starts to feel like your SQL.

```
aggregate()               -> ONE dict for the whole QuerySet          SELECT
SUM(...) FROM ...
annotate()                -> a value ON EACH ROW                       SELECT
..., COUNT(...) GROUP BY pk
values("x").annotate()    -> one row PER x                            GROUP
BY x
annotate(...).filter(n..) -> HAVING                                   HAVING
COUNT(...) > n
Count("id", filter=Q())   -> conditional aggregate
COUNT(id) FILTER (WHERE ...)
The order of values() and annotate() decides the GROUP BY.
```

A. Drills (1–10) — the aggregation toolkit

- [ ] **1.** `aggregate()` over calls: count, total and average duration, the first and last call.
- [ ] **2.** Aggregate an empty QuerySet. What comes back? Fix it with `default=`.
- [ ] **3.** `annotate()` each lead with its number of calls. Read the GROUP BY.
- [ ] **4.** `values("status").annotate(n=Count("id"))`: leads per status.
- [ ] **5.** Swap the order: `annotate(...).values(...)` vs `values(...).annotate(...)`. Compare the rows.
- [ ] **6.** `Count("lead", distinct=True)`: how many different leads each agent called.
- [ ] **7.** Conditional aggregates: FTDs, callbacks and lost leads per desk in one query with `filter=Q(...)`.
- [ ] **8.** HAVING: leads with more than 5 calls.
- [ ] **9.** Sum across two relations: approved deposit USD per lead (`accounts__deposits__...`).
- [ ] **10.** An aggregate of an annotation: the average number of calls per lead.

B. Core (11–20) — report building blocks

- [ ] **11.** Desk scoreboard: leads, FTDs and conversion % per desk, one query.
- [ ] **12.** Agent activity per day: calls, distinct leads, talk time.
- [ ] **13.** A pivot: deposit USD per method with one column per status.
- [ ] **14.** Broker revenue: approved deposit USD per broker, highest first.
- [ ] **15.** Statistics: `StdDev` and `Variance` of call duration per agent.
- [ ] **16.** PostgreSQL aggregates: `ArrayAgg`, `StringAgg`, `BoolOr`, `JSONBAgg`.
- [ ] **17.** Top 5 agents by FTD leads this month.
- [ ] **18.** Ratios without integer division: `Cast` and `ExpressionWrapper`.
- [ ] **19.** Grouping by `desk_id` vs `desk__name` vs `desk__slug`: the SQL difference, and the trap with names.
- [ ] **20.** `crm/reports.py`: report functions that return lists of dicts with stable keys.

C. Nexora CRM (21–30) — the weekly management pack

- [ ] **21.** FTDs per day for the last 30 days (days with zero included).
- [ ] **22.** Calls per outcome per agent this week, as a pivot.
- [ ] **23.** Average time from lead creation to first call.
- [ ] **24.** Approved deposit USD per desk this month, and the desk's share of the total.
- [ ] **25.** Brokers with more than 10 FTD leads.
- [ ] **26.** SOP breaches per agent split by severity.
- [ ] **27.** Closed trades per symbol: count, volume, total and average PnL, win rate.
- [ ] **28.** Leads and FTDs grouped by top-level broker (the Brf hierarchy root).
- [ ] **29.** The desk with the highest average approved deposit.
- [ ] **30.** Prove two reports right against hand-written SQL in tests. Commit `orm day 6`.

D. Challenge (31–40) — numbers that are wrong

- [ ] **31.** Count calls AND sum deposits per lead in one `annotate()`. Why are both wrong? Fix it.
- [ ] **32.** `values("desk").annotate(...).order_by("name")` suddenly returns one row per lead. Why?
- [ ] **33.** `Count("*")` vs `Count("id")` vs `Count("calls")` on leads with no calls.

- [ ] **34.** `Avg` of an integer column vs a decimal column: types and rounding.
- [ ] **35.** `Sum` in `annotate()` for leads without deposits gives `None`. Where does that break a report?
- [ ] **36.** `filter()` before `annotate()` vs after it on the same relation. Compare the counts.
- [ ] **37.** `aggregate()` on a sliced QuerySet (`[:100]`). What SQL does Django write?
- [ ] **38.** Grouping by `started_at` instead of its date. What do you get?
- [ ] **39.** Daily totals in Dubai time vs UTC. Which days move?
- [ ] **40.** A `Count` with `distinct=True` fixes task 31's counts. Does `Sum(..., distinct=True)` fix the sums?

Solutions

Imports used throughout: `from django.db.models import Avg, Count, DecimalField, F, FloatField, Max, Min, Q, Sum` and `from django.db.models.functions import Cast, Round, TruncDate`. Numbers in comments are examples — yours depend on the seed.

A. Drills

1–5.

```
# 1
CallLog.objects.aggregate(n=Count("id"), talk=Sum("duration_sec"),
avg=Avg("duration_sec"),
                          first=Min("started_at"), last=Max("started_at"))
# SELECT COUNT(id), SUM(duration_sec), AVG(duration_sec), MIN(started_at),
MAX(started_at) FROM crm_calllog
# {'n': 18034, 'talk': 8115300, 'avg': 450.0..., 'first': datetime(...),
'last': datetime(...)}

# 2
CallLog.objects.filter(duration_sec__gt=10**6).aggregate(n=Count("id"),
talk=Sum("duration_sec"))
# {'n': 0, 'talk': None}           COUNT gives 0, every other aggregate gives
NULL -> None
CallLog.objects.filter(duration_sec__gt=10**6).aggregate(talk=Sum("duration_s
ec", default=0))
# {'talk': 0}                      COALESCE(SUM(duration_sec), 0)   (default=
```

```
is Django 4.0+)

# 3
leads = Lead.objects.annotate(n_calls=Count("calls"))
# SELECT crm_lead.*, COUNT(crm_calllog.id) AS n_calls
# FROM crm_lead LEFT OUTER JOIN crm_calllog ON (crm_lead.id =
crm_calllog.lead_id)
# GROUP BY crm_lead.id       -- PostgreSQL lets Django group by the primary
key alone
leads[0].n_calls               # an attribute on each Lead object

# 4
Lead.objects.values("status").annotate(n=Count("id")).order_by("status")
# SELECT status, COUNT(id) AS n FROM crm_lead GROUP BY status ORDER BY status
# [{'status': 'callback', 'n': 1251}, {'status': 'ftd', 'n': 1003}, ...]
```

```
# 5
Lead.objects.values("status").annotate(n=Count("calls"))   -> 4 rows: calls
per STATUS
   values() FIRST = "group by these columns"
Lead.objects.annotate(n=Count("calls")).values("status", "n")   -> 5,000
rows: one per LEAD
   annotate() FIRST = computed per object (GROUP BY lead.id); values() then
just picks columns
```

6–10.

```
# 6
(CallLog.objects.values("agent__username")
 .annotate(calls=Count("id"), leads=Count("lead", distinct=True))
 .order_by("agent__username"))
# COUNT(DISTINCT lead_id) -- several calls to one lead count once

# 7 -- one pass over the table, three answers: COUNT(...) FILTER (WHERE ...)
Lead.objects.values("desk__name").annotate(
    total=Count("id"),
    ftd=Count("id", filter=Q(status="ftd")),
    callback=Count("id", filter=Q(status="callback")),
    lost=Count("id", filter=Q(status="lost")),
).order_by("desk__name")

# 8 -- HAVING
Lead.objects.annotate(n=Count("calls")).filter(n__gt=5)
```

```
# ... GROUP BY crm_lead.id HAVING COUNT(crm_calllog.id) > 5

# 9 -- through two relations (one multi-valued chain: lead -> accounts ->
deposits)
Lead.objects.annotate(deposited_usd=Sum("accounts__deposits__amount_usd",

filter=Q(accounts__deposits__status="approved"), default=0))

# 10 -- an aggregate OF an annotation: Django wraps the GROUP BY in a
subquery
Lead.objects.annotate(n=Count("calls")).aggregate(avg_calls=Avg("n"))
# SELECT AVG(n) FROM (SELECT ..., COUNT(crm_calllog.id) AS n ... GROUP BY
crm_lead.id) subquery
# check: CallLog.objects.count() / Lead.objects.count() gives the same number
```

B. Core

11–14.

```
# 11 -- desk scoreboard
PCT = DecimalField(max_digits=12, decimal_places=4)
(Lead.objects.values("desk__slug", "desk__name")
 .annotate(leads=Count("id"), ftd=Count("id", filter=Q(status="ftd")))
 .annotate(conversion_pct=Round(Cast("ftd", output_field=PCT) * 100 /
F("leads"), 1))
 .order_by("-conversion_pct"))
# [{'desk__slug': 'vip', 'desk__name': 'VIP', 'leads': 1688, 'ftd': 345,
'conversion_pct': Decimal('20.4')}, ...]
# Cast makes the division numeric (task 18); Round(..., 1) runs in SQL on a
numeric.


# 12 -- agent activity per (Dubai) day
(CallLog.objects.annotate(day=TruncDate("started_at"))        # in the
current time zone
 .values("agent__username", "day")
 .annotate(calls=Count("id"), leads=Count("lead", distinct=True),
talk_sec=Sum("duration_sec"))
 .order_by("day", "agent__username"))


# 13 -- pivot: one column per status, built from the choices
pivot = {status: Sum("amount_usd", filter=Q(status=status), default=0) for
```

```
status in Deposit.Status.values}
Deposit.objects.values("method").annotate(**pivot).order_by("method")
# [{'method': 'bank', 'approved': Decimal('1830500.00'), 'pending': ...,
'rejected': ...}, ...]


# 14 -- start from the table whose rows you're adding up
(Deposit.objects.filter(status="approved",
account__lead__broker__isnull=False)
 .values("account__lead__broker__code")
 .annotate(revenue_usd=Sum("amount_usd"))
 .order_by("-revenue_usd"))
# Broker.objects.annotate(Sum("leads__accounts__deposits__amount_usd", ...))
gives the same
# numbers, but starting from Deposit keeps the JOIN chain one-directional and
obvious.
```

15–17.

```
# 15
from django.db.models import StdDev, Variance

(CallLog.objects.values("agent__username")
 .annotate(avg=Avg("duration_sec"), sd=StdDev("duration_sec"),
var=Variance("duration_sec"))
 .order_by("-sd"))
# StDev/Variance default to the POPULATION versions (stddev_pop); sample=True
-> stddev_samp


# 16 -- add "django.contrib.postgres" to INSTALLED_APPS (Day 12 needs it too)
from django.contrib.postgres.aggregates import ArrayAgg, BoolOr, JSONBAgg,
StringAgg
from django.db.models import BooleanField, ExpressionWrapper

Desk.objects.annotate(agents=ArrayAgg("agents__user__username",
distinct=True))
# [{'agents': ['agent01', 'agent04', ...]}] -- a Python list from one SQL
array
Broker.objects.filter(parent__isnull=True).annotate(children=StringAgg("child
ren__code", delimiter=", "))
# 'BRF-0005, BRF-0006, BRF-0007'
Lead.objects.annotate(ever_interested=BoolOr(
```

```
    ExpressionWrapper(Q(calls__outcome="interested"),
output_field=BooleanField())))
# (the everyday equivalent: Count("calls",
filter=Q(calls__outcome="interested")) > 0)
Lead.objects.filter(pk=7).annotate(outcomes=JSONBAgg("calls__outcome"))
# one jsonb array per lead -- handy for API payloads


# 17 -- top 5 agents by leads who made an approved deposit this month
from crm.queries import day_start

month_start = day_start(timezone.localdate().replace(day=1))
(User.objects.filter(profile__isnull=False)
 .annotate(funded=Count("leads", distinct=True,

filter=Q(leads__accounts__deposits__status="approved",

leads__accounts__deposits__created_at__gte=month_start)))
 .order_by("-funded", "username")[:5])
# distinct=True: a lead with 3 deposits this month counts once. (A strict
"FIRST deposit this
# month" needs Day 8's subqueries.)
```

18–20.

```
# 18 -- the integer-division trap
qs = Lead.objects.values("desk__slug").annotate(leads=Count("id"),
ftd=Count("id", filter=Q(status="ftd")))
qs.annotate(pct=F("ftd") * 100 / F("leads"))
# ftd * 100 / leads in SQL: integer / integer -> 20 (truncated), not 20.44
qs.annotate(pct=ExpressionWrapper(F("ftd") * 100 / F("leads"),
output_field=FloatField()))
# STILL 20(.0): ExpressionWrapper only tells PYTHON the type -- the SQL is
unchanged
qs.annotate(pct=Cast("ftd", output_field=FloatField()) * 100 / F("leads"))
# ftd::double precision * 100 / leads -> 20.44...   Cast changes the SQL;
that's what fixes it
```

```
# 19
values("desk")              GROUP BY crm_lead.desk_id                  no
JOIN, integers: cheapest
values("desk__name")        JOIN crm_desk ... GROUP BY crm_desk.name   two
desks NAMED alike merge into one row!
values("desk_id", "desk__name")  GROUP BY both -> safe AND readable. Group by
ids; display names.
```

```
# crm/reports.py  (20)
from django.db.models import Count, DecimalField, F, Q, Sum
from django.db.models.functions import Cast, Round, TruncDate

from .models import CallLog, Deposit, Lead

PCT = DecimalField(max_digits=12, decimal_places=4)


def desk_scoreboard(leads=None):
    """[{desk_id, desk_name, leads, ftd, conversion_pct}] highest conversion
first."""
    leads = Lead.objects.all() if leads is None else leads
    return list(leads.values("desk_id", desk_name=F("desk__name"))
                .annotate(leads=Count("id"), ftd=Count("id",
filter=Q(status="ftd")))
                .annotate(conversion_pct=Round(Cast("ftd", output_field=PCT)
* 100 / F("leads"), 1))
                .order_by("-conversion_pct", "desk_name"))


def agent_daily(start, end):
    """[{day, agent_name, calls, leads, talk_sec}] for start <= started_at <
end."""
    return list(CallLog.objects.filter(started_at__gte=start,
started_at__lt=end)
                .annotate(day=TruncDate("started_at"))
                .values("day", agent_name=F("agent__username"))
                .annotate(calls=Count("id"), leads=Count("lead",
distinct=True), talk_sec=Sum("duration_sec"))
                .order_by("day", "agent_name"))


def deposit_pivot():
```

```
    """[{method, pending, approved, rejected}] in USD."""
    cols = {s: Sum("amount_usd", filter=Q(status=s), default=0) for s in
Deposit.Status.values}
    return
list(Deposit.objects.values("method").annotate(**cols).order_by("method"))
```

20: `values(desk_name=F("desk__name"))` renames a column in the output, so callers see `desk_name`, not `desk__name` — the keys are the report's contract with whoever consumes it (a template, a CSV, a BI tool). The alias must not clash with a model field: `values(desk=F("desk__name"))` raises `ValueError: The annotation 'desk' conflicts with a field on the model.` (the same goes for `agent` on `CallLog`). Returning `list(...)` is deliberate: a report is a finished result, unlike Day 5's query functions.

C. Nexora CRM

21–23.

```
# crm/reports.py (continued)
from collections import Counter
from datetime import timedelta
from decimal import Decimal

from django.db.models import Avg, DurationField, ExpressionWrapper, Min
from django.utils import timezone

from .queries import day_start


def ftd_per_day(days=30):                                                #
21
    """[{day, ftd}] for the last `days` Dubai days, zero days included. FTD =
first approved deposit."""
    today = timezone.localdate()
    start = day_start(today - timedelta(days=days - 1))
    firsts = (Lead.objects
              .annotate(ftd_at=Min("accounts__deposits__created_at",

filter=Q(accounts__deposits__status="approved")))
              .filter(ftd_at__gte=start)
              .values_list("ftd_at", flat=True))           # one timestamp
per FTD lead
```

```
    counts = Counter(timezone.localdate(ts) for ts in firsts)
    return [{"day": d, "ftd": counts.get(d, 0)}
            for d in (today - timedelta(days=i) for i in range(days - 1, -1,
-1))]
```

21: two honest limits of plain aggregation show up here. First, you can't GROUP BY a value that is itself an aggregate (each lead's first deposit) in the same query — so the database finds each lead's first deposit and Python buckets the few hundred timestamps. Day 8 does it all in SQL. Second, GROUP BY never invents days with no rows: zero-days must be filled in (here in Python; Day 12 shows `generate_series`).

```
def outcomes_this_week():                                                    #
22
    today = timezone.localdate()
    monday = day_start(today - timedelta(days=today.weekday()))
    cols = {o: Count("id", filter=Q(outcome=o))
            for o in ["no_answer", "callback", "interested",
"not_interested"]}
    return list(CallLog.objects.filter(started_at__gte=monday)
                .values(agent_name=F("agent__username"))
                .annotate(total=Count("id"), **cols).order_by("agent_name"))


def avg_wait_to_first_call():                                                #
23
    """timedelta: average of (first call - lead created), leads that were
called."""
    return (Lead.objects.annotate(first_call=Min("calls__started_at"))
            .filter(first_call__isnull=False)
            .aggregate(avg=Avg(ExpressionWrapper(F("first_call") -
F("created_at"),

output_field=DurationField())))["avg"])
# an aggregate OVER an aggregate annotation: Django wraps the GROUP BY in a
subquery
# (task 10). Returns e.g. timedelta(days=7, seconds=...). A MEDIAN needs
percentile_cont (Day 12).
```

24–27.

```
def desk_deposits_this_month():                                          #
24
    month_start = day_start(timezone.localdate().replace(day=1))
    rows = list(Deposit.objects.filter(status="approved",
created_at__gte=month_start)
                .values(desk_name=F("account__lead__desk__name"))
                .annotate(usd=Sum("amount_usd")).order_by("-usd"))
    total = sum((r["usd"] for r in rows), Decimal("0"))
    for r in rows:
        r["share_pct"] = (r["usd"] * 100 / total).quantize(Decimal("0.1")) if
total else Decimal("0")
    return rows
# the share needs the grand total next to each group's total -- Day 8's
Window(Sum(...)) does
# that in SQL; with 3 desks, Python is perfectly fine.


def brokers_with_ftds(minimum=10):                                       #
25
    return (Broker.objects.annotate(ftd_leads=Count("leads",
filter=Q(leads__status="ftd")))
            .filter(ftd_leads__gt=minimum).order_by("-ftd_leads"))       #
filter on aggregate = HAVING


def breaches_by_severity():                                              #
26
    S = SOPRule.Severity
    return list(SOPBreach.objects.values(agent_name=F("agent__username"))
                .annotate(total=Count("id"),
                          high=Count("id", filter=Q(rule__severity=S.HIGH)),
                          medium=Count("id",
filter=Q(rule__severity=S.MEDIUM)),
                          low=Count("id", filter=Q(rule__severity=S.LOW)))
                .order_by("-high", "-total"))


def symbol_performance():                                                #
27
    return
list(Trade.objects.filter(closed_at__isnull=False).values("symbol")
                .annotate(trades=Count("id"), volume=Sum("volume"),
pnl=Sum("pnl_usd"),
```

```
                           avg_pnl=Avg("pnl_usd"), wins=Count("id",
filter=Q(pnl_usd__gt=0)))
                .annotate(win_rate=Round(Cast("wins", output_field=PCT) * 100
/ F("trades"), 1))
                .order_by("-pnl"))
```

28–29.

```
from django.db.models.functions import Coalesce


def by_root_broker():                                                    #
28
    """Leads and FTDs per top-level broker (hierarchy is at most 3 levels
deep)."""
    return list(Lead.objects.filter(broker__isnull=False)
                .annotate(root=Coalesce("broker__parent__parent__code",
"broker__parent__code", "broker__code"))
                .values("root")
                .annotate(leads=Count("id"), ftd=Count("id",
filter=Q(status="ftd")))
                .order_by("root"))
# Coalesce takes the first non-NULL: the grandparent if there is one, else
the parent, else
# the broker itself. Fine for a FIXED depth; any depth needs a recursive CTE
(Day 14).


def best_desk_by_avg_deposit():                                          #
29
    return (Deposit.objects.filter(status="approved")
            .values(desk_name=F("account__lead__desk__name"))
            .annotate(avg_usd=Avg("amount_usd"), deposits=Count("id"))
            .order_by("-avg_usd").first())           # a dict, e.g.
{'desk_name': 'VIP', 'avg_usd': ...}
```

30 — prove reports against SQL.

```
# crm/tests.py
from django.db import connection

from .reports import deposit_pivot, desk_scoreboard
```

```
class ReportSqlTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=400, stdout=StringIO())

    def sql(self, query):
        with connection.cursor() as cur:
            cur.execute(query)
            return cur.fetchall()

    def test_desk_scoreboard_matches_sql(self):
        expected = dict(self.sql(
            "SELECT desk_id, COUNT(*) FILTER (WHERE status = 'ftd') FROM
crm_lead GROUP BY desk_id"))
        self.assertEqual({r["desk_id"]: r["ftd"] for r in desk_scoreboard()},
expected)

    def test_deposit_pivot_matches_sql(self):
        expected = dict(self.sql(
            """SELECT method, COALESCE(SUM(amount_usd) FILTER (WHERE status =
'approved'), 0)
               FROM nexora_deposits GROUP BY method"""))
        self.assertEqual({r["method"]: r["approved"] for r in
deposit_pivot()}, expected)
```

```
python manage.py test crm && git add -A && git commit -m "orm day 6"
```

30: the SQL in these tests is written by hand, from the question, not copied from `show()` — that's what makes it a second opinion. Day 15's capstone does this for 20 reports.

D. Challenge

31 + 40 — the JOIN multiplication bug.

```
wrong = Lead.objects.annotate(
    n_calls=Count("calls"),
    deposited=Sum("accounts__deposits__amount_usd"),
).get(pk=lead_with_4_calls_and_2_deposits)
wrong.n_calls, wrong.deposited          # (8, Decimal('4000.00'))   should be
(4, Decimal('1000.00'))
```

```
Why: both relations are LEFT JOINed in the SAME query, so each lead's rows
are
calls x deposits = 4 x 2 = 8. COUNT sees every call twice; SUM sees every
deposit 4 times.

40: Count("calls", distinct=True) -> 4. Correct: it counts distinct call ids.
    Sum("accounts__deposits__amount_usd", distinct=True) -> sums distinct
VALUES: two different
    500 USD deposits count once -> 500 instead of 1000. Still wrong -- never
use it for money.
```

```
# the fix: aggregate each relation separately, in a correlated subquery (Day
8 explains it fully)
from django.db.models import OuterRef, Subquery

deposits = (Deposit.objects.filter(account__lead=OuterRef("pk"))
            .order_by().values("account__lead")          # group by lead
inside the subquery
            .annotate(total=Sum("amount_usd")).values("total"))
Lead.objects.annotate(
    n_calls=Count("calls"),                                # the only JOIN
left in the outer query
    deposited=Coalesce(Subquery(deposits), Decimal("0"),
output_field=DecimalField()),
)
```

32–36.

```
# 32
Lead.objects.values("desk").annotate(n=Count("id")).order_by("name")
ORDER BY a column that isn't grouped -> Django adds it to the GROUP BY ->
GROUP BY desk_id, name:
one row per (desk, lead name). Order only by grouped columns or aggregates:
.order_by("-n").

# 33  a lead with ZERO calls, LEFT JOINed: one row, every crm_calllog column
NULL
Count("*")      COUNT(*)                -> 1   counts the row itself
Count("id")     COUNT(crm_lead.id)      -> 1   the lead's id isn't NULL
Count("calls")  COUNT(crm_calllog.id)   -> 0   correct: COUNT(col) skips
NULLs
```

```
# 34
Avg("duration_sec")  integer column -> Python float (450.08...)
Avg("amount_usd")    decimal column -> Decimal with many digits
(Decimal('1043.1764705882352941'))
Round or quantize for display; never compare raw averages with ==.

# 35
annotate(deposited=Sum(...)) is None for leads with no deposits.
Consequences:
- order_by("-deposited"): PostgreSQL sorts NULLs FIRST in DESC -> the "top
depositors" list
  starts with people who never deposited
- total += row.deposited -> TypeError: unsupported operand type(s) for +=:
'decimal.Decimal' and 'NoneType'
- a template prints "None"
Fix: Sum(..., default=0) (or Coalesce), or
F("deposited").desc(nulls_last=True) if NULL must stay.
```

```
# 36
a =
Lead.objects.filter(calls__outcome="interested").annotate(n=Count("calls"))
# filter FIRST: the annotation counts only the calls that passed the filter
(same JOIN)
b =
Lead.objects.annotate(n=Count("calls")).filter(calls__outcome="interested")
# annotate FIRST: the filter adds a SECOND join -> n = all calls x interested
calls (inflated)
# say what you mean with filter= instead:
Lead.objects.annotate(n=Count("calls"),
                      interested=Count("calls",
filter=Q(calls__outcome="interested"))
                      ).filter(interested__gt=0)
```

37–39.

```
# 37
Deposit.objects.order_by("-created_at")
[:100].aggregate(total=Sum("amount_usd"))
# SELECT SUM(amount_usd) FROM (SELECT ... ORDER BY created_at DESC LIMIT 100)
subquery
# correct: the last 100 deposits' total. Django wraps the slice so LIMIT
applies BEFORE SUM.

# 38
CallLog.objects.values("started_at").annotate(n=Count("id"))
# every timestamp is unique -> ~18,000 groups of 1. Group by a truncation
instead:
CallLog.objects.annotate(hour=TruncHour("started_at")).values("hour").annotat
e(n=Count("id"))

# 39
from zoneinfo import ZoneInfo

dubai =
CallLog.objects.annotate(d=TruncDate("started_at")).values("d").annotate(n=Co
unt("id"))
utc = (CallLog.objects.annotate(d=TruncDate("started_at",
tzinfo=ZoneInfo("UTC")))
       .values("d").annotate(n=Count("id")))
# every call made between 00:00 and 04:00 Dubai time (20:00-24:00 UTC the day
before) changes
# day. Pick the business time zone (Dubai) once, write it into the report's
docstring, and use
# the same one in SQL (AT TIME ZONE 'Asia/Dubai') when you cross-check.
```

Day 7 — Expressions

Goal: compute inside PostgreSQL instead of in Python loops — derived columns, labels, scores, buckets, commissions and bulk corrections, each as one SQL expression. By tonight you can turn any `CASE`, `COALESCE`, date or string function you'd write in SQL into its ORM form.

```
F("volume") * F("open_price")          volume * open_price          a column,
not a Python value
Value("direct")                         'direct'                     a
constant column
Case(When(amount_usd__gte=5000, then=Value("large")), default=Value("small"))
                                        CASE WHEN ... THEN ... ELSE ... END
Coalesce("last_contacted_at", "created_at")   COALESCE(a, b)
Cast("account_no", CharField())         account_no::varchar
Lead.objects.update(x=F("x") + 1)       UPDATE ... SET x = x + 1     no rows
travel to Python
```

A. Drills (1–10) — the expression toolkit

- [ ] **1.** Compare two columns of the same row with `F()`: EUR deposits where `amount_usd` is above `amount`; trades closed before they opened.
- [ ] **2.** Arithmetic in `annotate()`: a trade's notional (`volume × open_price`). When do you need `output_field`?
- [ ] **3.** `Value()`: add a constant column; mix it with a field in `Concat`.
- [ ] **4.** `Case` / `When`: label every deposit small, medium or large.
- [ ] **5.** `Coalesce`: a display name from `first_name` or `name`; a "last touch" time from `last_contacted_at` or `created_at`.
- [ ] **6.** `Cast`: an account number as text; a decimal as an integer.
- [ ] **7.** String functions: `Concat`, `Upper`, `Lower`, `Length`, `Left`, `Right`, `Substr`, `Replace`, `StrIndex`, `Trim`.
- [ ] **8.** Date functions: `TruncMonth`, `TruncWeek`, `ExtractHour`, `ExtractIsoWeekDay`, `Now()`, and subtracting two datetimes.
- [ ] **9.** Math functions: `Abs`, `Round`, `Greatest`, `Least`, `Mod`, `Power`.
- [ ] **10.** `update()` with expressions: add 1 to a counter; set a column from two others.

B. Core (11–20) — derived data, in SQL

- [ ] **11.** Each open lead's age in hours, computed in SQL. Then filter on it — and compare with the simpler filter.
- [ ] **12.** A lead priority score from status, source and last contact, as one annotation.
- [ ] **13.** Group deposits by size bucket: count and total per bucket.
- [ ] **14.** Trade holding time in hours, and average holding time per symbol.
- [ ] **15.** Commission per approved deposit: `amount_usd × broker.commission_rate`, following relations inside the expression.
- [ ] **16.** A masked phone (`+97150****07`) built in SQL.
- [ ] **17.** One UPDATE that moves several statuses at once with `Case`.
- [ ] **18.** A custom `Func` (`NULLIF`-style safe division) and a custom lookup (`__ne`).
- [ ] **19.** Order by an expression: a status priority with `Case`, and `F().desc(nulls_last=True)`.
- [ ] **20.** Expressions in constraints: a case-insensitive unique broker name; a CHECK built from `F()`.

C. Nexora CRM (21–30) — operations and finance

- [ ] **21.** The agent call queue: callbacks first, then new leads oldest first, then the rest.
- [ ] **22.** Days since last touch for every open lead (never-contacted leads count from creation).
- [ ] **23.** Deposits per currency: original amount, USD amount and the average rate used.
- [ ] **24.** Per account: realised PnL, win/loss count and average holding hours of closed trades.
- [ ] **25.** This month's commission statement per broker: deposits USD, rate, commission.
- [ ] **26.** SOP rule "first contact within 24 hours": a compliance flag per lead and the rate per desk.
- [ ] **27.** Clean all lead names in one UPDATE: trim spaces and collapse double spaces.
- [ ] **28.** Monthly cohorts: leads per creation month and how many of them became FTD.
- [ ] **29.** Calls per shift (morning, afternoon, evening, night) per desk, using Dubai hours.
- [ ] **30.** Tests for the expression helpers. Commit `orm day 7`.

D. Challenge (31–40) — expressions that surprise

- [ ] **31.** `lead.call_attempts = F("call_attempts") + 1; lead.save()`, then print the field. What do you see?
- [ ] **32.** Save that same instance twice. How much did the counter go up?
- [ ] **33.** `F("amount") * 1.5`. Copy the error and fix it two ways.
- [ ] **34.** A `Case` with no `default`, and two `When`s that both match. What does each row get?
- [ ] **35.** `Coalesce` of a `DateTimeField` and a `DateField`. Copy the error.
- [ ] **36.** A conversion rate where some groups have zero leads. What does PostgreSQL say? Make it safe.
- [ ] **37.** `TruncDay` vs `ExtractDay` vs `__date` around midnight: which time zone does each use, and how do you change it?
- [ ] **38.** `Concat` with a NULL part vs SQL's `||`. Which returns NULL?
- [ ] **39.** `filter(name__upper=...)`-style expressions on 50,000 leads with and without a matching functional index.
- [ ] **40.** Re-status 5,000 leads with a Python loop vs one `Case` UPDATE. Time both and count the queries.

Solutions

Imports used throughout:

```
from datetime import timedelta
from decimal import Decimal

from django.db.models import (Case, CharField, Count, DurationField,
ExpressionWrapper, F, IntegerField,
                              Q, Sum, Value, When)
from django.db.models.functions import (Abs, Cast, Coalesce, Concat, Extract,
ExtractHour, ExtractIsoWeekDay,
                                        Greatest, Left, Length, Least, Lower,
Mod, Now, NullIf, Power,
                                        Replace, Right, Round, StrIndex,
Substr, Trim, TruncMonth, TruncWeek,
                                        Upper)
from django.utils import timezone
```

A. Drills

1–5.

```
# 1 -- F() compares two columns of the SAME row
Deposit.objects.filter(currency="EUR", amount_usd__gt=F("amount"))   # WHERE
amount_usd > amount
Trade.objects.filter(closed_at__lt=F("opened_at"))                    #
empty: Day 4's CHECK forbids it

# 2
Trade.objects.annotate(notional=F("volume") * F("open_price"))       # volume
* open_price
# Decimal x Decimal -> Django infers DecimalField. output_field is needed
when the types mix in
# a way Django can't resolve (Decimal x float, task 33) or when you want a
different result
# type (datetime - datetime as a DurationField).

# 3
Lead.objects.annotate(channel=Value("crm", output_field=CharField()))
Lead.objects.annotate(label=Concat("name", Value(" · "), "phone",
output_field=CharField()))

# 4
SIZE = Case(
    When(amount_usd__gte=5000, then=Value("large")),
    When(amount_usd__gte=1000, then=Value("medium")),
    default=Value("small"),
)
Deposit.objects.annotate(size=SIZE)
# CASE WHEN amount_usd >= 5000 THEN 'large' WHEN amount_usd >= 1000 THEN
'medium' ELSE 'small' END

# 5
Lead.objects.annotate(display=Coalesce(NullIf("first_name", Value("")),
"name"))
Lead.objects.annotate(last_touch=Coalesce("last_contacted_at", "created_at"))
```

5: `first_name` is a text column, so "empty" is `''`, not NULL — and `COALESCE` only skips NULLs. `NullIf("first_name", Value(""))` turns `''` into NULL first. (Day 2's rule — text uses `''` — has a price here.)

6-9.

```
# 6
TradingAccount.objects.annotate(acc_text=Cast("account_no",
output_field=CharField()))
Deposit.objects.annotate(whole_usd=Cast("amount_usd",
output_field=IntegerField()))
# PostgreSQL ROUNDS numeric -> integer (2.5 -> 3); it doesn't truncate like
Python's int()

# 7 -- all evaluated in SQL; Substr and StrIndex are 1-based
Lead.objects.annotate(
    up=Upper("name"), low=Lower("name"), n=Length("name"),
    prefix=Left("phone", 6), last2=Right("phone", 2), mid=Substr("phone", 5,
3),
    digits=Replace("phone", Value("+"), Value("")),
first_space=StrIndex("name", Value(" ")),
    clean=Trim("name"),
)

# 8 -- truncations and extractions use the CURRENT time zone (Asia/Dubai)
CallLog.objects.annotate(month=TruncMonth("started_at"),
week=TruncWeek("started_at"),
                         hour=ExtractHour("started_at"),
dow=ExtractIsoWeekDay("started_at"))
Trade.objects.filter(closed_at__isnull=False).annotate(held=F("closed_at") -
F("opened_at"))
# datetime - datetime: Django infers a DurationField -> a Python timedelta
per row
Lead.objects.annotate(age=Now() - F("created_at"))       # Now() is the
DATABASE's clock

# 9
Trade.objects.annotate(
    abs_pnl=Abs("pnl_usd"), pnl_rounded=Round("pnl_usd"),
    capped=Least("pnl_usd", Value(Decimal("500"))),
wins_only=Greatest("pnl_usd", Value(Decimal("0"))),
    volume_sq=Power("volume", 2),
)
```

```
Lead.objects.annotate(test_group=Mod("id", 10))     # split leads into 10
stable A/B groups
# PostgreSQL's GREATEST/LEAST ignore NULLs (open trades give 500 / 0, not
NULL)
```

10.

```
# models.py -- Lead: the counter tasks 10, 31, 32 use
call_attempts = models.PositiveIntegerField(default=0)       # makemigrations
+ migrate

Lead.objects.filter(pk=7).update(call_attempts=F("call_attempts") + 1)
# UPDATE crm_lead SET call_attempts = call_attempts + 1 WHERE id = 7
# atomic in the database: two agents clicking at once give +2, never +1

CallLog.objects.filter(desk__isnull=True).update(desk=F("lead__desk"))
# FieldError: Joined field references are not permitted in this query
# update() can FILTER across relations, but its SET values can't JOIN.
# Day 8 does this with
Subquery(Lead.objects.filter(pk=OuterRef("lead")).values("desk")[:1]).
```

B. Core

11–14.

```
from crm.queries import open_leads

# 11
AGE = ExpressionWrapper(Now() - F("created_at"),
output_field=DurationField())
qs = open_leads().annotate(age_hours=Extract(AGE, "epoch") / 3600)       #
seconds -> hours
qs.filter(age_hours__gt=72)                                       # computes AGE
for EVERY open lead
open_leads().filter(created_at__lt=timezone.now() - timedelta(hours=72))  #
same rows; uses an index
# compute in SQL for DISPLAY; filter on the raw column whenever you can (Day
5, task 33).

# 12
stale = timezone.now() - timedelta(days=3)
SCORE = (
    Case(When(status="callback", then=Value(50)), When(status="new",
```

```
then=Value(30)), default=Value(0))
    + Case(When(source="broker", then=Value(20)), When(source="referral",
then=Value(15)), default=Value(0))
    + Case(When(last_contacted_at__isnull=True, then=Value(25)),
           When(last_contacted_at__lt=stale, then=Value(15)),
default=Value(0))
)
open_leads().annotate(score=SCORE).order_by("-score", "created_at")

# 13 -- GROUP BY an expression
(Deposit.objects.filter(status="approved").annotate(size=SIZE)
 .values("size").annotate(n=Count("id"),
usd=Sum("amount_usd")).order_by("size"))
# ... GROUP BY CASE WHEN ... END

# 14
HELD = ExpressionWrapper(F("closed_at") - F("opened_at"),
output_field=DurationField())
closed = Trade.objects.filter(closed_at__isnull=False)
closed.annotate(held_hours=Extract(HELD, "epoch") / 3600)
closed.values("symbol").annotate(avg_hold=Avg(HELD)).order_by("symbol")   #
Avg of an interval -> timedelta
```

11: inside `Extract`, a subtraction needs `ExpressionWrapper(..., output_field=DurationField())` — `Extract` must know it's extracting from an interval. Import `Avg` from `django.db.models` for task 14.

15–17.

```
# 15 -- F() can follow relations inside annotate() (not inside update())
(Deposit.objects.filter(status="approved",
account__lead__broker__isnull=False)
 .annotate(rate=F("account__lead__broker__commission_rate"),
           commission=Round(F("amount_usd") *
F("account__lead__broker__commission_rate"), 2)))

# 16
MASKED = Concat(Left("phone", 6), Value("****"), Right("phone", 2),
output_field=CharField())
Lead.objects.annotate(masked=MASKED).values_list("masked", flat=True)[:3]
# '+97150****07'

# 17 -- one UPDATE, several transitions; local columns only (no JOINs in SET,
task 10)
now = timezone.now()
(Lead.objects.filter(status__in=["new", "callback"])
 .update(status=Case(
            When(last_contacted_at__isnull=True, created_at__lt=now -
timedelta(days=30), then=Value("lost")),
            When(status="callback", last_contacted_at__lt=now -
timedelta(days=14), then=Value("new")),
            default=F("status")),                   # everything else keeps
its status
         updated_at=now))
# one statement, one transaction, every row decided by the same rules at the
same instant
```

18 — your own Func and lookup.

```
# crm/expressions.py
from django.db.models import CharField, DurationField, Field, Func, Lookup


class InitCap(Func):
    """PostgreSQL INITCAP('omar KHALID') -> 'Omar Khalid'."""
    function = "INITCAP"
    output_field = CharField()


class Age(Func):
    """PostgreSQL AGE(ts): interval from ts to today."""
    function = "AGE"
```

```
    output_field = DurationField()


@Field.register_lookup
class NotEqual(Lookup):
    """field__ne=value -> field <> value (NULL rows do NOT match, unlike
exclude())."""
    lookup_name = "ne"

    def as_sql(self, compiler, connection):
        lhs, lhs_params = self.process_lhs(compiler, connection)
        rhs, rhs_params = self.process_rhs(compiler, connection)
        return f"{lhs} <> {rhs}", lhs_params + rhs_params


# crm/apps.py -- make sure the module is imported once at startup
class CrmConfig(AppConfig):
    name = "crm"

    def ready(self):
        from . import expressions  # noqa: F401  (registers the lookup)
```

```
Lead.objects.annotate(nice=InitCap("name"))
Lead.objects.filter(source__ne="direct")      # WHERE source <> 'direct'
# safe division needs no custom class -- the built-in NullIf does it (task
36):
F("ftd") * 100 / NullIf(F("leads"), 0)        # x / NULLIF(y, 0) -> NULL
instead of an error
```

19–20.

```
# 19 -- order_by accepts expressions
PRIORITY = Case(When(status="callback", then=Value(0)), When(status="new",
then=Value(1)), default=Value(2))
Lead.objects.order_by(PRIORITY, F("last_contacted_at").asc(nulls_first=True),
"created_at")

# 20 -- expressions inside constraints (models.py, Broker.Meta.constraints)
models.UniqueConstraint(Lower("name"), name="broker_name_ci_unique",
                        violation_error_message="A broker with this name
already exists.")
# CREATE UNIQUE INDEX broker_name_ci_unique ON crm_broker ((LOWER(name)))
# 'Desert IB' and 'desert ib' can no longer both exist. Day 4's
# CHECK (closed_at IS NULL OR closed_at >= opened_at) is the F() version of
the same idea.
```

C. Nexora CRM

21–24.

```
# crm/queries.py (continued)
from django.db.models import Avg, BooleanField, DecimalField, Exists,
OuterRef
from django.db.models.functions import ExtractDay

QUEUE_PRIORITY = Case(When(status="callback", then=Value(0)),
When(status="new", then=Value(1)),
                      default=Value(2))


def call_queue(agent):                                                  #
21
    """An agent's open leads: callbacks first, then new leads oldest
first."""
    return (open_leads().filter(assigned_to=agent)
            .annotate(priority=QUEUE_PRIORITY).order_by("priority",
"created_at"))


LAST_TOUCH = Coalesce("last_contacted_at", "created_at")


def idle_days():                                                        #
```

```
22
    """Open leads with whole days since the last call (or since creation)."""
    idle = ExpressionWrapper(Now() - LAST_TOUCH,
output_field=DurationField())
    return open_leads().annotate(days_idle=ExtractDay(idle)).order_by("-
days_idle")
```

```
# crm/reports.py (continued)
def deposits_by_currency():                                              #
23
    return list(Deposit.objects.filter(status="approved").values("currency")
                .annotate(n=Count("id"), amount=Sum("amount"),
usd=Sum("amount_usd"))
                .annotate(avg_rate=Round(F("usd") / F("amount"), 6))
                .order_by("currency"))
# the rate that matches the totals is SUM(usd) / SUM(amount) -- a weighted
average.
# Avg("fx_rate") gives a 100-dollar deposit the same weight as a 5,000-dollar
one.


HELD = ExpressionWrapper(F("closed_at") - F("opened_at"),
output_field=DurationField())


def account_performance():                                               #
24
    return list(Trade.objects.filter(closed_at__isnull=False)
                .values("account__account_no")
                .annotate(pnl=Sum("pnl_usd"),
                          wins=Count("id", filter=Q(pnl_usd__gt=0)),
                          losses=Count("id", filter=Q(pnl_usd__lte=0)),
                          avg_hold_hours=Avg(Extract(HELD, "epoch")) / 3600)
                .order_by("-pnl"))
```

25–26.

```
RATE = F("account__lead__broker__commission_rate")


def commission_statement(start, end):                                    #
25
    """Per broker: approved deposit USD and commission, start <= created_at <
```

```
end."""
    return list(Deposit.objects.filter(status="approved",
created_at__gte=start, created_at__lt=end,
                                       account__lead__broker__isnull=False)
                .values(broker_code=F("account__lead__broker__code"),
rate=RATE)
                .annotate(deposits_usd=Sum("amount_usd"),
                          commission_usd=Round(Sum(F("amount_usd") * RATE),
2))
                .order_by("-commission_usd"))
# Sum(amount_usd x rate) per row, rounded once at the end -- not a Round per
deposit then summed


def first_contact_compliance():                                          #
26
    """Per desk: leads older than 24h, how many were called within 24h of
creation, and the %."""
    called_in_time = Exists(CallLog.objects.filter(
        lead=OuterRef("pk"), started_at__lte=OuterRef("created_at") +
timedelta(hours=24)))
    return list(Lead.objects.filter(created_at__lt=timezone.now() -
timedelta(hours=24))
                .annotate(ok=called_in_time)
                .values(desk_name=F("desk__name"))
                .annotate(leads=Count("id"), in_time=Count("id",
filter=Q(ok=True)))
                .annotate(pct=Round(Cast("in_time",
output_field=DecimalField(12, 4)) * 100 / F("leads"), 1))
                .order_by("pct"))
```

26: a per-lead flag built from `Min("calls__started_at")` would be an aggregate, and an aggregate can't be grouped again by desk in the same query (Day 6, task 21). `Exists` is a plain per-row boolean, so it groups fine — Day 8 covers it properly. Leads younger than 24 hours are left out: they haven't had their chance yet.

27–29.

```
# crm/expressions.py -- add
class RegexpReplace(Func):
    function = "REGEXP_REPLACE"
    output_field = CharField()
```

```
# 27 -- only rows that need it; one UPDATE
from crm.expressions import RegexpReplace

(Lead.objects.filter(name__regex=r"^\s|\s$|\s\s")
 .update(name=Trim(RegexpReplace("name", Value(r"\s+"), Value(" "),
Value("g")))))
# UPDATE crm_lead SET name = TRIM(REGEXP_REPLACE(name, '\s+', ' ', 'g'))
WHERE name ~ '^\s|\s$|\s\s'
# the WHERE keeps clean rows untouched: no needless row versions, smaller
locks


# 28
PCT = DecimalField(max_digits=12, decimal_places=4)
(Lead.objects.annotate(cohort=TruncMonth("created_at")).values("cohort")
 .annotate(leads=Count("id"), ftd=Count("id", filter=Q(status="ftd")))
 .annotate(ftd_pct=Round(Cast("ftd", output_field=PCT) * 100 / F("leads"),
1))
 .order_by("cohort"))


# 29 -- annotate the hour once, build the label from it, then group
SHIFT = Case(When(hour__lt=8, then=Value("night")), When(hour__lt=12,
then=Value("morning")),
             When(hour__lt=17, then=Value("afternoon")), When(hour__lt=22,
then=Value("evening")),
             default=Value("night"))
(CallLog.objects.annotate(hour=ExtractHour("started_at")).annotate(shift=SHIF
T)
 .values("desk__name",
"shift").annotate(calls=Count("id")).order_by("desk__name", "shift"))
```

30.

```
# crm/tests.py
from .expressions import InitCap


class ExpressionTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=300, stdout=StringIO())
```

```
    def test_masked_phone(self):
        lead = Lead.objects.order_by("pk").first()
        masked =
Lead.objects.filter(pk=lead.pk).annotate(m=MASKED).values_list("m",
flat=True).get()
        self.assertEqual(masked, lead.phone[:6] + "****" + lead.phone[-2:])

    def test_initcap(self):
        lead = Lead.objects.first()
        Lead.objects.filter(pk=lead.pk).update(name="omar KHALID")

self.assertEqual(Lead.objects.annotate(n=InitCap("name")).get(pk=lead.pk).n,
"Omar Khalid")

    def test_ne_lookup_skips_nulls_unlike_exclude(self):
        price =
Trade.objects.filter(close_price__isnull=False).values_list("close_price",
flat=True).first()
        ne = Trade.objects.filter(close_price__ne=price).count()
        ex = Trade.objects.exclude(close_price=price).count()
        self.assertEqual(ex - ne,
Trade.objects.filter(close_price__isnull=True).count())
```

```
python manage.py test crm && git add -A && git commit -m "orm day 7"
```

D. Challenge

31–36.

```
# 31
lead.call_attempts = F("call_attempts") + 1
lead.save(update_fields=["call_attempts"])
lead.call_attempts                # <CombinedExpression: F(call_attempts) +
Value(1)>  -- not a number
lead.refresh_from_db(fields=["call_attempts"])
lead.call_attempts                # 4

# 32 -- without that refresh, a second save() runs the expression AGAIN: +2
in total.
# The F() stays on the instance until you reload it. Prefer
#   Lead.objects.filter(pk=lead.pk).update(call_attempts=F("call_attempts") +
1)
# which never leaves an expression lying on an object.
```

```
# 33
Deposit.objects.annotate(x=F("amount") * 1.5)
FieldError: Cannot infer type of '*' expression involving these types:
DecimalField, FloatField.
You must set output_field.
fix 1: F("amount") * Value(Decimal("1.5"))                        -- stay in
Decimal (right for money)
fix 2: ExpressionWrapper(F("amount") * 1.5,
output_field=DecimalField(max_digits=16, decimal_places=4))

# 34
Case(When(amount_usd__gte=5000, then=Value("large")))     no default -> every
other row gets NULL
Case(When(amount_usd__gte=1000, then=Value("medium")),
     When(amount_usd__gte=5000, then=Value("large")), default=...)
-> the FIRST matching When wins: nothing is ever "large". Order Whens from
most specific.

# 35
Lead.objects.annotate(x=Coalesce("last_contacted_at",
"kyc_profile__passport_expiry"))
FieldError: Expression contains mixed types: DateTimeField, DateField. You
must set output_field.
fix: Cast the date first -- Coalesce("last_contacted_at",
Cast("kyc_profile__passport_expiry", DateTimeField()))

# 36
```

```
.annotate(pct=F("ftd") * 100 / F("leads")) on a group with leads = 0
django.db.utils.DataError: division by zero
fix: F("ftd") * 100 / NullIf(F("leads"), 0)   -> NULL for that group
     Coalesce(..., Value(0))                     -> or 0, if the report wants
a number
```

37–40.

```
# 37 -- TruncDay, ExtractDay and __date all use the CURRENT time zone
(Asia/Dubai)
from zoneinfo import ZoneInfo

CallLog.objects.annotate(d=TruncDay("started_at", tzinfo=ZoneInfo("UTC")))
# per expression
with timezone.override(ZoneInfo("UTC")):
# per block of code
    CallLog.objects.filter(started_at__date=date(2026, 10, 20)).count()
# a call at 21:30 UTC lands on different days in the two time zones (Day 6,
task 39)

# 38
Lead.objects.annotate(x=Concat("name", Value(" / "), "broker__code",
output_field=CharField()))
# Django on PostgreSQL wraps each part in COALESCE(part, ''): no broker ->
'Lead 00042 / '
# hand-written SQL: name || ' / ' || code -> NULL for the whole value when
code is NULL

# 39 -- 50,000 leads; Day 2's functional index lead_name_upper_idx =
UPPER(name)
explain(Lead.objects.annotate(u=Upper("name")).filter(u="LEAD 00042"))
#   Index Scan using lead_name_upper_idx   (the query expression matches the
index exactly)
# drop the index (or write the expression differently) and the same query is
a Seq Scan.
# Always confirm with explain() that the query's expression is the index's
expression.

# 40
import time

ids = list(Lead.objects.filter(status="callback").values_list("pk",
flat=True)[:5000])
```

```
t = time.perf_counter()
for lead in Lead.objects.filter(pk__in=ids):
    lead.status = "new"
    lead.save(update_fields=["status", "updated_at"])     # 1 SELECT + up to
5,000 UPDATEs
print(f"loop: {time.perf_counter() - t:.2f}s")          # seconds; 5,000
separate commits

t = time.perf_counter()
Lead.objects.filter(pk__in=ids).update(status=Case(When(status="new",
then=Value("callback")),
                                                   default=Value("new")),
updated_at=Now())
print(f"one UPDATE: {time.perf_counter() - t:.2f}s")    # milliseconds; all-
or-nothing
```

40: the loop is also not atomic — a failure at lead 3,000 leaves the first 2,999 changed. Use loops only when each row needs Python logic or `save()` side effects; otherwise one expression UPDATE.

Day 8 — Subqueries and window functions

Goal: answer the questions plain GROUP BY can't — "the latest call per lead", "each client's first deposit", "running totals", "top 3 per desk", "time since the previous call" — with correlated subqueries and window functions, in one query each. This is the day the ORM catches up with your analytics SQL.

```
Subquery(qs.filter(lead=OuterRef("pk")).values("x")[:1])   (SELECT x FROM ...
WHERE lead_id = outer.id LIMIT 1)
Exists(qs.filter(lead=OuterRef("pk")))                      EXISTS (SELECT 1
FROM ... WHERE lead_id = outer.id)
Window(RowNumber(), partition_by=["lead"], order_by="-started_at")
                                                            ROW_NUMBER() OVER
(PARTITION BY lead_id ORDER BY started_at DESC)
Window(Sum("amount_usd"), partition_by=["account"], order_by="created_at")
a running total
Lag("started_at")                                           the previous
row's value in the window
```

A. Drills (1–10) — subqueries and windows

- [ ] **1.** `Subquery` + `OuterRef`: annotate each lead with the outcome of its latest call.
- [ ] **2.** `Exists` and `~Exists`: leads with an approved deposit; leads with none.
- [ ] **3.** An aggregate inside a subquery: each lead's approved deposit USD (Day 6's fix, now explained line by line).
- [ ] **4.** A subquery in `filter()`: deposits larger than their account's average deposit.
- [ ] **5.** `OuterRef(OuterRef(...))`: two levels of nesting.
- [ ] **6.** `Window(RowNumber())`: number each lead's calls, newest first.
- [ ] **7.** A running total of approved deposits per account.
- [ ] **8.** `Lag`: the time since the previous call to the same lead.
- [ ] **9.** `Rank`, `DenseRank` and `NTile(4)`: rank brokers by revenue; split depositors into quartiles.
- [ ] **10.** Filter on a window annotation (Django 4.2+): keep only each lead's latest call.

B. Core (11–20) — the classic analytics patterns

- [ ] **11.** The latest call per lead, three ways: `Subquery`, `DISTINCT ON`, `Window` + filter. Compare plans — and beat Day 3's 50 queries.
- [ ] **12.** Each lead's FTD: date and amount of the first approved deposit.
- [ ] **13.** FTDs per day for 30 days, fully in SQL (Day 6, task 21, redone).
- [ ] **14.** Each deposit's share of its desk's monthly total, with `Window(Sum(...))`.
- [ ] **15.** Agent ranking within each desk by funded leads this month.
- [ ] **16.** Second deposit within 30 days of the FTD: which leads, and what share of FTDs.
- [ ] **17.** Contact gaps: calls that came more than 3 days after the previous call to the same lead.
- [ ] **18.** A correlated subquery vs JOIN + GROUP BY for the same answer: compare plans on 50,000 leads.
- [ ] **19.** Several values from one subquery: `JSONObject` (outcome, agent, time of the latest call).
- [ ] **20.** Move the reusable pieces into `crm/queries.py` and `crm/reports.py`.

C. Nexora CRM (21–30) — analyst questions

- [ ] **21.** An agent worklist: each open lead with its last call's outcome, agent and time.
- [ ] **22.** Brokers whose FTD rate beats the average of all brokers.
- [ ] **23.** The top 3 approved deposits per desk this month.
- [ ] **24.** Average days from FTD to second deposit, per desk.
- [ ] **25.** A rolling average PnL over each account's last 5 closed trades.
- [ ] **26.** The funnel per lead: created → first call → FTD, with both durations.
- [ ] **27.** FTD leads with no call in the 24 hours before their first deposit.
- [ ] **28.** Possible duplicates: leads sharing the last 7 digits of the phone, with group sizes.
- [ ] **29.** Backfill `Lead.last_contacted_at` and `call_attempts` for every lead in one UPDATE each.
- [ ] **30.** Tests that compare three answers against hand-written SQL. Commit `orm day 8`.

D. Challenge (31–40) — subquery and window traps

- [ ] **31.** A subquery without `[:1]`. Copy the PostgreSQL error.

- [ ] **32.** An aggregate subquery without `values("account__lead")` before `annotate()`. What goes wrong?
- [ ] **33.** Evaluate a queryset that contains `OuterRef` on its own. Copy the error.
- [ ] **34.** `annotate(rn=Window(...)).filter(status="new")` vs filtering first. Do the ranks change?
- [ ] **35.** `Window(Sum(...))` with and without `order_by`. Why does one give totals and the other running totals?
- [ ] **36.** `LastValue` returns the current row. Why, and how do you fix the frame?
- [ ] **37.** A correlated subquery on 200,000 calls that takes seconds. Rewrite it.
- [ ] **38.** `Exists` vs `__in` vs JOIN + `distinct()` for "leads with an approved deposit" on big data: compare plans.
- [ ] **39.** A subquery that finds no row returns NULL. Where does that break a sum or a sort?
- [ ] **40.** Ties: two calls at the same second. Which one is "latest"? Make the answer deterministic.

Solutions

Imports used throughout:

```
from datetime import timedelta
from decimal import Decimal

from django.contrib.auth import get_user_model
from django.db.models import (Avg, Count, DecimalField, Exists, F, OuterRef,
Q, Subquery, Sum, Value,
                              Window)
from django.db.models.functions import (Coalesce, DenseRank, FirstValue,
JSONObject, Lag, LastValue, NTile,
                                        Rank, Round, RowNumber, TruncDate)
from django.utils import timezone

USD = DecimalField(max_digits=16, decimal_places=2)
```

Add this index before you start (makemigrations + migrate): every "latest call of a lead" lookup below uses it.

```
# CallLog.Meta.indexes
models.Index(fields=["lead", "-started_at", "-id"],
name="call_lead_latest_idx")
```

A. Drills

1–5 — subqueries.

```
# 1
LATEST_CALL = CallLog.objects.filter(lead=OuterRef("pk")).order_by("-
started_at", "-id")
Lead.objects.annotate(last_outcome=Subquery(LATEST_CALL.values("outcome")
[:1]))
# (SELECT U0.outcome FROM crm_calllog U0 WHERE U0.lead_id = (crm_lead.id)
#  ORDER BY U0.started_at DESC, U0.id DESC LIMIT 1) AS last_outcome
# OuterRef("pk") = "the id of the lead in the OUTER query" -- correlated,
evaluated per lead.

# 2
APPROVED = Deposit.objects.filter(account__lead=OuterRef("pk"),
status="approved")
Lead.objects.filter(Exists(APPROVED))        # WHERE EXISTS (SELECT 1 ...
LIMIT 1)
Lead.objects.filter(~Exists(APPROVED))       # WHERE NOT EXISTS (...) -- the
anti-join, said plainly

# 3 -- an aggregate per outer row: five deliberate steps
DEPOSITED = (Deposit.objects
             .filter(account__lead=OuterRef("pk"), status="approved")   # 1
correlate with the outer lead
             .order_by()                                                # 2
drop any ordering (it'd join the GROUP BY)
             .values("account__lead")                                   # 3
group by lead INSIDE the subquery
             .annotate(total=Sum("amount_usd"))                         # 4
one sum per lead
             .values("total"))                                          # 5
select exactly one column
Lead.objects.annotate(deposited=Coalesce(Subquery(DEPOSITED),
Value(Decimal("0")), output_field=USD))
# no JOIN in the outer query -> nothing multiplies (Day 6, task 31)
```

```
# 4 -- a subquery in filter()
ACCOUNT_AVG = (Deposit.objects.filter(account=OuterRef("account")).order_by()
               .values("account").annotate(a=Avg("amount_usd")).values("a"))
Deposit.objects.filter(amount_usd__gt=Subquery(ACCOUNT_AVG))

# 5 -- OuterRef(OuterRef()): Desk -> Lead -> CallLog
same_desk_call = CallLog.objects.filter(lead=OuterRef("pk"),
# the lead (one level up)

agent__profile__desk=OuterRef(OuterRef("pk"))) # the desk (two up)
leads_called_by_own_desk =
(Lead.objects.filter(desk=OuterRef("pk")).filter(Exists(same_desk_call))

.order_by().values("desk").annotate(c=Count("id")).values("c"))
Desk.objects.annotate(own_desk_called=Coalesce(Subquery(leads_called_by_own_d
esk), 0))
```

6–10 — window functions.

```
# 6
NEWEST_FIRST = [F("started_at").desc(), F("id").desc()]
CallLog.objects.annotate(rn=Window(RowNumber(), partition_by=[F("lead")],
order_by=NEWEST_FIRST))
# ROW_NUMBER() OVER (PARTITION BY lead_id ORDER BY started_at DESC, id DESC)

# 7 -- running total: an aggregate used as a window, with order_by
(Deposit.objects.filter(status="approved")
 .annotate(running_usd=Window(Sum("amount_usd"), partition_by=[F("account")],
                              order_by=[F("created_at").asc(),
F("id").asc()])))

# 8
OLDEST_FIRST = [F("started_at").asc(), F("id").asc()]
(CallLog.objects
 .annotate(prev_at=Window(Lag("started_at"), partition_by=[F("lead")],
order_by=OLDEST_FIRST))
 .annotate(gap=F("started_at") - F("prev_at")))         # timedelta; None for
a lead's first call

# 9 -- rank over a per-row value (a subquery annotation), not over a GROUP BY
BROKER_REVENUE =
(Deposit.objects.filter(account__lead__broker=OuterRef("pk"),
status="approved")
```

```
.order_by().values("account__lead__broker").annotate(t=Sum("amount_usd")).val
ues("t"))
(Broker.objects.annotate(revenue=Coalesce(Subquery(BROKER_REVENUE),
Value(Decimal("0")), output_field=USD))
 .annotate(rank=Window(Rank(), order_by=F("revenue").desc()),
           dense=Window(DenseRank(), order_by=F("revenue").desc())))
# RANK: 1, 2, 2, 4   DENSE_RANK: 1, 2, 2, 3
(Lead.objects.annotate(deposited=Subquery(DEPOSITED)).filter(deposited__isnul
l=False)
 .annotate(quartile=Window(NTile(4), order_by=F("deposited").desc())))   # 1
= top 25% of depositors

# 10 -- filtering on a window (Django 4.2+): Django wraps the query and
filters outside
(CallLog.objects.annotate(rn=Window(RowNumber(), partition_by=[F("lead")],
order_by=NEWEST_FIRST))
 .filter(rn=1))
# SELECT * FROM (SELECT ..., ROW_NUMBER() OVER (...) AS rn FROM crm_calllog)
qualify WHERE rn = 1
```

B. Core

11 — the latest call per lead, three ways.

```
page = list(Lead.objects.order_by("-created_at")[:50])

# (a) Subquery per lead -- ONE query (Day 3's naive version took 50)
Lead.objects.filter(pk__in=[l.pk for l in page]).annotate(
    last_outcome=Subquery(LATEST_CALL.values("outcome")[:1]),
    last_at=Subquery(LATEST_CALL.values("started_at")[:1]))

# (b) DISTINCT ON (PostgreSQL only) -- the order_by must START with the
distinct field
CallLog.objects.filter(lead__in=page).order_by("lead_id", "-started_at", "-
id").distinct("lead_id")
# SELECT DISTINCT ON (lead_id) ... ORDER BY lead_id, started_at DESC, id DESC

# (c) Window + filter (any database with window functions)
(CallLog.objects.filter(lead__in=page)
 .annotate(rn=Window(RowNumber(), partition_by=[F("lead")],
order_by=NEWEST_FIRST)).filter(rn=1))
```

```
explain() on 50,000 leads / ~180,000 calls
(a) SubPlan per lead: Index Scan using call_lead_latest_idx + Limit -> 50
tiny lookups
(b) Unique + Index Scan (or Sort) over the matching calls
(c) WindowAgg over all matching calls, then the outer filter
Rule of thumb: a PAGE of leads -> (a); the WHOLE table -> (b) or (c), which
read the calls
once instead of once per lead. Two subqueries in (a) mean two lookups per
lead; task 19 packs
them into one.
```

12–14.

```
# 12 -- each lead's first approved deposit
FIRST_DEP = Deposit.objects.filter(account__lead=OuterRef("pk"),
status="approved").order_by("created_at", "id")
ftd_leads = (Lead.objects
             .annotate(ftd_at=Subquery(FIRST_DEP.values("created_at")[:1]),
                       ftd_usd=Subquery(FIRST_DEP.values("amount_usd")[:1]))
             .filter(ftd_at__isnull=False))

# 13 -- FTDs per day in ONE query: a subquery annotation is a plain column,
so it CAN be grouped
from crm.queries import day_start

start = day_start(timezone.localdate() - timedelta(days=29))
(Lead.objects.annotate(ftd_at=Subquery(FIRST_DEP.values("created_at")[:1]))
 .filter(ftd_at__gte=start)
 .annotate(day=TruncDate("ftd_at"))
 .values("day").annotate(ftd=Count("id")).order_by("day"))
# Day 6 couldn't group by Min(...) -- an aggregate. A Subquery isn't one.
(Zero-days: still Python.)

# 14 -- each deposit's share of its desk's month, with the desk total on
every row
month_start = day_start(timezone.localdate().replace(day=1))
(Deposit.objects.filter(status="approved", created_at__gte=month_start)
 .annotate(desk_name=F("account__lead__desk__name"),
           desk_total=Window(Sum("amount_usd"), partition_by=
```

```
[F("account__lead__desk")]))
 .annotate(share_pct=Round(F("amount_usd") * 100 / F("desk_total"), 2))
 .order_by("desk_name", "-share_pct"))
# no order_by in the window -> the SUM covers the whole partition (task 35)
```

15–17.

```
# 15 -- rank agents inside their desk
User = get_user_model()
funded_this_month =
Exists(Deposit.objects.filter(account__lead=OuterRef("pk"),
status="approved",

created_at__gte=month_start))
FUNDED =
(Lead.objects.filter(assigned_to=OuterRef("pk")).filter(funded_this_month)

.order_by().values("assigned_to").annotate(c=Count("id")).values("c"))
(User.objects.filter(profile__isnull=False)
 .annotate(funded=Coalesce(Subquery(FUNDED), 0),
desk_name=F("profile__desk__name"))
 .annotate(rank_in_desk=Window(Rank(), partition_by=[F("profile__desk")],
order_by=F("funded").desc()))
 .order_by("desk_name", "rank_in_desk"))
# the inner OuterRef("pk") belongs to the Lead query it sits in; FUNDED's
belongs to User


# 16 -- second deposit within 30 days of the first
repeat = (Lead.objects
          .annotate(ftd_at=Subquery(FIRST_DEP.values("created_at")[:1]),
                    second_at=Subquery(FIRST_DEP.values("created_at")[1:2]))
# OFFSET 1 LIMIT 1
          .filter(ftd_at__isnull=False))
repeat.filter(second_at__lte=F("ftd_at") + timedelta(days=30))
# which leads
repeat.aggregate(ftd=Count("id"),
                 repeat_30d=Count("id", filter=Q(second_at__lte=F("ftd_at") +
timedelta(days=30))))


# 17 -- gaps longer than 3 days (filtering on a window, Django 4.2+)
```

```
(CallLog.objects
 .annotate(prev_at=Window(Lag("started_at"), partition_by=[F("lead")],
order_by=OLDEST_FIRST))
 .filter(started_at__gt=F("prev_at") + timedelta(days=3)))
```

18–20.

```
# 18 -- the same numbers, two shapes (50,000 leads)
joined = Lead.objects.annotate(n=Count("calls"))                          #
LEFT JOIN + GROUP BY
CALLS_N = (CallLog.objects.filter(lead=OuterRef("pk")).order_by()
           .values("lead").annotate(c=Count("id")).values("c"))
correlated = Lead.objects.annotate(n=Coalesce(Subquery(CALLS_N), 0))      #
SubPlan per lead
# explain(): joined = one HashAggregate over all calls; correlated = SubPlan
with loops=50000.
# Whole table -> JOIN + GROUP BY. A page of 25 -> correlated (25 index
lookups, no big aggregate).
# Both are correct; only the shape of the work differs.


# 19 -- several values from ONE subquery
LAST_CALL_JSON = Subquery(LATEST_CALL.values(
    data=JSONObject(outcome="outcome", agent="agent__username",
at="started_at"))[:1])
lead = Lead.objects.annotate(last_call=LAST_CALL_JSON).first()
lead.last_call     # {'outcome': 'callback', 'agent': 'agent03', 'at': '2026-
10-19T09:12:00+00:00'}
# timestamps come back as ISO strings inside JSON -- parse them if you need
datetimes
```

20: move the building blocks into `crm/queries.py` — `LATEST_CALL`, `LAST_CALL_JSON`, `APPROVED`, `DEPOSITED`, `FIRST_DEP` — and the finished reports (`ftd_per_day` rewritten as task 13, `broker_ranking`, `repeat_rate`) into `crm/reports.py`. A subquery built with `OuterRef` is just an unevaluated QuerySet, so module-level constants are safe to share.

C. Nexora CRM

21–23.

```
# crm/queries.py (continued)
from django.db.models import DurationField, ExpressionWrapper, Max, RowRange
from django.db.models.functions import NullIf, Right, Cast


def worklist(agent):                                                    #
21
    """Day 7's call queue + the last call's outcome, agent and time, in ONE
query."""
    return
call_queue(agent).annotate(last_call=LAST_CALL_JSON).select_related("desk",
"broker")


# 22 -- two honest queries: the average, then the brokers above it
PCT = DecimalField(max_digits=12, decimal_places=4)
brokers = (Broker.objects
           .annotate(leads_n=Count("leads"), ftd_n=Count("leads",
filter=Q(leads__status="ftd")))
           .annotate(rate=Cast("ftd_n", output_field=PCT) * 100 /
NullIf(F("leads_n"), 0)))
avg_rate = brokers.aggregate(a=Avg("rate"))["a"]            # the unweighted
average of broker rates
brokers.filter(rate__gt=avg_rate).order_by("-rate")


# 23 -- top 3 per group: number the rows inside each desk, keep 1..3
(Deposit.objects.filter(status="approved", created_at__gte=month_start)
 .annotate(desk_name=F("account__lead__desk__name"),
           rn=Window(RowNumber(), partition_by=[F("account__lead__desk")],
                     order_by=[F("amount_usd").desc(), F("created_at").asc(),
F("id").asc()]))
 .filter(rn__lte=3).order_by("desk_name", "rn"))
# ROW_NUMBER: exactly 3 per desk. RANK would let ties produce 4 or more.
```

24–26.

```
# 24 -- group by desk on top of subquery annotations (task 16's `repeat`)
GAP = ExpressionWrapper(F("second_at") - F("ftd_at"),
output_field=DurationField())
(repeat.filter(second_at__isnull=False)
 .values(desk_name=F("desk__name"))
 .annotate(leads=Count("id"), avg_gap=Avg(GAP)).order_by("avg_gap"))
```

```
# avg_gap is a timedelta: .days on it for "average days"


# 25 -- a moving window: this trade and the 4 before it
(Trade.objects.filter(closed_at__isnull=False)
 .annotate(rolling_pnl=Window(Avg("pnl_usd"), partition_by=[F("account")],
                              order_by=[F("closed_at").asc(), F("id").asc()],
                              frame=RowRange(start=-4, end=0))))
# AVG(pnl_usd) OVER (PARTITION BY account_id ORDER BY closed_at, id
#                    ROWS BETWEEN 4 PRECEDING AND CURRENT ROW)


# 26 -- the funnel per lead, then the averages
FIRST_CALL_AT =
CallLog.objects.filter(lead=OuterRef("pk")).order_by("started_at",
"id").values("started_at")[:1]
funnel = (Lead.objects
          .annotate(first_call_at=Subquery(FIRST_CALL_AT),
                    ftd_at=Subquery(FIRST_DEP.values("created_at")[:1]))
          .annotate(to_first_call=ExpressionWrapper(F("first_call_at") -
F("created_at"), output_field=DurationField()),
                    call_to_ftd=ExpressionWrapper(F("ftd_at") -
F("first_call_at"), output_field=DurationField())))
funnel.aggregate(avg_to_first_call=Avg("to_first_call"),
avg_call_to_ftd=Avg("call_to_ftd"))
# NULLs (never called / never deposited) are skipped by AVG -- say so on the
report
```

27–29.

```
# 27 -- OuterRef can point at an ANNOTATION of the outer query (ftd_at)
no_call_before_ftd = ~Exists(CallLog.objects.filter(
    lead=OuterRef("pk"),
    started_at__gte=OuterRef("ftd_at") - timedelta(hours=24),
    started_at__lte=OuterRef("ftd_at")))
ftd_leads.filter(no_call_before_ftd)          # task 12's ftd_leads


# 28 -- duplicates by phone tail: a COUNT window keeps every row and shows
its group size
(Lead.objects.annotate(tail=Right("phone", 7))
 .annotate(group_size=Window(Count("id"), partition_by=[F("tail")]))
 .filter(group_size__gt=1).order_by("tail", "created_at"))
```

```
# the seed's phones never collide -- make a pair first:
#   Lead.objects.filter(pk=2).update(phone="+97155" +
Lead.objects.get(pk=1).phone[-7:])


# 29 -- backfills: a correlated subquery inside UPDATE ... SET
LAST_CALL_AT = (CallLog.objects.filter(lead=OuterRef("pk")).order_by()
                .values("lead").annotate(m=Max("started_at")).values("m"))
Lead.objects.update(last_contacted_at=Subquery(LAST_CALL_AT),
                    call_attempts=Coalesce(Subquery(CALLS_N), 0))
# UPDATE crm_lead SET last_contacted_at = (SELECT MAX(...) ... WHERE lead_id
= crm_lead.id), ...
# and the update Day 7 (task 10) couldn't do with F("lead__desk"):
CallLog.objects.filter(desk__isnull=True).update(
    desk=Subquery(Lead.objects.filter(pk=OuterRef("lead")).values("desk")
[:1]))
```

29: on millions of rows, batch this by id range (Day 4, task 13) — one UPDATE touching every lead holds every row lock until it commits.

30 — three answers checked against SQL.

```
# crm/tests.py
from .queries import FIRST_DEP, LATEST_CALL


class SubqueryWindowTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=400, stdout=StringIO())

    def sql(self, query):
        with connection.cursor() as cur:
            cur.execute(query)
            return cur.fetchall()

    def test_latest_outcome(self):
        expected = dict(self.sql("""SELECT DISTINCT ON (lead_id) lead_id,
outcome FROM crm_calllog
                                    ORDER BY lead_id, started_at DESC, id
DESC"""))
        got =
dict(Lead.objects.annotate(o=Subquery(LATEST_CALL.values("outcome")[:1]))
```

```
                    .filter(o__isnull=False).values_list("pk", "o"))
        self.assertEqual(got, expected)

    def test_ftd_per_day(self):
        expected = dict(self.sql("""
            WITH firsts AS (
              SELECT a.lead_id, MIN(d.created_at) AS ftd_at
              FROM nexora_deposits d JOIN crm_tradingaccount a ON a.id =
d.account_id
              WHERE d.status = 'approved' GROUP BY a.lead_id)
            SELECT (ftd_at AT TIME ZONE 'Asia/Dubai')::date, COUNT(*) FROM
firsts GROUP BY 1"""))
        got = {r["day"]: r["ftd"] for r in

Lead.objects.annotate(ftd_at=Subquery(FIRST_DEP.values("created_at")[:1]))

.filter(ftd_at__isnull=False).annotate(day=TruncDate("ftd_at"))
                .values("day").annotate(ftd=Count("id"))}
        self.assertEqual(got, expected)

    def test_top3_per_desk_has_at_most_three(self):
        top = (Deposit.objects.filter(status="approved")
               .annotate(rn=Window(RowNumber(), partition_by=
[F("account__lead__desk")],
                                   order_by=[F("amount_usd").desc(),
F("id").asc()]))
               .filter(rn__lte=3))
        per_desk =
top.values("account__lead__desk").annotate(n=Count("id")).values_list("n",
flat=True)
        self.assertTrue(all(n <= 3 for n in per_desk))
```

```
python manage.py test crm && git add -A && git commit -m "orm day 8"
```

D. Challenge

31–34.

```
# 31
Lead.objects.annotate(o=Subquery(LATEST_CALL.values("outcome")))         --
no [:1]
django.db.utils.ProgrammingError: more than one row returned by a subquery
```

```
used as an expression
A subquery used as a VALUE must return at most one row: slice it, or
aggregate it.

# 32  DEPOSITED without .values("account__lead") before .annotate()
The Sum groups by deposit id -> one row PER DEPOSIT -> the same error as 31.
Add [:1] to "fix" the error and it silently returns ONE deposit's amount
instead of the total:
worse, because now nothing fails.

# 33
list(CallLog.objects.filter(lead=OuterRef("pk")))
ValueError: This queryset contains a reference to an outer query and may only
be used in a subquery.

# 34
Django keeps plain conditions in the INNER WHERE, which runs BEFORE window
functions:
  ranked.filter(rn=1, outcome="interested")   -> the latest INTERESTED call
per lead
not "leads whose latest call was interested". For that, rank first, then
filter on the result:
  Lead.objects.annotate(o=Subquery(LATEST_CALL.values("outcome")
[:1])).filter(o="interested")
or CallLog.objects.filter(pk__in=ranked.filter(rn=1).values("pk"),
outcome="interested")
```

35–37.

```
# 35
Window(Sum("amount_usd"), partition_by=[F("account")])                        -
> the account TOTAL on every row
Window(Sum("amount_usd"), partition_by=[F("account")], order_by="created_at")
-> a RUNNING total
With ORDER BY, the default frame is RANGE BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW.
RANGE treats ties as one step: two deposits with the same created_at show the
SAME running
total. Add a unique tiebreaker (id) to order_by, or use
frame=RowRange(start=None, end=0).

# 36
Window(LastValue("outcome"), partition_by=[F("lead")], order_by="started_at")
-> the frame stops at the CURRENT row, so the "last" value is the current
row's own outcome.
fix: frame=RowRange(start=None, end=None)   -- ROWS BETWEEN UNBOUNDED
PRECEDING AND UNBOUNDED FOLLOWING
simpler: FirstValue("outcome") with order_by=NEWEST_FIRST

# 37  drop call_lead_latest_idx and annotate all 50,000 leads with
LATEST_CALL
Every SubPlan now uses only the plain lead_id index and SORTS that lead's
calls: 50,000 sorts.
With (lead_id, started_at DESC, id DESC) each SubPlan is an index probe +
LIMIT 1.
For the WHOLE table, one pass is better than 50,000 probes: DISTINCT ON (task
11b),
or Window + rn=1 (task 10).
```

38–40.

```
# 38 -- one question, three shapes, identical counts
a = Lead.objects.filter(Exists(APPROVED))
# Hash Semi Join
b =
Lead.objects.filter(pk__in=Deposit.objects.filter(status="approved").values("
account__lead"))

# usually ALSO a semi-join
c = Lead.objects.filter(accounts__deposits__status="approved").distinct()
# Hash Join + Unique/HashAggregate
a.count() == b.count() == c.count()
# True
# c builds the full multiplied JOIN first, then removes duplicates: more
rows, more memory.
# Exists says "at least one" and lets PostgreSQL stop at the first match.


# 39 -- a subquery that finds nothing is NULL
Lead.objects.annotate(deposited=Subquery(DEPOSITED)).order_by("-deposited")
[:10]
# NULLs sort FIRST in DESC: the top-10 depositors are 10 non-depositors
# F("deposited") - F("withdrawn") is NULL when either side is NULL
# Python: None > Decimal("5") -> TypeError
# fix: Coalesce(Subquery(...), Value(Decimal("0")), output_field=USD) -- as
in task 3
```

```
# 40
Two calls to lead 7 at exactly 2026-10-20 09:15:00 (an import, a dialer
retry).
ORDER BY started_at DESC alone -> PostgreSQL may return either one, and may
change its mind
between runs -- the Subquery and the DISTINCT ON versions can even disagree
with each other.
Every "latest" or "first" ordering in this program ends with a unique column:
("-started_at", "-id"), ("created_at", "id"). The index includes id for the
same reason.
```

Day 9 — Performance

Goal: make every query path fast and keep it fast. Find N+1 loops, load related data in a fixed number of queries, fetch only what you use, stream big exports, write in bulk, design indexes from `EXPLAIN ANALYZE`, and lock query counts with tests. Seed big for this day: `python manage.py seed_nexora --leads 50000` (~180,000 calls), then `ANALYZE` in psql.

```
The five questions for any slow page or job
1 How many queries?        CaptureQueriesContext / assertNumQueries      ->
N+1?
2 How many rows?           LIMIT, filters, values()                      ->
fetching what you show?
3 How many columns?        only() / values() / defer()                   ->
dragging JSON and text?
4 How is each one run?     explain(analyze=True)                         ->
Seq Scan on a big table?
5 How many round trips?    bulk_create / bulk_update / update()          ->
writing row by row?
```

A. Drills (1–10) — the performance toolkit

- [ ] **1.** Print 100 leads with their desk name. Count the queries; fix it with `select_related`.
- [ ] **2.** Print 100 leads with their number of calls by looping over `lead.calls.all()`. Fix it with `prefetch_related`.
- [ ] **3.** `Prefetch` with a filtered queryset and `to_attr`: only each lead's interested calls.
- [ ] **4.** Nested prefetch: leads → accounts → deposits, in a fixed number of queries.
- [ ] **5.** `select_related` two levels deep across a nullable FK: `broker__parent`. Which JOIN type?
- [ ] **6.** `only()` and `defer()`: compare the SELECT lists; touch a deferred field and count queries.
- [ ] **7.** Model instances vs `values()` vs `values_list()` for 50,000 rows: time and memory.
- [ ] **8.** `iterator(chunk_size=2000)`: stream 180,000 calls. What does PostgreSQL use underneath?
- [ ] **9.** `bulk_create` with `batch_size`, and an upsert with `update_conflicts=True`.
- [ ] **10.** `count()` vs `len()`, `exists()` vs `bool()`, `in_bulk()` vs a loop of `get()`.

B. Core (11–20) — performance as a habit

- [ ] **11.** `assertNumQueries` tests for five functions in `crm/queries.py` and `crm/reports.py`.
- [ ] **12.** A `query_budget(n)` decorator for any function: logs a warning when it runs more than n queries.
- [ ] **13.** Find your three slowest queries on the big seed; fix each with an index and prove it with `explain()`.
- [ ] **14.** A covering index (`include=[...]`) that turns an Index Scan into an Index Only Scan.
- [ ] **15.** A partial index for the agents' hot query: open leads by assignee.
- [ ] **16.** OFFSET vs keyset pagination at page 1, 100 and 1,000. Time all six.
- [ ] **17.** Cache an expensive report with Django's cache framework, with a short timeout and a clear key.
- [ ] **18.** Import 50,000 trades as an idempotent upsert: run it twice, nothing duplicates.
- [ ] **19.** "Latest call per lead" as a sliced `Prefetch` (Django 4.2+): two queries for any number of leads.
- [ ] **20.** Enable `pg_stat_statements` and list the queries that cost the most total time.

C. Nexora CRM (21–30) — real screens and jobs

- [ ] **21.** The agent worklist (25 leads with desk, broker, last call and account count) in at most 3 queries.
- [ ] **22.** The lead detail page (lead, 20 latest calls with agents, accounts with their deposits) in at most 5 queries.
- [ ] **23.** Commission statements for all brokers: rewrite a per-broker loop as one or two queries.
- [ ] **24.** Export 50,000 leads to CSV with flat memory. Measure peak memory with `tracemalloc`.
- [ ] **25.** The nightly `last_contacted_at` refresh: a Python loop vs Day 8's one UPDATE. Time both.
- [ ] **26.** Reassign 5,000 leads round-robin to 12 agents: `bulk_update` vs a `Case` UPDATE.
- [ ] **27.** Write a daily desk snapshot into `reports.DailySnapshot` from one aggregate query.
- [ ] **28.** Time every report in `crm/reports.py` on the big seed and fix the worst one.

- [ ] **29.** An index audit: unused indexes and duplicate indexes, from PostgreSQL's statistics.
- [ ] **30.** Query-budget tests for tasks 21–23 pass. Commit `orm day 9`.

D. Challenge (31–40) — performance traps

- [ ] **31.** `select_related("calls")`. Copy the error.
- [ ] **32.** Prefetch calls, then `lead.calls.filter(outcome="interested")` in the loop. How many queries?
- [ ] **33.** `only("name")`, then print `lead.phone` for 100 leads. How many queries?
- [ ] **34.** `iterator()` together with `prefetch_related()` without `chunk_size`. What happens?
- [ ] **35.** `bulk_create`: which of `save()`, signals, auto pk and `auto_now` still happen?
- [ ] **36.** `bulk_update` of 50,000 rows in one batch. What does the SQL look like, and why batch it?
- [ ] **37.** `Lead.objects.count()` on 50 million rows. Why is it slow, and what can a UI show instead?
- [ ] **38.** Four reasons PostgreSQL ignores an index you made. Reproduce two.
- [ ] **39.** `select_related` on five relations at once vs `prefetch_related`. When does a JOIN become the slow part?
- [ ] **40.** `list()` of 200,000 model instances vs a `values_list` iterator: measure the memory.

Solutions

Shell setup used throughout:

```
import time
import tracemalloc

from django.db import connection
from django.db.models import Count, Prefetch, Q
from django.test.utils import CaptureQueriesContext

from crm.models import CallLog, Deposit, Lead, TradingAccount, Trade
from crm.sqltools import explain


def n_queries(fn):
    with CaptureQueriesContext(connection) as ctx:
        fn()
    return len(ctx)
```

Timings and memory figures depend on your machine; the RATIOS between the versions are the lesson.

A. Drills

1–5 — loading related data.

```
# 1
def show_desks(qs):
    for lead in qs:
        lead.desk.name

n_queries(lambda: show_desks(Lead.objects.order_by("-created_at")[:100]))
# 101
n_queries(lambda: show_desks(Lead.objects.select_related("desk").order_by("-
created_at")[:100])) # 1
# select_related = a JOIN: the desk columns arrive in the same rows

# 2
def count_calls(qs):
    for lead in qs:
        len(lead.calls.all())

n_queries(lambda: count_calls(Lead.objects.order_by("-created_at")[:100]))
# 101
n_queries(lambda:
```

```
count_calls(Lead.objects.prefetch_related("calls").order_by("-created_at")
[:100]))  # 2
# prefetch_related = a SECOND query: SELECT ... FROM crm_calllog WHERE
lead_id IN (<100 ids>),
# then Python attaches each call to its lead.
# If you only need the NUMBER: annotate(n=Count("calls")) -- 1 query, no call
rows loaded.

# 3
interested = Prefetch("calls",
queryset=CallLog.objects.filter(outcome="interested").order_by("-
started_at"),
                      to_attr="interested_calls")
lead = Lead.objects.prefetch_related(interested).first()
lead.interested_calls          # a plain list; lead.calls.all() still means
ALL calls (and queries)

# 4 -- one query per level, whatever the number of leads
approved = Prefetch("deposits",
queryset=Deposit.objects.filter(status="approved").order_by("-created_at"))
qs = (Lead.objects.filter(status="ftd")
      .prefetch_related(Prefetch("accounts",
queryset=TradingAccount.objects.prefetch_related(approved)))[:100])
n_queries(lambda: [d for l in qs for a in l.accounts.all() for d in
a.deposits.all()])     # 3

# 5
show(Lead.objects.select_related("broker__parent"))
# LEFT OUTER JOIN crm_broker ON (lead.broker_id = broker.id)
# LEFT OUTER JOIN crm_broker T3 ON (broker.parent_id = T3.id)
# nullable FK -> LEFT OUTER (leads without a broker stay); the NOT NULL desk
FK -> INNER JOIN
```

6–10.

```
# 6
show(Lead.objects.only("id", "name", "status"))    # SELECT id, name, status
FROM crm_lead
show(Lead.objects.defer("kyc"))                    # every column except the
JSON
lead = Lead.objects.only("name").first()
n_queries(lambda: lead.phone)                      # 1 -- a deferred field
loads on first touch
```

```
# 7
def measure(label, fn):
    tracemalloc.start()
    t = time.perf_counter()
    rows = fn()
    secs = time.perf_counter() - t
    peak = tracemalloc.get_traced_memory()[1]
    tracemalloc.stop()
    print(f"{label:<12} {len(rows):>7,} rows  {secs:6.2f}s  peak {peak /
2**20:7.1f} MB")

measure("instances", lambda: list(Lead.objects.all()))
measure("values", lambda: list(Lead.objects.values("id", "name", "phone",
"status")))
measure("values_list", lambda: list(Lead.objects.values_list("id", "name",
"phone", "status")))
# instances: every column + a Python object with its state per row -> several
times the time and
# memory of tuples. For read-only exports and reports, values_list() is the
default choice.


# 8 -- stream instead of loading
total = 0
for duration in CallLog.objects.values_list("duration_sec",
flat=True).iterator(chunk_size=2000):
    total += duration
# On PostgreSQL, iterator() uses a SERVER-SIDE cursor: rows arrive 2,000 at a
time and no
# result cache is kept. (Behind PgBouncer in transaction mode, set
# DISABLE_SERVER_SIDE_CURSORS = True in DATABASES.) For a plain total,
aggregate() is better still.


# 9
CallLog.objects.bulk_create(calls, batch_size=1000)        # one INSERT per
1,000 rows
TradingAccount.objects.bulk_create(
    rows, update_conflicts=True,
    unique_fields=["platform", "account_no"],              # must match a
```

```
unique constraint
    update_fields=["currency", "leverage"])
# INSERT ... ON CONFLICT (platform, account_no)
#   DO UPDATE SET currency = EXCLUDED.currency, leverage = EXCLUDED.leverage
```

```
# 10
qs.count()           SELECT COUNT(*)                  vs len(qs): loads
every row first
qs.exists()          SELECT 1 ... LIMIT 1             vs bool(qs): loads
every row first
Lead.objects.in_bulk(ids)   ONE query: WHERE id IN (...) -> {id: lead}
[Lead.objects.get(pk=i) for i in ids]   len(ids) queries
```

B. Core

11–12 — lock the query counts.

```
# crm/tests.py
from .queries import filter_leads, leads_page, worklist
from .reports import desk_scoreboard, ftd_per_day


class QueryBudgetTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=300, stdout=StringIO())
        cls.agent = get_user_model().objects.get(username="agent01")

    def test_building_a_queryset_runs_nothing(self):
        with self.assertNumQueries(0):
            filter_leads({"status": "new", "desk": "vip", "q": "0500"})

    def test_worklist_is_one_query(self):
        with self.assertNumQueries(1):
            rows = list(worklist(self.agent)[:25])
            for r in rows:                    # touch everything a template
would touch
                r.desk.name, r.broker and r.broker.code, r.last_call

    def test_keyset_page_is_one_query(self):
        with self.assertNumQueries(1):
            leads_page()
```

```
    def test_reports_are_one_query_each(self):
        with self.assertNumQueries(1):
            desk_scoreboard()
        with self.assertNumQueries(1):
            ftd_per_day()
```

```
# crm/sqltools.py -- 12
import functools
import logging

logger = logging.getLogger("crm.queries")


def query_budget(limit):
    """@query_budget(3): warn when the decorated function runs more than
`limit` queries."""
    def decorate(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            with CaptureQueriesContext(connection) as ctx:
                result = fn(*args, **kwargs)
            if len(ctx) > limit:
                logger.warning("%s ran %d queries (budget %d)",
fn.__qualname__, len(ctx), limit)
            return result
        return wrapper
    return decorate
```

12: it only sees queries that run INSIDE the function — a function that returns a lazy QuerySet always reports 0. Put it on functions that return finished data (reports, export jobs). `CaptureQueriesContext` records every SQL string, so enable it in development and tests, not on every production request.

13–15 — indexes from evidence.

```
# 13 -- time each function on the big seed, then explain the worst
from crm import queries, reports

for fn in [lambda: list(queries.stale_open_leads()[:50]),
           lambda: list(queries.long_calls_this_week("conversion")),
           lambda: list(queries.call_queue(agent)[:25]),
           reports.desk_scoreboard, reports.symbol_performance]:
    t = time.perf_counter(); fn(); print(f"{time.perf_counter() - t:.3f}s")
```

```
Typical findings on 50,000 leads / 180,000 calls, and the fix for each:
long_calls_this_week   Seq Scan on crm_calllog, Filter: desk_id + started_at
                       -> Index(fields=["desk", "started_at"],
name="call_desk_time_idx")
stale_open_leads       Seq Scan on crm_lead, Filter: status IN (...) AND
(last_contacted_at < ... OR IS NULL)
                       -> Index(fields=["status", "last_contacted_at"],
name="lead_status_touch_idx")
call_queue             Sort on top of a Seq Scan                        ->
the partial index in task 15
After each index: makemigrations, migrate, ANALYZE, explain() again. Keep an
index only if
the plan uses it AND the time drops; every index slows every INSERT and
UPDATE a little.
```

```
# 14 -- covering index: the answer is IN the index, the table is never
visited
# CallLog.Meta.indexes: REPLACE Day 8's call_lead_latest_idx with
models.Index(fields=["lead", "-started_at", "-id"], include=["outcome"],
name="call_lead_latest_cov_idx")
# psql: VACUUM ANALYZE crm_calllog;   (Index Only Scans need an up-to-date
visibility map)
explain(Lead.objects.filter(pk__lte=100).annotate(o=Subquery(LATEST_CALL.valu
es("outcome")[:1])))
#   -> Index Only Scan using call_lead_latest_cov_idx ... Heap Fetches: 0


# 15 -- a partial index for the agents' hottest query (Lead.Meta.indexes)
models.Index(fields=["assigned_to", "created_at"], condition=Q(status__in=
["new", "callback"]),
             name="lead_open_by_agent_idx")
# serves call_queue(): WHERE status IN ('new','callback') AND assigned_to_id
= X ORDER BY created_at
# it holds only OPEN leads -- a fraction of the table -- and the query's
WHERE implies its condition
```

16–17.

```
# 16
from django.core.paginator import Paginator

from crm.queries import leads_page

base = Lead.objects.order_by("-created_at", "-id")
for page in (1, 100, 1000):
    t = time.perf_counter()
    list(Paginator(base, 25).page(page).object_list)                #
COUNT(*) + OFFSET
    offset_s = time.perf_counter() - t

    cursor_row = base[(page - 1) * 25 - 1] if page > 1 else None     # setup,
not timed
    cursor = (cursor_row.created_at, cursor_row.pk) if cursor_row else None
    t = time.perf_counter()
    leads_page(cursor)                                               # keyset
    print(f"page {page:>5}: offset {offset_s:.3f}s   keyset
{time.perf_counter() - t:.3f}s")
# OFFSET grows with the page (page 1,000 reads and throws away 24,975 rows,
```

```
plus a COUNT(*)
# per page); keyset stays flat -- it starts from an index position, not from
row 1.


# 17 -- crm/reports.py
from django.core.cache import cache


def cached_desk_scoreboard():
    """desk_scoreboard(), cached for 5 minutes per Dubai day."""
    key = f"report:desk_scoreboard:v1:{timezone.localdate():%Y%m%d}"
    return cache.get_or_set(key, desk_scoreboard, timeout=300)    # a
callable: only runs on a miss
```

17: the default cache is in-memory PER PROCESS — with 4 gunicorn workers there are 4 caches. For shared caching use Redis (`django.core.cache.backends.redis.RedisCache`, Django 4.0+). Put a version (`v1`) in the key so a changed report shape never reads old cached rows, and show "as of" on the page — 5 minutes stale is a product decision, not a bug.

18–20.

```
# 18 -- an idempotent import needs a natural key: the platform's ticket
number
# Trade:  ticket = models.BigIntegerField(null=True, unique=True)
(makemigrations + migrate)
def import_trades(rows):
    """rows: dicts from the trading platform. Safe to run any number of
times."""
    objs = (Trade(ticket=r["ticket"], account_id=r["account_id"],
symbol=r["symbol"], side=r["side"],
                  volume=r["volume"], open_price=r["open_price"],
opened_at=r["opened_at"],
                  close_price=r.get("close_price"),
closed_at=r.get("closed_at"), pnl_usd=r.get("pnl_usd"))
            for r in rows)
    return Trade.objects.bulk_create(objs, batch_size=2000,
update_conflicts=True, unique_fields=["ticket"],
                                     update_fields=["close_price",
"closed_at", "pnl_usd"])
# second run: zero new rows; trades closed since the first run get their
close data.
```

```
# (bulk_create needs a list to batch; it materialises the generator -- feed
it in chunks for millions.)


# 19 -- a SLICED Prefetch (Django 4.2+): N latest per parent in ONE extra
query
recent = Prefetch("calls",
queryset=CallLog.objects.select_related("agent").order_by("-started_at", "-
id")[:3],
                  to_attr="recent_calls")
leads = list(Lead.objects.prefetch_related(recent)[:500])     # 2 queries for
500 leads
leads[0].recent_calls                                         # up to 3
calls, newest first
# Django builds it with ROW_NUMBER() OVER (PARTITION BY lead_id ...) -- Day
8's pattern, automated.
```

```
-- 20  postgresql.conf: shared_preload_libraries = 'pg_stat_statements'
(restart PostgreSQL;
--     on a managed database it's a parameter-group setting)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SELECT calls, round(total_exec_time) AS total_ms, round(mean_exec_time, 2) AS
mean_ms, rows,
       left(query, 100) AS query
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;
-- sort by TOTAL time: a 2 ms query run 400,000 times a day (an N+1 inside a
loop) usually
-- costs more than the one 3-second report everyone complains about.
```

C. Nexora CRM

21–23 — screens with fixed query counts.

```
# crm/queries.py (continued)
def worklist_page(agent, size=25):                                          #
21
    """25 open leads with desk, broker, last call and number of accounts: 1
query."""
    return list(worklist(agent).annotate(accounts_n=Count("accounts"))
[:size])
# one multi-valued JOIN (accounts) + one JSON subquery (last call) -> nothing
multiplies


def lead_detail(pk):                                                        #
22
    """Lead + 20 latest calls with agents + accounts with deposits: 4
queries."""
    recent = Prefetch("calls",
queryset=CallLog.objects.select_related("agent")
                        .order_by("-started_at", "-id")[:20],
to_attr="recent_calls")
    accounts = Prefetch("accounts",
queryset=TradingAccount.objects.order_by("-is_primary", "opened_at")
                          .prefetch_related(Prefetch("deposits",
queryset=Deposit.objects.order_by("-created_at"))))
    return (Lead.objects.select_related("desk", "broker", "assigned_to")
            .prefetch_related(recent, accounts).get(pk=pk))
# 1 lead (+ desk, broker, agent JOINed)  2 calls (+ agent)  3 accounts  4
deposits
```

```
# 23 -- before: 1 + 40 queries
for broker in Broker.objects.all():
    Deposit.objects.filter(account__lead__broker=broker, status="approved",
                           created_at__gte=start,
created_at__lt=end).aggregate(t=Sum("amount_usd"))

# after: 2 queries, and brokers with no deposits still appear
from crm.reports import commission_statement          # Day 7, task 25: one
GROUP BY query

def all_broker_statements(start, end):
    by_code = {r["broker_code"]: r for r in commission_statement(start, end)}
    zero = {"deposits_usd": Decimal("0"), "commission_usd": Decimal("0")}
    return [{"broker_code": b.code, "rate": b.commission_rate,
**by_code.get(b.code, zero)}
            for b in Broker.objects.order_by("code")]
```

24–26 — jobs.

```
# 24 -- crm/exports.py
import csv


def export_leads_csv(path):
    rows = (Lead.objects.order_by("id")
            .values_list("id", "name", "phone", "status", "desk__slug",
"broker__code", "created_at")
            .iterator(chunk_size=5000))                  # tuples, streamed
from a server-side cursor
    with open(path, "w", newline="") as fh:
        writer = csv.writer(fh)
        writer.writerow(["id", "name", "phone", "status", "desk", "broker",
"created_at"])
        writer.writerows(rows)                           # consumes the
iterator lazily

# tracemalloc around this vs a version that builds list(Lead.objects.all())
first:
# the streamed one stays at a few MB whatever the row count; the list version
grows with it.
```

```
# 25 -- the nightly refresh, row by row (DON'T)
from django.db.models import Max

t = time.perf_counter()
for lead in Lead.objects.only("pk").iterator(chunk_size=2000):
    last = lead.calls.aggregate(m=Max("started_at"))["m"]                 #
1 query per lead
    Lead.objects.filter(pk=lead.pk).update(last_contacted_at=last)        # +
1 more
print(f"loop: {time.perf_counter() - t:.0f}s")      # 100,000 queries:
minutes
# Day 8, task 29: ONE UPDATE with a correlated subquery -- seconds.


# 26 -- round-robin reassignment of 5,000 new leads to 12 agents
from django.db.models.functions import Mod
from django.db.models.lookups import Exact

agents =
list(get_user_model().objects.filter(profile__isnull=False).order_by("usernam
e")
              .values_list("pk", flat=True))
leads = list(Lead.objects.filter(status="new").order_by("id")[:5000])
for i, lead in enumerate(leads):
    lead.assigned_to_id = agents[i % len(agents)]
Lead.objects.bulk_update(leads, ["assigned_to"], batch_size=1000)   # 5
UPDATE ... CASE statements

# or ONE statement, no rows in Python (spread by id, so near-even rather than
exact):
ids = [lead.pk for lead in leads]
Lead.objects.filter(pk__in=ids).update(assigned_to=Case(
    *[When(Exact(Mod("id", len(agents)), i), then=Value(pk)) for i, pk in
enumerate(agents)]))
# lookups as expressions inside When(): Django 4.0+
```

27–29.

```
# 27 -- reports/models.py: add Meta.constraints = [UniqueConstraint(fields=
["desk", "day"], name="one_snapshot_per_desk_day")]
from crm.queries import FIRST_DEP, day_start
from reports.models import DailySnapshot


def snapshot(day):
    start, end = day_start(day), day_start(day + timedelta(days=1))
    ftd =
dict(Lead.objects.annotate(ftd_at=Subquery(FIRST_DEP.values("created_at")
[:1]))
               .filter(ftd_at__gte=start, ftd_at__lt=end)
               .values("desk").annotate(n=Count("id")).values_list("desk",
"n"))
    DailySnapshot.objects.bulk_create(
        [DailySnapshot(desk_id=desk_id, day=day, ftd_count=ftd.get(desk_id,
0))
         for desk_id in Desk.objects.values_list("pk", flat=True)],
# zero-FTD desks too
        update_conflicts=True, unique_fields=["desk", "day"], update_fields=
["ftd_count"])
# re-running a day overwrites it -- safe for a nightly job that sometimes
runs twice.
# (INSERT ... SELECT with no rows through Python is Day 12's raw SQL.)
```

```
# 28 -- time every report
from crm import reports

for name in ["desk_scoreboard", "deposit_pivot", "ftd_per_day",
"symbol_performance",
             "breaches_by_severity", "by_root_broker",
"first_contact_compliance"]:
    t = time.perf_counter(); getattr(reports, name)(); print(f"{name:<26}
{time.perf_counter() - t:.3f}s")

# A typical worst offender: Day 6's ftd_per_day finds the FIRST deposit of
EVERY lead (50,000)
# just to keep the last 30 days. Start from the recent deposits instead and
keep only those
# with no earlier approved deposit for the same lead:
from django.db.models.functions import TruncDate

earlier = Deposit.objects.filter(
    account__lead=OuterRef("account__lead"), status="approved").filter(
    Q(created_at__lt=OuterRef("created_at")) |
Q(created_at=OuterRef("created_at"), id__lt=OuterRef("id")))
(Deposit.objects.filter(status="approved",
created_at__gte=start).filter(~Exists(earlier))

.annotate(day=TruncDate("created_at")).values("day").annotate(ftd=Count("id")
).order_by("day"))
# scans ~30 days of deposits through deposit_approved_idx instead of every
lead's history
```

```
-- 29  never-used indexes (since the statistics were last reset)
SELECT relname AS table_name, indexrelname AS index_name, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
WHERE schemaname = 'public' AND idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;

-- exact duplicates: same table, same column list
SELECT indrelid::regclass AS table_name, array_agg(indexrelid::regclass) AS
indexes
FROM pg_index GROUP BY indrelid, indkey HAVING count(*) > 1;
```

29: read these on PRODUCTION statistics gathered over weeks, not on a laptop. Never drop primary-key or unique indexes for being "unused" — they enforce constraints. Prefix redundancy needs your eyes: Django's automatic `crm_calllog_lead_id` index is a prefix of `call_lead_latest_cov_idx`, so `db_index=False` on `CallLog.lead` would remove the extra one.

30.

```
# crm/tests.py
from .queries import lead_detail, worklist_page


class ScreenBudgetTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=300, stdout=StringIO())
        cls.agent = get_user_model().objects.get(username="agent01")

    def test_worklist_page(self):
        with self.assertNumQueries(1):
            for r in worklist_page(self.agent):
                r.desk.name, r.broker and r.broker.code, r.last_call,
r.accounts_n

    def test_lead_detail(self):
        pk = Lead.objects.filter(status="ftd").values_list("pk",
flat=True).first()
        with self.assertNumQueries(4):
            lead = lead_detail(pk)
            [c.agent.username for c in lead.recent_calls]
            [d.amount_usd for a in lead.accounts.all() for d in
a.deposits.all()]
```

```
python manage.py test crm && git add -A && git commit -m "orm day 9"
```

D. Challenge

31–35.

```
# 31
Lead.objects.select_related("calls")
FieldError: Invalid field name(s) given in select_related: 'calls'. Choices
are: desk, broker, assigned_to
select_related follows to-ONE relations (FK, one-to-one) with a JOIN. To-many
-> prefetch_related.

# 32
for lead in Lead.objects.prefetch_related("calls")[:100]:
    lead.calls.filter(outcome="interested")     # a NEW query every time: 2 +
100
The prefetch cache only serves .all(). Filter in the Prefetch (task 3), or in
Python:
    [c for c in lead.calls.all() if c.outcome == "interested"]

# 33
for lead in Lead.objects.only("name")[:100]: print(lead.phone)    # 1 + 100
queries
only()/defer() trade columns for queries: list exactly what the loop touches.

# 34
for lead in Lead.objects.prefetch_related("calls").iterator(): ...
Django 5.0+: ValueError -- chunk_size must be provided when using
QuerySet.iterator() after
prefetch_related(). With iterator(chunk_size=2000) the prefetch runs once per
chunk.
(Older versions silently skipped the prefetch -- an N+1 nobody saw.)

# 35  bulk_create(objs)
save()                 NOT called (no phone normalisation, no custom logic)
pre_save / post_save   NOT sent
primary keys           SET on the objects on PostgreSQL (INSERT ... RETURNING
id);
                       not set when ignore_conflicts=True
auto_now / auto_now_add  applied (they're field pre_save hooks, not
Model.save())
multi-table inheritance  not supported
```

36–38.

```
# 36  Lead.objects.bulk_update(fifty_thousand, ["assigned_to"])
UPDATE crm_lead SET assigned_to_id = CASE WHEN id = 1 THEN 5 WHEN id = 2 THEN
9 ... (50,000 WHENs)
WHERE id IN (1, 2, ..., 50000)
One enormous statement: slow to build in Python, slow to parse and plan, big
in memory, and it
holds 50,000 row locks until it finishes. batch_size=1000 -> 50 ordinary
statements (still
inside one transaction).
```

```
# 37 -- COUNT(*) must visit every visible row (PostgreSQL keeps no stored row
count)
n = Lead.objects.filter(status="new")[:10_001].count()      # SELECT COUNT(*)
FROM (... LIMIT 10001)
label = "10,000+" if n > 10_000 else f"{n:,}"               # stops counting
at 10,001
# or the planner's estimate for a whole table, instantly:
with connection.cursor() as cur:
    cur.execute("SELECT reltuples::bigint FROM pg_class WHERE relname =
'crm_lead'")
    estimate = cur.fetchone()[0]                            # "about 50
million leads"
```

```
# 38  why PostgreSQL ignores your index
1 low selectivity  WHERE status = 'lost' matches ~20% of rows -> a Seq Scan
IS cheaper (correct!)
2 a function on the column  started_at::date, UPPER(name) without a matching
functional index
3 a leading wildcard  LIKE '%omar%' -- a B-tree can't search from the middle
4 stale statistics  right after a bulk load the planner thinks the table is
tiny -> ANALYZE
5 the index's leading column isn't in the WHERE  (desk, started_at) can't
serve started_at alone
Reproduce 1: explain(Lead.objects.filter(status="lost"))
Reproduce 2: explain(CallLog.objects.filter(started_at__date=today))   (Day
5, task 33)
```

39–40.

```
# 39
select_related("desk", "broker__parent", "assigned_to__profile")  -> ONE row
per lead carrying
every column of every joined table. Fine for a page of 25. Over 50,000 leads
the 3 desk rows are
repeated 50,000 times on the wire, and a wide joined row (JSON, long text)
multiplies that cost.
prefetch_related("desk") instead: 2 queries, 3 desk rows. Rule:
select_related for to-one data
on pages; prefetch for to-many, and for to-one targets that repeat heavily on
big result sets.
```

```
# 40 -- task 7's measure() on ~180,000 calls
measure("instances", lambda: list(CallLog.objects.all()))
measure("tuples", lambda: list(CallLog.objects.values_list("id", "lead_id",
"duration_sec")))
# and the streamed version keeps nothing:
tracemalloc.start()
sum(d for d in CallLog.objects.values_list("duration_sec",
flat=True).iterator(chunk_size=5000))
print(f"streamed peak {tracemalloc.get_traced_memory()[1] / 2**20:.1f} MB");
tracemalloc.stop()
# instances: hundreds of MB; tuples: a fraction; streamed: a few MB whatever
the table size.
```

Day 10 — Transactions and concurrency

Goal: keep Nexora's data correct when 12 agents, the dialer, the payment webhook and nightly jobs all write at once. Group writes into transactions, lock exactly the rows that need it, make conflicting writes fail loudly instead of silently, and never call the outside world from inside a transaction.

```
autocommit (Django's default)      every statement commits on its own
with transaction.atomic():         BEGIN ... COMMIT, or ROLLBACK on an
exception
    with transaction.atomic():     a SAVEPOINT: the inner block can fail
alone
transaction.on_commit(fn)          fn runs after COMMIT -- never after a
ROLLBACK
select_for_update()                SELECT ... FOR UPDATE: others wait for
your COMMIT
select_for_update(skip_locked=True)   ... SKIP LOCKED: others take the NEXT
free row
F("x") + 1, WHERE version = n      writes that can't lose another writer's
change
```

For the two-shell drills, open two terminals with `python manage.py shell` side by side. Shell A and shell B below mean those two.

A. Drills (1–10) — transactions and locks

- [ ] **1.** Autocommit: update a lead in shell A; read it in shell B straight away. Then do the same inside `atomic()` in A, before and after the block ends.
- [ ] **2.** `atomic` as a decorator and as a context manager; raise inside it and prove nothing was written.
- [ ] **3.** Nested `atomic`: the inner block fails, the outer one continues. Read the SAVEPOINT SQL.
- [ ] **4.** Catch an `IntegrityError` inside one `atomic` block without a savepoint, then run another query. Copy the error.
- [ ] **5.** `transaction.on_commit`: print after commit; show it never runs after a rollback.
- [ ] **6.** `select_for_update()`: shell A locks a lead; shell B tries to lock it too, then tries with `nowait=True`, then with `skip_locked=True`.

- [ ] **7.** `select_for_update(of=("self",))` with `select_related("desk")`: lock the lead rows but not the desk rows.
- [ ] **8.** Two threads add 1 to `call_attempts` 500 times each: read-modify-write vs `F()`. Compare the totals.
- [ ] **9.** Isolation levels: PostgreSQL's default, how to change it in `DATABASES`, and what SERIALIZABLE adds.
- [ ] **10.** `connection.in_atomic_block` and `atomic(durable=True)`: guard a function that must be the outermost transaction.

B. Core (11–20) — safe write patterns

- [ ] **11.** `claim_next_lead(agent)`: the oldest unassigned new lead, claimed exactly once even with 12 agents clicking together.
- [ ] **12.** Optimistic locking: a `version` column on `Lead` and `update_lead(pk, version, **changes)` that refuses stale edits.
- [ ] **13.** `approve_deposit(pk)`: lock the deposit, check it's pending, approve it, mark the lead FTD if it's the first; notify on commit.
- [ ] **14.** `move_lead(lead, desk, by)`: change the desk and write a `LeadTransfer` audit row — both or neither.
- [ ] **15.** Race two threads through `KycProfile.objects.get_or_create(lead=...)`. Why exactly one row?
- [ ] **16.** An idempotent event import with a unique key and `ignore_conflicts=True`.
- [ ] **17.** `retry_on_conflict()`: retry a function on serialization failures and deadlocks, with a limit.
- [ ] **18.** A PostgreSQL advisory lock so only one copy of a nightly job runs.
- [ ] **19.** Reproduce a deadlock with two shells; fix it by always locking in primary-key order.
- [ ] **20.** Test concurrency with `TransactionTestCase` and threads; close each thread's connection.

C. Nexora CRM (21–30) — the busy-hour flows

- [ ] **21.** Dialer hook: insert the call, add 1 to `call_attempts`, set `last_contacted_at` — one transaction.
- [ ] **22.** Batch claim: give an agent up to 20 free new leads, skipping rows another agent is claiming.

- [ ] **23.** Monthly broker payout: one `BrokerPayout` per broker and month, safe to run twice.
- [ ] **24.** KYC approval as a check-and-set UPDATE: approve only if still pending, without a lock.
- [ ] **25.** An outbox: write the message row in the same transaction; a sender claims rows with `skip_locked`.
- [ ] **26.** Deposit status transitions enforced by the UPDATE itself (pending → approved/rejected only).
- [ ] **27.** Make Day 9's snapshot job single-runner and idempotent together.
- [ ] **28.** Merge two duplicate leads: move calls, accounts and breaches to the survivor, delete the other — locked, ordered, atomic.
- [ ] **29.** Find long transactions and blocked queries in `pg_stat_activity` and `pg_locks`.
- [ ] **30.** Tests for 11, 12, 13 and 22. Commit `orm day 10`.

D. Challenge (31–40) — concurrency bugs

- [ ] **31.** `try: ... except IntegrityError: pass` inside a request-wide transaction. What breaks next?
- [ ] **32.** An `on_commit` callback in a `TestCase` never runs. Why? Test it properly.
- [ ] **33.** `select_for_update()` outside a transaction. Copy the error.
- [ ] **34.** You built a `select_for_update()` QuerySet but the row isn't locked. Why?
- [ ] **35.** `select_for_update()` with `select_related("broker")` (nullable). Copy the PostgreSQL error and fix it.
- [ ] **36.** An HTTP call to the payment provider inside `atomic()`. List three things that go wrong.
- [ ] **37.** Two agents edit the same lead's name and status with `save()`. Show the lost update; fix it three ways.
- [ ] **38.** `get_or_create` on a field with no unique constraint, under two threads. What do you get?
- [ ] **39.** Run a transfer under SERIALIZABLE in two shells until one fails. Copy the error; retry it.
- [ ] **40.** Two `bulk_update` jobs touching overlapping leads in different orders deadlock. Fix it.

Solutions

New models used today (add, `makemigrations`, `migrate`):

```
# crm/models.py
class Lead(models.Model):
    ...
    version = models.PositiveIntegerField(default=1)                        #
12

    class Meta:
        indexes = [..., models.Index(fields=["created_at", "id"],
                                     condition=Q(status="new",
assigned_to__isnull=True),
                                     name="lead_unclaimed_idx")]             #
11, 22


class LeadTransfer(models.Model):                                           #
14
    lead = models.ForeignKey(Lead, on_delete=models.CASCADE,
related_name="transfers")
    from_desk = models.ForeignKey(Desk, on_delete=models.PROTECT,
related_name="+")
    to_desk = models.ForeignKey(Desk, on_delete=models.PROTECT,
related_name="+")
    by = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.PROTECT, related_name="+")
    at = models.DateTimeField(default=timezone.now)


class DialerEvent(models.Model):                                            #
16
    event_id = models.CharField(max_length=64, unique=True)
    payload = models.JSONField()
    received_at = models.DateTimeField(default=timezone.now)


class BrokerPayout(models.Model):                                           #
23
    broker = models.ForeignKey(Broker, on_delete=models.PROTECT,
related_name="payouts")
    period = models.DateField()                       # first day of the
```

```
month
    deposits_usd = models.DecimalField(max_digits=16, decimal_places=2)
    commission_usd = models.DecimalField(max_digits=16, decimal_places=2)
    created_at = models.DateTimeField(default=timezone.now)

    class Meta:
        constraints = [models.UniqueConstraint(fields=["broker", "period"],
name="one_payout_per_broker_month")]


class OutboxMessage(models.Model):                                      #
25
    topic = models.CharField(max_length=50)
    payload = models.JSONField()
    created_at = models.DateTimeField(default=timezone.now)
    sent_at = models.DateTimeField(null=True, blank=True)
    attempts = models.PositiveIntegerField(default=0)

    class Meta:
        indexes = [models.Index(fields=["created_at", "id"],
condition=Q(sent_at__isnull=True),
                                name="outbox_unsent_idx")]
```

All write functions below go in `crm/services.py`, with these imports:

```
import functools
import random
import time
from contextlib import contextmanager
from datetime import timedelta

from django.db import IntegrityError, OperationalError, connection,
transaction
from django.db.models import F, Q, Value
from django.db.models.functions import Greatest, Now
from django.utils import timezone
from psycopg import errors as pg_errors

from .models import (Broker, BrokerPayout, CallLog, Deposit, KycProfile,
Lead, LeadTransfer,
                     OutboxMessage, SOPBreach, TradingAccount)
```

A. Drills

1–5.

```
# 1
Autocommit: shell A's update() is visible to shell B immediately -- it
committed on its own.
Inside `with transaction.atomic():` in A, B keeps seeing the OLD value until
A's block ends;
then B sees the new value. PostgreSQL never shows uncommitted data to other
sessions.
```

```
# 2
@transaction.atomic
def rename(pk, name):
    Lead.objects.filter(pk=pk).update(name=name)

with transaction.atomic():
    Lead.objects.filter(pk=7).update(name="Changed")
    raise RuntimeError("boom")         # ROLLBACK -- lead 7 keeps its old
name

# 3 -- a savepoint lets one part fail
with transaction.atomic():                                  # BEGIN
    Lead.objects.filter(pk=7).update(status="callback")
    try:
        with transaction.atomic():                          # SAVEPOINT
"s..._x1"
            Deposit.objects.create(account=acc, amount=0, currency="USD",
fx_rate=1, method="card")
    except IntegrityError:                                  # ROLLBACK TO
SAVEPOINT
        pass
# COMMIT -- the status change is kept, the bad deposit isn't

# 5
with transaction.atomic():
    Lead.objects.filter(pk=7).update(status="ftd")
    transaction.on_commit(lambda: print("notify Finance"))
    print("still inside")
# prints "still inside", THEN "notify Finance". Raise inside the block:
"notify Finance" never prints.
```

```
# 4  catching an IntegrityError with NO inner atomic, then querying again
django.db.transaction.TransactionManagementError: An error occurred in the
current transaction.
You can't execute queries until the end of the 'atomic' block.
PostgreSQL marks the whole transaction failed after any error. Only rolling
back -- the whole
thing, or to a savepoint -- makes it usable again. That's what the inner
atomic() in 3 is for.
```

6–10.

```
# 6 -- shell A
with transaction.atomic():
    lead = Lead.objects.select_for_update().get(pk=7)   # SELECT ... FOR
UPDATE
    input("holding the lock -- press Enter to commit")

# shell B, while A waits
with transaction.atomic():
    Lead.objects.select_for_update().get(pk=7)           # blocks until A
commits
with transaction.atomic():
    Lead.objects.select_for_update(nowait=True).get(pk=7)
# OperationalError: could not obtain lock on row in relation "crm_lead"
with transaction.atomic():
    Lead.objects.select_for_update(skip_locked=True).filter(pk=7).first()   #
None: skipped

# 7
Lead.objects.select_related("desk").select_for_update(of=("self",))
# ... FOR UPDATE OF "crm_lead" -- the desk rows stay unlocked for everyone
else
```

```
# 8
import threading


def bump_naive(pk, n):
    for _ in range(n):
        lead = Lead.objects.get(pk=pk)
        lead.call_attempts += 1                              # read in Python
...
```

```
        lead.save(update_fields=["call_attempts"])          # ... write back:
the other thread's +1 can vanish
    connection.close()                                      # each thread has
its own connection


def bump_f(pk, n):
    for _ in range(n):
        Lead.objects.filter(pk=pk).update(call_attempts=F("call_attempts") +
1)
    connection.close()


for fn in (bump_naive, bump_f):
    Lead.objects.filter(pk=7).update(call_attempts=0)
    threads = [threading.Thread(target=fn, args=(7, 500)) for _ in range(2)]
    [t.start() for t in threads]; [t.join() for t in threads]
    print(fn.__name__, Lead.objects.get(pk=7).call_attempts)
# bump_naive  ~700-990 (varies)   bump_f  1000, every time
```

```
# 9 -- PostgreSQL's default is READ COMMITTED. To change it for a connection:
from django.db.backends.postgresql.psycopg_any import IsolationLevel

DATABASES["default"]["OPTIONS"] = {"isolation_level":
IsolationLevel.SERIALIZABLE}
# SERIALIZABLE makes concurrent transactions behave as if they ran one after
another -- by
# FAILING one of them when they'd conflict (task 39), so every caller must be
ready to retry.
# Common practice: keep READ COMMITTED globally; use constraints, F(),
version checks and row
# locks; switch a single transaction with SET TRANSACTION ISOLATION LEVEL
SERIALIZABLE.

# 10
def run_month_end():
    with transaction.atomic(durable=True):     # must be the OUTERMOST
transaction
        ...
# called inside another atomic(): RuntimeError: A durable atomic block cannot
be nested within
# another atomic block. Use it for code that must really commit when it says
it does.
```

B. Core

11–14.

```
def claim_next_lead(agent):                                                 #
11
    """Oldest unassigned new lead, claimed exactly once, or None."""
    with transaction.atomic():
        lead = (Lead.objects.select_for_update(skip_locked=True)
                .filter(status="new", assigned_to__isnull=True)
                .order_by("created_at", "id").first())
        if lead is None:
            return None
        Lead.objects.filter(pk=lead.pk).update(assigned_to=agent,
updated_at=Now())
        lead.assigned_to = agent
        return lead
# 12 agents at once: each locks a DIFFERENT row; nobody waits, nobody gets
```

```
the same lead.


class StaleEdit(Exception):
    pass


def update_lead(pk, version, **changes):                                  #
12
    """Apply changes only if nobody saved the lead since `version` was
read."""
    n = (Lead.objects.filter(pk=pk, version=version)
         .update(**changes, version=F("version") + 1, updated_at=Now()))
    if n == 0:
        raise StaleEdit(f"Lead {pk} was changed by someone else -- reload and
try again.")
# the check and the write are ONE statement: there's no gap for another
writer to slip into


def approve_deposit(pk, notify=print):                                    #
13
    """Approve a pending deposit; mark the lead FTD if it's their first
approved one."""
    lead_id = Deposit.objects.filter(pk=pk).values_list("account__lead_id",
flat=True).get()
    with transaction.atomic():
        Lead.objects.select_for_update().filter(pk=lead_id).get()         #
lock order: lead, then deposit
        dep = Deposit.objects.select_for_update().get(pk=pk)
        if dep.status != "pending":
            raise ValueError(f"Deposit {pk} is {dep.status}, not pending.")
        first = not Deposit.objects.filter(account__lead_id=lead_id,
status="approved").exists()
        Deposit.objects.filter(pk=pk).update(status="approved")
        if first:
            Lead.objects.filter(pk=lead_id).update(status="ftd",
updated_at=Now())
        transaction.on_commit(lambda: notify(f"deposit {pk} approved (first=
{first})"))
    return first
# locking the LEAD serialises approvals per client: two first deposits
approved at the same
```

```
# moment can't both see "no approved deposit yet" and both count as the FTD.


def move_lead(lead_pk, desk, by):                                       #
14
    with transaction.atomic():
        lead = Lead.objects.select_for_update().get(pk=lead_pk)
        if lead.desk_id == desk.pk:
            return lead
        LeadTransfer.objects.create(lead=lead, from_desk_id=lead.desk_id,
to_desk=desk, by=by)
        lead.desk = desk
        lead.save(update_fields=["desk", "updated_at"])
        return lead
```

15–17.

```
# 15  two threads: KycProfile.objects.get_or_create(lead=lead)
Both SELECT, both find nothing, both INSERT. KycProfile.lead is one-to-one =
UNIQUE, so the
second INSERT fails; get_or_create catches that IntegrityError (inside its
own savepoint) and
re-reads the row the first thread created. One row -- because of the
CONSTRAINT, not the method
(task 38 removes it).
```

```
# 16 -- the dialer may send the same event twice; the unique event_id makes
it harmless
DialerEvent.objects.bulk_create(
    [DialerEvent(event_id=e["id"], payload=e) for e in events],
ignore_conflicts=True)
# INSERT ... ON CONFLICT DO NOTHING


RETRYABLE = (pg_errors.SerializationFailure, pg_errors.DeadlockDetected)


def retry_on_conflict(attempts=3):                                      #
17
    """Run fn in its own transaction; retry on serialization failure or
deadlock."""
    def decorate(fn):
        @functools.wraps(fn)
```

```
        def wrapper(*args, **kwargs):
            for attempt in range(attempts):
                try:
                    with transaction.atomic(durable=True):     # retrying
INSIDE an outer
                        return fn(*args, **kwargs)             # transaction
can't work
                except OperationalError as exc:
                    if not isinstance(exc.__cause__, RETRYABLE) or attempt ==
attempts - 1:
                        raise
                    time.sleep(0.05 * 2 ** attempt + random.random() * 0.05)
# back off, with jitter
        return wrapper
    return decorate
# Django re-raises psycopg's errors as django.db.utils.OperationalError; the
original is in __cause__.
```

18–20.

```
@contextmanager
def single_runner(name):                                                  #
18
    """Yield True if this process got the job's lock, False if another run
holds it."""
    with transaction.atomic():
        with connection.cursor() as cur:
            cur.execute("SELECT pg_try_advisory_xact_lock(hashtext(%s))",
[name])
            got = cur.fetchone()[0]
        yield got                       # the lock is released automatically
at COMMIT / ROLLBACK


with single_runner("nightly-snapshot") as got:
    if got:
        snapshot(yesterday)             # Day 9, task 27
```

```
# 19  shell A: lock lead 1, then lead 2       shell B: lock lead 2, then lead
1
Each waits for the other. After deadlock_timeout (1s) PostgreSQL kills one
transaction:
OperationalError: deadlock detected
Fix: never lock rows one by one in caller-chosen order. Lock them together,
in one order:
  Lead.objects.select_for_update().filter(pk__in=[a, b]).order_by("pk")
```

```
# 20 -- crm/tests/test_concurrency.py
from concurrent.futures import ThreadPoolExecutor

from django.contrib.auth import get_user_model
from django.db import connection
from django.test import TransactionTestCase

from crm.models import Desk, Lead
from crm.services import claim_next_lead


class ClaimRaceTest(TransactionTestCase):      # real COMMITs, so threads see
each other's data
    def test_every_lead_is_claimed_once(self):
        desk = Desk.objects.create(slug="t", name="T")
        Lead.objects.bulk_create([Lead(name=f"L{i}",
phone=f"+97150000{i:04d}", desk=desk) for i in range(5)])
        User = get_user_model()
        agents = [User.objects.create(username=f"a{i}").pk for i in
range(10)]

        def claim(agent_pk):
            try:
                lead = claim_next_lead(User.objects.get(pk=agent_pk))
                return lead.pk if lead else None
            finally:
                connection.close()           # threads open their own
connections

        with ThreadPoolExecutor(max_workers=10) as pool:
            claimed = [pk for pk in pool.map(claim, agents) if pk]
```

```
        self.assertEqual(sorted(claimed), sorted(set(claimed)))      # no
lead twice
        self.assertEqual(len(claimed), 5)                              # and
none left behind
```

C. Nexora CRM

21–24.

```
def record_call(lead_pk, agent, started_at, duration_sec, outcome):        #
21
    with transaction.atomic():
        desk_id = Lead.objects.values_list("desk_id",
flat=True).get(pk=lead_pk)
        call = CallLog.objects.create(lead_id=lead_pk, agent=agent,
desk_id=desk_id, started_at=started_at,
                                      duration_sec=duration_sec,
outcome=outcome)
        Lead.objects.filter(pk=lead_pk).update(
            call_attempts=F("call_attempts") + 1,
            last_contacted_at=Greatest("last_contacted_at",
Value(started_at)),   # never moves backwards
            updated_at=Now())
    return call
# GREATEST ignores NULL in PostgreSQL, so the first call sets it; a late-
arriving older call
# from the dialer can't overwrite a newer one.


def claim_batch(agent, size=20):                                            #
22
    with transaction.atomic():
        ids = list(Lead.objects.select_for_update(skip_locked=True)
                   .filter(status="new", assigned_to__isnull=True)
                   .order_by("created_at", "id").values_list("pk", flat=True)
[:size])
        Lead.objects.filter(pk__in=ids).update(assigned_to=agent,
updated_at=Now())
    return ids


def run_payouts(period):                                                    #
23
```

```
    """Create BrokerPayout rows for the month starting `period`. Re-running
changes nothing."""
    from .queries import day_start
    from .reports import commission_statement
    next_month = (period.replace(day=28) + timedelta(days=4)).replace(day=1)
    with single_runner(f"payouts-{period:%Y-%m}") as got:
        if not got:
            return "another payout run is active"
        rows = commission_statement(day_start(period), day_start(next_month))
        ids = dict(Broker.objects.filter(code__in=[r["broker_code"] for r in
rows]).values_list("code", "pk"))
        BrokerPayout.objects.bulk_create(
            [BrokerPayout(broker_id=ids[r["broker_code"]], period=period,
                          deposits_usd=r["deposits_usd"],
commission_usd=r["commission_usd"]) for r in rows],
            ignore_conflicts=True)               # an issued payout is never
silently rewritten
    return "done"


def approve_kyc(lead_pk):                                                #
24
    """Check-and-set in one UPDATE. False = it wasn't pending any more."""
    return KycProfile.objects.filter(lead_id=lead_pk,
status="pending").update(status="approved") == 1
```

25–29.

```
# 25 -- producer: inside approve_deposit's atomic block, next to the status
change
OutboxMessage.objects.create(topic="deposit.approved", payload={"deposit":
pk, "lead": lead_id, "first": first})


def send_pending(send, batch=50):
    """Worker: deliver unsent messages. Several workers can run side by
side."""
    with transaction.atomic():
        msgs = list(OutboxMessage.objects.select_for_update(skip_locked=True)
                    .filter(sent_at__isnull=True).order_by("created_at",
"id")[:batch])
        for m in msgs:
            try:
```

```
                send(m.topic, {**m.payload, "message_id": m.pk})
                m.sent_at = timezone.now()
            except Exception:
                m.attempts += 1
        OutboxMessage.objects.bulk_update(msgs, ["sent_at", "attempts"])
# the message exists IFF the business change committed. Delivery is at-least-
once (a crash
# after send() but before COMMIT resends), so consumers de-duplicate on
message_id.


ALLOWED_FROM = {"approved": ["pending"], "rejected": ["pending"]}


def set_deposit_status(pk, new):                                        #
26
    if not Deposit.objects.filter(pk=pk,
status__in=ALLOWED_FROM[new]).update(status=new):
        raise ValueError(f"Deposit {pk} can't move to {new}.")
# the WHERE is the state machine: "approved -> rejected" simply matches no
row


def nightly_snapshot(day):                                              #
27
    with single_runner("nightly-snapshot") as got:      # one runner at a
time ...
        if got:
            snapshot(day)                               # ... and Day 9's
upsert makes re-runs safe


def merge_leads(keep_pk, drop_pk):                                      #
28
    with transaction.atomic():
        list(Lead.objects.select_for_update().filter(pk__in=[keep_pk,
drop_pk]).order_by("pk"))
        # calls at the same instant as the survivor's would break Day 4's
(lead, started_at) unique key
        CallLog.objects.filter(lead_id=drop_pk,
started_at__in=CallLog.objects.filter(
            lead_id=keep_pk).values("started_at")).delete()
        CallLog.objects.filter(lead_id=drop_pk).update(lead_id=keep_pk)
```

```
TradingAccount.objects.filter(lead_id=drop_pk).update(lead_id=keep_pk,
is_primary=False)
        SOPBreach.objects.filter(lead_id=drop_pk).update(lead_id=keep_pk)
        KycProfile.objects.filter(lead_id=drop_pk).delete()   # one-to-one:
keep the survivor's
        Lead.objects.filter(pk=drop_pk).delete()
```

```
-- 29  long or idle transactions, oldest first
SELECT pid, state, now() - xact_start AS xact_age, wait_event_type,
left(query, 80) AS query
FROM pg_stat_activity WHERE xact_start IS NOT NULL ORDER BY xact_start;
-- who blocks whom
SELECT blocked.pid, blocking.pid AS blocked_by, left(blocked.query, 60) AS
waiting_query
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking ON blocking.pid = ANY
(pg_blocking_pids(blocked.pid));
-- state = 'idle in transaction' = the app opened a transaction and is doing
something else
-- (an HTTP call, task 36). Guard rail: ALTER DATABASE nexora_orm SET
idle_in_transaction_session_timeout = '60s';
```

30.

```
# crm/tests.py
from .services import StaleEdit, approve_deposit, claim_batch, update_lead


class ServiceTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=300, stdout=StringIO())

    def test_stale_edit_is_refused(self):
        lead = Lead.objects.first()
        update_lead(lead.pk, lead.version, name="First")
        with self.assertRaises(StaleEdit):
            update_lead(lead.pk, lead.version, name="Second")      # still
holds the OLD version

    def test_first_approval_marks_ftd_and_notifies_after_commit(self):
        lead = Lead.objects.filter(status="new",
```

```
accounts__isnull=False).first()
        acc = lead.accounts.first()
        dep = Deposit.objects.create(account=acc, amount=500,
currency=acc.currency, fx_rate=1, method="card")
        sent = []
        with self.captureOnCommitCallbacks(execute=True):
            self.assertTrue(approve_deposit(dep.pk, notify=sent.append))
        lead.refresh_from_db()
        self.assertEqual((lead.status, len(sent)), ("ftd", 1))

    def test_batches_never_overlap(self):
        a, b = get_user_model().objects.filter(username__in=["agent01",
"agent02"])
        Lead.objects.filter(status="new").update(assigned_to=None)
        self.assertFalse(set(claim_batch(a)) & set(claim_batch(b)))
```

```
python manage.py test crm && git add -A && git commit -m "orm day 10"
```

D. Challenge

31–36.

```
# 31  ATOMIC_REQUESTS = True (or any outer atomic) + a bare except around a
failing INSERT
The NEXT query raises TransactionManagementError (task 4) -- often far from
the real error.
Wrap the risky statement in its own `with transaction.atomic():` so the
failure rolls back
only to its savepoint.

# 32
TestCase runs each test inside a transaction that is ROLLED BACK at the end -
- nothing ever
commits, so on_commit callbacks never fire. Use
self.captureOnCommitCallbacks(execute=True)
(task 30), or TransactionTestCase when you need real commits (task 20).

# 33
Lead.objects.select_for_update().get(pk=7)        -- in autocommit
TransactionManagementError: select_for_update cannot be used outside of a
transaction.
(A lock that's released the instant the statement ends would protect
```

```
nothing.)

# 34
qs = Lead.objects.select_for_update().filter(pk=7)   -- builds SQL, locks
NOTHING
The lock is taken when the QuerySet is EVALUATED (get(), first(), list(), a
for loop), and it
lasts until the transaction ends.

# 35
Lead.objects.select_related("broker").select_for_update().get(pk=7)
NotSupportedError: FOR UPDATE cannot be applied to the nullable side of an
outer join
fix: select_for_update(of=("self",)) -- lock the lead, not the (LEFT JOINed)
broker

# 36  calling the payment provider inside atomic()
1 Locks are held for the whole network round trip; a 30-second timeout blocks
every agent
  touching those rows for 30 seconds.
2 The provider succeeds, then the transaction rolls back (a later error):
money moved, the
  CRM says nothing happened. The reverse also happens.
3 Long transactions pin connections (pool exhaustion) and stop VACUUM; a
retry calls the
  provider twice.
Fix: commit first, then call -- transaction.on_commit, or the outbox (task
25).
```

37–40.

```
# 37  agent A: lead.name = "Omar K."; lead.save()     agent B: lead.status =
"callback"; lead.save()
B's save() writes ALL columns from its stale copy: A's name change
disappears.
Fix 1  save(update_fields=["status"]) -- each writes only its own column
(different fields only)
Fix 2  F() / single-statement updates for counters and computed values
Fix 3  version check (task 12) -- the second writer gets StaleEdit and
reloads

# 38  get_or_create(campaign="autumn") on a column WITHOUT a unique
constraint, two threads
```

```
Both SELECT nothing, both INSERT, both succeed: two rows. Later, get() raises
MultipleObjectsReturned. get_or_create is only race-safe ON TOP OF a unique
constraint.

# 39  in both shells: atomic() + cursor.execute("SET TRANSACTION ISOLATION
LEVEL SERIALIZABLE")
#     as the first statement; each reads both desks' lead counts, then moves
a lead
OperationalError: could not serialize access due to read/write dependencies
among transactions
Nothing is wrong with your code: SERIALIZABLE chose a victim. Retry the whole
transaction --
that's what retry_on_conflict (task 17) is for.

# 40  job A: bulk_update(leads in id order)   job B: bulk_update(leads in
score order)
Each UPDATE locks rows as it goes; overlapping rows in opposite orders ->
deadlock detected.
Fix: sort before writing -- objs.sort(key=lambda o: o.pk) -- in every job
that writes the same
table. Same rule as task 19: one global lock order.
```

Day 11 — Managers and custom QuerySets

Goal: give the ORM Nexora's vocabulary. Instead of repeating filters in every view, report and job, write `Lead.objects.visible_to(user).open().stale(3).with_last_call()` — one tested definition of "open", "stale" and "visible" that every caller shares. Then add soft delete without breaking uniqueness or reports.

```
QuerySet method   chainable rule about WHICH rows / WHAT extra columns
Lead.objects.open().stale(3)
Manager method    an entry point that isn't a filter (create, bulk ops)
Lead.objects.create_lead(...)
Model method      behaviour of ONE row
lead.is_open, lead.mark_status()
Service function  a business action touching several models / a txn
approve_deposit(pk)   (Day 10)
```

A. Drills (1–10) — managers and QuerySets

- [ ] **1.** `type(Lead.objects)` vs `type(Lead.objects.all())`. What can a manager do that a QuerySet can't, and the reverse?
- [ ] **2.** A `LeadQuerySet` with `open()` and `stale(days)`, attached with `as_manager()`.
- [ ] **3.** Chain your methods with built-in ones in any order: `filter`, `open`, `order_by`, `stale`.
- [ ] **4.** `Manager.from_queryset(LeadQuerySet)` plus a manager-only `create_lead()`.
- [ ] **5.** Override `get_queryset()` to hide archived leads; add an `all_objects` manager that doesn't.
- [ ] **6.** Which manager does each use: `Lead.objects`, `desk.leads`, `call.lead`, the admin, `dumpdata`?
- [ ] **7.** `use_in_migrations = True`: call a manager method inside a data migration.
- [ ] **8.** Methods that add columns: `with_last_call()`, `with_deposited()`.
- [ ] **9.** Related managers get your methods: `desk.leads.open().stale(3)`.
- [ ] **10.** Sort six pieces of logic into model method, QuerySet method, manager method or service.

B. Core (11–20) — Nexora's vocabulary

- [ ] **11.** The full `LeadQuerySet`: open, stale, for_desk, for_agent, search, queue_order, with_last_call, with_deposited, funded, with_ftd, ftd_between.
- [ ] **12.** `DepositQuerySet`: approved, in_period, usd_total, by_method.
- [ ] **13.** `CallLogQuerySet`: today, long, for_agent, latest_per_lead.
- [ ] **14.** `TradeQuerySet` (open, closed, stale, for_symbol, performance) and `SOPBreachQuerySet` (high, recent, for_desk).
- [ ] **15.** Soft delete: `archived_at`, `archive()` / `restore()`, managers, and a phone uniqueness rule that ignores archived rows.
- [ ] **16.** `visible_to(user)`: agents see their leads, desk managers their desks, superusers everything — one query.
- [ ] **17.** `create_lead()`: normalise the phone, validate with `full_clean()`, save.
- [ ] **18.** `with_recent_calls(n)`: a sliced `Prefetch` behind a method.
- [ ] **19.** Rewrite `crm/queries.py` on top of the new methods. What's left in the file?
- [ ] **20.** Unit tests for the QuerySet methods.

C. Nexora CRM (21–30) — the vocabulary at work

- [ ] **21.** The agent worklist as one readable chain.
- [ ] **22.** A desk dashboard (open, stale, funded this month, deposits USD) from method chains.
- [ ] **23.** `Broker.objects.downline_of("BRF-0001")` and leads from that downline.
- [ ] **24.** `SOPBreach.objects.high().recent(30).for_desk("conversion")`.
- [ ] **25.** `Trade.objects.open().stale(7).for_symbol("XAUUSD")`.
- [ ] **26.** Archive every lost lead older than 90 days in one statement.
- [ ] **27.** Restore an archived lead; create a new lead with an archived lead's phone.
- [ ] **28.** Admin: show archived leads with a filter, using `all_objects`.
- [ ] **29.** Make every report in `crm/reports.py` use the methods (one definition of "approved").
- [ ] **30.** Tests pass. Commit `orm day 11`.

D. Challenge (31–40) — manager traps

- [ ] **31.** After archiving, where do archived leads still appear — and where do they vanish?
- [ ] **32.** A method that returns `list(...)`. What breaks for its callers?
- [ ] **33.** A QuerySet method named `count` or `update`. What did you just break?
- [ ] **34.** `get_queryset()` that always adds `select_related` and an annotation. What does `count()` cost now?
- [ ] **35.** A ModelForm adds a lead whose phone belongs to an archived lead. Who complains, and with what?
- [ ] **36.** A data migration calls `Lead.objects.open()` without `use_in_migrations`. Copy the error.
- [ ] **37.** A manager on an abstract base model: what do the child models inherit?
- [ ] **38.** Your methods after `union()`. Copy the error.
- [ ] **39.** A `with_everything()` method used on every page. Measure what it costs a simple count.
- [ ] **40.** Two chained methods each filter across `accounts__deposits`. Why can the result be wrong?

Solutions

The finished file comes first because almost every task touches it; the task numbers in the comments show where each piece is introduced.

```
# crm/querysets.py
from datetime import timedelta
from decimal import Decimal

from django.apps import apps
from django.db import models
from django.db.models import Count, DecimalField, Exists, OuterRef, Prefetch,
Q, Subquery, Sum, Value
from django.db.models.functions import Coalesce
from django.utils import timezone

OPEN = ["new", "callback"]
USD = DecimalField(max_digits=16, decimal_places=2)
```

```
def _model(name):
    """Late lookup: models.py imports this module, so it can't import
models.py back."""
    return apps.get_model("crm", name)


class LeadQuerySet(models.QuerySet):
    def open(self):
# 2
        return self.filter(status__in=OPEN)

    def stale(self, days=3):
# 2
        cutoff = timezone.now() - timedelta(days=days)
        return self.filter(Q(last_contacted_at__lt=cutoff) |
Q(last_contacted_at__isnull=True))

    def for_desk(self, slug):
# 11
        return self.filter(desk__slug=slug)

    def for_agent(self, user):
        return self.filter(assigned_to=user)

    def search(self, text):
        from .queries import search_q                # Day 5's Q builder
        return self.filter(search_q(text))

    def queue_order(self):
        from .queries import QUEUE_PRIORITY          # Day 7
        return self.annotate(priority=QUEUE_PRIORITY).order_by("priority",
"created_at", "id")

    def with_last_call(self):
# 8
        from .queries import LAST_CALL_JSON          # Day 8
        return self.annotate(last_call=LAST_CALL_JSON)

    def with_deposited(self):
# 8
        from .queries import DEPOSITED
        return self.annotate(deposited=Coalesce(Subquery(DEPOSITED),
Value(Decimal("0")), output_field=USD))
```

```
    def funded(self):
        from .queries import APPROVED
        return self.filter(Exists(APPROVED))

    def with_ftd(self):
        from .queries import FIRST_DEP
        return self.annotate(ftd_at=Subquery(FIRST_DEP.values("created_at")
[:1]))

    def ftd_between(self, start, end):
        return self.with_ftd().filter(ftd_at__gte=start, ftd_at__lt=end)

    def visible_to(self, user):
# 16
        if user.is_superuser:
            return self
        manages_desk =
Exists(_model("DeskMembership").objects.filter(desk=OuterRef("desk"),
user=user))
        return self.filter(Q(assigned_to=user) | manages_desk)

    def with_recent_calls(self, n=5):
# 18
        calls = _model("CallLog").objects.select_related("agent").order_by("-
started_at", "-id")[:n]
        return self.prefetch_related(Prefetch("calls", queryset=calls,
to_attr="recent_calls"))

    def archive(self):
# 15
        return self.update(archived_at=timezone.now())


class LeadManager(models.Manager.from_queryset(LeadQuerySet)):              #
4, 5
    def get_queryset(self):
        return super().get_queryset().filter(archived_at__isnull=True)

    def create_lead(self, *, name, phone, desk, **extra):
# 17
        lead = self.model(name=name.strip(),
phone=self.model.normalize_phone(phone), desk=desk, **extra)
```

```
        lead.full_clean()             # fields, clean(), constraints --
before the database sees it
        lead.save()
        return lead


class DepositQuerySet(models.QuerySet):                                  #
12
    def approved(self):
        return self.filter(status="approved")

    def in_period(self, start, end):
        return self.filter(created_at__gte=start, created_at__lt=end)

    def usd_total(self):              # TERMINAL: ends the chain with a
Decimal
        return self.aggregate(t=Sum("amount_usd", default=Decimal("0")))["t"]

    def by_method(self):
        return self.values("method").annotate(n=Count("id"),
usd=Sum("amount_usd")).order_by("method")


class CallLogQuerySet(models.QuerySet):                                  #
13
    def today(self):
        from .queries import today_bounds
        start, end = today_bounds()
        return self.filter(started_at__gte=start, started_at__lt=end)

    def long(self, minutes=10):
        return self.filter(duration_sec__gt=minutes * 60)

    def for_agent(self, user):
        return self.filter(agent=user)

    def latest_per_lead(self):
        return self.order_by("lead_id", "-started_at", "-
id").distinct("lead_id")


class TradeQuerySet(models.QuerySet):                                    #
14
```

```
    def open(self):
        return self.filter(closed_at__isnull=True)

    def closed(self):
        return self.filter(closed_at__isnull=False)

    def stale(self, days=7):
        return self.open().filter(opened_at__lt=timezone.now() -
timedelta(days=days))

    def for_symbol(self, symbol):
        return self.filter(symbol=symbol)

    def performance(self):
        return (self.closed().values("symbol")
                .annotate(trades=Count("id"), pnl=Sum("pnl_usd"),
wins=Count("id", filter=Q(pnl_usd__gt=0)))
                .order_by("-pnl"))


class SOPBreachQuerySet(models.QuerySet):                                  #
14
    def high(self):
        return self.filter(rule__severity=3)

    def recent(self, days=30):
        return self.filter(occurred_at__gte=timezone.now() -
timedelta(days=days))

    def for_desk(self, slug):
        return self.filter(agent__profile__desk__slug=slug)


class BrokerQuerySet(models.QuerySet):                                     #
23
    def downline_of(self, code, include_self=True):
        q = Q(parent__code=code) | Q(parent__parent__code=code)       # the
3-level Brf tree
        return self.filter(q | Q(code=code) if include_self else q)
```

```
# crm/models.py -- wiring (makemigrations + migrate)
from .querysets import (BrokerQuerySet, CallLogQuerySet, DepositQuerySet,
LeadManager, LeadQuerySet,
                        SOPBreachQuerySet, TradeQuerySet)


class Lead(models.Model):
    phone = models.CharField(max_length=20)               # unique=True
REMOVED -- see the constraint
    archived_at = models.DateTimeField(null=True, blank=True)

    objects = LeadManager()                    # first manager = the DEFAULT
manager
    all_objects = LeadQuerySet.as_manager()    # same methods, archived rows
included

    class Meta:
        constraints = [...,
                       models.UniqueConstraint(fields=["phone"],
condition=Q(archived_at__isnull=True),

name="lead_phone_unique_active")]          # 15

    def archive(self):
        self.archived_at = timezone.now()
        self.save(update_fields=["archived_at", "updated_at"])

    def restore(self):
        self.archived_at = None
        self.save(update_fields=["archived_at", "updated_at"])


# and on the others:
#   Deposit.objects = DepositQuerySet.as_manager()     CallLog.objects =
CallLogQuerySet.as_manager()
#   Trade.objects = TradeQuerySet.as_manager()         SOPBreach.objects =
SOPBreachQuerySet.as_manager()
#   Broker.objects = BrokerQuerySet.as_manager()
```

A. Drills

```
# 1
type(Lead.objects)        -> crm.querysets.LeadManager   the ENTRY POINT; not
itself a query
type(Lead.objects.all())  -> crm.querysets.LeadQuerySet  lazy, chainable,
evaluable
Only a manager can be a class attribute (Lead.objects) and own non-query
entry points
(create_lead). Only a QuerySet can be sliced, iterated, or chained after a
filter.

# 3  every order works -- each method returns a new QuerySet of the same
class
Lead.objects.filter(desk__slug="vip").open().order_by("-created_at").stale(3)
Lead.objects.stale(3).open().filter(desk__slug="vip")       # same SQL, same
rows

# 6  which manager?
Lead.objects        LeadManager -> archived HIDDEN
desk.leads          built from Lead's DEFAULT manager class -> hidden, with
your methods
call.lead           Lead._base_manager: a plain Manager by default ->
archived VISIBLE
                    (leave base_manager_name alone: following an FK must
never fail)
admin changelist    the default manager unless ModelAdmin.get_queryset() says
otherwise
dumpdata            the default manager; dumpdata --all uses the base manager

# 7
class LeadManager(...):
    use_in_migrations = True
Historical models in migrations then get this manager -- imported from
TODAY's code, so the
migration's behaviour can change when you edit the manager. Prefer plain
filters inside
migrations (Day 4, task 33); use this only for stable helpers.

# 9
desk = Desk.objects.get(slug="vip")
desk.leads.open().stale(3)       # WHERE desk_id = 3 AND archived_at IS NULL
AND status IN (...) AND ...
```

```
# 10
lead.is_open, lead.masked_phone()     model method        -- one row's own
data
leads that are open / stale           QuerySet method     -- WHICH rows
leads a user may see                  QuerySet method     -- WHICH rows
(security rule, one place)
create a lead with normalised phone   manager method      -- an entry point,
not a filter
approve a deposit + mark FTD          service (Day 10)    -- several models +
a transaction
desk dashboard numbers                report function     -- combines several
queries (task 22)
```

```
# 8
lead = Lead.objects.with_last_call().with_deposited().get(pk=7)
lead.last_call, lead.deposited       # {'outcome': ..., 'agent': ..., 'at':
...}, Decimal('1250.00')
```

B. Core

```
# 15 -- soft delete in use
lead = Lead.objects.get(pk=7)
lead.archive()
Lead.objects.filter(pk=7).exists()          # False
Lead.all_objects.get(pk=7).restore()        # back
# phone uniqueness now only counts ACTIVE leads: an archived lead's phone can
be reused (task 27)

# 16
agent = User.objects.get(username="agent01")
Lead.objects.visible_to(agent).count()                  # only agent01's
leads
Lead.objects.visible_to(User.objects.get(username="manager1")).values("desk__
slug").distinct()
# -> retention, conversion: the desks manager1 manages. One query either way;
Exists never
#    multiplies rows, so no distinct() is needed on the result.

# 17
Lead.objects.create_lead(name="  Sara Ali ", phone="050 123 4567", desk=desk)
# saved as 'Sara Ali' / '+971501234567'; a duplicate ACTIVE phone raises
```

```
ValidationError from
# full_clean() (the constraint's check), before any IntegrityError can happen

# 18
for lead in Lead.objects.open().with_recent_calls(3)[:25]:      # 2 queries
in total
    lead.recent_calls
```

19: `crm/queries.py` keeps the BUILDING BLOCKS — `day_start`, `today_bounds`, `search_q`, `QUEUE_PRIORITY`, `LATEST_CALL`, `LAST_CALL_JSON`, `APPROVED`, `DEPOSITED`, `FIRST_DEP` — and loses most functions: `open_leads()` is now `Lead.objects.open()`, `stale_open_leads()` is `Lead.objects.open().stale()`, `call_queue(agent)` is task 21's chain. Keep a function only when it combines several models (Day 9's `lead_detail`).

```
# 20 -- crm/tests.py
class LeadQuerySetTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=300, stdout=StringIO())
        U = get_user_model()
        cls.agent, cls.manager = U.objects.get(username="agent01"),
U.objects.get(username="manager1")

    def test_open_stale_matches_the_long_hand(self):
        cutoff = timezone.now() - timedelta(days=3)
        expected = Lead.objects.filter(status__in=["new",
"callback"]).filter(
            Q(last_contacted_at__lt=cutoff) |
Q(last_contacted_at__isnull=True))
        self.assertEqual(set(Lead.objects.open().stale(3)), set(expected))

    def test_agent_sees_only_own_leads(self):

self.assertFalse(Lead.objects.visible_to(self.agent).exclude(assigned_to=self
.agent).exists())

    def test_manager_sees_only_managed_desks(self):
        desks =
set(Lead.objects.visible_to(self.manager).values_list("desk__slug",
flat=True))
        self.assertEqual(desks, {"retention", "conversion"})
```

```
    def test_archived_lead_is_hidden_but_reachable(self):
        call = CallLog.objects.select_related("lead").first()
        call.lead.archive()
        self.assertFalse(Lead.objects.filter(pk=call.lead_id).exists())
        self.assertEqual(CallLog.objects.get(pk=call.pk).lead.pk,
call.lead_id)   # base manager
```

C. Nexora CRM

```
# 21 -- the worklist reads like the requirement
def worklist(agent, size=25):
    return list(Lead.objects.visible_to(agent).for_agent(agent).open()
                .queue_order().with_last_call().select_related("desk",
"broker")[:size])


# 22
from crm.queries import day_start


def desk_dashboard(slug):
    now = timezone.now()
    month_start = day_start(timezone.localdate().replace(day=1))
    leads = Lead.objects.for_desk(slug)
    return {
        "open": leads.open().count(),
        "stale": leads.open().stale(3).count(),
        "funded_this_month": leads.ftd_between(month_start, now).count(),
        "deposits_usd": Deposit.objects.approved().in_period(month_start,
now)

.filter(account__lead__desk__slug=slug).usd_total(),
    }
# 4 small queries, each self-explanatory. (One query with Count(...,
filter=Q(...)) per number
# is possible -- Day 6 -- when the dashboard is hot enough to matter.)

# 23
downline = Broker.objects.downline_of("BRF-0001")              # BRF-0001 and
2 levels below
Lead.objects.filter(broker__in=downline).funded().count()      # one query,
subquery inside
```

```
# 24, 25
SOPBreach.objects.high().recent(30).for_desk("conversion").select_related("ag
ent", "rule")
Trade.objects.open().stale(7).for_symbol("XAUUSD")             # stale()
already implies open()

# 26 -- one UPDATE
Lead.objects.filter(status="lost", updated_at__lt=timezone.now() -
timedelta(days=90)).archive()

# 27
old = Lead.all_objects.get(pk=7)                               # archived
Lead.objects.create_lead(name="New Owner", phone=old.phone, desk=old.desk)
# allowed: old is archived
old.restore()
# IntegrityError: duplicate key value violates unique constraint
"lead_phone_unique_active"
# -> restoring must check first:
Lead.objects.filter(phone=old.phone).exists()
```

```
# 28 -- crm/admin.py
class ArchivedFilter(admin.SimpleListFilter):
    title = "archived"
    parameter_name = "archived"

    def lookups(self, request, model_admin):
        return [("no", "Active"), ("yes", "Archived")]

    def queryset(self, request, qs):
        if self.value() == "yes":
            return qs.filter(archived_at__isnull=False)
        if self.value() == "no":
            return qs.filter(archived_at__isnull=True)
        return qs


@admin.register(Lead)
class LeadAdmin(admin.ModelAdmin):
    list_filter = (ArchivedFilter, "status", "desk")
```

```
    def get_queryset(self, request):
        return Lead.all_objects.select_related("desk", "broker",
"assigned_to")
```

29: rewrite each report to start from the vocabulary — `Deposit.objects.approved().in_period(start, end)` in `commission_statement`, `Trade.objects.performance()` instead of `symbol_performance`'s own filters, `Lead.objects.funded()` wherever "has an approved deposit" appears. The day Finance redefines "approved" (say, excluding chargebacks), one method changes and every report follows. 30: run the whole suite — the Day 5–9 tests now exercise the new methods underneath — then commit `orm day 11`.

D. Challenge

```
# 31  after lead.archive()
VANISHES   Lead.objects..., desk.leads, broker.leads, the admin (unless
overridden), dumpdata,
           every report that STARTS from Lead.objects
STAYS      Lead.all_objects, call.lead / account.lead (base manager), and
every query that
           starts from ANOTHER table and joins to leads:
Deposit.objects.filter(account__lead__...)
           does NOT apply Lead's manager -- archived leads' deposits stay in
revenue.
Decide per report and write it down: account__lead__archived_at__isnull=True
where it matters.

# 32  def stale(self): return list(self.filter(...))
Callers lose chaining (.open(), .count(), pagination, select_related), the
query runs at once,
and related managers stop composing. Methods return QuerySets; only
explicitly TERMINAL ones
(usd_total) return values -- and their names say so.

# 33  def count(self): ...   /   def update(self): ...
You've replaced methods Django itself calls: Paginator, the admin, exists
checks, update(),
delete() cascades. Bugs surface far away. Never shadow QuerySet's own names.

# 34  get_queryset() = super().get_queryset().select_related("desk",
"broker").annotate(n=Count("calls"))
```

```
Lead.objects.count() becomes SELECT COUNT(*) FROM (SELECT ... JOIN ... GROUP
BY ...) subquery,
and every .exists(), .update() and admin page pays for the JOIN and GROUP BY.
Keep default
managers minimal; put extras behind named methods the caller opts into.

# 35  plain unique=True on phone + a filtering default manager
ModelForm validation checks uniqueness through the DEFAULT manager, which
hides archived
leads -> "valid" -> save() -> IntegrityError -> a 500 page. The conditional
constraint makes
the database rule and the validation rule the same rule.

# 36  RunPython: Lead.objects.open()
AttributeError: 'Manager' object has no attribute 'open'
Historical models get a plain Manager unless use_in_migrations = True (task
7).

# 37  managers declared on an abstract base model
Every child inherits them; the first manager in the MRO is the child's
default. A child that
declares its own `objects` overrides the inherited one -- check
_default_manager when in doubt.

# 38
Lead.objects.open().union(Lead.objects.funded()).stale(3)
NotSupportedError: Calling QuerySet.filter() after union() is not supported.
Your methods are filters underneath. Combine with | on QuerySets of the same
model instead:
  Lead.objects.open().stale(3) | Lead.objects.funded().stale(3)
```

```
# 39
def with_everything(self):
    return (self.with_last_call().with_deposited().with_ftd()
            .select_related("desk", "broker__parent",
"assigned_to__profile").with_recent_calls(5))

t = time.perf_counter(); Lead.objects.with_everything().count();
print(time.perf_counter() - t)
t = time.perf_counter(); Lead.objects.count(); print(time.perf_counter() - t)
# the first keeps the subquery annotations (Django can't prove they don't
matter) and is many
# times slower. Methods are cheap to chain -- so chain what THIS caller
```

```
needs, nothing more.


# 40
def has_eur_account(self):
    return self.filter(accounts__currency="EUR")

def has_big_deposit(self):
    return self.filter(accounts__deposits__amount_usd__gt=1000)

Lead.objects.has_eur_account().has_big_deposit()
# two filter() calls = two separate JOINs to accounts (Day 3, task 34): "has
a EUR account" AND
# "has SOME account with a big deposit" -- maybe a USD one -- and leads
repeat once per match.
# Write such methods with Exists(): no duplicates, and the meaning can't
drift when chained.
# If the rule is "a big deposit INTO a EUR account", that's one method with
one Exists.
```

Day 12 — Raw SQL and PostgreSQL features

Goal: know where the ORM ends and use PostgreSQL directly there — safely. Raw queries with parameters, `generate_series`, recursive CTEs, `LATERAL`, `INSERT ... SELECT`, `COPY`; and the PostgreSQL features Django wraps: arrays, JSON paths, full-text search, trigrams, ranges and exclusion constraints. You can also call the functions you write in your PostgreSQL program from Django.

```
Reach for raw SQL when the ORM can't say it (recursive CTE, LATERAL,
generate_series,
INSERT ... SELECT, COPY, ordered-set aggregates) -- not because SQL feels
more familiar.
The one unbreakable rule: VALUES go in params, never into the SQL string.
  cur.execute("SELECT ... WHERE phone = %s", [phone])        safe
  cur.execute(f"SELECT ... WHERE phone = '{phone}'")         SQL injection
```

Setup: `django.contrib.postgres` is in `INSTALLED_APPS` (Day 6). The extensions below are installed by migrations as you go.

A. Drills (1–10) — raw SQL and PostgreSQL types

- [ ] **1.** `Lead.objects.raw()`: leads with their call count, mapped to Lead objects with an extra attribute.
- [ ] **2.** `connection.cursor()`: run a query with parameters; turn rows into dicts.
- [ ] **3.** Build a search with an f-string, then attack it with a crafted name. Fix it with parameters; quote an identifier safely.
- [ ] **4.** `RawSQL` inside `annotate()` and `filter()`. Why is `extra()` the last resort?
- [ ] **5.** `ArrayField`: campaign tags on leads; `contains`, `overlap`, `len`, index lookups.
- [ ] **6.** JSON deeper: nested keys, `has_keys`, `has_any_keys`, `KT()` for text values.
- [ ] **7.** Full-text search over call notes: `SearchVector`, `SearchQuery`, `SearchRank`.
- [ ] **8.** Trigram similarity on lead names: the extension, `TrigramSimilarity`, `trigram_similar`.
- [ ] **9.** `DateTimeRangeField`: agent shifts; `contains` and `overlap` lookups.
- [ ] **10.** Call a PostgreSQL function from Django two ways: as a `Func` in a QuerySet, and with a cursor.

B. Core (11–20) — SQL the ORM can't write

- [ ] **11.** `crm/sql.py`: `fetch_dicts()` and `fetch_value()` helpers used for the rest of the day.
- [ ] **12.** FTDs per day for 30 days with zero days included, using `generate_series`.
- [ ] **13.** The whole Brf downline at ANY depth with `WITH RECURSIVE`, used inside a normal QuerySet.
- [ ] **14.** The 3 latest calls of each lead on a page with `LATERAL`.
- [ ] **15.** Day 9's daily snapshot as one `INSERT ... SELECT ... ON CONFLICT`.
- [ ] **16.** A ranked search box: trigram on lead names plus full-text on call notes.
- [ ] **17.** A stored search vector with a GIN index, kept current by a trigger.
- [ ] **18.** A GIN index on tags; check the plan of an `overlap` query.
- [ ] **19.** `ExclusionConstraint`: an agent can't have overlapping shifts.
- [ ] **20.** A `Percentile` aggregate (`percentile_cont`) usable in `annotate()`.

C. Nexora CRM (21–30) — PostgreSQL earns its keep

- [ ] **21.** Approved deposit USD per top-level broker across its whole downline (recursive CTE).
- [ ] **22.** Likely duplicate leads: similar names on the same desk.
- [ ] **23.** Compliance search: call notes mentioning guaranteed returns or "risk-free".
- [ ] **24.** Leads and FTDs per campaign tag (`unnest`).
- [ ] **25.** Median and 90th-percentile approved deposit per desk.
- [ ] **26.** Agents on shift right now, and calls made outside the caller's shift.
- [ ] **27.** KYC passports expiring in the next 60 days (JSON text → date).
- [ ] **28.** Export 180,000 calls to CSV with `COPY ... TO STDOUT`. Compare with Day 9's iterator export.
- [ ] **29.** `fn_lead_score(lead_id)` as a SQL function in a migration, used to order the call queue.
- [ ] **30.** Tests for the raw helpers and constraints. Commit `orm day 12`.

D. Challenge (31–40) — raw SQL traps

- [ ] **31.** `raw()` without the primary key in the SELECT. Copy the error.

- [ ] **32.** Task 3's injection: what exactly reaches PostgreSQL in each version?
- [ ] **33.** `LIKE` with parameters: where does the `%` go? And a literal `%` in the SQL text?
- [ ] **34.** Parameterise a table name. Why can't you, and what do you do instead?
- [ ] **35.** Group raw SQL by day without `AT TIME ZONE`. Which days are wrong?
- [ ] **36.** What Python types come back from a cursor for numeric, timestamptz, jsonb and arrays?
- [ ] **37.** `RawSQL` in `order_by()` built from a query-string parameter. What's the risk?
- [ ] **38.** Raw SQL vs your managers: what does a raw query ignore?
- [ ] **39.** `update(notes=...)` leaves the stored search vector stale — unless? Prove it.
- [ ] **40.** A cursor opened without `with`. What leaks, and when does it matter?

Solutions

Model and migration additions for today:

```
# the migration that adds these must create the extensions FIRST -- put them
at the top of
# its operations list (or in an earlier migration):
from django.contrib.postgres.operations import BtreeGistExtension,
TrigramExtension

operations = [TrigramExtension(), BtreeGistExtension(), ...]
# pg_trgm for gin_trgm_ops and similarity(); btree_gist so an exclusion
constraint can use "="
# on a plain integer column (agent_id). Creating extensions needs a superuser
or the
# database owner on most servers.
```

```
# crm/models.py
from django.contrib.postgres.constraints import ExclusionConstraint
from django.contrib.postgres.fields import ArrayField, DateTimeRangeField,
RangeOperators
from django.contrib.postgres.indexes import GinIndex
from django.contrib.postgres.search import SearchVectorField


class Lead(models.Model):
    tags = ArrayField(models.CharField(max_length=30), default=list,
blank=True)        # 5
```

```
    class Meta:
        indexes = [...,
                   GinIndex(fields=["tags"], name="lead_tags_gin"),
# 18
                   GinIndex(fields=["name"], opclasses=["gin_trgm_ops"],
name="lead_name_trgm")]  # 8


class CallLog(models.Model):
    notes = models.TextField(blank=True)
# 7
    search = SearchVectorField(null=True)
# 17

    class Meta:
        indexes = [..., GinIndex(fields=["search"], name="call_search_gin")]


class AgentShift(models.Model):
# 9, 19
    agent = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name="shifts")
    period = DateTimeRangeField()

    class Meta:
        constraints = [ExclusionConstraint(
            name="no_overlapping_shifts",
            expressions=[("agent", RangeOperators.EQUAL), ("period",
RangeOperators.OVERLAPS)])]
```

A. Drills

1–4.

```
# 1 -- the SELECT must include the primary key; extra columns become
attributes
leads = Lead.objects.raw("""
    SELECT l.id, l.name, l.phone, COUNT(c.id) AS n_calls
    FROM crm_lead l LEFT JOIN crm_calllog c ON c.lead_id = l.id
    WHERE l.desk_id = %s
    GROUP BY l.id ORDER BY n_calls DESC LIMIT 10""", [desk.pk])
for lead in leads:
```

```
    lead.name, lead.n_calls          # Lead objects; columns you didn't
select load lazily (1 query each)


# 2 -- crm/sql.py
from django.db import connection


def fetch_dicts(sql, params=None):
    """Run SQL with parameters; return a list of dicts."""
    with connection.cursor() as cur:
        cur.execute(sql, params or [])
        cols = [col[0] for col in cur.description]
        return [dict(zip(cols, row)) for row in cur.fetchall()]


def fetch_value(sql, params=None):
    """Run SQL; return the first column of the first row (or None)."""
    with connection.cursor() as cur:
        cur.execute(sql, params or [])
        row = cur.fetchone()
        return row[0] if row else None
```

```
# 3
name = "x' OR '1'='1"
fetch_dicts(f"SELECT id FROM crm_lead WHERE name = '{name}'")    # returns
EVERY lead -- injected
fetch_dicts("SELECT id FROM crm_lead WHERE name = %s", [name])     # []: the
value stays a value

# identifiers (table / column names) can't be parameters -- compose them with
psycopg.sql
from psycopg import sql as pgsql

query = pgsql.SQL("SELECT COUNT(*) FROM
{}").format(pgsql.Identifier("crm_lead"))
with connection.cursor() as cur:
    cur.execute(query.as_string(cur.connection))    # the raw psycopg
connection quotes it
    cur.fetchone()
# better still: choose from a fixed allow-list ({"leads": "crm_lead", ...}
[key]) -- never user text
```

```
# 4
from django.db.models.expressions import RawSQL

Lead.objects.annotate(digits=RawSQL("regexp_replace(phone, '[^0-9]', '',
'g')", []))
Lead.objects.filter(id__in=RawSQL("SELECT lead_id FROM crm_calllog WHERE
duration_sec > %s", [600]))
# RawSQL keeps params separate and composes with the ORM. extra() predates
it, has odd
# rules, and is deprecated in spirit -- Django's docs ask you to file a
ticket if you need it.
```

5–10.

```
# 5
Lead.objects.filter(pk__in=[1, 2]).update(tags=["autumn-promo", "webinar"])
Lead.objects.filter(tags__contains=["autumn-promo"])            # tags @>
ARRAY['autumn-promo']
Lead.objects.filter(tags__overlap=["autumn-promo", "referral"]) # tags &&
ARRAY[...]
Lead.objects.filter(tags__len=0)                                # untagged
Lead.objects.filter(tags__0="webinar")                          # first
element

# 6
from django.db.models.fields.json import KT

Lead.objects.filter(kyc__address__city="Dubai")                  # nested key
Lead.objects.filter(kyc__has_keys=["passport_no", "passport_expiry"])      #
all of them
Lead.objects.filter(kyc__has_any_keys=["passport_no", "emirates_id"])      #
any of them
Lead.objects.annotate(nationality=KT("kyc__nationality"))       # ->> : text,
not JSON (Django 4.2+)

# 7
from django.contrib.postgres.search import SearchQuery, SearchRank,
SearchVector

vector = SearchVector("notes", config="english")
query = SearchQuery("guaranteed profit", search_type="websearch",
config="english")
```

```
(CallLog.objects.annotate(rank=SearchRank(vector,
query)).filter(rank__gt=0).order_by("-rank"))
# 'english' stems words: "guaranteed" also finds "guarantee", "guarantees"

# 8
from django.contrib.postgres.search import TrigramSimilarity

Lead.objects.annotate(sim=TrigramSimilarity("name", "omar
khaled")).filter(sim__gt=0.3).order_by("-sim")
Lead.objects.filter(name__trigram_similar="omar khaled")    # the % operator,
threshold 0.3; uses lead_name_trgm
```

```
# 9
from datetime import timedelta

from django.db.backends.postgresql.psycopg_any import DateTimeTZRange

start = timezone.now().replace(minute=0, second=0, microsecond=0)
AgentShift.objects.create(agent=agent, period=DateTimeTZRange(start, start +
timedelta(hours=8)))   # [start, end)
AgentShift.objects.filter(period__contains=timezone.now())                 #
on shift now
AgentShift.objects.filter(period__overlap=DateTimeTZRange(start, start +
timedelta(hours=1)))


# 10 -- a function from your PostgreSQL program, called from Django (it's
created in task 29)
from django.db.models import Func, IntegerField


class LeadScore(Func):
    function = "fn_lead_score"
    output_field = IntegerField()


Lead.objects.open().annotate(score=LeadScore("id")).order_by("-score")   # in
a QuerySet
fetch_value("SELECT fn_lead_score(%s)", [7])                             #
with a cursor
# a PROCEDURE: with connection.cursor() as cur: cur.execute("CALL
sp_recalc_scores(%s)", [desk_id])
```

B. Core

12–15 — SQL only PostgreSQL can say.

```
# crm/sql.py (continued)
FIRSTS_CTE = """firsts AS (
    SELECT a.lead_id, MIN(d.created_at) AS ftd_at
    FROM nexora_deposits d JOIN crm_tradingaccount a ON a.id = d.account_id
    WHERE d.status = 'approved'
    GROUP BY a.lead_id)"""


def ftd_per_day_filled(start_day, end_day):                                  #
12
    """[{day, ftd}] for every Dubai day from start_day to end_day
inclusive."""
    return fetch_dicts(f"""
        WITH {FIRSTS_CTE},
        days AS (SELECT generate_series(%(start)s::date, %(end)s::date,
interval '1 day')::date AS day)
        SELECT days.day, COUNT(firsts.lead_id) AS ftd
        FROM days
        LEFT JOIN firsts ON (firsts.ftd_at AT TIME ZONE 'Asia/Dubai')::date =
days.day
        GROUP BY days.day ORDER BY days.day""", {"start": start_day, "end":
end_day})
# the f-string only inserts OUR constant CTE text; every value is still a %
(name)s parameter


DOWNLINE_IDS = """
    WITH RECURSIVE downline AS (
        SELECT id, 0 AS depth FROM crm_broker WHERE code = %s
        UNION ALL
        SELECT b.id, d.depth + 1 FROM crm_broker b JOIN downline d ON
b.parent_id = d.id
        WHERE d.depth < 20                          -- guard against a
referral loop (Day 3, task 37)
    )
    SELECT id FROM downline"""


def downline_leads(code):                                                    #
```

```
13
    """Leads under a broker at ANY depth -- one query, still a lazy
QuerySet."""
    return Lead.objects.filter(broker_id__in=RawSQL(DOWNLINE_IDS, [code]))


def latest_calls_for(lead_ids, n=3):                                    #
14
    return fetch_dicts("""
        SELECT l.id AS lead_id, l.name, c.started_at, c.outcome
        FROM crm_lead l
        CROSS JOIN LATERAL (
            SELECT started_at, outcome FROM crm_calllog
            WHERE lead_id = l.id ORDER BY started_at DESC, id DESC LIMIT %s
        ) c
        WHERE l.id = ANY(%s)
        ORDER BY l.id, c.started_at DESC""", [n, list(lead_ids)])
# LATERAL = "for each lead, run this subquery" -- each one an index probe on
call_lead_latest_cov_idx.
# (A Python list becomes a PostgreSQL array for ANY(...).)


def snapshot_sql(day):                                                  #
15
    with connection.cursor() as cur:
        cur.execute(f"""
            WITH {FIRSTS_CTE}
            INSERT INTO reports_dailysnapshot (desk_id, day, ftd_count)
            SELECT d.id, %(day)s, COUNT(f.lead_id)
            FROM crm_desk d
            LEFT JOIN crm_lead l ON l.desk_id = d.id
            LEFT JOIN firsts f ON f.lead_id = l.id
                              AND (f.ftd_at AT TIME ZONE 'Asia/Dubai')::date
= %(day)s
            GROUP BY d.id
            ON CONFLICT (desk_id, day) DO UPDATE SET ftd_count =
EXCLUDED.ftd_count""", {"day": day})
        return cur.rowcount
# no rows travel through Python; Day 9's version fetched the counts and sent
them back
```

16–20.

```
# 16 -- crm/queries.py: one search box, two engines
def search_everything(text, limit=20):
    leads = (Lead.objects.annotate(score=TrigramSimilarity("name", text))
             .filter(score__gt=0.25).order_by("-score")[:limit])
    calls = (CallLog.objects.filter(search=SearchQuery(text,
search_type="websearch", config="english"))
             .annotate(score=SearchRank("search", SearchQuery(text,
search_type="websearch", config="english")))
             .select_related("lead").order_by("-score")[:limit])
    return {"leads": list(leads), "calls": list(calls)}
# trigram tolerates typos in NAMES ("Omar Khaled" ~ "Omar Khalid"); full-text
understands WORDS
# in notes. Their scores aren't comparable, so show two lists rather than
merging them.
```

```
# 17 -- fill once, then let a trigger keep it current (a RunSQL migration)
CallLog.objects.update(search=SearchVector("notes", config="english"))

migrations.RunSQL(
    sql="""CREATE TRIGGER calllog_search_update
           BEFORE INSERT OR UPDATE OF notes ON crm_calllog
           FOR EACH ROW EXECUTE FUNCTION tsvector_update_trigger(search,
'pg_catalog.english', notes)""",
    reverse_sql="DROP TRIGGER calllog_search_update ON crm_calllog",
)
# the trigger fires for save(), update(), bulk_create, raw SQL -- every path
(task 39).
# (A GeneratedField over to_tsvector('english', notes) is the declarative
alternative to try on
#  recent Django versions: the config makes the expression immutable, which
PostgreSQL requires.)


# 18
explain(Lead.objects.filter(tags__overlap=["autumn-promo"]))
#   Bitmap Heap Scan on crm_lead -> Bitmap Index Scan on lead_tags_gin (tags
&& '{autumn-promo}')
# (on a table where few rows match; with most rows tagged, a Seq Scan is the
right plan)


# 19
```

```
from django.db import IntegrityError

AgentShift.objects.create(agent=agent, period=DateTimeTZRange(start, start +
timedelta(hours=8)))
AgentShift.objects.create(agent=agent, period=DateTimeTZRange(start +
timedelta(hours=4),
                                                              start +
timedelta(hours=12)))
# IntegrityError: conflicting key value violates exclusion constraint
"no_overlapping_shifts"
# a UNIQUE constraint compares equality; an EXCLUSION constraint compares
with any operator --
# here "same agent AND overlapping period" may never be true for two rows.
Two agents may overlap.


# 20 -- crm/expressions.py
from django.db.models import Aggregate, FloatField


class Percentile(Aggregate):
    """percentile_cont(p) WITHIN GROUP (ORDER BY expr). p is OUR constant,
never user input."""
    function = "PERCENTILE_CONT"
    name = "percentile"
    output_field = FloatField()
    template = "%(function)s(%(p)s) WITHIN GROUP (ORDER BY %(expressions)s)"

    def __init__(self, expression, p, **extra):
        if not 0 <= float(p) <= 1:
            raise ValueError("p must be between 0 and 1")
        super().__init__(expression, p=float(p), **extra)
```

C. Nexora CRM

```
# 21 -- revenue per top-level broker, whole downline, any depth
fetch_dicts("""
    WITH RECURSIVE tree AS (
        SELECT id, id AS root_id, code AS root_code, 0 AS depth FROM
crm_broker WHERE parent_id IS NULL
        UNION ALL
        SELECT b.id, t.root_id, t.root_code, t.depth + 1
```

```
        FROM crm_broker b JOIN tree t ON b.parent_id = t.id WHERE t.depth <
20
    )
    SELECT t.root_code, COUNT(DISTINCT l.id) AS leads,
COALESCE(SUM(d.amount_usd), 0) AS revenue_usd
    FROM tree t
    JOIN crm_lead l ON l.broker_id = t.id
    LEFT JOIN crm_tradingaccount a ON a.lead_id = l.id
    LEFT JOIN nexora_deposits d ON d.account_id = a.id AND d.status =
'approved'
    GROUP BY t.root_code ORDER BY revenue_usd DESC""")
# Day 6's Coalesce trick needed a fixed depth; the recursive CTE doesn't.
COUNT(DISTINCT ...)
# because deposits multiply lead rows -- SUM is safe here: each deposit row
appears once.


# 22 -- similar names on the same desk (a self-join on the trigram operator)
fetch_dicts("""
    SELECT a.id AS lead_a, b.id AS lead_b, a.name, b.name AS other_name,
           similarity(a.name, b.name) AS sim
    FROM crm_lead a JOIN crm_lead b
      ON a.desk_id = b.desk_id AND a.id < b.id AND a.name %% b.name
    WHERE a.archived_at IS NULL AND b.archived_at IS NULL
    ORDER BY sim DESC LIMIT 100""")
# %% is a literal % (the trigram operator) -- in Django SQL with params, %
must be doubled (task 33)


# 23
query = SearchQuery('"guaranteed returns" OR "risk-free" OR "guaranteed
profit"',
                    search_type="websearch", config="english")
CallLog.objects.filter(search=query).select_related("agent",
"lead").order_by("-started_at")

# 24
fetch_dicts("""
    SELECT tag, COUNT(*) AS leads, COUNT(*) FILTER (WHERE status = 'ftd') AS
ftd
    FROM crm_lead, unnest(tags) AS tag
    WHERE archived_at IS NULL
    GROUP BY tag ORDER BY leads DESC""")
```

```
# 25
from crm.expressions import Percentile

(Deposit.objects.approved().values(desk_name=F("account__lead__desk__name"))
 .annotate(median=Percentile("amount_usd", 0.5), p90=Percentile("amount_usd",
0.9), n=Count("id"))
 .order_by("desk_name"))
```

```
# 26
now = timezone.now()
AgentShift.objects.filter(period__contains=now).select_related("agent")
# on shift now
on_shift = AgentShift.objects.filter(agent=OuterRef("agent"),
period__contains=OuterRef("started_at"))
CallLog.objects.filter(~Exists(on_shift))
# calls outside a shift

# 27
from datetime import date

from django.db.models import DateField
from django.db.models.functions import Cast

soon = timezone.localdate() + timedelta(days=60)
(Lead.objects.filter(kyc__passport_expiry__regex=r"^\d{4}-\d{2}-\d{2}$")
# only well-formed dates
 .annotate(expiry=Cast(KT("kyc__passport_expiry"), DateField()))
 .filter(expiry__lte=soon).order_by("expiry"))
# without the regex guard, ONE malformed value makes the whole query fail
with a DataError

# 28 -- COPY streams straight from PostgreSQL to the file
with connection.cursor() as cur, open("calls.csv", "wb") as fh:
    with cur.copy("""COPY (SELECT id, lead_id, agent_id, started_at,
duration_sec, outcome
                          FROM crm_calllog ORDER BY id) TO STDOUT WITH (FORMAT
csv, HEADER)""") as copy:
        for block in copy:
            fh.write(block)
```

```
# psycopg 3's copy API. Usually several times faster than Day 9's iterator +
csv.writer: no
# Python objects per row at all. (COPY ... FROM STDIN is the bulk-import
twin.)
```

```
# 29 -- a migration owns the function
migrations.RunSQL(
    sql="""
    CREATE OR REPLACE FUNCTION fn_lead_score(p_lead_id bigint) RETURNS
integer
    LANGUAGE sql STABLE AS $$
        SELECT CASE status WHEN 'callback' THEN 50 WHEN 'new' THEN 30 ELSE 0
END
             + CASE WHEN last_contacted_at IS NULL THEN 25
                    WHEN last_contacted_at < now() - interval '3 days' THEN
15 ELSE 0 END
        FROM crm_lead WHERE id = p_lead_id
    $$;""",
    reverse_sql="DROP FUNCTION IF EXISTS fn_lead_score(bigint);",
)
# then: Lead.objects.open().annotate(score=LeadScore("id")).order_by("-
score", "created_at")
# Put a rule in SQL when other systems (BI, the dialer) must use the SAME
definition; keep it in
# Python (Day 7's SCORE) when only Django needs it. Two copies of one rule is
the thing to avoid.
```

```
# 30 -- crm/tests.py
from .sql import downline_leads, fetch_dicts, ftd_per_day_filled


class RawSqlTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=300, stdout=StringIO())

    def test_filled_series_has_every_day(self):
        today = timezone.localdate()
        rows = ftd_per_day_filled(today - timedelta(days=29), today)
        self.assertEqual(len(rows), 30)

    def test_downline_any_depth_matches_fixed_depth(self):
```

```
        top = Broker.objects.filter(parent__isnull=True).first()
        self.assertEqual(set(downline_leads(top.code)),

set(Lead.objects.filter(broker__in=Broker.objects.downline_of(top.code))))

    def test_shifts_cannot_overlap(self):
        agent = get_user_model().objects.get(username="agent01")
        start = timezone.now()
        AgentShift.objects.create(agent=agent, period=DateTimeTZRange(start,
start + timedelta(hours=8)))
        with self.assertRaises(IntegrityError), transaction.atomic():
            AgentShift.objects.create(agent=agent,
period=DateTimeTZRange(start + timedelta(hours=1),

start + timedelta(hours=9)))
```

```
python manage.py test crm && git add -A && git commit -m "orm day 12"
```

D. Challenge

```
# 31
Lead.objects.raw("SELECT name FROM crm_lead")
FieldDoesNotExist: Raw query must include the primary key
(raw() builds model instances, and an instance without its pk can't be one.)

# 32
f-string:   ... WHERE name = 'x' OR '1'='1'      -- the input REWROTE the
query
parameters: ... WHERE name = $1   with $1 = "x' OR '1'='1"   -- the input
stays one string value
The driver sends the value separately (or escapes it); it can never become
SQL.

# 33
cur.execute("SELECT ... WHERE name LIKE %s", [f"%{text}%"])      -- the %
belongs in the VALUE
"WHERE name LIKE '%%' || %s || '%%'"                              -- a
literal % in the SQL text
                                                                    must be
written %% when params are used
(text containing % or _ still acts as a wildcard -- escape it, as the ORM's
contains does.)
```

```
# 34
Placeholders stand for VALUES only; a table or column name is part of the
query's structure.
Use a fixed allow-list mapping, or psycopg.sql.Identifier (task 3). Never
user text in SQL.

# 35
GROUP BY created_at::date in a session whose time zone is UTC (Django's
connections are)
-> every row created 00:00-04:00 Dubai time lands on the PREVIOUS day.
Write (created_at AT TIME ZONE 'Asia/Dubai')::date, exactly as tasks 12 and
15 do.

# 36
numeric -> Decimal   timestamptz -> aware datetime (UTC)   jsonb -> dict /
list   text[] -> list
bigint -> int   date -> date   NULL -> None

# 37
qs.order_by(RawSQL(request.GET["sort"], []))
The client now writes SQL into your ORDER BY -- injection, plus sorting by
any column (or
expression) they like. Map allowed keys to fields: {"newest": "-created_at",
"name": "name"}.

# 38
Raw SQL skips managers and model logic: archived leads (Day 11) come back,
scoped access
(visible_to) is gone, save() and signals don't run, field conversions don't
apply.
Every raw query must restate the rules it needs: archived_at IS NULL, desk
scoping, ...

# 39
Without the trigger: CallLog.objects.filter(pk=7).update(notes="guaranteed
profit")
-> the search column keeps the OLD vector; task 23's search can't find call
7.
With task 17's trigger: BEFORE UPDATE OF notes recomputes it -- for update(),
save(),
bulk_update and raw SQL alike. That's the argument for doing it in the
database.
```

```
# 40
cur = connection.cursor(); cur.execute(...)       -- never closed
The cursor (and a server-side cursor's resources) live until garbage
collection or the
connection closes. Inside a long-running worker or a loop that adds up.
`with` closes it
even when an exception is raised -- every helper above uses it.
```

Day 13 — Model design

Goal: design models that stay right as Nexora grows — shared bases, the three kinds of inheritance and when each fits, validation at the right layer, side effects in services instead of hidden signals, a custom field that makes the phone rule impossible to skip, and a data dictionary analysts can read.

```
abstract base     shared COLUMNS, no table of its own
TimeStampedModel
multi-table       a child TABLE joined 1:1 to the parent       rarely worth
the JOIN on every query
proxy             same table, different PYTHON behaviour      VipLead: own
manager, admin, methods
one-to-one        an optional extension table you control     BrokerProfile,
KycProfile (Day 4)
Where rules live: DB constraint (always) > custom field (every write path) >
service (the
business action) > clean() / forms (friendly messages) > signals (decoupled
reactions only)
```

A. Drills (1–10) — design tools

- [ ] **1.** An abstract `TimeStampedModel`; inherit it and confirm with `sqlmigrate` that no new table appears.
- [ ] **2.** `Meta` inheritance from an abstract base; `%(class)s` and `%(app_label)s` in `related_name` and index names.
- [ ] **3.** Multi-table inheritance: `ExtendedLead(Lead)`. Read the table it creates and the JOIN in every query. Then remove it.
- [ ] **4.** A proxy model `VipLead` with its own manager and `Meta.verbose_name`.
- [ ] **5.** `full_clean()` step by step: `clean_fields`, `clean`, `validate_unique`, `validate_constraints`.
- [ ] **6.** The `save()` override contract: `*args, **kwargs`, `super()`, and respecting `update_fields`.
- [ ] **7.** A `post_save` receiver registered in `AppConfig.ready()`. Which writes fire it and which don't?
- [ ] **8.** `@property` vs `@cached_property` vs an annotation for a computed value.

- [ ] **9.** A custom `PhoneNumberField` that normalises on every save AND every query.
- [ ] **10.** `GenericForeignKey`: a `Note` that can attach to a lead or a broker.

B. Core (11–20) — a design that holds

- [ ] **11.** Apply `TimeStampedModel` to the existing models without changing a single column.
- [ ] **12.** Status as `TextChoices` or as a lookup table? Decide for leads, deposits and SOP rules, with reasons.
- [ ] **13.** `LeadHistory`: record field changes from a service, not a signal.
- [ ] **14.** Model-level rules: a deposit's currency must match its account; a trade can't close before it opens.
- [ ] **15.** Replace `Lead.save()`'s phone logic with `PhoneNumberField`; migrate.
- [ ] **16.** Make approved deposits immutable (amount, currency, fx rate) — in Python and in the database.
- [ ] **17.** A custom signal `deposit_approved`, sent on commit by the service, received by the `reports` app.
- [ ] **18.** Custom permissions: `export_leads`, `view_kyc`; check them in code.
- [ ] **19.** An unmanaged model over a SQL view (`v_lead_summary`).
- [ ] **20.** Broker extras (IB agreement date, payout email): multi-table, proxy or one-to-one? Build the right one.

C. Nexora CRM (21–30) — design in practice

- [ ] **21.** The VIP desk: `VipLead` proxy with its own admin and default ordering.
- [ ] **22.** Notes on leads and brokers: create, list a lead's notes, search note text.
- [ ] **23.** A lead timeline: history, calls, transfers and notes merged in time order.
- [ ] **24.** Validate deposits at the service boundary with friendly messages; keep the constraint as the backstop.
- [ ] **25.** `lead.can_delete()` and a guarded delete in the service.
- [ ] **26.** A custom system check: no `FloatField` anywhere in `crm`.
- [ ] **27.** `__str__` and `__repr__` that make logs readable without leaking phone numbers.
- [ ] **28.** A data dictionary command: table, column, type, nullability and comment for every model.
- [ ] **29.** Decide where each of 8 rules lives (DB, field, service, clean, signal) and move them.

- [ ] **30.** Tests pass. Commit `orm day 13`.

D. Challenge (31–40) — design traps

- [ ] **31.** A FK with a fixed `related_name` on an abstract base used by two models. Copy the error.
- [ ] **32.** Multi-table inheritance: what `Lead.objects.all()` returns, and what deleting a child deletes.
- [ ] **33.** Save a `VipLead` with a non-VIP desk. Did the proxy stop you?
- [ ] **34.** An audit `post_save` receiver plus `QuerySet.update()`. What's missing from the audit?
- [ ] **35.** A `post_save` receiver that calls `instance.save()`. What happens?
- [ ] **36.** `save()` normalises the phone but the caller passes `update_fields=["status"]`. What's in the database?
- [ ] **37.** `full_clean()` inside every `save()`. List the costs.
- [ ] **38.** A broker with notes is deleted. What happens to the notes — with and without `GenericRelation`?
- [ ] **39.** A custom field with a new `__init__` argument and no `deconstruct()`. What does `makemigrations` do?
- [ ] **40.** A `cached_property` read, then `refresh_from_db()`, then read again. Which value do you get?

Solutions

A. Drills

1–4 — the inheritance options.

```
# crm/models.py
class TimeStampedModel(models.Model):                                    #
1
    created_at = models.DateTimeField(default=timezone.now)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True                    # no table: the fields are COPIED
into each child
```

```
        get_latest_by = "created_at"


class Note(TimeStampedModel):
    ...
# sqlmigrate: CREATE TABLE crm_note (..., created_at ..., updated_at ...) --
no crm_timestampedmodel


# 2 -- placeholders keep inherited names unique per child
class Owned(models.Model):
    owner = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.PROTECT,
                              related_name="%(app_label)s_%(class)s_owned")
# crm_note_owned, ...

    class Meta:
        abstract = True
        indexes = [models.Index(fields=["owner"], name="%(app_label)s_%
(class)s_owner_idx")]
# a child's own Meta must say `class Meta(Owned.Meta):` to keep the parent's
Meta options


# 3 -- multi-table inheritance (drill only -- delete it afterwards)
class ExtendedLead(Lead):
    relationship_manager = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.PROTECT, related_name="+")
# CREATE TABLE crm_extendedlead (lead_ptr_id bigint PRIMARY KEY REFERENCES
crm_lead (id), ...)
# ExtendedLead.objects.all() -> ... FROM crm_extendedlead INNER JOIN crm_lead
ON (...)  -- EVERY query


# 4 -- proxy: same table, different Python
class VipLeadManager(LeadManager):
    def get_queryset(self):
        return super().get_queryset().filter(desk__slug="vip")


class VipLead(Lead):
    objects = VipLeadManager()
```

```
    class Meta:
        proxy = True
        verbose_name = "VIP lead"
        ordering = ["-created_at"]
# makemigrations writes a CreateModel with proxy=True -- and no SQL at all
```

5-8.

```
# 5  lead.full_clean() runs, in order:
clean_fields()          each field: blank / null / max_length / choices /
validators
clean()                 your cross-field rules (Day 1, task 18)
validate_unique()       unique=True and unique_together, by querying the
database
validate_constraints()  Meta.constraints: CHECKs evaluated, UniqueConstraints
queried (4.1+)
Errors are collected per step into one ValidationError {field: [messages]}.
```

```
# 6 -- the save() contract
def save(self, *args, **kwargs):
    # 1 accept and pass on everything (update_fields, force_insert, using,
...)
    # 2 if you change a field here, make sure it's written even when
update_fields is given:
    update_fields = kwargs.get("update_fields")
    if update_fields is not None and "name" in update_fields:
        self.name = self.name.strip()
    super().save(*args, **kwargs)          # 3 never forget this line


# 7 -- crm/receivers.py, imported from CrmConfig.ready()
from django.db.models.signals import post_save
from django.dispatch import receiver


@receiver(post_save, sender=Deposit)
def log_deposit(sender, instance, created, **kwargs):
    print("deposit saved", instance.pk, created)
# fires for: save(), objects.create(), get_or_create/update_or_create when
they write
# silent for: QuerySet.update(), bulk_create(), bulk_update(), raw SQL,
loaddata (raw=True)
```

```
# 8
from django.utils.functional import cached_property


class Lead(models.Model):
    @property
    def is_open(self):                      # cheap, from this row:
recomputed every access
        return self.status in {"new", "callback"}

    @cached_property
    def deposited_usd(self):                # one query, cached on the
instance (task 40's trap)
        return
Deposit.objects.approved().filter(account__lead=self).usd_total()
# for a LIST of leads, neither: Lead.objects.with_deposited() computes it in
the same query
```

9–10.

```
# crm/phones.py
import re


def normalize_phone(raw):
    digits = re.sub(r"\D", "", raw or "")
    if not digits:
        return ""
    if digits.startswith("00"):
        digits = digits[2:]
    elif digits.startswith("0"):
        digits = "971" + digits[1:]
    return "+" + digits


# crm/fields.py
from django.db import models

from .phones import normalize_phone


class PhoneNumberField(models.CharField):
    description = "A phone number stored as +<digits>"
```

```
    def __init__(self, *args, **kwargs):
        kwargs.setdefault("max_length", 20)
        super().__init__(*args, **kwargs)

    def deconstruct(self):                        # keep migrations free of
our defaults (task 39)
        name, path, args, kwargs = super().deconstruct()
        if kwargs.get("max_length") == 20:
            del kwargs["max_length"]
        return name, path, args, kwargs

    def to_python(self, value):                   # forms, full_clean()
        value = super().to_python(value)
        return normalize_phone(value) if value else value

    def get_prep_value(self, value):              # EVERY write path and exact
/ in lookups
        value = super().get_prep_value(value)
        return normalize_phone(value) if value else value
```

9: `get_prep_value` runs for `save()`, `update()`, `bulk_create()`, `bulk_update()` and for `filter(phone=...)` / `phone__in` — so `Lead.objects.filter(phone="050 000 0007")` finds `+971500000007`. Pattern lookups (`contains`, `startswith`) skip it, which keeps Day 5's digit search working. Day 1's three ways around `save()` are now closed.

```
# 10 -- crm/models.py
from django.contrib.contenttypes.fields import GenericForeignKey,
GenericRelation
from django.contrib.contenttypes.models import ContentType


class Note(TimeStampedModel):
    content_type = models.ForeignKey(ContentType, on_delete=models.CASCADE)
    object_id = models.PositiveBigIntegerField()
    target = GenericForeignKey("content_type", "object_id")
    author = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.PROTECT, related_name="+")
    body = models.TextField()

    class Meta:
        indexes = [models.Index(fields=["content_type", "object_id"],
```

```
name="note_target_idx")]


# on the models that can have notes:
class Lead(models.Model):
    notes = GenericRelation(Note)        # lead.notes.all(),
Lead.objects.filter(notes__body__icontains=...)

class Broker(models.Model):
    notes = GenericRelation(Note)

Note.objects.create(target=lead, author=agent, body="Prefers WhatsApp after
6pm")
```

B. Core

```
# 11  class Lead(TimeStampedModel): ... -- delete Lead's own created_at /
updated_at lines
Identical field definitions -> makemigrations finds NOTHING to do. Only a
difference
(an added db_index, a new default) produces an AlterField -- read it before
accepting.

# 12  choices or a lookup table?
Code branches on the value (if status == "ftd")          -> TextChoices + a
CHECK constraint
Non-developers add values, or values carry attributes     -> a table with a
FK (PROTECT)
lead status        TextChoices  -- the code's logic depends on every value
deposit method     TextChoices  -- rarely changes; a migration per new method
is fine
SOP rules          a table      -- Compliance edits them; each has max_hours
and severity
```

```
# 13 -- crm/models.py + crm/services.py
class LeadHistory(models.Model):
    lead = models.ForeignKey(Lead, on_delete=models.CASCADE,
related_name="history")
    field = models.CharField(max_length=50)
    old_value = models.JSONField(null=True)
    new_value = models.JSONField(null=True)
    by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
```

```
on_delete=models.SET_NULL, related_name="+")
    at = models.DateTimeField(default=timezone.now)


def change_lead(pk, by, **changes):
    """Change tracked columns (status=..., desk_id=..., assigned_to_id=...)
and record each change."""
    with transaction.atomic():
        lead = Lead.objects.select_for_update().get(pk=pk)
        history = []
        for attname, new in changes.items():
            old = getattr(lead, attname)
            if old != new:
                history.append(LeadHistory(lead=lead, field=attname,
old_value=old, new_value=new, by=by))
                setattr(lead, attname, new)
        if history:
            lead.save(update_fields=[*changes, "updated_at"])      # attnames
like desk_id are accepted
            LeadHistory.objects.bulk_create(history)
        return lead
# a service knows WHO made the change and why; a post_save signal knows
neither, and misses update().


# 14 -- clean() for friendly messages
class Deposit(models.Model):
    def clean(self):
        if self.account_id and self.currency != self.account.currency:
            raise ValidationError({"currency": f"Account {self.account} is in
{self.account.currency}."})

class Trade(models.Model):
    def clean(self):
        if self.closed_at and self.closed_at < self.opened_at:
            raise ValidationError({"closed_at": "A trade can't close before
it opens."})
# Trade already has the CHECK (Day 4, task 37). The currency rule crosses
tables, so a CHECK
# can't express it -- the service validates it (task 24); a trigger could
enforce it.
```

```
# 15 -- swap the field
phone = PhoneNumberField()          # was CharField(max_length=20) +
normalisation in save()
# makemigrations -> AlterField; the column is unchanged (varchar(20)), so no
SQL rewrite.
# delete the phone lines from Lead.save(); Lead.normalize_phone can now call
phones.normalize_phone.


# 16 -- immutability: Python for the message, the database for the guarantee
class Deposit(models.Model):
    FROZEN = ("amount", "currency", "fx_rate")

    def save(self, *args, **kwargs):
        if self.pk:
            old = Deposit.objects.filter(pk=self.pk).values("status",
*self.FROZEN).first()
            if old and old["status"] == "approved" and any(old[f] !=
getattr(self, f) for f in self.FROZEN):
                raise ValidationError("An approved deposit can't be changed.
Create an adjustment instead.")
        super().save(*args, **kwargs)
```

```
-- 16 -- RunSQL migration: also covers update(), bulk_update and raw SQL
CREATE OR REPLACE FUNCTION deposits_freeze_approved() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF OLD.status = 'approved'
     AND (NEW.amount, NEW.currency, NEW.fx_rate) IS DISTINCT FROM
(OLD.amount, OLD.currency, OLD.fx_rate) THEN
    RAISE EXCEPTION 'approved deposit % is immutable', OLD.id USING ERRCODE =
'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER deposits_freeze BEFORE UPDATE ON nexora_deposits
FOR EACH ROW EXECUTE FUNCTION deposits_freeze_approved();
-- ERRCODE check_violation reaches Django as an IntegrityError, like any
CHECK.
-- reverse_sql: DROP TRIGGER deposits_freeze ON nexora_deposits; DROP
FUNCTION deposits_freeze_approved();
```

```
# 17 -- crm/signals.py
from django.dispatch import Signal

deposit_approved = Signal()        # kwargs: deposit_id, lead_id, first

# crm/services.py, inside approve_deposit (Day 10):
transaction.on_commit(lambda: deposit_approved.send(sender=Deposit,
deposit_id=pk, lead_id=lead_id, first=first))

# reports/receivers.py -- imported in ReportsConfig.ready()
from django.core.cache import cache
from django.dispatch import receiver
from django.utils import timezone

from crm.signals import deposit_approved


@receiver(deposit_approved)
def drop_cached_scoreboard(sender, deposit_id, lead_id, first, **kwargs):
    if first:                          # Day 9's key: exactly today's cached
scoreboard
        cache.delete(f"report:desk_scoreboard:v1:
{timezone.localdate():%Y%m%d}")
# crm doesn't import reports; reports listens. Sent from the SERVICE, on
commit: it fires exactly
# once per real approval -- unlike post_save, which fires for every save of
any deposit.
```

```
# 18
class Lead(models.Model):
    class Meta:
        permissions = [("export_leads", "Can export leads"), ("view_kyc",
"Can view KYC data")]
# migrate creates them; give them to groups in the admin or a data migration
if not request.user.has_perm("crm.export_leads"):
    raise PermissionDenied


# 19 -- a view, owned by a migration; a model to read it
migrations.RunSQL(
    sql="""CREATE VIEW v_lead_summary AS
           SELECT l.id AS lead_id, l.desk_id, COUNT(DISTINCT c.id) AS calls,
```

```
                  MAX(c.started_at) AS last_call_at
           FROM crm_lead l LEFT JOIN crm_calllog c ON c.lead_id = l.id
           GROUP BY l.id""",
    reverse_sql="DROP VIEW v_lead_summary",
)


class LeadSummary(models.Model):
    lead = models.OneToOneField(Lead, on_delete=models.DO_NOTHING,
primary_key=True, related_name="summary")
    desk = models.ForeignKey(Desk, on_delete=models.DO_NOTHING,
related_name="+")
    calls = models.IntegerField()
    last_call_at = models.DateTimeField(null=True)

    class Meta:
        managed = False                  # Django never creates, alters or
drops it
        db_table = "v_lead_summary"
# LeadSummary.objects.filter(desk__slug="vip", calls=0) -- an ordinary read-
only model


# 20 -- extras most brokers won't have, owned by Broker -> a one-to-one table
class BrokerProfile(TimeStampedModel):
    broker = models.OneToOneField(Broker, on_delete=models.CASCADE,
related_name="profile")
    ib_agreement_date = models.DateField(null=True, blank=True)
    payout_email = models.EmailField(blank=True)
# multi-table would JOIN on every Broker query; a proxy can't add columns.
One-to-one costs a
# JOIN only when you ask for it: Broker.objects.select_related("profile").
```

C. Nexora CRM

```
# 21 -- crm/admin.py
@admin.register(VipLead)
class VipLeadAdmin(admin.ModelAdmin):
    list_display = ("name", "phone", "status", "assigned_to", "created_at")
    list_select_related = ("assigned_to",)
# the admin uses VipLead's default manager -> only VIP leads, with its own
permissions
```

```
# (crm.view_viplead ...) to give the VIP team access to just this list.


# 22
Note.objects.create(target=lead, author=agent, body="Asked for a callback
after Eid")
lead.notes.order_by("-created_at")
Lead.objects.filter(notes__body__icontains="callback").distinct()
Note.objects.filter(content_type=ContentType.objects.get_for_model(Broker))
# all broker notes


# 23
def lead_timeline(lead):
    events = [(h.at, "change", f"{h.field}: {h.old_value} -> {h.new_value}")
for h in lead.history.all()]
    events += [(c.started_at, "call", f"{c.agent.username}: {c.outcome}")
               for c in lead.calls.select_related("agent")]
    events += [(t.at, "transfer", f"desk {t.from_desk_id} -> {t.to_desk_id}")
for t in lead.transfers.all()]
    events += [(n.created_at, "note", n.body[:80]) for n in lead.notes.all()]
    return sorted(events, key=lambda e: e[0], reverse=True)
# 4 queries; one per source. (A UNION ALL of values() QuerySets with
identical columns can do
# it in one -- worth it only for long timelines that need database-side
pagination.)


# 24 -- crm/services.py
def create_deposit(account, amount, currency, fx_rate, method):
    dep = Deposit(account=account, amount=amount, currency=currency,
fx_rate=fx_rate, method=method)
    dep.full_clean()          # friendly: {'currency': ['Account mt5:880042
is in EUR.']}
    dep.save()                # the CHECKs and FKs are still the backstop
    return dep


# 25
class Lead(models.Model):
    def can_delete(self):
        return not self.accounts.exists() and not self.sop_breaches.exists()
```

```
# service: if not lead.can_delete(): raise ValidationError("Archive this lead
instead.")
# PROTECT (Day 2) still blocks it in the database if the check is ever
skipped.
```

```
# 26 -- crm/checks.py, imported in CrmConfig.ready()
from django.apps import apps
from django.core import checks
from django.db import models


@checks.register(checks.Tags.models)
def no_float_fields(app_configs, **kwargs):
    errors = []
    for model in apps.get_app_config("crm").get_models():
        for field in model._meta.concrete_fields:
            if isinstance(field, models.FloatField):
                errors.append(checks.Error(
                    f"{model.__name__}.{field.name} is a FloatField.",
                    hint="Use DecimalField for money, rates and anything that
must add up exactly.",
                    obj=field, id="crm.E001"))
    return errors
# python manage.py check -- and every runserver / migrate / test run -- now
enforces it


# 27
class Lead(models.Model):
    def __str__(self):
        return f"{self.name} ({self.phone[:6]}****{self.phone[-2:]})"

    def __repr__(self):
        return f"<Lead {self.pk} {self.status} desk={self.desk_id}>"     #
ids, never personal data
```

```
# 28 -- crm/management/commands/data_dictionary.py
from django.apps import apps
from django.core.management.base import BaseCommand
from django.db import connection


class Command(BaseCommand):
    help = "Print every crm table and column with its type, nullability and
comment"

    def handle(self, *args, **opts):
        for model in apps.get_app_config("crm").get_models():
            meta = model._meta
            if not meta.managed or meta.proxy:
                continue
            self.stdout.write(f"\n{meta.db_table}  -- {meta.db_table_comment
or meta.verbose_name}")
            for f in meta.concrete_fields:
                self.stdout.write(f"  {f.column:<24} {f.db_type(connection)
or '':<28} "
                                  f"{'NULL' if f.null else 'NOT NULL':<9}
{f.db_comment or f.help_text or ''}")
```

```
# 29  where each rule lives
phone format                      custom field (every write path)
status is one of the choices      DB CHECK (+ TextChoices for the code)
one primary account per lead      DB conditional unique constraint
deposit currency = account's      service + clean() (crosses tables)
approved deposits immutable       DB trigger + Python guard (task 16)
only managers reassign leads      service + permission check
history of lead changes           service (change_lead)
refresh dashboards on approval    custom signal, sent on commit (decoupled
reaction)
```

```
# 30 -- crm/tests.py
class DesignTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=300, stdout=StringIO())

    def test_phone_normalised_on_every_write_path(self):
        lead = Lead.objects.first()
```

```
        Lead.objects.filter(pk=lead.pk).update(phone="050 999 0001")
        self.assertTrue(Lead.objects.filter(pk=lead.pk,
phone="+971509990001").exists())

    def test_approved_deposit_is_frozen_in_the_database(self):
        dep = Deposit.objects.filter(status="approved").first()
        with self.assertRaises(IntegrityError), transaction.atomic():
            Deposit.objects.filter(pk=dep.pk).update(amount=dep.amount + 1)

    def test_change_lead_records_history(self):
        lead = Lead.objects.filter(status="new").first()
        change_lead(lead.pk, by=None, status="callback")
        self.assertEqual(lead.history.get().new_value, "callback")

    def test_no_system_check_errors(self):
        from .checks import no_float_fields
        self.assertEqual(no_float_fields(None), [])
```

```
python manage.py test crm && git add -A && git commit -m "orm day 13"
```

D. Challenge

```
# 31  related_name="owned_items" on an abstract base used by Note and
Attachment
crm.Note.owner: (fields.E304) Reverse accessor 'User.owned_items' for
'crm.Note.owner' clashes
with reverse accessor for 'crm.Attachment.owner'.
fix: related_name="%(app_label)s_%(class)s_owned" (task 2), or "+" for no
reverse accessor.

# 32  multi-table
Lead.objects.all() returns Lead objects, children included as plain Leads.
lead.extendedlead
reaches the child -- and raises DoesNotExist for leads that aren't one.
Creating a child INSERTs
two rows; deleting a child deletes BOTH rows (the parent too), unless
delete(keep_parents=True).

# 33
VipLead.objects.create(name="X", phone="050 1",
desk=Desk.objects.get(slug="retention"))
Saved without complaint -- and now invisible through VipLead.objects. A proxy
```

```
changes Python
behaviour, never the data rules; enforce "VIP means desk=vip" in the service
or a constraint.

# 34  audit via post_save + Lead.objects.filter(...).update(status="lost")
The bulk change writes NO audit rows: update() sends no signals. Days later
the history says
the leads never changed. Must-happen side effects belong in services (task
13) or triggers.

# 35  @receiver(post_save, sender=Lead) def f(...): instance.save()
post_save -> save() -> post_save -> ... -> RecursionError: maximum recursion
depth exceeded.
In a receiver, write with Lead.objects.filter(pk=instance.pk).update(...) --
no signal fires.

# 36  save() normalises phone; caller does lead.save(update_fields=
["status"])
The normalised phone exists only in memory; the database keeps the old
format. In-memory and
stored values disagree silently. Task 15's field removes the problem at its
root.

# 37  full_clean() inside save()
ModelForms already validated -> duplicate unique/constraint queries on every
save; bulk paths
skip save() anyway, so it isn't a guarantee; code that expects IntegrityError
gets
ValidationError; admin and fixtures pay too. Validate at the boundaries
(forms, services,
create_lead); let constraints guarantee.

# 38  deleting a broker that has notes
With GenericRelation(Note) on Broker: Django deletes the broker's notes too
(it knows the link).
Without it: the notes stay, pointing at an object_id that no longer exists --
orphans. A generic
FK has NO database foreign key, so PostgreSQL can't help. Also:
Note.objects.filter(target__...)
is impossible -- generic relations can't be JOINed from the Note side.

# 39  PhoneNumberField(region="SA") with region missing from deconstruct()
Migrations record the field WITHOUT region; loading them builds the field
```

```
with the default
region; changing region never shows up in makemigrations. If __init__
REQUIRES region,
loading old migrations fails with TypeError. deconstruct() must return every
non-default
argument -- and drop defaults (task 9) so migrations don't churn.

# 40
lead.deposited_usd                # Decimal('1250.00') -- computed and
cached
(a new approved deposit arrives)
lead.refresh_from_db()
lead.deposited_usd                # STILL 1250.00: refresh_from_db() reloads
fields and related
                                  # caches, not cached_property values
del lead.deposited_usd            # clears it; or use @property / an
annotation instead
```

Day 14 — Reporting with the ORM

Goal: do analytics engineering inside a Django project. Views and materialised views behind read-only models, a separate analytics database with facts and a date dimension, an incremental nightly load, cohorts and funnels, data-quality tests and reconciliation. This is the day that points straight at DIFC data roles.

```
crm (default database)                          warehouse (analytics database)
  OLTP tables: leads, calls, deposits     --->   dim_date          one row
per day
  v_broker_tree       (view)        nightly      fact_deposit      one row
per APPROVED deposit
  mv_daily_desk_kpis  (materialised)   ETL       etl_state         watermarks
Grain first: every fact table answers "one row per WHAT?" before it gets a
single column.
```

Setup: `createdb nexora_analytics`, then `python manage.py startapp warehouse` and add it to `INSTALLED_APPS`. Tasks 1–2 wire the second database.

A. Drills (1–10) — the analytics toolkit

- [ ] **1.** A second database alias `analytics`; run a query on it with `.using("analytics")`.
- [ ] **2.** A database router: `warehouse` lives in `analytics`, everything else in `default`; migrations follow.
- [ ] **3.** `DimDate`: a date dimension filled for 2024–2027 by a data migration on the right database.
- [ ] **4.** `FactDeposit`: grain, keys, measures — no foreign keys across databases.
- [ ] **5.** `EtlState`: a watermark per job.
- [ ] **6.** `v_broker_tree`: Day 12's recursive CTE as a view, read through an unmanaged `BrokerTree` model.
- [ ] **7.** `mv_daily_desk_kpis`: a materialised view with a unique index, read through an unmanaged model.
- [ ] **8.** Refresh it with `REFRESH MATERIALIZED VIEW CONCURRENTLY` from a command.
- [ ] **9.** Add `Deposit.approved_at`; set it in `approve_deposit` and backfill it.
- [ ] **10.** Zero-filled daily FTDs with the ORM alone: `DimDate` + a correlated subquery.

B. Core (11–20) — the warehouse

- [ ] **11.** `load_fact_deposits(since, until)`: build fact rows from crm, upsert them into analytics.
- [ ] **12.** The FTD flag and FTD month on every fact row.
- [ ] **13.** An incremental load: from the watermark, with a 3-day reprocess window for late changes.
- [ ] **14.** Monthly depositor cohorts: FTD month × months since FTD → active depositors.
- [ ] **15.** A retention matrix as rows of percentages.
- [ ] **16.** The FTD funnel by source: leads → contacted → account → FTD.
- [ ] **17.** Root-broker revenue in pure ORM, through `BrokerTree`.
- [ ] **18.** Data-quality checks: not null, accepted values, uniqueness, relationships, freshness.
- [ ] **19.** Reconciliation: fact totals vs source totals for each of the last 7 days.
- [ ] **20.** A report registry: name, grain, owner, description and function for each report.

C. Nexora CRM (21–30) — the data jobs

- [ ] **21.** The weekly management pack from the materialised view and the facts.
- [ ] **22.** Cohort retention for the last 6 FTD months.
- [ ] **23.** This quarter's source funnel with conversion rates.
- [ ] **24.** Root-broker league table: revenue, FTDs, average first deposit.
- [ ] **25.** Agent leaderboard: FTDs, calls, SOP breaches per 100 calls.
- [ ] **26.** `nightly_etl`: lock, load, refresh, check, advance the watermark — idempotent.
- [ ] **27.** Late approvals: a deposit created last week, approved today. Does it reach the facts?
- [ ] **28.** Backfill facts for any date range from the command line.
- [ ] **29.** Export a fact table for BI as CSV (and Parquet, if pandas is available).
- [ ] **30.** Tests across both databases. Commit `orm day 14`.

D. Challenge (31–40) — analytics traps

- [ ] **31.** `migrate` without `--database=analytics`. Where do the warehouse tables go?
- [ ] **32.** A `ForeignKey` from `FactDeposit` to `crm.Lead`. What breaks?

- [ ] **33.** `REFRESH MATERIALIZED VIEW` without `CONCURRENTLY` during office hours.
- [ ] **34.** Do views and materialised views exist in the TEST database? Why?
- [ ] **35.** Facts keyed by UTC date vs Dubai date. Which numbers move?
- [ ] **36.** A watermark on `created_at`. Which deposits are never loaded?
- [ ] **37.** Build facts by joining deposits to ALL of a lead's accounts and calls. What happens to the totals?
- [ ] **38.** A dashboard reads the materialised view at 15:00. How old is the data, and how do you show it?
- [ ] **39.** Reading from a replica right after writing to the primary. What do you see?
- [ ] **40.** Where should this logic live long-term — Django, SQL views, or a tool like dbt? Draw the line.

Solutions

A. Drills

1–2 — two databases.

```
# django_project/settings.py
DATABASES["analytics"] = {**DATABASES["default"],
                          "NAME": os.environ.get("PG_ANALYTICS_DB",
"nexora_analytics")}
DATABASE_ROUTERS = ["django_project.routers.WarehouseRouter"]


# django_project/routers.py
class WarehouseRouter:
    """warehouse -> analytics; every other app -> default."""
    LABEL = "warehouse"

    def db_for_read(self, model, **hints):
        return "analytics" if model._meta.app_label == self.LABEL else
"default"

    def db_for_write(self, model, **hints):
        return self.db_for_read(model)

    def allow_relation(self, obj1, obj2, **hints):
        return obj1._state.db == obj2._state.db           # never relate rows
```

```
across databases

    def allow_migrate(self, db, app_label, model_name=None, **hints):
        if app_label == self.LABEL:
            return db == "analytics"
        return db == "default"
```

```
python manage.py migrate                          # default: crm, reports,
auth ... (warehouse skipped)
python manage.py migrate --database=analytics    # analytics: warehouse only
```

3-5 — the warehouse models.

```
# warehouse/models.py
from django.db import models


class DimDate(models.Model):                                              #
3
    day = models.DateField(primary_key=True)
    year = models.PositiveSmallIntegerField()
    month = models.PositiveSmallIntegerField()
    month_start = models.DateField(db_index=True)
    iso_week = models.PositiveSmallIntegerField()
    iso_weekday = models.PositiveSmallIntegerField()          # 1 = Monday
    is_weekend = models.BooleanField()                        # UAE:
Saturday, Sunday


class FactDeposit(models.Model):                                          #
4
    """Grain: one row per APPROVED deposit. Keys are plain ids -- no FKs
across databases."""
    deposit_id = models.BigIntegerField(primary_key=True)     # the source's
natural key
    day = models.DateField(db_index=True)                     # Dubai date of
approval
    approved_at = models.DateTimeField()
    lead_id = models.BigIntegerField(db_index=True)
    desk_id = models.BigIntegerField()
    broker_id = models.BigIntegerField(null=True)
    root_broker_code = models.CharField(max_length=20, null=True)
    agent_id = models.BigIntegerField(null=True)
```

```
    source = models.CharField(max_length=10)
    amount_usd = models.DecimalField(max_digits=16, decimal_places=2)
    is_ftd = models.BooleanField()
    ftd_month = models.DateField(db_index=True)               # the lead's
cohort
    loaded_at = models.DateTimeField(auto_now=True)


class EtlState(models.Model):                                          #
5
    name = models.CharField(max_length=50, primary_key=True)
    watermark = models.DateTimeField()
    updated_at = models.DateTimeField(auto_now=True)


# warehouse/migrations/0002_fill_dim_date.py                           #
3
from datetime import date, timedelta


def fill(apps, schema_editor):
    DimDate = apps.get_model("warehouse", "DimDate")
    db = schema_editor.connection.alias                   # write to the
database being migrated
    d, end, rows = date(2024, 1, 1), date(2027, 12, 31), []
    while d <= end:
        iso = d.isocalendar()
        rows.append(DimDate(day=d, year=d.year, month=d.month,
month_start=d.replace(day=1),
                            iso_week=iso.week, iso_weekday=iso.weekday,
is_weekend=iso.weekday >= 6))
        d += timedelta(days=1)
    DimDate.objects.using(db).bulk_create(rows, ignore_conflicts=True)


class Migration(migrations.Migration):
    dependencies = [("warehouse", "0001_initial")]
    operations = [migrations.RunPython(fill, migrations.RunPython.noop)]
```

6-8 — views behind models.

```
# crm/migrations/00xx_views.py -- RunSQL in a crm migration (default
database)
migrations.RunSQL(
    sql="""
    CREATE VIEW v_broker_tree AS
    WITH RECURSIVE tree AS (
        SELECT id, id AS root_id, code AS root_code, 0 AS depth FROM
crm_broker WHERE parent_id IS NULL
        UNION ALL
        SELECT b.id, t.root_id, t.root_code, t.depth + 1
        FROM crm_broker b JOIN tree t ON b.parent_id = t.id WHERE t.depth <
20)
    SELECT id AS broker_id, root_id, root_code, depth FROM tree;

    CREATE MATERIALIZED VIEW mv_daily_desk_kpis AS
    SELECT (c.started_at AT TIME ZONE 'Asia/Dubai')::date AS day, c.desk_id,
           COUNT(*) AS calls, SUM(c.duration_sec) AS talk_sec, COUNT(DISTINCT
c.lead_id) AS leads_called
    FROM crm_calllog c GROUP BY 1, 2;
    CREATE UNIQUE INDEX mv_daily_desk_kpis_key ON mv_daily_desk_kpis (day,
desk_id);""",
    reverse_sql="DROP MATERIALIZED VIEW mv_daily_desk_kpis; DROP VIEW
v_broker_tree;",
)


# crm/models.py
class BrokerTree(models.Model):                                          #
6
    broker = models.OneToOneField(Broker, on_delete=models.DO_NOTHING,
primary_key=True, related_name="tree")
    root = models.ForeignKey(Broker, on_delete=models.DO_NOTHING,
related_name="+")
    root_code = models.CharField(max_length=20)
    depth = models.IntegerField()

    class Meta:
        managed = False
        db_table = "v_broker_tree"


class DailyDeskKpi(models.Model):                                        #
7
```

```
    pk = models.CompositePrimaryKey("day", "desk")          # Django 5.2+
    day = models.DateField()
    desk = models.ForeignKey(Desk, on_delete=models.DO_NOTHING,
related_name="+")
    calls = models.IntegerField()
    talk_sec = models.BigIntegerField()
    leads_called = models.IntegerField()

    class Meta:
        managed = False
        db_table = "mv_daily_desk_kpis"
# before 5.2: add ROW_NUMBER() OVER (ORDER BY day, desk_id) AS id to the view
and use id as the pk
```

```
# 8 -- crm/management/commands/refresh_kpis.py
from django.core.management.base import BaseCommand
from django.db import connection


class Command(BaseCommand):
    help = "Refresh mv_daily_desk_kpis without blocking readers"

    def handle(self, *args, **opts):
        with connection.cursor() as cur:
            cur.execute("REFRESH MATERIALIZED VIEW CONCURRENTLY
mv_daily_desk_kpis")
# CONCURRENTLY builds the new contents beside the old and swaps rows in, so
dashboards keep
# reading during the refresh. It REQUIRES a unique index on the view
(mv_daily_desk_kpis_key).
```

9-10.

```
# 9 -- crm/models.py: Deposit.approved_at = models.DateTimeField(null=True,
blank=True, db_index=True)
# backfill (data migration): Deposit.objects.filter(status="approved",
approved_at__isnull=True)
#
.update(approved_at=F("created_at"))
# crm/services.py approve_deposit (Day 10): .update(status="approved",
approved_at=Now())
# the seed: set approved_at=created_at for approved deposits


# 10 -- zero-filled days without raw SQL: one row per DimDate day, a count
per day
from django.db.models import Count, OuterRef, Subquery
from django.db.models.functions import Coalesce

from warehouse.models import DimDate, FactDeposit

FTD_ON_DAY = (FactDeposit.objects.filter(day=OuterRef("day"), is_ftd=True)

.order_by().values("day").annotate(n=Count("deposit_id")).values("n"))
(DimDate.objects.filter(day__range=(start, end))
 .annotate(ftd=Coalesce(Subquery(FTD_ON_DAY), 0)).values("day",
"ftd").order_by("day"))
# both models route to analytics, so the subquery runs in the same database.
The date
# dimension is what GROUP BY could never give you: the days with NO rows.
```

B. Core

11–13 — the load.

```
# warehouse/etl.py
from datetime import timedelta

from django.db.models import F, OuterRef, Subquery
from django.utils import timezone

from crm.models import Deposit
from warehouse.models import EtlState, FactDeposit

REPROCESS = timedelta(days=3)
```

```
# the lead's first approved deposit -- Day 8's FIRST_DEP, correlated through
the deposit's lead
FIRST_OF_LEAD =
(Deposit.objects.filter(account__lead=OuterRef("account__lead"),
status="approved")
                .order_by("created_at", "id"))


def load_fact_deposits(since, until, batch=2000):                          #
11, 12
    rows = (Deposit.objects.approved().filter(approved_at__gte=since,
approved_at__lt=until)
            .annotate(first_id=Subquery(FIRST_OF_LEAD.values("id")[:1]),
                      first_at=Subquery(FIRST_OF_LEAD.values("created_at")
[:1]))
            .values("id", "approved_at", "amount_usd", "first_id",
"first_at",
                    lead=F("account__lead_id"),
desk=F("account__lead__desk_id"),
                    broker=F("account__lead__broker_id"),
root=F("account__lead__broker__tree__root_code"),
                    agent=F("account__lead__assigned_to_id"),
src=F("account__lead__source"))
            .iterator(chunk_size=batch))
    facts, n = [], 0
    for r in rows:
        facts.append(FactDeposit(
            deposit_id=r["id"], approved_at=r["approved_at"],
day=timezone.localdate(r["approved_at"]),
            lead_id=r["lead"], desk_id=r["desk"], broker_id=r["broker"],
root_broker_code=r["root"],
            agent_id=r["agent"], source=r["src"], amount_usd=r["amount_usd"],
            is_ftd=r["id"] == r["first_id"],
            ftd_month=timezone.localdate(r["first_at"]).replace(day=1)))
        if len(facts) == batch:
            n += _upsert(facts); facts = []
    return n + _upsert(facts)


def _upsert(facts):
    FactDeposit.objects.using("analytics").bulk_create(
        facts, update_conflicts=True, unique_fields=["deposit_id"],
```

```
        update_fields=["day", "approved_at", "desk_id", "broker_id",
"root_broker_code", "agent_id",
                       "source", "amount_usd", "is_ftd", "ftd_month"])
    return len(facts)


def incremental_load():                                                  #
13
    state, _ = EtlState.objects.get_or_create(name="fact_deposit",
                                              defaults={"watermark":
timezone.now() - timedelta(days=400)})
    until = timezone.now()
    n = load_fact_deposits(state.watermark - REPROCESS, until)      # re-read
the last 3 days too
    state.watermark = until
    state.save(update_fields=["watermark", "updated_at"])
    return n
```

11–13: reads come from `default` (crm), writes go to `analytics` (the router, plus an explicit `.using()` for clarity). The upsert on `deposit_id` makes every run idempotent — reprocessing 3 days rewrites those rows instead of duplicating them. A crash before the watermark is saved simply repeats the window next time.

14–17.

```
# warehouse/reports.py
from collections import defaultdict

from django.db.models import Count, Exists, OuterRef, Q, Sum
from django.db.models.functions import TruncMonth


def cohorts(first_month, last_month):                                    #
14
    """{ftd_month: {months_since: active_depositors}}"""
    rows = (FactDeposit.objects.filter(ftd_month__range=(first_month,
last_month))
            .annotate(month=TruncMonth("day")).values("ftd_month", "month")
            .annotate(active=Count("lead_id", distinct=True)))
    out = defaultdict(dict)
    for r in rows:
        since = (r["month"].year - r["ftd_month"].year) * 12 +
r["month"].month - r["ftd_month"].month
```

```
        out[r["ftd_month"]][since] = r["active"]
    return dict(out)


def retention_matrix(first_month, last_month):                                  #
15
    """[[cohort, size, m0%, m1%, ...]] -- month 0 is the cohort itself
(100%)."""
    table = []
    for cohort, months in sorted(cohorts(first_month, last_month).items()):
        size = months.get(0, 0)
        table.append([cohort, size] + [round(100 * months.get(m, 0) / size,
1) if size else None
                                       for m in range(max(months) + 1)])
    return table
```

```
# crm/reports.py
def source_funnel(start, end):                                                  #
16
    has_account = Exists(TradingAccount.objects.filter(lead=OuterRef("pk")))
    funded =
Exists(Deposit.objects.approved().filter(account__lead=OuterRef("pk")))
    return list(Lead.objects.filter(created_at__gte=start,
created_at__lt=end)
                .annotate(acc=has_account, ftd=funded)
                .values("source")
                .annotate(leads=Count("id"),
                          contacted=Count("id",
filter=Q(last_contacted_at__isnull=False)),
                          with_account=Count("id", filter=Q(acc=True)),
                          ftds=Count("id", filter=Q(ftd=True)))
                .order_by("source"))


def root_broker_revenue():                                                      #
17 -- pure ORM via the view
    return list(Deposit.objects.approved()

.values(root_code=F("account__lead__broker__tree__root_code"))
                .exclude(root_code=None)
                .annotate(revenue_usd=Sum("amount_usd"),
```

```
leads=Count("account__lead", distinct=True))
                .order_by("-revenue_usd"))
# Day 12 needed raw SQL for this; with the view behind a model it's an
ordinary JOIN.
```

18–20.

```
# warehouse/quality.py
from datetime import timedelta

from django.db.models import Count, Sum
from django.utils import timezone

from crm.models import Deposit, Lead

from .models import EtlState, FactDeposit


def run_checks():                                                    #
18
    """Return a list of failure messages; empty means healthy."""
    f = FactDeposit.objects
    failures = []
    checks = {
        "amount_usd is positive": f.filter(amount_usd__lte=0).count(),
        "source has an accepted value":
f.exclude(source__in=Lead.Source.values).count(),
        "at most one FTD per lead": f.filter(is_ftd=True).values("lead_id")

.annotate(n=Count("deposit_id")).filter(n__gt=1).count(),
        "every FTD month is a month start":
f.exclude(ftd_month__day=1).count(),
    }
    failures += [f"{name}: {bad} bad rows" for name, bad in checks.items() if
bad]

    # relationship across databases: every fact's lead must exist in crm
(checked in chunks)
    fact_leads = set(f.values_list("lead_id", flat=True).distinct())
    known = set(Lead.all_objects.filter(pk__in=fact_leads).values_list("pk",
flat=True))
    if fact_leads - known:
        failures.append(f"{len(fact_leads - known)} fact lead_ids missing
```

```
from crm")

    # freshness
    state = EtlState.objects.filter(name="fact_deposit").first()
    if not state or state.watermark < timezone.now() - timedelta(hours=26):
        failures.append("fact_deposit is stale (last load more than 26 hours
ago)")
    return failures


def reconcile(days=7):                                                  #
19
    """[(day, source_usd, fact_usd)] for days where the two disagree."""
    from crm.queries import day_start
    bad = []
    for i in range(1, days + 1):
        d = timezone.localdate() - timedelta(days=i)
        src = (Deposit.objects.approved()
               .filter(approved_at__gte=day_start(d),
approved_at__lt=day_start(d + timedelta(days=1)))
               .usd_total())
        fact =
FactDeposit.objects.filter(day=d).aggregate(t=Sum("amount_usd", default=0))
["t"]
        if src != fact:
            bad.append((d, src, fact))
    return bad
```

```
# 20 -- crm/registry.py
from crm import reports as crm_reports
from warehouse import reports as wh_reports

REPORTS = {
    "desk_scoreboard": {"fn": crm_reports.desk_scoreboard, "grain": "desk",
"owner": "Sales ops",
                        "description": "Leads, FTDs and conversion % per desk
(all time)."},
    "source_funnel": {"fn": crm_reports.source_funnel, "grain": "lead
source", "owner": "Marketing",
                      "description": "Leads -> contacted -> account -> FTD
for leads created in a period."},
    "retention": {"fn": wh_reports.retention_matrix, "grain": "FTD month x
months since", "owner": "Finance",
                  "description": "% of each FTD cohort that deposited again
in later months."},
    "root_broker_revenue": {"fn": crm_reports.root_broker_revenue, "grain":
"top-level broker",
                            "owner": "Partnerships", "description": "Approved
USD across each Brf tree."},
}
# one place answers "what reports exist, at what grain, who owns them" -- and
a `report <name>`
# command can run any of them by name.
```

C. Nexora CRM

```
# 21 -- weekly management pack
def weekly_pack(week_start):
    week_end = week_start + timedelta(days=7)
    return {
        "calls": list(DailyDeskKpi.objects.filter(day__gte=week_start,
day__lt=week_end)
                      .values("desk__name").annotate(calls=Sum("calls"),
talk_h=Sum("talk_sec") / 3600)
                      .order_by("desk__name")),
        "deposits": list(FactDeposit.objects.filter(day__gte=week_start,
day__lt=week_end)
                         .values("desk_id").annotate(usd=Sum("amount_usd"),
                                                     ftds=Count("deposit_id",
filter=Q(is_ftd=True)))
```

```
                          .order_by("desk_id")),
    }
# two databases, two queries: desk NAMES join in crm; facts carry desk_id --
map ids to names in
# Python with dict(Desk.objects.values_list("pk", "name")).

# 22
today = timezone.localdate()
first = (today.replace(day=1) - timedelta(days=150)).replace(day=1)        #
~6 months back
retention_matrix(first, today.replace(day=1))

# 23 -- this quarter, with rates
q_start = date(today.year, 3 * ((today.month - 1) // 3) + 1, 1)
for row in source_funnel(day_start(q_start), timezone.now()):
    row["ftd_pct"] = round(100 * row["ftds"] / row["leads"], 1) if
row["leads"] else None

# 24 -- league table from the facts (one row per approved deposit -> FTD rows
are first deposits)
(FactDeposit.objects.exclude(root_broker_code=None).values("root_broker_code"
)
 .annotate(revenue_usd=Sum("amount_usd"), ftds=Count("deposit_id",
filter=Q(is_ftd=True)),
           avg_first_usd=Avg("amount_usd", filter=Q(is_ftd=True)))
 .order_by("-revenue_usd"))

# 25 -- agents: three sources, each grouped once, merged in Python
ftds = dict(FactDeposit.objects.filter(is_ftd=True, day__gte=month_start_day)

.values("agent_id").annotate(n=Count("deposit_id")).values_list("agent_id",
"n"))
calls = dict(CallLog.objects.filter(started_at__gte=month_start)
             .values("agent").annotate(n=Count("id")).values_list("agent",
"n"))
breaches = dict(SOPBreach.objects.filter(occurred_at__gte=month_start)
                .values("agent").annotate(n=Count("id")).values_list("agent",
"n"))
board = sorted(({"agent_id": a, "ftds": ftds.get(a, 0), "calls": calls.get(a,
0),
```

```
                 "breaches_per_100_calls": round(100 * breaches.get(a, 0) /
calls[a], 2) if calls.get(a) else None}
                for a in set(calls) | set(ftds)), key=lambda r: -r["ftds"])
# separate GROUP BYs can't multiply each other (Day 6, task 31)
```

```
# 26 -- warehouse/management/commands/nightly_etl.py
from contextlib import contextmanager

from django.core.management import call_command
from django.core.management.base import BaseCommand, CommandError
from django.db import connection

from warehouse.etl import incremental_load
from warehouse.quality import reconcile, run_checks


@contextmanager
def job_lock(name):
    """A SESSION advisory lock: held across many transactions, released in
finally."""
    with connection.cursor() as cur:
        cur.execute("SELECT pg_try_advisory_lock(hashtext(%s))", [name])
        got = cur.fetchone()[0]
    try:
        yield got
    finally:
        if got:
            with connection.cursor() as cur:
                cur.execute("SELECT pg_advisory_unlock(hashtext(%s))",
[name])


class Command(BaseCommand):
    help = "Load facts, refresh views, check quality. Safe to re-run."

    def handle(self, *args, **opts):
        with job_lock("nightly-etl") as got:
            if not got:
                self.stdout.write("another nightly_etl is running --
exiting")
                return
            n = incremental_load()
            call_command("refresh_kpis")
```

```
            failures = run_checks() + [f"reconcile {d}: crm {s} vs facts {f}"
for d, s, f in reconcile()]
            self.stdout.write(f"loaded {n} fact rows")
            if failures:
                raise CommandError("data quality failed:\n" +
"\n".join(failures))   # non-zero exit -> alert
# Day 10's single_runner held ONE transaction for the whole job; here each
step commits on its
# own, so the lock has to be session-level -- and explicitly released.
```

```
# 27  a deposit created 7 days ago, approved today
The load filters on approved_at (task 9) -> it's in tonight's window and
lands in the facts on
TODAY's day. With a created_at watermark it would sit behind the watermark
forever (task 36).
```

```
# 28 -- warehouse/management/commands/backfill_facts.py
class Command(BaseCommand):
    def add_arguments(self, parser):
        parser.add_argument("start", type=date.fromisoformat)
        parser.add_argument("end", type=date.fromisoformat)

    def handle(self, *args, start, end, **opts):
        from crm.queries import day_start
        n = load_fact_deposits(day_start(start), day_start(end +
timedelta(days=1)))
        self.stdout.write(f"{n} rows upserted for {start}..{end}")
# python manage.py backfill_facts 2026-07-01 2026-09-30 -- idempotent thanks
to the upsert


# 29
import csv

with open("fact_deposit.csv", "w", newline="") as fh:
    w = csv.writer(fh)
    cols = ["deposit_id", "day", "lead_id", "desk_id", "root_broker_code",
"source", "amount_usd", "is_ftd", "ftd_month"]
    w.writerow(cols)

w.writerows(FactDeposit.objects.order_by("deposit_id").values_list(*cols).ite
rator(chunk_size=5000))
```

```
# with pandas + pyarrow installed:
#   pd.DataFrame.from_records(FactDeposit.objects.values(*cols).iterator(),
columns=cols).to_parquet("fact_deposit.parquet")
# (Day 12's COPY on the analytics connection is the fastest route for big
tables.)
```

```
# 30 -- warehouse/tests.py
class WarehouseTests(TestCase):
    databases = {"default", "analytics"}          # create and use BOTH test
databases

    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=300, stdout=StringIO())
        Deposit.objects.filter(status="approved",
approved_at__isnull=True).update(approved_at=F("created_at"))
        load_fact_deposits(timezone.now() - timedelta(days=400),
timezone.now())

    def test_facts_match_source(self):
        self.assertEqual(FactDeposit.objects.count(),
Deposit.objects.approved().count())
        self.assertEqual(FactDeposit.objects.aggregate(t=Sum("amount_usd"))
["t"], Deposit.objects.approved().usd_total())

    def test_one_ftd_per_funded_lead(self):
        self.assertEqual(FactDeposit.objects.filter(is_ftd=True).count(),
Lead.objects.funded().count())

    def test_reload_is_idempotent(self):
        before = FactDeposit.objects.count()
        load_fact_deposits(timezone.now() - timedelta(days=400),
timezone.now())
        self.assertEqual(FactDeposit.objects.count(), before)

    def test_quality_checks_pass_after_load(self):
        EtlState.objects.create(name="fact_deposit",
watermark=timezone.now())
        self.assertEqual(run_checks(), [])
```

```
python manage.py test && git add -A && git commit -m "orm day 14"
```

D. Challenge

```
# 31  python manage.py migrate   (no --database)
Only `default` is migrated, and the router's allow_migrate says "warehouse:
not here" -- so the
warehouse tables are created NOWHERE until you run migrate --
database=analytics. (Without a
router they'd land in default -- the classic mistake.) Deploy scripts run
both.

# 32  FactDeposit.lead = ForeignKey("crm.Lead")
A FOREIGN KEY constraint can't point into another database: migrate fails
creating it; with
db_constraint=False, joins still can't cross databases, and the router's
allow_relation refuses.
Facts carry plain ids (lead_id); join across databases in Python (task 25).

# 33  REFRESH MATERIALIZED VIEW mv_daily_desk_kpis   (no CONCURRENTLY)
Takes an ACCESS EXCLUSIVE lock: every dashboard query on the view WAITS until
the refresh ends.
CONCURRENTLY lets reads continue -- at the cost of a slower refresh and the
unique index.

# 34
Yes: views and materialised views are created by RunSQL in migrations, and
the test runner runs
migrations. managed = False only stops Django from creating a TABLE for the
model -- it doesn't
skip your RunSQL. (A materialised view is EMPTY until refreshed -- refresh it
in setUpTestData.)

# 35  day = approved_at.date() in UTC instead of Dubai
Every deposit approved 00:00-04:00 Dubai time moves to the previous day;
daily totals,
reconciliation (task 19 uses Dubai days) and month boundaries all disagree.
timezone.localdate(approved_at) at load time, and document the time zone on
the table.

# 36  watermark on created_at
A deposit created before the watermark but approved after it is never
selected again -- it
never reaches the facts. Watermark on the column that marks the CHANGE you
load (approved_at),
```

```
and reprocess a window for anything that can still change.

# 37  joining deposits to ALL of the lead's accounts and calls while building
facts
Each deposit row repeats once per call (and per extra account): amount_usd is
summed several
times -- the grain is broken (Day 6, task 31). Build a fact from rows AT its
grain (deposits),
and bring in other tables only through to-ONE paths or subqueries.

# 38
The view holds whatever existed at the last refresh (e.g. 02:00) -- 13 hours
old at 15:00.
Store the refresh time (a refreshed_at row, or EtlState) and print "as of
02:00" on the
dashboard. Stale-and-labelled is a choice; stale-and-unlabelled is a bug.

# 39  write to the primary, read the replica straight away
Replication is asynchronous: the replica can be milliseconds -- or minutes,
under load --
behind. The agent saves a lead and the next page (read from the replica)
shows the old value.
Read-after-write goes to the primary; replicas serve reports that tolerate a
little lag.

# 40  where the logic belongs
Django ORM    operational queries the app runs (screens, APIs, services) and
small reports
SQL views     shared definitions other tools must read the same way (broker
tree, KPIs)
dbt / SQL     the warehouse itself: staged, tested, documented models built
from raw tables,
  models      owned by data engineers -- the next step after this program,
and the one
              DIFC data teams will expect you to know. Django then becomes
one SOURCE for it.
```

Day 15 — Capstone: the Nexora query library

Goal: build what an analytics engineer is actually trusted with — a library of 20 Nexora business questions, each answered in the ORM, checked against an independent hand-written SQL answer, held to a query budget, timed, documented and reproducible. When you finish, every number the CRM reports has a second opinion.

```
Every question in the library has:
  id + title + definition    "Q03 First-contact SLA: share of leads called
within 24h"
  grain + key columns        one row per desk  (key: desk_name)
  orm(as_of) -> rows         the answer the app uses
  sql                        a SECOND answer, written from the definition,
never from show()
  budget                     max queries orm() may run
  owner                      who to ask when the definition changes
verify = run both, compare row by row, check the budget, record the timings.
```

Work on the full seed (`seed_nexora --leads 50000`) for timings and the small one (`--leads 300`) for tests. Solve each question in the ORM FIRST, then write its SQL without looking at your ORM's SQL.

A. Drills (1–10) — build the library

- [ ] **1.** `crm/library/registry.py` : a frozen `Question` dataclass and a `@question(...)` decorator that registers functions.
- [ ] **2.** `crm/library/__init__.py` : the conventions every question follows, written once (time zone, `as_of` , approved, FTD, archived, money).
- [ ] **3.** Time helpers that take `as_of` : Dubai day bounds, month bounds, N months back.
- [ ] **4.** `verify(question, as_of)` : run the ORM inside a query counter, run the SQL, normalise both, diff them.
- [ ] **5.** Normalisation rules: row keys, Decimal vs float, rounding half-up, dates vs datetimes.
- [ ] **6.** A `Result` record: rows, queries, ORM time, SQL time, problems.
- [ ] **7.** `python manage.py library list | run Q07 | verify [--ids Q01,Q02]` .
- [ ] **8.** One test that verifies every registered question with `subTest` .
- [ ] **9.** A markdown catalogue generated from the registry (id, title, grain, owner, budget, definition).

- [ ] **10.** Freeze `as_of` in tests so results are reproducible run after run.

B. Operations questions (11–20) — Q01–Q10

- [ ] **11.** Q01 Open leads and stale open leads per agent.
- [ ] **12.** Q02 Today's calls per agent: calls, talk seconds, distinct leads.
- [ ] **13.** Q03 First-contact SLA: share of leads called within 24 hours, per desk, last 30 days.
- [ ] **14.** Q04 Callback backlog by age bucket (0–1d, 1–3d, 3–7d, 7d+).
- [ ] **15.** Q05 Open leads by the outcome of their latest call (including never called).
- [ ] **16.** Q06 Calls, FTDs and FTDs per 100 calls per agent this month.
- [ ] **17.** Q07 Open leads with no call in the last 7 days, per desk.
- [ ] **18.** Q08 Possible duplicate groups (same last 7 phone digits) per desk.
- [ ] **19.** Q09 Call heatmap: desk × Dubai hour, last 7 days.
- [ ] **20.** Q10 Lead transfers per (from desk, to desk) this month.

C. Finance, broker and compliance questions (21–30) — Q11–Q20

- [ ] **21.** Q11 FTDs per Dubai day, last 30 days, zero days included.
- [ ] **22.** Q12 Approved USD per desk per month, last 6 months.
- [ ] **23.** Q13 Average first deposit and FTD count by lead source.
- [ ] **24.** Q14 FTDs and second deposits within 30 days, per desk.
- [ ] **25.** Q15 Revenue and funded leads per root broker (whole Brf tree).
- [ ] **26.** Q16 Last month's commission statement per broker.
- [ ] **27.** Q17 Top 3 depositors per desk this month.
- [ ] **28.** Q18 Median and 90th-percentile approved deposit per desk.
- [ ] **29.** Q19 High-severity breaches per agent (30 days) and breaches per 100 calls.
- [ ] **30.** Q20 Open trades older than 7 days per symbol, with exposure.

D. Hardening (31–40) — make it trustworthy

- [ ] **31.** Run `library verify` on the 50,000-lead seed. Record timings; fix the two slowest questions.
- [ ] **32.** Read the plan of every question over 200 ms; justify every Seq Scan that remains.
- [ ] **33.** Add only the indexes the library proves it needs; remove one that nothing uses.

- [ ] **34.** Edge data: an empty desk, a lead with no calls, an archived lead, a lead without a broker. Verify still passes.
- [ ] **35.** A call at 23:30 and one at 00:30 Dubai time: make sure ORM and SQL put them on the same days.
- [ ] **36.** Change request: a new deposit status `chargeback` must stop counting as approved. Change one place, re-verify.
- [ ] **37.** Change request: the business week now starts on Sunday. Where does that live?
- [ ] **38.** Export every question's result for one `as_of` as a CSV pack.
- [ ] **39.** Write your 12 rules for Django ORM work at Nexora.
- [ ] **40.** Self-assessment: the 15 things you can now do from a blank editor.

Solutions

The library is a package:

```
crm/library/
  __init__.py      conventions + exports (task 2)
  registry.py      Question, @question, REGISTRY (task 1)
  timeutil.py      as_of helpers (task 3)
  verify.py        normalise, verify, Result (tasks 4-6)
  ops.py           Q01-Q10
  finance.py       Q11-Q20
crm/management/commands/library.py   (tasks 7, 9)
```

A. Drills — the library

1–3.

```
# crm/library/registry.py
from dataclasses import dataclass
from typing import Callable


@dataclass(frozen=True)
class Question:
    id: str
    title: str
    definition: str
    grain: str
```

```
    key: tuple            # the columns that identify one row (the grain, as
column names)
    owner: str
    budget: int           # max queries orm() may run
    orm: Callable         # orm(as_of) -> iterable of dicts
    sql: str              # independent SQL; uses %(as_of)s


REGISTRY = {}


def question(id, title, *, definition, grain, key, owner, sql, budget=1):
    def register(fn):
        if id in REGISTRY:
            raise ValueError(f"duplicate question id {id}")
        REGISTRY[id] = Question(id, title, definition, grain, tuple(key),
owner, budget, fn, sql)
        return fn
    return register
```

```
# crm/library/__init__.py
"""Nexora query library. Conventions that EVERY question follows:

- Time zone: Asia/Dubai. "Day", "month" and hour buckets are Dubai calendar
units,
  in SQL written as (ts AT TIME ZONE 'Asia/Dubai').
- Every question takes `as_of` (an aware datetime) and never calls
timezone.now(): the same
  as_of always gives the same answer on the same data.
- Approved deposit = DepositQuerySet.approved(); in SQL: status = 'approved'.
- FTD = a lead's first approved deposit ordered by (created_at, id); the FTD
happens at that
  deposit's created_at (when the client paid).
- Deposit VOLUME (revenue, commission, medians) belongs to approved_at; calls
to started_at;
  leads to created_at.
- Archived leads are excluded everywhere -- including their deposits, calls
and trades when a
  question starts from those tables (Lead's manager doesn't apply across
joins: filter
  account__lead__archived_at__isnull=True explicitly).
- Money stays Decimal; rounding (half-up, 2 places) happens only for display
and comparison.
```

```
"""
from .registry import REGISTRY, question          # noqa: F401
from .verify import verify                        # noqa: F401


def load_all():
    """Import the question modules so they register themselves."""
    from . import finance, ops                    # noqa: F401
    return REGISTRY
```

```
# crm/library/timeutil.py
from datetime import datetime, time, timedelta

from django.utils import timezone


def day_start(d):
    return timezone.make_aware(datetime.combine(d, time.min))      # 00:00
Dubai on date d


def day_bounds(as_of):
    d = timezone.localdate(as_of)
    return day_start(d), day_start(d + timedelta(days=1))


def month_first(as_of, back=0):
    """First day (a date) of as_of's Dubai month, `back` months earlier."""
    d = timezone.localdate(as_of).replace(day=1)
    for _ in range(back):
        d = (d - timedelta(days=1)).replace(day=1)
    return d
```

4-6.

```
# crm/library/verify.py
import time
from dataclasses import dataclass, field
from datetime import datetime
from decimal import ROUND_HALF_UP, Decimal

from django.db import connection
from django.test.utils import CaptureQueriesContext
```

```
from crm.sql import fetch_dicts

CENT = Decimal("0.01")


def _norm(value):                                                       #
5
    if isinstance(value, bool) or value is None:
        return value
    if isinstance(value, (int, float, Decimal)):
        return Decimal(str(value)).quantize(CENT, rounding=ROUND_HALF_UP)   #
7 == 7.00 == 7.0
    if isinstance(value, datetime):
        return value.isoformat()
    return value                                                    # str,
date stay as they are


def normalise(rows, key):
    out = {}
    for row in rows:
        k = tuple(_norm(row[c]) for c in key)
        if k in out:
            raise ValueError(f"duplicate key {k}: the rows don't match the
declared grain")
        out[k] = {c: _norm(v) for c, v in row.items()}
    return out


@dataclass
class Result:                                                           #
6
    id: str
    rows: int
    queries: int
    orm_ms: float
    sql_ms: float
    problems: list = field(default_factory=list)

    @property
    def ok(self):
        return not self.problems
```

```
def verify(q, as_of):                                                    #
4
    with CaptureQueriesContext(connection) as ctx:
        t = time.perf_counter()
        orm_rows = list(q.orm(as_of))
        orm_ms = (time.perf_counter() - t) * 1000
    t = time.perf_counter()
    sql_rows = fetch_dicts(q.sql, {"as_of": as_of})
    sql_ms = (time.perf_counter() - t) * 1000

    res = Result(q.id, len(orm_rows), len(ctx), orm_ms, sql_ms)
    if len(ctx) > q.budget:
        res.problems.append(f"{len(ctx)} queries (budget {q.budget})")
    if orm_rows and sql_rows and set(orm_rows[0]) != set(sql_rows[0]):
        res.problems.append(f"columns differ: orm={sorted(orm_rows[0])} sql=
{sorted(sql_rows[0])}")
    try:
        a, b = normalise(orm_rows, q.key), normalise(sql_rows, q.key)
    except ValueError as exc:
        res.problems.append(str(exc))
        return res
    for k in sorted(a.keys() | b.keys(), key=str):
        if a.get(k) != b.get(k):
            res.problems.append(f"{k}: orm={a.get(k)} sql={b.get(k)}")
    return res
```

5: half-up rounding to cents on BOTH sides, so `Decimal('20.445')` from PostgreSQL and a Python float `20.445` compare equal. Without it, Python's round-half-even and PostgreSQL's half-up disagree on exactly the values that matter in money.

7–10.

```
# crm/management/commands/library.py
from datetime import datetime

from django.core.management.base import BaseCommand, CommandError
from django.utils import timezone

from crm.library import load_all, verify
```

```
class Command(BaseCommand):
    help = "Nexora query library: list | run | verify | catalogue"

    def add_arguments(self, parser):
        parser.add_argument("action", choices=["list", "run", "verify",
"catalogue"])
        parser.add_argument("ids", nargs="*")
        parser.add_argument("--as-of", type=datetime.fromisoformat)

    def handle(self, *args, action, ids, as_of, **opts):
        registry = load_all()
        as_of = as_of or timezone.now()
        if timezone.is_naive(as_of):
            as_of = timezone.make_aware(as_of)             # read as Dubai
time
        questions = [registry[i] for i in ids] if ids else [registry[k] for k
in sorted(registry)]

        if action == "list":
            for q in questions:
                self.stdout.write(f"{q.id}  {q.title}  [grain: {q.grain};
owner: {q.owner}; budget {q.budget}]")
        elif action == "run":
            for q in questions:
                self.stdout.write(f"\n{q.id} {q.title} (as of {as_of:%Y-%m-%d
%H:%M})")
                for row in q.orm(as_of):
                    self.stdout.write(f"  {row}")
        elif action == "catalogue":                                       #
9
            self.stdout.write("| id | title | grain | owner | queries |\n| --
- | --- | --- | --- | --- |")
            for q in questions:
                self.stdout.write(f"| {q.id} | {q.title} | {q.grain} |
{q.owner} | {q.budget} |")
            for q in questions:
                self.stdout.write(f"\n**{q.id} {q.title}.** {q.definition}")
        else:
            failed = 0
            for q in questions:
                r = verify(q, as_of)
                self.stdout.write(f"{'OK  ' if r.ok else 'FAIL'} {r.id}
rows={r.rows:<5} queries={r.queries}  "
```

```
                                f"orm={r.orm_ms:7.1f}ms  sql=
{r.sql_ms:7.1f}ms")
                for p in r.problems[:5]:
                    self.stdout.write(f"       {p}")
                failed += not r.ok
            if failed:
                raise CommandError(f"{failed} question(s) failed
verification")
```

```
python manage.py library list
python manage.py library run Q07
python manage.py library verify --as-of 2026-10-20T18:00
python manage.py library catalogue > docs/query-library.md
```

```
# crm/tests/test_library.py  (8, 10)
from io import StringIO

from django.core.management import call_command
from django.test import TestCase
from django.utils import timezone

from crm.library import load_all, verify


class LibraryTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        call_command("seed_nexora", leads=300, stdout=StringIO())
        cls.as_of = timezone.now()     # ONE frozen instant for every
question in this run

    def test_every_question_verifies(self):
        for qid, q in sorted(load_all().items()):
            with self.subTest(qid):
                self.assertEqual(verify(q, self.as_of).problems, [])
```

10: freezing `as_of` per run makes all 20 answers describe the same moment, so they're consistent with each other. For numbers that are identical on every machine and every day, give the seed an anchor too (a `--anchor 2026-10-20T00:00` option used instead of `timezone.now()` inside `seed_nexora`) and pass the same instant as `as_of`.

B. Operations questions — Q01–Q10

```
# crm/library/ops.py
from datetime import timedelta
from decimal import ROUND_HALF_UP, Decimal

from django.db.models import Case, Count, Exists, F, OuterRef, Q, Subquery,
Sum, Value, When
from django.db.models.functions import Coalesce, ExtractHour, Right

from crm.models import CallLog, Lead, LeadTransfer

from .registry import question
from .timeutil import day_bounds, day_start, month_first

DUBAI_MONTH_START = "(date_trunc('month', %(as_of)s AT TIME ZONE
'Asia/Dubai') AT TIME ZONE 'Asia/Dubai')"


@question("Q01", "Open and stale leads per agent", grain="agent", key=
["agent_name"], owner="Sales ops",
          definition="Active new/callback leads created before as_of, per
assigned agent. Stale = never "
                     "contacted, or last contact more than 3 days before
as_of.",
          sql="""
    SELECT u.username AS agent_name, COUNT(*) AS open_leads,
           COUNT(*) FILTER (WHERE l.last_contacted_at IS NULL
                            OR l.last_contacted_at < %(as_of)s - interval
'3 days') AS stale
    FROM crm_lead l JOIN auth_user u ON u.id = l.assigned_to_id
    WHERE l.status IN ('new', 'callback') AND l.archived_at IS NULL AND
l.created_at < %(as_of)s
    GROUP BY u.username""")
def q01(as_of):
    cutoff = as_of - timedelta(days=3)
    return (Lead.objects.open().filter(created_at__lt=as_of,
assigned_to__isnull=False)
            .values(agent_name=F("assigned_to__username"))
            .annotate(open_leads=Count("id"),
                      stale=Count("id",
filter=Q(last_contacted_at__isnull=True) | Q(last_contacted_at__lt=cutoff))))
```

```
@question("Q02", "Calls per agent today", grain="agent", key=["agent_name"],
owner="Sales ops",
          definition="Calls started on as_of's Dubai day, up to as_of: count,
talk seconds, distinct leads.",
          sql="""
    SELECT u.username AS agent_name, COUNT(*) AS calls, SUM(c.duration_sec)
AS talk_sec,
           COUNT(DISTINCT c.lead_id) AS leads
    FROM crm_calllog c JOIN auth_user u ON u.id = c.agent_id
    WHERE (c.started_at AT TIME ZONE 'Asia/Dubai')::date = (%(as_of)s AT TIME
ZONE 'Asia/Dubai')::date
      AND c.started_at < %(as_of)s
    GROUP BY u.username""")
def q02(as_of):
    start, _ = day_bounds(as_of)
    return (CallLog.objects.filter(started_at__gte=start,
started_at__lt=as_of)
            .values(agent_name=F("agent__username"))
            .annotate(calls=Count("id"), talk_sec=Sum("duration_sec"),
leads=Count("lead", distinct=True)))


@question("Q03", "First-contact SLA per desk", grain="desk", key=
["desk_name"], owner="Compliance",
          definition="Leads created between 31 days and 1 day before as_of;
in_time = has a call within "
                     "24 hours of creation.",
          sql="""
    SELECT d.name AS desk_name, COUNT(*) AS leads,
           COUNT(*) FILTER (WHERE EXISTS (
               SELECT 1 FROM crm_calllog c
               WHERE c.lead_id = l.id AND c.started_at <= l.created_at +
interval '24 hours')) AS in_time
    FROM crm_lead l JOIN crm_desk d ON d.id = l.desk_id
    WHERE l.archived_at IS NULL
      AND l.created_at >= %(as_of)s - interval '31 days' AND l.created_at < %
(as_of)s - interval '1 day'
    GROUP BY d.name""")
def q03(as_of):
    called = Exists(CallLog.objects.filter(lead=OuterRef("pk"),

started_at__lte=OuterRef("created_at") + timedelta(hours=24)))
```

```
    return (Lead.objects.filter(created_at__gte=as_of - timedelta(days=31),
created_at__lt=as_of - timedelta(days=1))
            .annotate(ok=called).values(desk_name=F("desk__name"))
            .annotate(leads=Count("id"), in_time=Count("id",
filter=Q(ok=True))))


@question("Q04", "Callback backlog by age", grain="age bucket", key=
["bucket"], owner="Sales ops",
          definition="Active callback leads created before as_of, bucketed by
time since last touch "
                     "(last contact, else creation): 0-1d, 1-3d, 3-7d, 7d+.",
          sql="""
    SELECT CASE WHEN t.touch >= %(as_of)s - interval '1 day' THEN '0-1d'
                WHEN t.touch >= %(as_of)s - interval '3 days' THEN '1-3d'
                WHEN t.touch >= %(as_of)s - interval '7 days' THEN '3-7d'
                ELSE '7d+' END AS bucket,
           COUNT(*) AS leads
    FROM (SELECT COALESCE(last_contacted_at, created_at) AS touch FROM
crm_lead
          WHERE status = 'callback' AND archived_at IS NULL AND created_at <
%(as_of)s) t
    GROUP BY 1""")
def q04(as_of):
    bucket = Case(When(touch__gte=as_of - timedelta(days=1), then=Value("0-
1d")),
                  When(touch__gte=as_of - timedelta(days=3), then=Value("1-
3d")),
                  When(touch__gte=as_of - timedelta(days=7), then=Value("3-
7d")),
                  default=Value("7d+"))
    return (Lead.objects.filter(status="callback", created_at__lt=as_of)
            .annotate(touch=Coalesce("last_contacted_at",
"created_at")).annotate(bucket=bucket)
            .values("bucket").annotate(leads=Count("id")))


@question("Q05", "Open leads by latest call outcome", grain="outcome", key=
["last_outcome"], owner="Sales ops",
          definition="Active new/callback leads, grouped by the outcome of
their latest call before as_of "
                     "('never_called' if none).",
          sql="""
```

```
    SELECT COALESCE(lc.outcome, 'never_called') AS last_outcome, COUNT(*) AS
leads
    FROM crm_lead l
    LEFT JOIN LATERAL (
        SELECT outcome FROM crm_calllog c
        WHERE c.lead_id = l.id AND c.started_at < %(as_of)s
        ORDER BY c.started_at DESC, c.id DESC LIMIT 1) lc ON true
    WHERE l.status IN ('new', 'callback') AND l.archived_at IS NULL AND
l.created_at < %(as_of)s
    GROUP BY 1""")
def q05(as_of):
    latest = (CallLog.objects.filter(lead=OuterRef("pk"),
started_at__lt=as_of)
              .order_by("-started_at", "-id").values("outcome")[:1])
    return (Lead.objects.open().filter(created_at__lt=as_of)
            .annotate(last_outcome=Coalesce(Subquery(latest),
Value("never_called")))
            .values("last_outcome").annotate(leads=Count("id")))
```

```
# crm/library/ops.py (continued)
def _per_100(n, d):
    return (Decimal(100 * n) / Decimal(d)).quantize(Decimal("0.01"),
rounding=ROUND_HALF_UP) if d else None


@question("Q06", "Calls and FTDs per agent this month", grain="agent", key=
["agent_name"], owner="Sales ops",
          budget=2,
          definition="From the start of as_of's Dubai month to as_of: calls
made by the agent, FTDs of leads "
                     "assigned to the agent, and FTDs per 100 calls.",
          sql=f"""
    WITH b AS (SELECT {DUBAI_MONTH_START} AS m_start),
    calls AS (
        SELECT agent_id, COUNT(*) AS n FROM crm_calllog, b
        WHERE started_at >= b.m_start AND started_at < %(as_of)s GROUP BY
agent_id),
    firsts AS (
        SELECT DISTINCT ON (a.lead_id) a.lead_id, d.created_at
        FROM nexora_deposits d JOIN crm_tradingaccount a ON a.id =
d.account_id
        WHERE d.status = 'approved' ORDER BY a.lead_id, d.created_at, d.id),
    ftds AS (
```

```
        SELECT l.assigned_to_id AS agent_id, COUNT(*) AS n
        FROM firsts f JOIN crm_lead l ON l.id = f.lead_id, b
        WHERE l.archived_at IS NULL AND l.assigned_to_id IS NOT NULL
          AND f.created_at >= b.m_start AND f.created_at < %(as_of)s
        GROUP BY 1)
    SELECT u.username AS agent_name, COALESCE(c.n, 0) AS calls, COALESCE(f.n,
0) AS ftds,
           ROUND(100.0 * COALESCE(f.n, 0) / NULLIF(c.n, 0), 2) AS
ftd_per_100_calls
    FROM auth_user u LEFT JOIN calls c ON c.agent_id = u.id LEFT JOIN ftds f
ON f.agent_id = u.id
    WHERE c.n IS NOT NULL OR f.n IS NOT NULL""")
def q06(as_of):
    start = day_start(month_first(as_of))
    calls = dict(CallLog.objects.filter(started_at__gte=start,
started_at__lt=as_of)
                 .values_list("agent__username").annotate(n=Count("id")))
    ftds = dict(Lead.objects.ftd_between(start,
as_of).filter(assigned_to__isnull=False)

.values_list("assigned_to__username").annotate(n=Count("id")))
    return [{"agent_name": a, "calls": calls.get(a, 0), "ftds": ftds.get(a,
0),
             "ftd_per_100_calls": _per_100(ftds.get(a, 0), calls.get(a, 0))}
            for a in calls.keys() | ftds.keys()]


@question("Q07", "Open leads with no call in 7 days", grain="desk", key=
["desk_name"], owner="Sales ops",
          definition="Active new/callback leads created before as_of with no
call in the 7 days before as_of.",
          sql="""
    SELECT d.name AS desk_name, COUNT(*) AS leads
    FROM crm_lead l JOIN crm_desk d ON d.id = l.desk_id
    WHERE l.status IN ('new', 'callback') AND l.archived_at IS NULL AND
l.created_at < %(as_of)s
      AND NOT EXISTS (SELECT 1 FROM crm_calllog c WHERE c.lead_id = l.id
                      AND c.started_at >= %(as_of)s - interval '7 days' AND
c.started_at < %(as_of)s)
    GROUP BY d.name""")
def q07(as_of):
    recent = CallLog.objects.filter(lead=OuterRef("pk"),
started_at__gte=as_of - timedelta(days=7),
```

```
                                    started_at__lt=as_of)
    return
(Lead.objects.open().filter(created_at__lt=as_of).filter(~Exists(recent))
            .values(desk_name=F("desk__name")).annotate(leads=Count("id")))


@question("Q08", "Possible duplicate groups", grain="desk", key=
["desk_name"], owner="Data quality",
          definition="Active leads created before as_of grouped by desk and
the last 7 phone digits; "
                     "groups = tails shared by 2+ leads, leads = leads in
those groups.",
          sql="""
    SELECT desk_name, COUNT(*) AS groups, SUM(n) AS leads FROM (
        SELECT d.name AS desk_name, right(l.phone, 7) AS tail, COUNT(*) AS n
        FROM crm_lead l JOIN crm_desk d ON d.id = l.desk_id
        WHERE l.archived_at IS NULL AND l.created_at < %(as_of)s
        GROUP BY 1, 2 HAVING COUNT(*) > 1) g
    GROUP BY desk_name""")
def q08(as_of):
    groups = (Lead.objects.filter(created_at__lt=as_of)
              .values(desk_name=F("desk__name"), tail=Right("phone", 7))
              .annotate(n=Count("id")).filter(n__gt=1))
    out = {}
    for g in groups:                          # a second GROUP BY over a few
rows: Python is fine
        row = out.setdefault(g["desk_name"], {"desk_name": g["desk_name"],
"groups": 0, "leads": 0})
        row["groups"] += 1
        row["leads"] += g["n"]
    return list(out.values())


@question("Q09", "Call heatmap: desk x hour", grain="desk, Dubai hour", key=
["desk_name", "hour"],
          owner="Workforce planning",
          definition="Calls in the 7 days before as_of by the desk at call
time and the Dubai hour.",
          sql="""
    SELECT d.name AS desk_name, EXTRACT(HOUR FROM c.started_at AT TIME ZONE
'Asia/Dubai')::int AS hour,
           COUNT(*) AS calls
    FROM crm_calllog c JOIN crm_desk d ON d.id = c.desk_id
```

```
    WHERE c.started_at >= %(as_of)s - interval '7 days' AND c.started_at < %
(as_of)s
    GROUP BY 1, 2""")
def q09(as_of):
    return (CallLog.objects.filter(started_at__gte=as_of - timedelta(days=7),
started_at__lt=as_of)
            .annotate(hour=ExtractHour("started_at"))
            .values("hour",
desk_name=F("desk__name")).annotate(calls=Count("id")))


@question("Q10", "Lead transfers between desks this month", grain="desk
pair",
          key=["from_desk_name", "to_desk_name"], owner="Sales ops",
          definition="LeadTransfer rows from the start of as_of's Dubai month
to as_of.",
          sql=f"""
    SELECT fd.name AS from_desk_name, td.name AS to_desk_name, COUNT(*) AS
transfers
    FROM crm_leadtransfer t
    JOIN crm_desk fd ON fd.id = t.from_desk_id JOIN crm_desk td ON td.id =
t.to_desk_id
    WHERE t.at >= {DUBAI_MONTH_START} AND t.at < %(as_of)s
    GROUP BY 1, 2""")
def q10(as_of):
    return
(LeadTransfer.objects.filter(at__gte=day_start(month_first(as_of)),
at__lt=as_of)
            .values(from_desk_name=F("from_desk__name"),
to_desk_name=F("to_desk__name"))
            .annotate(transfers=Count("id")))
```

B notes: Q02 deliberately writes the day condition differently in each answer (a range in the ORM, a Dubai-date comparison in SQL) — two routes to the same rows is what makes the check worth having. Q08 and Q10 return no rows on the seed (no duplicate tails, no transfers); create a pair of duplicates and a transfer to see them verify with data. Q06's SQL keeps the `f"..."` prefix only to insert the constant `DUBAI_MONTH_START` text; `as_of` remains a parameter.

C. Finance, broker and compliance questions — Q11–Q20

```
# crm/library/finance.py
from datetime import timedelta

from django.contrib.auth import get_user_model
from django.db.models import Avg, Count, DateField, F, OuterRef, Q, Subquery,
Sum, Window
from django.db.models.functions import Coalesce, RowNumber, TruncDate,
TruncMonth
from django.utils import timezone

from crm.expressions import Percentile
from crm.models import CallLog, Deposit, Lead, SOPBreach, Trade

from .ops import _per_100
from .registry import question
from .timeutil import day_start, month_first

# the lead's approved deposits in FTD order (ORM) and each active lead's FTD
row (SQL)
FIRST_DEP =
Deposit.objects.approved().filter(account__lead=OuterRef("pk")).order_by("cre
ated_at", "id")
FIRSTS_SQL = """firsts AS (
    SELECT DISTINCT ON (a.lead_id) a.lead_id, d.id AS deposit_id,
d.created_at, d.amount_usd
    FROM nexora_deposits d
    JOIN crm_tradingaccount a ON a.id = d.account_id
    JOIN crm_lead l ON l.id = a.lead_id AND l.archived_at IS NULL
    WHERE d.status = 'approved'
    ORDER BY a.lead_id, d.created_at, d.id)"""
LOCAL_MONTH = "date_trunc('month', %(as_of)s AT TIME ZONE 'Asia/Dubai')"
# a local timestamp
ACTIVE_DEPOSITS = """nexora_deposits d
    JOIN crm_tradingaccount a ON a.id = d.account_id
    JOIN crm_lead l ON l.id = a.lead_id AND l.archived_at IS NULL"""


@question("Q11", "FTDs per day, last 30 days", grain="Dubai day", key=
["day"], owner="Finance",
          definition="For each of the 30 Dubai days ending on as_of's day:
FTDs that happened that day "
```

```
                    "before as_of. Days with none show 0.",
          sql=f"""
    WITH {FIRSTS_SQL},
    days AS (SELECT generate_series((%(as_of)s AT TIME ZONE
'Asia/Dubai')::date - 29,
                                    (%(as_of)s AT TIME ZONE
'Asia/Dubai')::date, interval '1 day')::date AS day)
    SELECT days.day, COUNT(f.lead_id) AS ftds
    FROM days LEFT JOIN firsts f
      ON (f.created_at AT TIME ZONE 'Asia/Dubai')::date = days.day AND
f.created_at < %(as_of)s
    GROUP BY days.day""")
def q11(as_of):
    first = timezone.localdate(as_of) - timedelta(days=29)
    counts =
dict(Lead.objects.annotate(ftd_at=Subquery(FIRST_DEP.values("created_at")
[:1]))
                 .filter(ftd_at__gte=day_start(first), ftd_at__lt=as_of)

.annotate(day=TruncDate("ftd_at")).values("day").annotate(n=Count("id"))
                 .values_list("day", "n"))
    days = [first + timedelta(days=i) for i in range(30)]
    return [{"day": d, "ftds": counts.get(d, 0)} for d in days]


@question("Q12", "Approved USD per desk per month", grain="month, desk", key=
["month", "desk_name"],
          owner="Finance",
          definition="Approved deposit USD by Dubai month of approved_at,
from 5 months before as_of's "
                     "month to as_of.",
          sql=f"""
    SELECT date_trunc('month', d.approved_at AT TIME ZONE 'Asia/Dubai')::date
AS month,
           k.name AS desk_name, SUM(d.amount_usd) AS usd
    FROM {ACTIVE_DEPOSITS} JOIN crm_desk k ON k.id = l.desk_id
    WHERE d.status = 'approved'
      AND d.approved_at >= ({LOCAL_MONTH} - interval '5 months') AT TIME ZONE
'Asia/Dubai'
      AND d.approved_at < %(as_of)s
    GROUP BY 1, 2""")
def q12(as_of):
    return (Deposit.objects.approved()
```

```
            .filter(approved_at__gte=day_start(month_first(as_of, back=5)),
approved_at__lt=as_of,
                    account__lead__archived_at__isnull=True)
            .annotate(month=TruncMonth("approved_at",
output_field=DateField()))
            .values("month", desk_name=F("account__lead__desk__name"))
            .annotate(usd=Sum("amount_usd")))


@question("Q13", "Average first deposit by source", grain="lead source", key=
["source"], owner="Marketing",
          definition="FTDs before as_of per lead source, and the average FTD
amount in USD.",
          sql=f"""
    WITH {FIRSTS_SQL}
    SELECT l.source, COUNT(*) AS ftds, AVG(f.amount_usd) AS avg_first_usd
    FROM firsts f JOIN crm_lead l ON l.id = f.lead_id
    WHERE f.created_at < %(as_of)s
    GROUP BY l.source""")
def q13(as_of):
    return
(Lead.objects.annotate(ftd_at=Subquery(FIRST_DEP.values("created_at")[:1]),

ftd_usd=Subquery(FIRST_DEP.values("amount_usd")[:1]))
            .filter(ftd_at__lt=as_of).values("source")
            .annotate(ftds=Count("id"), avg_first_usd=Avg("ftd_usd")))


@question("Q14", "Second deposit within 30 days of FTD", grain="desk", key=
["desk_name"], owner="Retention",
          definition="Per desk: FTDs before as_of, and how many of those
leads made their second approved "
                     "deposit within 30 days of the first (and before
as_of).",
          sql="""
    WITH ranked AS (
        SELECT a.lead_id, d.created_at,
               ROW_NUMBER() OVER (PARTITION BY a.lead_id ORDER BY
d.created_at, d.id) AS rn
        FROM nexora_deposits d JOIN crm_tradingaccount a ON a.id =
d.account_id
        WHERE d.status = 'approved'),
    pairs AS (
```

```
        SELECT r1.lead_id, r1.created_at AS ftd_at, r2.created_at AS
second_at
        FROM ranked r1 LEFT JOIN ranked r2 ON r2.lead_id = r1.lead_id AND
r2.rn = 2
        WHERE r1.rn = 1)
    SELECT k.name AS desk_name, COUNT(*) AS ftds,
           COUNT(*) FILTER (WHERE p.second_at <= p.ftd_at + interval '30
days'
                            AND p.second_at < %(as_of)s) AS repeat_30d
    FROM pairs p JOIN crm_lead l ON l.id = p.lead_id AND l.archived_at IS
NULL
    JOIN crm_desk k ON k.id = l.desk_id
    WHERE p.ftd_at < %(as_of)s
    GROUP BY k.name""")
def q14(as_of):
    return
(Lead.objects.annotate(ftd_at=Subquery(FIRST_DEP.values("created_at")[:1]),

second_at=Subquery(FIRST_DEP.values("created_at")[1:2]))
            .filter(ftd_at__lt=as_of).values(desk_name=F("desk__name"))
            .annotate(ftds=Count("id"),
                      repeat_30d=Count("id",
filter=Q(second_at__lte=F("ftd_at") + timedelta(days=30),
                                                    second_at__lt=as_of))))


@question("Q15", "Revenue per root broker", grain="root broker", key=
["root_code"], owner="Partnerships",
          definition="Approved deposit USD (approved before as_of) and funded
leads for every lead under "
                     "each top-level broker, at any depth of the Brf tree.",
          sql=f"""
    WITH RECURSIVE tree AS (
        SELECT id, code AS root_code, 0 AS depth FROM crm_broker WHERE
parent_id IS NULL
        UNION ALL
        SELECT b.id, t.root_code, t.depth + 1 FROM crm_broker b JOIN tree t
ON b.parent_id = t.id
        WHERE t.depth < 20)
    SELECT t.root_code, SUM(d.amount_usd) AS revenue_usd, COUNT(DISTINCT
l.id) AS funded_leads
    FROM {ACTIVE_DEPOSITS} JOIN tree t ON t.id = l.broker_id
    WHERE d.status = 'approved' AND d.approved_at < %(as_of)s
```

```
    GROUP BY t.root_code""")
def q15(as_of):
    return (Deposit.objects.approved()
            .filter(approved_at__lt=as_of,
account__lead__archived_at__isnull=True,
                    account__lead__broker__isnull=False)
            .values(root_code=F("account__lead__broker__tree__root_code"))
# Day 14's view
            .annotate(revenue_usd=Sum("amount_usd"),
funded_leads=Count("account__lead", distinct=True)))


@question("Q16", "Last month's commission per broker", grain="broker", key=
["broker_code"], owner="Finance",
          definition="Deposits approved in the Dubai month before as_of's
month, per introducing broker: "
                     "USD and commission (sum of amount_usd x the broker's
rate).",
          sql=f"""
    SELECT br.code AS broker_code, SUM(d.amount_usd) AS deposits_usd,
           SUM(d.amount_usd * br.commission_rate) AS commission_usd
    FROM {ACTIVE_DEPOSITS} JOIN crm_broker br ON br.id = l.broker_id
    WHERE d.status = 'approved'
      AND d.approved_at >= ({LOCAL_MONTH} - interval '1 month') AT TIME ZONE
'Asia/Dubai'
      AND d.approved_at < {LOCAL_MONTH} AT TIME ZONE 'Asia/Dubai'
    GROUP BY br.code""")
def q16(as_of):
    rate = F("account__lead__broker__commission_rate")
    return (Deposit.objects.approved()
            .filter(approved_at__gte=day_start(month_first(as_of, back=1)),
                    approved_at__lt=day_start(month_first(as_of)),
                    account__lead__archived_at__isnull=True,
account__lead__broker__isnull=False)
            .values(broker_code=F("account__lead__broker__code"))
            .annotate(deposits_usd=Sum("amount_usd"),
commission_usd=Sum(F("amount_usd") * rate)))


@question("Q17", "Top 3 depositors per desk this month", grain="desk, rank",
key=["desk_name", "rank"],
          owner="VIP team",
          definition="Per desk, the 3 active leads with the most approved USD
```

```
approved from the start of "
                     "as_of's Dubai month to as_of (ties broken by lead
id).",
          sql=f"""
    WITH sums AS (
        SELECT l.id AS lead_id, l.desk_id, SUM(d.amount_usd) AS month_usd
        FROM {ACTIVE_DEPOSITS}
        WHERE d.status = 'approved' AND d.approved_at >= {LOCAL_MONTH} AT
TIME ZONE 'Asia/Dubai'
          AND d.approved_at < %(as_of)s
        GROUP BY l.id, l.desk_id),
    ranked AS (
        SELECT s.*, ROW_NUMBER() OVER (PARTITION BY s.desk_id ORDER BY
s.month_usd DESC, s.lead_id) AS rank
        FROM sums s)
    SELECT k.name AS desk_name, r.rank, r.lead_id, r.month_usd
    FROM ranked r JOIN crm_desk k ON k.id = r.desk_id
    WHERE r.rank <= 3""")
def q17(as_of):
    month_usd = (Deposit.objects.approved()
                 .filter(account__lead=OuterRef("pk"),
approved_at__gte=day_start(month_first(as_of)),
                         approved_at__lt=as_of)

.order_by().values("account__lead").annotate(t=Sum("amount_usd")).values("t")
)
    return
(Lead.objects.annotate(month_usd=Subquery(month_usd)).filter(month_usd__isnul
l=False)
            .annotate(rank=Window(RowNumber(), partition_by=[F("desk")],
                                  order_by=[F("month_usd").desc(),
F("id").asc()]))
            .filter(rank__lte=3)
            .values("rank", "month_usd", desk_name=F("desk__name"),
lead_id=F("id")))


@question("Q18", "Median and p90 deposit per desk", grain="desk", key=
["desk_name"], owner="Finance",
          definition="Approved deposits approved in the 90 days before as_of:
count, median and 90th "
                     "percentile USD (continuous percentiles).",
          sql=f"""
```

```
    SELECT k.name AS desk_name, COUNT(*) AS n,
           percentile_cont(0.5) WITHIN GROUP (ORDER BY d.amount_usd) AS
median_usd,
           percentile_cont(0.9) WITHIN GROUP (ORDER BY d.amount_usd) AS
p90_usd
    FROM {ACTIVE_DEPOSITS} JOIN crm_desk k ON k.id = l.desk_id
    WHERE d.status = 'approved' AND d.approved_at >= %(as_of)s - interval '90
days'
      AND d.approved_at < %(as_of)s
    GROUP BY k.name""")
def q18(as_of):
    return (Deposit.objects.approved()
            .filter(approved_at__gte=as_of - timedelta(days=90),
approved_at__lt=as_of,
                    account__lead__archived_at__isnull=True)
            .values(desk_name=F("account__lead__desk__name"))
            .annotate(n=Count("id"), median_usd=Percentile("amount_usd",
0.5),
                      p90_usd=Percentile("amount_usd", 0.9)))


@question("Q19", "High-severity breaches per agent", grain="agent", key=
["agent_name"], owner="Compliance",
          definition="Agents with high-severity SOP breaches in the 30 days
before as_of: breaches, calls in "
                     "the same window, breaches per 100 calls.",
          sql="""
    WITH b AS (
        SELECT s.agent_id, COUNT(*) AS n FROM crm_sopbreach s JOIN
crm_soprule r ON r.id = s.rule_id
        WHERE r.severity = 3 AND s.occurred_at >= %(as_of)s - interval '30
days' AND s.occurred_at < %(as_of)s
        GROUP BY s.agent_id),
    c AS (
        SELECT agent_id, COUNT(*) AS n FROM crm_calllog
        WHERE started_at >= %(as_of)s - interval '30 days' AND started_at < %
(as_of)s GROUP BY agent_id)
    SELECT u.username AS agent_name, b.n AS high_breaches, COALESCE(c.n, 0)
AS call_count,
           ROUND(100.0 * b.n / NULLIF(c.n, 0), 2) AS breaches_per_100_calls
    FROM b JOIN auth_user u ON u.id = b.agent_id LEFT JOIN c ON c.agent_id =
b.agent_id""")
def q19(as_of):
```

```
    since = as_of - timedelta(days=30)
    breaches = (SOPBreach.objects.high().filter(agent=OuterRef("pk"),
occurred_at__gte=since, occurred_at__lt=as_of)

.order_by().values("agent").annotate(n=Count("id")).values("n"))
    calls = (CallLog.objects.filter(agent=OuterRef("pk"),
started_at__gte=since, started_at__lt=as_of)
             .order_by().values("agent").annotate(n=Count("id")).values("n"))
    rows = (get_user_model().objects
            .annotate(high_breaches=Coalesce(Subquery(breaches), 0),
call_count=Coalesce(Subquery(calls), 0))
            .filter(high_breaches__gt=0)
            .values("high_breaches", "call_count", agent_name=F("username")))
    return [{**r, "breaches_per_100_calls": _per_100(r["high_breaches"],
r["call_count"])} for r in rows]
# `calls` as an annotation name would clash with User's reverse relation
`calls` -- hence call_count


@question("Q20", "Stale open trades per symbol", grain="symbol", key=
["symbol"], owner="Dealing desk",
          definition="Trades of active leads opened more than 7 days before
as_of and still open at as_of; "
                     "exposure = sum of volume x open_price.",
          sql="""
    SELECT t.symbol, COUNT(*) AS trades, SUM(t.volume * t.open_price) AS
exposure
    FROM crm_trade t JOIN crm_tradingaccount a ON a.id = t.account_id
    JOIN crm_lead l ON l.id = a.lead_id AND l.archived_at IS NULL
    WHERE t.opened_at < %(as_of)s - interval '7 days'
      AND (t.closed_at IS NULL OR t.closed_at >= %(as_of)s)
    GROUP BY t.symbol""")
def q20(as_of):
    return (Trade.objects.filter(opened_at__lt=as_of - timedelta(days=7),
account__lead__archived_at__isnull=True)
            .filter(Q(closed_at__isnull=True) | Q(closed_at__gte=as_of))
# open AT as_of, not today
            .values("symbol").annotate(trades=Count("id"),
exposure=Sum(F("volume") * F("open_price"))))
```

C notes: Q20 doesn't use Day 11's `Trade.objects.stale()` — that method reads `timezone.now()`, which breaks the `as_of` rule. Library questions take time as an argument; convenience methods for screens may use "now". Q15 compares Day 14's view

(ORM side) with a fresh recursive CTE (SQL side): if someone edits the view, verify notices.

D. Hardening

31–33 — speed, from evidence.

```
# 31  python manage.py library verify      (50,000 leads; typical shape of
the output)
OK   Q01  rows=12   queries=1  orm=   38.0ms  sql=   31.2ms
...
OK   Q11  rows=30   queries=1  orm= 1410.0ms  sql=  690.4ms   <- slowest
OK   Q06  rows=12   queries=2  orm= 1190.6ms  sql=  702.9ms   <- second
Both compute the FIRST deposit of EVERY lead (50,000 subqueries) to keep one
month's FTDs.
```

```
# 31 -- the fix (Day 9, task 28): start from recent deposits, keep those with
no earlier approved one
from django.db.models import Exists


def ftd_deposits(start, end):
    """Approved deposits created in [start, end) that are their (active)
lead's FTD."""
    earlier =
Deposit.objects.approved().filter(account__lead=OuterRef("account__lead")).fi
lter(
        Q(created_at__lt=OuterRef("created_at")) |
Q(created_at=OuterRef("created_at"), id__lt=OuterRef("id")))
    return (Deposit.objects.approved()
            .filter(created_at__gte=start, created_at__lt=end,
account__lead__archived_at__isnull=True)
            .filter(~Exists(earlier)))


# q11's counts become:
counts = dict(ftd_deposits(day_start(first),
as_of).annotate(day=TruncDate("created_at"))
              .values("day").annotate(n=Count("id")).values_list("day", "n"))
# and q06's FTDs: ftd_deposits(start,
as_of).values_list("account__lead__assigned_to__username")
#                                      .annotate(n=Count("id"))  (exclude
```

```
NULL agents)
# re-run verify: same rows, a fraction of the time -- the SQL side is
untouched, so the check
# proves the rewrite changed the SPEED and nothing else.
```

```
# 32  plans of everything over 200 ms -- Seq Scans that are fine, and why
crm_desk, crm_broker, crm_soprule      tiny tables: a Seq Scan is the fastest
plan
Q13 / Q15 all-time aggregates          they read every approved deposit by
definition
Anything else on crm_calllog or nexora_deposits with a selective time filter
should be an
Index or Bitmap scan -- if not, check the filter is on the raw column (Day 5,
task 33).

# 33  indexes the library proves it needs
Deposit.Meta.indexes += [models.Index(fields=["approved_at"],
condition=Q(status="approved"),
                                      name="deposit_approved_at_idx")]   #
Q12, Q15-Q18 filter on it
Then pg_stat_user_indexes after a week of verify runs and real traffic: an
index with
idx_scan = 0 that no constraint needs (Day 9, task 29) is removed in its own
migration.
```

34–37 — edges and change requests.

```
# 34  edge data -- add, then verify
- a desk with no leads: absent from both answers (GROUP BY has no rows for
it) -- consistent
- a lead with no calls: Q05 counts it as never_called on both sides
- an archived lead WITH deposits: Q12-Q18 must exclude it on both sides. If
one side forgets the
  explicit archived filter (Day 11, task 31), verify FAILS -- exactly the bug
it exists to catch.
- a lead without a broker: excluded from Q15/Q16 by both (inner joins /
broker__isnull=False)
Review finding: Q02, Q09 and Q10 start from calls and transfers and still
COUNT archived
leads' rows on both sides -- verify passes because the two answers agree with
each other, not
with the convention. Add lead__archived_at__isnull=True (and the matching
JOIN) to both.
```

```
Tests check consistency; only reading the definitions checks the definitions.

# 35  calls at 23:30 and 00:30 Dubai time around as_of's day boundary
Q02 (ORM: range from Dubai midnight; SQL: AT TIME ZONE 'Asia/Dubai') and Q09
(ExtractHour vs
EXTRACT ... AT TIME ZONE) put both calls on the same days and hours -- the
shared convention
made them agree. Rerun with the SQL's AT TIME ZONE removed and verify fails
on those rows.

# 36  change request: a new boolean Deposit.charged_back; charged-back
deposits stop counting
1 DepositQuerySet.approved(): self.filter(status="approved",
charged_back=False) -- one place for
  the whole ORM side, every screen and report included.
2 Run verify: every deposit question FAILS -- the SQL strings still say
status = 'approved'.
3 Update FIRSTS_SQL, ACTIVE_DEPOSITS and each WHERE deliberately, re-verify,
done.
The failure is the feature: the definition can't silently differ between the
two answers.

# 37  the business week now starts on Sunday
One helper in timeutil.py -- week_start(as_of) -> the Sunday on or before
as_of's Dubai date --
and one SQL expression next to DUBAI_MONTH_START:
  date_trunc('week', (%(as_of)s AT TIME ZONE 'Asia/Dubai') + interval '1
day') - interval '1 day'
(PostgreSQL's date_trunc('week') is Monday-based; shifting by a day makes it
Sunday-based.)
Write it into the conventions docstring; every weekly question uses only
these two.
```

38–40.

```
# 38 -- crm/library/export.py
import csv
from pathlib import Path

from . import load_all


def export_pack(as_of, folder):
    """One CSV per question, all for the same as_of."""
    folder = Path(folder)
    folder.mkdir(parents=True, exist_ok=True)
    for qid, q in sorted(load_all().items()):
        rows = list(q.orm(as_of))
        with open(folder / f"{qid}.csv", "w", newline="") as fh:
            if rows:
                writer = csv.DictWriter(fh, fieldnames=list(rows[0]))
                writer.writeheader()
                writer.writerows(rows)
    (folder / "README.txt").write_text(f"Nexora query library pack, as of
{as_of.isoformat()}\n")
```

```
# 39  12 rules for Django ORM work at Nexora
 1  Read the SQL. Every new query gets show() or explain() before it ships.
 2  Rules that must always hold live in PostgreSQL: constraints, then
triggers. Python gives the
    friendly message; the database gives the guarantee.
 3  Money is Decimal end to end. Round once, at the edge, half-up.
 4  Time is aware and Dubai-based for business days. Filter on raw columns
with ranges.
 5  Define the grain before writing an aggregate. Never aggregate two to-many
joins together.
 6  Every "latest", "first" and "top N" ordering ends with a unique
tiebreaker.
 7  Business words live once: QuerySet methods (open, stale, approved,
visible_to).
 8  Screens and reports have query budgets, locked by tests.
 9  Writes that must agree happen in one transaction; HTTP and notifications
happen on commit.
10  Concurrency is designed: F(), version checks, skip_locked, unique keys
for idempotency.
11  Migrations are reviewed SQL: expand / migrate / contract on big tables,
CONCURRENTLY for indexes.
```

```
12  Numbers people act on have a second, independent answer -- and the two
are compared.

# 40  from a blank editor, you can now:
 1  model a domain with the right fields, constraints, indexes and
relationships
 2  write, read, reverse and squash migrations, including data migrations and
big-table changes
 3  express any WHERE clause with lookups, Q, F and Exists -- and predict its
SQL
 4  build GROUP BY reports with conditional aggregates and HAVING
 5  use subqueries and window functions for latest / first / top-N / running
totals
 6  find and remove N+1 queries; design indexes from EXPLAIN ANALYZE
 7  stream and bulk-load large data without running out of memory
 8  make concurrent writes correct with transactions, locks and idempotency
 9  give the ORM a domain vocabulary with custom QuerySets and managers
10  drop to raw SQL safely for CTEs, LATERAL, generate_series and COPY
11  use PostgreSQL features: arrays, JSON, search, trigrams, ranges,
exclusion constraints
12  choose between abstract, proxy, multi-table and one-to-one designs
13  build views, materialised views, a second database, facts and an
incremental load
14  test data: quality checks, reconciliation, and ORM-vs-SQL verification
15  explain all of it -- in Django and in SQL -- to an interviewer at a DIFC
data team
```

```
python manage.py library verify && python manage.py test && git add -A && git
commit -m "orm day 15: query library"
```